

ソーラーカー EMS live / live_wifi 実装・計算完全解説

PowerShell 入口から速度指令まで

Contents

1	実行入口	1
1.1	PowerShell から ROS 2 まで	1
1.2	標準コマンド	1
1.3	profile path の解決	1
1.4	live 採用 profile の境界	1
2	起動 node と条件	2
3	時系列	3
3.1	process 開始から定常周期まで	3
3.2	並行性	3
4	UDP 文字列から状態量まで	4
4.1	受信 packet	4
4.2	timestamp 判定	4
4.3	一次遅れ filter と slew limit	4
5	距離と勾配	5
5.1	距離積分	5
5.2	勾配	5
6	天候取得と風補正	6
6.1	予報 CSV	6
6.2	cell 温度近似	6
6.3	向かい風成分	6
6.4	距離相関補正	6
7	車両物理 model	7
7.1	空気密度	7
7.2	走行抵抗	7
7.3	motor map	7
7.4	太陽電池	7
7.5	pack 電力と電池	7
7.6	補機電力	8
8	上位 MPC	9
8.1	距離離散化	9
8.2	状態遷移	9
8.3	目的関数	9
8.4	探索	9
9	下位 MPC と指令	10
9.1	reference	10
9.2	下位目的関数	10
9.3	外部指令 gate	10

10 MHE と自動校正	11
10.1 battery MHE	11
10.2 autocal	11
11 保存と表示	12
11.1 CSV logger	12
11.2 内蔵 dashboard	12
11.3 Grafana	12
12 1 周期の数値計算	13
12.1 入力	13
12.2 抵抗と電力	13
12.3 電池と SoC	13
13 故障時遷移	14
14 監査で修正した箇所	15
15 live 到達 file 一覧	16
16 ROS interface 完全表	18
17 全関数の入出力・呼出関係	24
17.1 launch/solar_race_live.launch.py	24
17.2 launch/solar_race_live_wifi.launch.py	24
17.3 mpc_solarcar/dashboard_node.py	24
17.4 mpc_solarcar/distance_node.py	28
17.5 mpc_solarcar/estimator.py	29
17.6 mpc_solarcar/forecast_grid.py	29
17.7 mpc_solarcar/grade_node.py	30
17.8 mpc_solarcar/live_launch.py	31
17.9 mpc_solarcar/model.py	31
17.10mpc_solarcar/mpc_node.py	34
17.11mpc_solarcar/path_utils.py	40
17.12mpc_solarcar/route_utils.py	40
17.13mpc_solarcar/schedule_utils.py	41
17.14mpc_solarcar/signal_utils.py	42
17.15mpc_solarcar/solar_autocal_logic.py	43
17.16mpc_solarcar/solar_autocal_node.py	43
17.17mpc_solarcar/solar_logger_node.py	44
17.18mpc_solarcar/solar_preflight_logic.py	45
17.19mpc_solarcar/solar_preflight_node.py	46
17.20mpc_solarcar/solar_profile.py	46
17.21mpc_solarcar/speed_command_bridge_node.py	47
17.22mpc_solarcar/telemetry_protocol.py	48
17.23mpc_solarcar/telemetry_text_bridge_node.py	48
17.24mpc_solarcar/upper_cost.py	50
17.25mpc_solarcar/upper_horizon.py	51
17.26mpc_solarcar/upper_policy.py	51
17.27mpc_solarcar/upper_solver.py	52
17.28mpc_solarcar/utils_maps.py	52
17.29mpc_solarcar/weather_fetch_node.py	53
17.30mpc_solarcar/weather_utils.py	53
17.31mpc_solarcar/wind_correction_node.py	54

17.32	setup.py	55
A	live 到達 source 全文	56
A.1	SolarSim.ps1	56
A.2	scripts/solar_control.sh	59
A.3	launch/solar_race_live.launch.py	61
A.4	launch/solar_race_live_wifi.launch.py	62
A.5	mpc_solarcar/live_launch.py	62
A.6	mpc_solarcar/solar_profile.py	64
A.7	mpc_solarcar/path_utils.py	65
A.8	mpc_solarcar/mpc_node.py	66
A.9	mpc_solarcar/model.py	91
A.10	mpc_solarcar/utils_maps.py	96
A.11	mpc_solarcar/route_utils.py	96
A.12	mpc_solarcar/schedule_utils.py	97
A.13	mpc_solarcar/forecast_grid.py	100
A.14	mpc_solarcar/upper_policy.py	101
A.15	mpc_solarcar/upper_cost.py	102
A.16	mpc_solarcar/upper_horizon.py	104
A.17	mpc_solarcar/upper_solver.py	106
A.18	mpc_solarcar/estimator.py	111
A.19	mpc_solarcar/signal_utils.py	112
A.20	mpc_solarcar/solar_preflight_logic.py	113
A.21	mpc_solarcar/solar_preflight_node.py	114
A.22	mpc_solarcar/distance_node.py	115
A.23	mpc_solarcar/grade_node.py	116
A.24	mpc_solarcar/weather_utils.py	117
A.25	mpc_solarcar/weather_fetch_node.py	118
A.26	mpc_solarcar/solar_autocal_logic.py	119
A.27	mpc_solarcar/solar_autocal_node.py	120
A.28	mpc_solarcar/speed_command_bridge_node.py	121
A.29	mpc_solarcar/telemetry_protocol.py	123
A.30	mpc_solarcar/telemetry_text_bridge_node.py	124
A.31	mpc_solarcar/wind_correction_node.py	127
A.32	mpc_solarcar/solar_logger_node.py	129
A.33	mpc_solarcar/dashboard_node.py	131
A.34	dashboard/index.html	135
A.35	dashboard/app.js	137
A.36	dashboard/style.css	139
A.37	grafana/docker-compose.yml	142
A.38	grafana/prometheus.yml	143
A.39	grafana/provisioning/dashboards/dashboards.yml	143
A.40	grafana/provisioning/datasources/prometheus.yml	143
A.41	grafana/dashboards/solarcar-ems.json	143
A.42	config/solar/bwsc_2027_demo.yaml	145
A.43	package.xml	148
A.44	setup.py	148

Chapter 1

実行入口

1.1 PowerShell から ROS 2 まで

1. Windows PowerShell が SolarSim.ps1 の引数を検証する。
2. Resolve-WslDistro が WSL distribution を決める。
3. Windows path を wslpath -a で Linux path へ変換する。
4. WSL 内で scripts/solar_control.sh up live_wifi <profile> を実行する。
5. colcon build --packages-select mpc_solarcar を実行する。
6. 既存 launch process を停止する。
7. ros2 launch mpc_solarcar solar_race_live_wifi.launch.py profile_yaml:=... を別 process group で起動する。
8. dashboard API が応答するまで Windows 側が最大 90 秒待つ。
9. Docker があれば Prometheus と Grafana を起動し、なければ内蔵 dashboard を開く。
10. rqt graph exporter が実際に存在する node/topic を取得する。

1.2 標準コマンド

```
1 powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 `
2   -Action up -Mode live_wifi `
3   -Profile project_packages/<vehicle>/profile.yaml
```

-Mode live は ROS topic を外部装置が直接 publish する構成である。-Mode live_wifi は UDP JSON 文字列を telemetry_text_bridge_node が ROS topic へ変換する構成である。

1.3 profile path の解決

相対 path は profile YAML 自身の directory を基準にする。root 基準ではない。計算は

$$p_{\text{resolved}} = \text{resolve}(p_{\text{profile.parent}}, p_{\text{yaml}})$$

である。値の優先順位は、node 内既定値、profile の model/mpc/live、launch が明示する runtime 値、ROS launch override の順である。

1.4 live 採用 profile の境界

config/solar/bwsc_2027_demo.yaml1 と project_packages/bwsc2027_template/profile.yaml1 は入力 schema であり、車両定数 0 のままでは preflight を通さない。bwsc2025_fitted_mle13_grounded_segmented は履歴再現用で、受入資料の high_precision_claim_allowed=false により live 採用外である。live 投入条件は、同定後の受入 YAML で fullsim_adoption_gate_pass=true かつ high_precision_claim_allowed=true、静的監査 ERROR 0、実機 preflight RUNNING の全成立である。

Chapter 2

起動 node と条件

node	起動条件	処理
solar_preflight_node	常時	必須 telemetry と planner status の鮮度を 1 秒周期で判定する。
mpc_node	常時	1 時間級上位再計画、1 秒級下位追従、物理 model、制約判定を実行する。
dashboard_node	常時	ROS 値を共有 state へ格納し HTTP /api/state と Prometheus /metrics を公開する。
solar_logger_node	常時	profile 指定 rate で CSV へ時系列を書き込む。
speed_command_bridge_node	command_bridge.enabled	planner 指令を安全 gate、平滑化、rate limit 後に ROS/UDP へ出す。
distance_node	use_distance_node	実測速度を時間積分して距離を publish する。
grade_node	use_grade_node	高度差と距離差から勾配を計算する。
weather_fetch_node	weather.enabled	chase GPS 地点の予報を取得して runtime CSV を atomic replace する。
solar_autocal_node	autocal.enabled	太陽光 gain、走行 power gain、補機 power を限定条件下で逐次更新する。
telemetry_text_bridge_node	live_wifi かつ wifi_bridge.enabled	UDP 受信、timestamp 検査、値域/rate 検査、filter、ROS 変換、返信を行う。
wind_correction_node	live_wifi かつ wind_model.enabled	実測向かい風誤差を距離相関で予報 CSV 全体へ伝播する。

Chapter 3

時系列

3.1 process 開始から定常周期まで

時刻	実行主体	処理
$t = 0$	launch	profile 読込、予報 runtime file の初期 copy、node action 生成。
$t \approx 0$	各 node constructor	parameter 宣言、map/route/weather 読込、publisher/subscriber/timer 生成。
$t = 0-10\text{ s}$	preflight	STARTING。command gate は安全速度を出す。
20 Hz	Wi-Fi bridge	non-blocking UDP socket を poll する。
5 Hz 標準	command bridge	gate、一次遅れ、加減速 limit、量子化、publish。
2 Hz 標準	logger/distance	CSV 保存、速度積分距離。
1 Hz	preflight	missing/stale 判定。
1 Hz 標準	lower MPC	現在速度から短期 torque 列を最適化して先頭指令を publish。
1 s 監視	upper callback	初回、再計画時刻、予報更新条件で長距離 MPC を実行。
30 s 標準	autocal/wind	calibration 更新、風補正 CSV 更新。
3600 s 標準	weather	最新予報取得。
200 ms	browser	/api/state を取得して画面更新。
1 s	Prometheus	/metrics を scrape し Grafana へ供給。

3.2 並行性

mpc_node は 4 thread executor を使う。callback group は次の 4 本である。

```
telemetry_callback_group : sensor reception
upper_callback_group      : long-horizon MPC
lower_callback_group      : short-horizon MPC
command_callback_group    : command publication
```

各 group 内は直列、異なる group は並行である。長い上位探索中も telemetry callback と下位指令周期を継続する。

Chapter 4

UDP 文字列から状態量まで

4.1 受信 packet

UTF-8 JSON を 1 datagram 1 packet で送る。最低限 timestamp を含める。

```
1 {"source":"solar","ts_unix":181000000.25,  
2  "vehicle":{"speed_kmh":70.2,"soc":0.73,"batt_voltage_v":89.1,  
3  "batt_current_a":8.7,"batt_temp_c":28.4,"solar_power_w":430.0,  
4  "latitude":-20.1,"longitude":133.2,"altitude_m":310.0}}
```

chase 側は source=chase とし、GPS、風速、風向、進行方向を送る。packet は missing timestamp、古すぎる、未来、duplicate、許容外 out-of-order の順に拒否される。

4.2 timestamp 判定

$$a = t_{\text{receive}} - t_{\text{source}}$$

$$\text{accept} \iff -a_{\text{future,max}} \leq a \leq a_{\text{max}} \wedge t_{\text{source}} > t_{\text{last}} - \Delta t_{\text{reorder}}$$

同一 timestamp は duplicate として拒否する。PC、solar Raspberry Pi、chase Raspberry Pi は NTP で UTC を同期する。

4.3 一次遅れ filter と slew limit

入力 x 、前回出力 y_{k-1} 、時刻差 Δt 、時定数 τ に対し

$$\alpha = 1 - \exp(-\Delta t / \tau), \quad \tilde{y}_k = y_{k-1} + \alpha(x_k - y_{k-1}).$$

上昇率 r_+ 、下降率 r_- を使い

$$y_k = y_{k-1} + \text{clip}(\tilde{y}_k - y_{k-1}, -r_- \Delta t, r_+ \Delta t)$$

とする。最後に値域 clip、deadband、量子化を適用する。

Chapter 5

距離と勾配

5.1 距離積分

速度 v_k [km/h] を受けた時刻差を Δt [s] として

$$s_{k+1} = s_k + \frac{v_k}{3600} \min(\Delta t, \Delta t_{\max}).$$

timer は保存済み s を publish する。reset service は $s = 0$ へ戻す。

5.2 勾配

高度を EMA で

$$h_f \leftarrow h_f + \alpha_h(h - h_f)$$

とし、 Δs が閾値以上かつ速度が閾値以上のとき

$$g\% = 100 \frac{h_{f,k} - h_{f,k-1}}{1000(s_k - s_{k-1})}$$

を publish する。GPS 高度と専用高度 topic の後着値が入力になる。

Chapter 6

天候取得と風補正

6.1 予報 CSV

weather node は現在の chase GPS、未受信時は fallback 緯度経度を使う。Open-Meteo 応答を指定 step へ補間し、temporary file へ書いてから置換する。MPC は `forecast_reload_sec` ごとに mtime を確認し、変更時だけ再読込する。

6.2 cell 温度近似

$$T_{\text{cell}} = T_{\text{amb}} + k_T G_{\text{POA}}$$

である。 k_T は profile の `live.weather.tcell_gain` から入る。

6.3 向かい風成分

風向 ψ_w 、進行方向 ψ_c 、風速 u_w に対し、source 定義に合わせて

$$u_h = u_w \cos(\psi_w - \psi_c)$$

を得る。符号規約は正を向かい風とし、bridge で `max_abs_headwind_ms` へ clip する。

6.4 距離相関補正

現在地点 s_0 の残差 $e_0 = u_{h,\text{meas}} - u_{h,\text{forecast}}$ を

$$e(s) = e_0 \exp\left(-\frac{|s - s_0|}{L_c}\right)$$

で伝播する。予報分散は予測時間と共に増やし、指定 quantile で planner 用風速を作る。

Chapter 7

車両物理 model

7.1 空気密度

constant mode では ρ を直接使う。ideal-gas-altitude mode では

$$p(h) = p_0 \left(1 - \frac{0.0065h}{288.15}\right)^{5.255877}, \quad \rho = \frac{p(h)}{287.05(T_{\text{amb}} + 273.15)}.$$

7.2 走行抵抗

$$\begin{aligned} v_r &= \max(0, v + u_h), \quad \theta = \tan^{-1}(g\%/100) \\ F_a &= \frac{1}{2}\rho C_d A v_r^2, \quad F_r = C_{rr} m g \cos \theta, \quad F_g = m g \sin \theta. \\ P_{\text{road}} &= (F_a + F_r + F_g)v, \quad P_{\text{wheel}} = P_{\text{road}} + P_{\text{inertia}}. \end{aligned}$$

空力抵抗は対気速度で求め、仕事率は対地速度を掛ける。

7.3 motor map

$$\omega_w = v/r_w, \quad T_w = P_{\text{wheel}}/(\omega_w + 10^{-3}), \quad T_m = T_w/i_g.$$

CSV 格子 (v, T) の 4 点から双線形補間して η_d または η_r を得る。駆動時

$$P_{dc,d} = \frac{\max(P_{\text{wheel}}, 0)}{\eta_d \eta_g \eta_{inv}},$$

回生時

$$P_{dc,r} = u_r \eta_r \eta_g \eta_{inv} \max(-P_{\text{wheel}}, 0).$$

7.4 太陽電池

map があれば (G, T_c) で η_p と η_{mppt} を双線形補間する。なければ

$$\begin{aligned} \eta_p &= \max\{0, \eta_{ref}[1 + \mu_p(T_c - 25)]\} g_p. \\ P_{pv,raw} &= \eta_p A_{pv} G, \quad P_{pv} = \min(P_{pv,raw} \eta_{mppt}, P_{pv,max}). \end{aligned}$$

7.5 pack 電力と電池

$$\begin{aligned} P_{\text{pack}} &= P_{dc,d} - P_{dc,r} + P_{aux} - P_{pv}. \\ R_t &= R_{int}(T, z) + R_{line} + R_p, \quad P_{\text{pack}} = I(V_{oc} - IR_t). \end{aligned}$$

低電流側の根を選び

$$I = \frac{V_{oc} - \sqrt{V_{oc}^2 - 4R_t P_{\text{pack}}}}{2R_t}, \quad V = V_{oc} - IR_t.$$

$Q_{nom} > 0$ なら coulomb counting

$$z_{k+1} = z_k - \eta_c \frac{I_k \Delta t}{3600 Q_{nom}},$$

未設定の legacy profile だけ energy integration を使う。

7.6 補機電力

走行中は P_{aux} 、停止かつ夜間閾値以下は $P_{aux,night}$ 、停止昼間は $P_{aux,stopped}$ を使う。夜間 0 W 設定はここで実現する。

Chapter 8

上位 MPC

8.1 距離離散化

残距離を固定 4 分割しない。upper_horizon.py が最小/最大距離刻み、成長率、control point、最大 step 数から境界 s_i を生成する。各区間の決定変数は速度 v_i である。

$$\Delta t_i = \frac{1000(s_{i+1} - s_i)}{v_i/3.6}.$$

route 勾配、標高、速度制限は区間平均または補間、予報は (t, s) 格子補間で得る。

8.2 状態遷移

各区間で物理 model から P_{pack}, I, V を計算し、SoC と温度を進める。走行可能時間外は schedule が次の開始まで待機時間を挿入し、停止時太陽光と補機で SoC を進める。control stop は stop 定義の hold 時間を挿入する。

8.3 目的関数

objective_mode=fastest_feasible では

$$J = \sum_i (\Delta t_{travel,i} + w_{wait} \Delta t_{wait,i}) + c \sum_j [\max(0, g_j)]^2$$

である。 g_j は速度、SoC、電流、電圧、温度、走行時間帯、終端条件の違反量である。weighted mode はさらに速度平滑、電流二乗、pack energy、Joule 損、空力 energy、機械 energy、pack power slew、SoC barrier 等を加える。全項の実装は本書の upper_cost.py 全文と関数表を参照する。

8.4 探索

1. 前回解または initial upper policy を horizon へ補間する。
2. 決定論的 seed と物理 seed を生成する。
3. CEM が bounded population を評価し elite 平均・分散を更新する。
4. 上位候補を L-BFGS-B で局所 refine する。
5. 設定時は SHGO または有限 grid 証明を追加する。
6. 目的値最小の finite feasible candidate を採用する。

MPC は一度作った 2025 速度列を 2027 へ固定適用しない。vehicle model は同定済み資産として共用するが、2027 予報と現在状態で上位問題を再構成する。

Chapter 9

下位 MPC と指令

9.1 reference

上位距離計画 $v^U(s)$ を現在距離から 1 秒刻みで sample し、加速上限、減速上限、deadband で整形して $v_{0:N}^{ref}$ を作る。

9.2 下位目的関数

制御変数は motor torque 列 $u_{0:N-1}$ である。

$$J_L = \sum_{k=0}^{N-1} \left[w_t (v_k - v_k^{ref})^2 + w_u u_k^2 + w_{\Delta u} (u_k - u_{k-1})^2 \right].$$

$$v_{k+1} = \max \left(0, v_k + \frac{F_{trac}(u_k) - F_{res}(v_k)}{m} \Delta t \right).$$

L-BFGS-B に torque bounds を渡し、先頭 torque から速度指令、throttle、drive mode を publish する。

9.3 外部指令 gate

許可条件は

$$t - t_0 \geq t_{hold}, \quad a_{speed} \leq a_{speed,max}, \quad state = \text{RUNNING}, \quad a_{state} \leq a_{state,max}.$$

一つでも偽なら target は `safe_speed_kmh`、mode は `stop` である。速度と mode の受信時刻は別に保持する。平滑化後に ROS output topic と任意 UDP へ出す。

Chapter 10

MHE と自動校正

10.1 battery MHE

window 内の入力 P_{pack}, T_{amb} と観測 V, T_b を使い、初期 $z_0, T_{b,0}$ を bounded optimization する。

$$J_{MHE} = \sum_k w_V (V_k - \hat{V}_k)^2 + w_T (T_{b,k} - \hat{T}_{b,k})^2 + w_z (z_0 - z_{prior})^2.$$

推定失敗または観測不足時は filter 済み実測/内部状態を維持する。

10.2 autocal

停止昼間で太陽光 power の比から solar gain、十分な走行速度で実測 pack power と model metric の比から drive gain、条件成立時だけ補機推定を EMA 更新する。各 gain は YAML の min/max で clip する。校正 topic を MPC が受け、model override へ反映する。

Chapter 11

保存と表示

11.1 CSV logger

ROS callback は共有 dictionary を更新し、timer だけが 1 行を書き込む。1 行には UTC、距離、上位/下位/出力/実測速度、SoC、温度、電圧、電流、環境、power、system state、diagnostic を含む。書込周期は `logging.log_rate_hz` である。

11.2 内蔵 dashboard

dashboard node は ROS 値を lock 付き state へ保存する。HTTP thread は JSON snapshot を返す。browser は 200 ms 周期で取得する。表示値は制御入力ではない。

11.3 Grafana

dashboard node の `/metrics` を Prometheus が 1 秒周期で取得し、Grafana provisioned dashboard が表示する。Grafana 停止や browser 停止は ROS 制御 process へ影響しない。

Chapter 12

1 周期の数値計算

12.1 入力

$m = 235 \text{ kg}$ 、 $C_d A = 0.11 \text{ m}^2$ 、 $C_{rr} = 0.008$ 、 $\rho = 1.18 \text{ kg/m}^3$ 、 $v = 70 \text{ km/h}$ 、向かい風 2 m/s 、勾配 0 、 $\eta_d = 0.92$ 、 $\eta_g = 1$ 、 $\eta_{inv} = 0.98$ 、 $P_{aux} = 21 \text{ W}$ 、 $P_{pv} = 400 \text{ W}$ とする。

12.2 抵抗と電力

$$v = 70/3.6 = 19.444 \text{ m/s}, \quad v_r = 21.444 \text{ m/s}$$

$$F_a = 0.5 \times 1.18 \times 0.11 \times 21.444^2 = 29.82 \text{ N}$$

$$F_r = 0.008 \times 235 \times 9.80665 = 18.44 \text{ N}$$

$$P_{road} = (29.82 + 18.44) \times 19.444 = 938.4 \text{ W}$$

$$P_{dc,d} = 938.4 / (0.92 \times 0.98) = 1040.9 \text{ W}$$

$$P_{pack} = 1040.9 + 21 - 400 = 661.9 \text{ W}.$$

12.3 電池と SoC

$V_{oc} = 92 \text{ V}$ 、 $R_t = 0.10 \Omega$ とすると

$$I = \frac{92 - \sqrt{92^2 - 4 \times 0.10 \times 661.9}}{0.20} = 7.25 \text{ A}, \quad V = 91.28 \text{ V}.$$

$Q_{nom} = 33 \text{ Ah}$ 、 $\Delta t = 1 \text{ s}$ なら

$$\Delta z = -\frac{7.25}{3600 \times 33} = -6.10 \times 10^{-5}.$$

この結果を制約、logger、dashboard へ同じ周期で渡す。

Chapter 13

故障時遷移

事象	検出	出力
速度 packet 欠落	speed age > timeout	safe speed へ rate 制限、mode stop。
preflight DEGRADED	system state 不一致	同上。
古い UDP packet	source timestamp age	packet 破棄、status へ理由。
異常 jump	value/rate filter	clip または破棄、前回 filter 値維持。
予報 API 失敗	exception/status	既存 runtime forecast を保持。
上位 solver 失敗	finite/feasible 検査	前回解または fallback。
下位 solver 失敗	optimizer result 検査	reference/fallback 指令。
dashboard/Grafana 停止	HTTP/Docker のみ	planner と logger は継続。

Chapter 14

監査で修正した箇所

修正前	修正	効果
配布生成時に巨大 external docs を copy 後削除	copy 段階で ignore	disk 不足による生成失敗を防止。
現行 MPC 更新と配布 snapshot が不一致	solar 依存だけ同期	forecast grid、route 平均、initial policy 等を配布へ反映。
command bridge の enabled 未使用	launch 条件へ反映	false 指定 node を起動しない。
wifi bridge の enabled 未使用	launch 条件へ反映	false 指定 socket を開かない。
mode 受信で速度鮮度も更新	受信時刻を分離	古い速度指令の延命を防止。
preflight は表示だけ	command gate へ接続	DEGRADED 時の速度出力を防止。
launch 依存未宣言	package.xml へ追加	clean ROS 環境での依存解決を明示。
dashboard の年表示固定	vehicle 非依存表記	2027 運用時の誤認防止。
相対 import の同梱検査なし	AST で全 local import を解決	mpc_node.py から estimator.py 等の欠落を ERROR 化。
PDF/fullsim の存在だけで fitted model を配布	受入 YAML 二重 gate を必須化	低精度 MLE13/MLE19 の live 誤採用を防止。
旧 MLE13 を検証済み live 入口と表記	歴史再現用へ明確化	物理契約 15 警告を未解決のまま運用しない。

Chapter 15

live 到達 file 一覽

path	lines	bytes
SolarSim.ps1	298	11207
scripts/solar_control.sh	255	5904
launch/solar_race_live.launch.py	22	745
launch/solar_race_live_wifi.launch.py	68	2517
mpc_solarcar/live_launch.py	175	8209
mpc_solarcar/solar_profile.py	118	3900
mpc_solarcar/path_utils.py	37	1241
mpc_solarcar/mpc_node.py	2573	127651
mpc_solarcar/model.py	477	20449
mpc_solarcar/utils_maps.py	33	1387
mpc_solarcar/route_utils.py	139	5471
mpc_solarcar/schedule_utils.py	217	8038
mpc_solarcar/forecast_grid.py	130	3937
mpc_solarcar/upper_policy.py	49	2097
mpc_solarcar/upper_cost.py	288	11307
mpc_solarcar/upper_horizon.py	145	5148
mpc_solarcar/upper_solver.py	473	21120
mpc_solarcar/estimator.py	111	3762
mpc_solarcar/signal_utils.py	155	5709
mpc_solarcar/solar_preflight_logic.py	68	2217
mpc_solarcar/solar_preflight_node.py	89	3358
mpc_solarcar/distance_node.py	60	1928
mpc_solarcar/grade_node.py	95	3581
mpc_solarcar/weather_utils.py	128	4484
mpc_solarcar/weather_fetch_node.py	144	6662
mpc_solarcar/solar_autocal_logic.py	24	701
mpc_solarcar/solar_autocal_node.py	158	7058
mpc_solarcar/speed_command_bridge_node.py	167	7947
mpc_solarcar/telemetry_protocol.py	76	2644
mpc_solarcar/telemetry_text_bridge_node.py	354	17681
mpc_solarcar/wind_correction_node.py	159	7951
mpc_solarcar/solar_logger_node.py	199	7439
mpc_solarcar/dashboard_node.py	418	18128
dashboard/index.html	150	5378
dashboard/app.js	218	7930
dashboard/style.css	340	7314
grafana/docker-compose.yml	39	1111
grafana/prometheus.yml	12	241
grafana/provisioning/dashboards/dashboards.yml	12	229
grafana/provisioning/datasources/prometheus.yml	13	255

path	lines	bytes
grafana/dashboards/solarcar-ems.json	153	11849
config/solar/bwsc_2027_demo.yaml	254	7843
package.xml	31	1106
setup.py	76	2923

Chapter 16

ROS interface 完全表

file	line	kind	type/period	topic/name	callback/default
mpe_solarcar/dashboard_node.py	58	parameter		'host'	'0.0.0.0'
mpe_solarcar/dashboard_node.py	59	parameter		'port'	8080
mpe_solarcar/dashboard_node.py	60	parameter		'static_dir'	''
mpe_solarcar/dashboard_node.py	61	parameter		'dummy_csv'	''
mpe_solarcar/dashboard_node.py	62	parameter		'dummy_rate_hz'	5.0
mpe_solarcar/dashboard_node.py	117	subscription	Float32	'/planner/speed_cmd'	self._on_speed_cmd
mpe_solarcar/dashboard_node.py	118	subscription	Float32	'/planner/upper_speed_cmd'	self._on_upper_speed_cmd
mpe_solarcar/dashboard_node.py	119	subscription	Float32	'/vehicle/speed_kmh'	self._on_speed_meas
mpe_solarcar/dashboard_node.py	120	subscription	Float32	'/planner/throttle_cmd_pct'	self._on_throttle_cmd
mpe_solarcar/dashboard_node.py	121	subscription	String	'/planner/drive_mode'	self._on_drive_mode
mpe_solarcar/dashboard_node.py	122	subscription	Float32	'/vehicle/batt_soc'	self._on_soc
mpe_solarcar/dashboard_node.py	123	subscription	Float32	'/vehicle/batt_temp_c'	self._on_tb
mpe_solarcar/dashboard_node.py	124	subscription	Float32	'/vehicle/batt_current_a'	self._on_ibatt
mpe_solarcar/dashboard_node.py	125	subscription	Float32	'/vehicle/batt_voltage_v'	self._on_vbatt
mpe_solarcar/dashboard_node.py	126	subscription	Float32	'/vehicle/s_kmh'	self._on_s_kmh
mpe_solarcar/dashboard_node.py	127	subscription	Float32MultiArray	'/planner/status'	self._on_status
mpe_solarcar/dashboard_node.py	128	subscription	Float32MultiArray	'/planner/env'	self._on_env
mpe_solarcar/dashboard_node.py	129	subscription	Float32MultiArray	'/planner/metrics'	self._on_metrics
mpe_solarcar/dashboard_node.py	130	subscription	Float32MultiArray	'/planner/upper_plan'	self._on_upper_plan
mpe_solarcar/dashboard_node.py	131	subscription	Float32MultiArray	'/planner/lower_plan'	self._on_lower_plan
mpe_solarcar/dashboard_node.py	132	subscription	String	'/system/state'	self._on_sys_state
mpe_solarcar/dashboard_node.py	133	subscription	String	'/system/diag'	self._on_sys_diag
mpe_solarcar/dashboard_node.py	134	subscription	String	'/system/mpc_state'	self._on_mpc_state
mpe_solarcar/dashboard_node.py	135	subscription	Float32	'/system/health'	self._on_health
mpe_solarcar/dashboard_node.py	136	subscription	NavSatFix	'/sim/gps'	self._on_gps
mpe_solarcar/dashboard_node.py	137	subscription	Path	'/planner/trajectory'	self._on_path
mpe_solarcar/dashboard_node.py	392	timer	period[s]	period	self._tick_dummy
mpe_solarcar/distance_node.py	12	parameter		'max_dt_sec'	2.5
mpe_solarcar/distance_node.py	13	parameter		'publish_rate_hz'	1.0
mpe_solarcar/distance_node.py	22	subscription	Float32	'/vehicle/speed_kmh'	self._on_speed
mpe_solarcar/distance_node.py	23	publisher	Float32	'/vehicle/s_kmh'	
mpe_solarcar/distance_node.py	26	timer	period[s]	1.0/publish_rate_hz	self._publish
mpe_solarcar/grade_node.py	13	parameter		'min_speed_kmh'	5.0
mpe_solarcar/grade_node.py	14	parameter		'altitude_alpha'	0.2
mpe_solarcar/grade_node.py	15	parameter		'min_delta_s_kmh'	0.01
mpe_solarcar/grade_node.py	16	parameter		'gps_topic'	'/sim/gps'
mpe_solarcar/grade_node.py	17	parameter		'altitude_topic'	'/vehicle/altitude_m'
mpe_solarcar/grade_node.py	34	subscription	Float32	'/vehicle/s_kmh'	self._on_s_kmh
mpe_solarcar/grade_node.py	35	subscription	Float32	'/vehicle/speed_kmh'	self._on_speed
mpe_solarcar/grade_node.py	36	subscription	Float32	self.altitude_topic	self._on_altitude
mpe_solarcar/grade_node.py	37	subscription	NavSatFix	self.gps_topic	self._on_gps
mpe_solarcar/grade_node.py	39	publisher	Float32	'/vehicle/grade'	
mpe_solarcar/grade_node.py	40	timer	period[s]	1.0	self._step
mpe_solarcar/mpc_node.py	278	parameter		'forecast_csv'	'inputs/forecast_10min.csv'
mpe_solarcar/mpc_node.py	279	parameter		'maps_dir'	'maps'
mpe_solarcar/mpc_node.py	280	parameter		'drive_eff_map'	''
mpe_solarcar/mpc_node.py	281	parameter		'regen_eff_map'	''
mpe_solarcar/mpc_node.py	282	parameter		'rint_map'	''
mpe_solarcar/mpc_node.py	283	parameter		'drive_map_eco'	''
mpe_solarcar/mpc_node.py	284	parameter		'drive_map_power'	''
mpe_solarcar/mpc_node.py	285	parameter		'regen_map_eco'	''
mpe_solarcar/mpc_node.py	286	parameter		'regen_map_power'	''
mpe_solarcar/mpc_node.py	287	parameter		'panel_eff_map'	''
mpe_solarcar/mpc_node.py	288	parameter		'mppt_eff_map'	''
mpe_solarcar/mpc_node.py	289	parameter		'ocv_soc_map'	''
mpe_solarcar/mpc_node.py	290	parameter		'dt'	600.0
mpe_solarcar/mpc_node.py	291	parameter		'horizon_steps'	9
mpe_solarcar/mpc_node.py	292	parameter		'v_max_kmh'	110.0
mpe_solarcar/mpc_node.py	293	parameter		'terminal_soc_min'	0.10
mpe_solarcar/mpc_node.py	294	parameter		'stop_yaml'	'inputs/stop_points.yaml'
mpe_solarcar/mpc_node.py	295	parameter		'forecast_time_mode'	'auto'
mpe_solarcar/mpc_node.py	296	parameter		'forecast_time_tz'	'UTC'
mpe_solarcar/mpc_node.py	297	parameter		'forecast_start_time_utc'	''
mpe_solarcar/mpc_node.py	298	parameter		'forecast_time_offset_sec'	0.0
mpe_solarcar/mpc_node.py	299	parameter		'forecast_reload_sec'	60.0
mpe_solarcar/mpc_node.py	300	parameter		'replan_on_forecast_reload'	False
mpe_solarcar/mpc_node.py	301	parameter		'params_yaml'	''
mpe_solarcar/mpc_node.py	302	parameter		'profile_runtime_mode'	''
mpe_solarcar/mpc_node.py	303	parameter		'route_profile_csv'	''
mpe_solarcar/mpc_node.py	304	parameter		'speed_profile_csv'	''
mpe_solarcar/mpc_node.py	305	parameter		'drive_schedule_yaml'	''
mpe_solarcar/mpc_node.py	306	parameter		'initial_upper_policy_csv'	''
mpe_solarcar/mpc_node.py	307	parameter		'use_measured_s'	True
mpe_solarcar/mpc_node.py	308	parameter		'use_measured_speed'	True

file	line	kind	type/period	topic/name	callback/default
mpe_solarcar/mpc_node.py	309	parameter		'soc0'	0.95
mpe_solarcar/mpc_node.py	310	parameter		'Tb0'	30.0
mpe_solarcar/mpc_node.py	311	parameter		's0_km'	0.0
mpe_solarcar/mpc_node.py	312	parameter		'speed_meas_timeout_sec'	3.0
mpe_solarcar/mpc_node.py	313	parameter		'distance_meas_timeout_sec'	5.0
mpe_solarcar/mpc_node.py	314	parameter		'battery_meas_timeout_sec'	15.0
mpe_solarcar/mpc_node.py	315	parameter		'speed_meas_filter_tau_sec'	0.6
mpe_solarcar/mpc_node.py	316	parameter		'speed_meas_max_accel_kmhps'	12.0
mpe_solarcar/mpc_node.py	317	parameter		'speed_meas_max_decel_kmhps'	20.0
mpe_solarcar/mpc_node.py	318	parameter		'distance_meas_max_rate_kmhps'	0.06
mpe_solarcar/mpc_node.py	319	parameter		'distance_meas_max_backtrack_km'	0.02
mpe_solarcar/mpc_node.py	320	parameter		'battery_meas_filter_tau_sec'	1.0
mpe_solarcar/mpc_node.py	321	parameter		'w_dv'	0.05
mpe_solarcar/mpc_node.py	322	parameter		'w_dv_limit'	2.0
mpe_solarcar/mpc_node.py	323	parameter		'dv_max_kmhps'	5.0
mpe_solarcar/mpc_node.py	324	parameter		'w_T'	5.0
mpe_solarcar/mpc_node.py	325	parameter		'w_speed_limit'	50.0
mpe_solarcar/mpc_node.py	326	parameter		'w_drive_window'	1.0e5
mpe_solarcar/mpc_node.py	327	parameter		'w_current'	0.02
mpe_solarcar/mpc_node.py	328	parameter		'soc_target'	0.5
mpe_solarcar/mpc_node.py	329	parameter		'soc_band'	0.1
mpe_solarcar/mpc_node.py	330	parameter		'w_soc_target'	2.0
mpe_solarcar/mpc_node.py	331	parameter		'w_soc_band'	50.0
mpe_solarcar/mpc_node.py	332	parameter		'soc_day_end_max'	-1.0
mpe_solarcar/mpc_node.py	333	parameter		'soc_day_end_target'	-1.0
mpe_solarcar/mpc_node.py	334	parameter		'w_soc_day_max'	1.0e4
mpe_solarcar/mpc_node.py	335	parameter		'w_soc_day_track'	0.0
mpe_solarcar/mpc_node.py	336	parameter		'soc_day_end_tol'	0.02
mpe_solarcar/mpc_node.py	337	parameter		'soc_finish_target'	-1.0
mpe_solarcar/mpc_node.py	338	parameter		'soc_finish_tol'	0.02
mpe_solarcar/mpc_node.py	339	parameter		'w_soc_progress'	1.0e5
mpe_solarcar/mpc_node.py	340	parameter		'w_soc_terminal'	1.0e5
mpe_solarcar/mpc_node.py	341	parameter		'race_km'	3035.5
mpe_solarcar/mpc_node.py	342	parameter		'upper_mode'	'time'
mpe_solarcar/mpc_node.py	343	parameter		'upper_ds_km'	20.0
mpe_solarcar/mpc_node.py	344	parameter		'upper_horizon_km'	1500.0
mpe_solarcar/mpc_node.py	345	parameter		'upper_max_steps'	200
mpe_solarcar/mpc_node.py	346	parameter		'upper_horizon_mode'	'fixed'
mpe_solarcar/mpc_node.py	347	parameter		'upper_adaptive_min_ds_km'	10.0
mpe_solarcar/mpc_node.py	348	parameter		'upper_adaptive_max_ds_km'	200.0
mpe_solarcar/mpc_node.py	349	parameter		'upper_adaptive_growth'	1.18
mpe_solarcar/mpc_node.py	350	parameter		'upper_ctrl_km'	0.0
mpe_solarcar/mpc_node.py	351	parameter		'upper_vmin_kmh'	1.0
mpe_solarcar/mpc_node.py	352	parameter		'upper_replan_km'	0.0
mpe_solarcar/mpc_node.py	353	parameter		'upper_replan_sec'	0.0
mpe_solarcar/mpc_node.py	354	parameter		'upper_max_iter'	120
mpe_solarcar/mpc_node.py	355	parameter		'upper_global_search_enabled'	True
mpe_solarcar/mpc_node.py	356	parameter		'upper_global_search_mode'	'auto'
mpe_solarcar/mpc_node.py	357	parameter		'upper_cem_generations'	4
mpe_solarcar/mpc_node.py	358	parameter		'upper_cem_population'	16
mpe_solarcar/mpc_node.py	359	parameter		'upper_cem_elite'	4
mpe_solarcar/mpc_node.py	360	parameter		'upper_local_refine_topk'	4
mpe_solarcar/mpc_node.py	361	parameter		'upper_seed_library_mode'	'full'
mpe_solarcar/mpc_node.py	362	parameter		'upper_shgo_samples'	256
mpe_solarcar/mpc_node.py	363	parameter		'upper_shgo_iters'	2
mpe_solarcar/mpc_node.py	364	parameter		'upper_cert_grid_levels'	0
mpe_solarcar/mpc_node.py	365	parameter		'upper_cert_max_evaluations'	250000
mpe_solarcar/mpc_node.py	366	parameter		'upper_cert_workers'	1
mpe_solarcar/mpc_node.py	367	parameter		'upper_day_end_soc_min'	0.2
mpe_solarcar/mpc_node.py	368	parameter		'soc_guard_margin'	0.01
mpe_solarcar/mpc_node.py	369	parameter		'soc_guard_mode'	'stop'
mpe_solarcar/mpc_node.py	370	parameter		'drive_mode'	'auto'
mpe_solarcar/mpc_node.py	371	parameter		'drive_mode_tau_margin'	0.0
mpe_solarcar/mpc_node.py	372	parameter		'hierarchical'	True
mpe_solarcar/mpc_node.py	373	parameter		'lower_dt'	1.0
mpe_solarcar/mpc_node.py	374	parameter		'lower_horizon_steps'	20
mpe_solarcar/mpc_node.py	375	parameter		'lower_rate_hz'	5.0
mpe_solarcar/mpc_node.py	379	parameter		'lower_max_iter'	0
mpe_solarcar/mpc_node.py	380	parameter		'lower_maxfun'	80
mpe_solarcar/mpc_node.py	381	parameter		'lower_skip_optimization_during_upper_solve'	True
mpe_solarcar/mpc_node.py	382	parameter		'w_track'	5.0
mpe_solarcar/mpc_node.py	383	parameter		'w_throttle'	0.2
mpe_solarcar/mpc_node.py	384	parameter		'throttle_rate_limit'	30.0
mpe_solarcar/mpc_node.py	385	parameter		'mhie_enabled'	True

file	line	kind	type/period	topic/name	callback/default
mpe_solarcar/mpc_node.py	386	parameter		'mhe_horizon_steps'	12
mpe_solarcar/mpc_node.py	387	parameter		'mhe_w_soc'	50.0
mpe_solarcar/mpc_node.py	388	parameter		'mhe_w_tb'	5.0
mpe_solarcar/mpc_node.py	389	parameter		'mhe_w_i'	1.0
mpe_solarcar/mpc_node.py	390	parameter		'mhe_w_v'	1.0
mpe_solarcar/mpc_node.py	391	parameter		'mhe_w_prior'	5.0
mpe_solarcar/mpc_node.py	392	parameter		'soc_min'	0.05
mpe_solarcar/mpc_node.py	393	parameter		'soc_max'	0.98
mpe_solarcar/mpc_node.py	394	parameter		'T_min'	-5.0
mpe_solarcar/mpc_node.py	395	parameter		'T_max'	55.0
mpe_solarcar/mpc_node.py	396	parameter		'solar_gain'	1.0
mpe_solarcar/mpc_node.py	397	parameter		'poa_gain_drive'	1.0
mpe_solarcar/mpc_node.py	398	parameter		'poa_gain_stop'	1.0
mpe_solarcar/mpc_node.py	399	parameter		'stop_tilt_fraction'	1.0
mpe_solarcar/mpc_node.py	400	parameter		'control_stop_tilt_fraction'	0.0
mpe_solarcar/mpc_node.py	401	parameter		'control_stop_arrival_tolerance_km'	0.20
mpe_solarcar/mpc_node.py	402	parameter		'control_stop_stationary_speed_kmh'	2.0
mpe_solarcar/mpc_node.py	403	parameter		'control_stop_brake_decel_kmhps'	3.5
mpe_solarcar/mpc_node.py	404	parameter		'control_stop_brake_margin_km'	0.03
mpe_solarcar/mpc_node.py	708	publisher	Float32	'/planner/speed_cmd'	
mpe_solarcar/mpc_node.py	709	publisher	Float32	'/planner/upper_speed_cmd'	
mpe_solarcar/mpc_node.py	710	publisher	Float32	'/planner/throttle_cmd_pct'	
mpe_solarcar/mpc_node.py	711	publisher	String	'/planner/drive_mode'	
mpe_solarcar/mpc_node.py	712	publisher	Path	'/planner/trajectory'	
mpe_solarcar/mpc_node.py	713	publisher	Float32MultiArray	'/planner/upper_plan'	
mpe_solarcar/mpc_node.py	714	publisher	Float32MultiArray	'/planner/lower_plan'	
mpe_solarcar/mpc_node.py	715	publisher	Float32MultiArray	'/planner/env'	
mpe_solarcar/mpc_node.py	716	publisher	Float32MultiArray	'/planner/metrics'	
mpe_solarcar/mpc_node.py	717	publisher	Float32MultiArray	'/planner/status'	
mpe_solarcar/mpc_node.py	720	subscription	Float32	'/vehicle/s_km'	self_on_s_km_solar
mpe_solarcar/mpc_node.py	721	subscription	Float32	'/vehicle/speed_kmh'	self_on_speed_solar
mpe_solarcar/mpc_node.py	722	subscription	Float32	'/vehicle/batt_soc'	self_on_soc_solar
mpe_solarcar/mpc_node.py	723	subscription	Float32	'/vehicle/batt_temp_c'	self_on_tb_solar
mpe_solarcar/mpc_node.py	724	subscription	Float32	'/vehicle/batt_current_a'	self_on_i_solar
mpe_solarcar/mpc_node.py	725	subscription	Float32	'/vehicle/batt_voltage_v'	self_on_v_solar
mpe_solarcar/mpc_node.py	726	subscription	Float32	'/calib/solar_gain'	self_on_calib_solar_gain
mpe_solarcar/mpc_node.py	727	subscription	Float32	'/calib/drive_power_gain'	self_on_calib_drive_power_gain
mpe_solarcar/mpc_node.py	728	subscription	Float32	'/calib/aux_power_w'	self_on_calib_aux_power
mpe_solarcar/mpc_node.py	731	timer	period[s]	1.0	self_step_solar
mpe_solarcar/mpc_node.py	738	timer	period[s]	1.0/self.lower_rate_hz	self_step_lower
mpe_solarcar/mpc_node.py	741	timer	period[s]	1.0/self.lower_rate_hz	self_publish_lower_command_cycle
mpe_solarcar/solar_autocal_node.py	20	parameter		"publish_period_sec"	30.0
mpe_solarcar/solar_autocal_node.py	21	parameter		"stationary_speed_kmh"	2.0
mpe_solarcar/solar_autocal_node.py	22	parameter		"drive_speed_kmh"	25.0
mpe_solarcar/solar_autocal_node.py	23	parameter		"day_ghi_threshold"	150.0
mpe_solarcar/solar_autocal_node.py	24	parameter		"alpha"	0.2
mpe_solarcar/solar_autocal_node.py	25	parameter		"solar_gain_init"	1.0
mpe_solarcar/solar_autocal_node.py	26	parameter		"drive_power_gain_init"	1.0
mpe_solarcar/solar_autocal_node.py	27	parameter		"aux_power_w_init"	8.0
mpe_solarcar/solar_autocal_node.py	28	parameter		"solar_gain_min"	0.5
mpe_solarcar/solar_autocal_node.py	29	parameter		"solar_gain_max"	1.5
mpe_solarcar/solar_autocal_node.py	30	parameter		"drive_power_gain_min"	0.7
mpe_solarcar/solar_autocal_node.py	31	parameter		"drive_power_gain_max"	1.4
mpe_solarcar/solar_autocal_node.py	32	parameter		"aux_power_w_min"	0.0
mpe_solarcar/solar_autocal_node.py	33	parameter		"aux_power_w_max"	300.0
mpe_solarcar/solar_autocal_node.py	57	publisher	Float32	"/calib/solar_gain"	
mpe_solarcar/solar_autocal_node.py	58	publisher	Float32	"/calib/drive_power_gain"	
mpe_solarcar/solar_autocal_node.py	59	publisher	Float32	"/calib/aux_power_w"	
mpe_solarcar/solar_autocal_node.py	60	publisher	String	"/calib/status"	
mpe_solarcar/solar_autocal_node.py	62	subscription	Float32	"/vehicle/speed_kmh"	self_on_speed
mpe_solarcar/solar_autocal_node.py	63	subscription	Float32	"/vehicle/batt_current_a"	self_on_current
mpe_solarcar/solar_autocal_node.py	64	subscription	Float32	"/vehicle/batt_voltage_v"	self_on_voltage
mpe_solarcar/solar_autocal_node.py	65	subscription	Float32	"/vehicle/solar_power_w"	self_on_solar
mpe_solarcar/solar_autocal_node.py	66	subscription	Float32MultiArray	"/planner/env"	self_on_env
mpe_solarcar/solar_autocal_node.py	67	subscription	Float32MultiArray	"/planner/metrics"	self_on_metrics
mpe_solarcar/solar_autocal_node.py	71	timer	period[s]	self.period	self_step
mpe_solarcar/solar_logger_node.py	88	parameter		"log_dir"	"outputs/logs"
mpe_solarcar/solar_logger_node.py	89	parameter		"file_prefix"	"solar_live"
mpe_solarcar/solar_logger_node.py	90	parameter		"log_rate_hz"	2.0
mpe_solarcar/solar_logger_node.py	91	parameter		"flush_every_rows"	1
mpe_solarcar/solar_logger_node.py	92	parameter		"output_speed_topic"	"/vehicle/speed_cmd_kmh"
mpe_solarcar/solar_logger_node.py	93	parameter		"output_drive_mode_topic"	"/vehicle/drive_mode_cmd"
mpe_solarcar/solar_logger_node.py	115	subscription	Float32	topic	self_set_float(field)
mpe_solarcar/solar_logger_node.py	117	subscription	Float64	topic	self_set_float(field)

file	line	kind	type/period	topic/name	callback/default
mpe_solarcar/solar_logger_node.py	119	subscription	String	topic	self._set_string(field)
mpe_solarcar/solar_logger_node.py	121	subscription	Float32MultiArray	topic	self._set_array(fields)
mpe_solarcar/solar_logger_node.py	126	subscription	Float32	output_speed_topic	self._set_float("output_speed_cmd_kmh")
mpe_solarcar/solar_logger_node.py	129	subscription	String	output_mode_topic	self._set_string("output_drive_mode")
mpe_solarcar/solar_logger_node.py	136	timer	period[s]	1.0/rate_hz	self._write_row
mpe_solarcar/solar_preflight_node.py	17	parameter		"startup_grace_sec"	10.0
mpe_solarcar/solar_preflight_node.py	18	parameter		"measurement_timeout_sec"	3.0
mpe_solarcar/solar_preflight_node.py	19	parameter		"require_speed"	True
mpe_solarcar/solar_preflight_node.py	20	parameter		"require_distance"	True
mpe_solarcar/solar_preflight_node.py	21	parameter		"require_battery"	True
mpe_solarcar/solar_preflight_node.py	22	parameter		"require_planner"	True
mpe_solarcar/solar_preflight_node.py	27	publisher	String	"/system/state"	
mpe_solarcar/solar_preflight_node.py	28	publisher	Float32	"/system/health"	
mpe_solarcar/solar_preflight_node.py	29	publisher	String	"/system/diag"	
mpe_solarcar/solar_preflight_node.py	31	subscription	Float32	"/vehicle/speed_kmh"	self._seen("speed")
mpe_solarcar/solar_preflight_node.py	32	subscription	Float32	"/vehicle/s_kmh"	self._seen("distance")
mpe_solarcar/solar_preflight_node.py	33	subscription	Float32	"/vehicle/batt_soc"	self._seen("batt_soc")
mpe_solarcar/solar_preflight_node.py	34	subscription	Float32	"/vehicle/batt_voltage_v"	self._seen("batt_voltage")
mpe_solarcar/solar_preflight_node.py	35	subscription	Float32	"/vehicle/batt_current_a"	self._seen("batt_current")
mpe_solarcar/solar_preflight_node.py	36	subscription	Float32MultiArray	"/planner/status"	self._seen("planner")
mpe_solarcar/solar_preflight_node.py	38	timer	period[s]	1.0	self._step
mpe_solarcar/speed_command_bridge_node.py	20	parameter		"output_speed_topic"	"/vehicle/speed_cmd_kmh"
mpe_solarcar/speed_command_bridge_node.py	21	parameter		"output_drive_mode_topic"	"/vehicle/drive_mode_cmd"
mpe_solarcar/speed_command_bridge_node.py	22	parameter		"udp_enabled"	False
mpe_solarcar/speed_command_bridge_node.py	23	parameter		"udp_host"	"127.0.0.1"
mpe_solarcar/speed_command_bridge_node.py	24	parameter		"udp_port"	50050
mpe_solarcar/speed_command_bridge_node.py	25	parameter		"publish_rate_hz"	5.0
mpe_solarcar/speed_command_bridge_node.py	26	parameter		"input_timeout_sec"	3.0
mpe_solarcar/speed_command_bridge_node.py	27	parameter		"safe_speed_kmh"	0.0
mpe_solarcar/speed_command_bridge_node.py	28	parameter		"startup_hold_sec"	2.0
mpe_solarcar/speed_command_bridge_node.py	29	parameter		"filter_tau_sec"	1.0
mpe_solarcar/speed_command_bridge_node.py	30	parameter		"accel_limit_kmhps"	1.2
mpe_solarcar/speed_command_bridge_node.py	31	parameter		"decel_limit_kmhps"	3.5
mpe_solarcar/speed_command_bridge_node.py	32	parameter		"speed_deadband_kmh"	0.1
mpe_solarcar/speed_command_bridge_node.py	33	parameter		"speed_quantize_step_kmh"	0.1
mpe_solarcar/speed_command_bridge_node.py	34	parameter		"max_output_speed_kmh"	120.0
mpe_solarcar/speed_command_bridge_node.py	35	parameter		"drive_mode_min_hold_sec"	5.0
mpe_solarcar/speed_command_bridge_node.py	36	parameter		"require_system_running"	True
mpe_solarcar/speed_command_bridge_node.py	37	parameter		"system_state_timeout_sec"	2.5
mpe_solarcar/speed_command_bridge_node.py	66	publisher	Float32	self.output_speed_topic	
mpe_solarcar/speed_command_bridge_node.py	67	publisher	String	self.output_drive_mode_topic	
mpe_solarcar/speed_command_bridge_node.py	68	subscription	Float32	"/planner/speed_cmd"	self._on_speed_cmd
mpe_solarcar/speed_command_bridge_node.py	69	subscription	Float32	"/planner/upper_speed_cmd"	self._on_upper_speed_cmd
mpe_solarcar/speed_command_bridge_node.py	70	subscription	String	"/planner/drive_mode"	self._on_drive_mode
mpe_solarcar/speed_command_bridge_node.py	71	subscription	String	"/system/state"	self._on_system_state
mpe_solarcar/speed_command_bridge_node.py	86	timer	period[s]	1.0/self.publish_rate_hz	self._step
mpe_solarcar/telemetry_text_bridge_node.py	82	parameter		"enable_inbound"	True
mpe_solarcar/telemetry_text_bridge_node.py	83	parameter		"enable_outbound"	True
mpe_solarcar/telemetry_text_bridge_node.py	84	parameter		"bind_host"	"0.0.0.0"
mpe_solarcar/telemetry_text_bridge_node.py	85	parameter		"bind_port"	52001
mpe_solarcar/telemetry_text_bridge_node.py	86	parameter		"publish_period_sec"	1.0
mpe_solarcar/telemetry_text_bridge_node.py	87	parameter		"solar_remote_host"	"192.168.50.21"
mpe_solarcar/telemetry_text_bridge_node.py	88	parameter		"solar_remote_port"	52002
mpe_solarcar/telemetry_text_bridge_node.py	89	parameter		"chase_remote_host"	"192.168.50.22"
mpe_solarcar/telemetry_text_bridge_node.py	90	parameter		"chase_remote_port"	52003
mpe_solarcar/telemetry_text_bridge_node.py	91	parameter		"send_to_solar"	True
mpe_solarcar/telemetry_text_bridge_node.py	92	parameter		"send_to_chase"	True
mpe_solarcar/telemetry_text_bridge_node.py	93	parameter		"prefer_direct_distance"	False
mpe_solarcar/telemetry_text_bridge_node.py	94	parameter		"speed_filter_tau_sec"	0.6
mpe_solarcar/telemetry_text_bridge_node.py	95	parameter		"speed_max_kmh"	130.0
mpe_solarcar/telemetry_text_bridge_node.py	96	parameter		"speed_max_accel_kmhps"	12.0
mpe_solarcar/telemetry_text_bridge_node.py	97	parameter		"speed_max_decel_kmhps"	20.0
mpe_solarcar/telemetry_text_bridge_node.py	98	parameter		"distance_max_rate_kmps"	0.06
mpe_solarcar/telemetry_text_bridge_node.py	99	parameter		"distance_max_backtrack_km"	0.02
mpe_solarcar/telemetry_text_bridge_node.py	100	parameter		"battery_filter_tau_sec"	1.0
mpe_solarcar/telemetry_text_bridge_node.py	101	parameter		"wind_filter_tau_sec"	1.0
mpe_solarcar/telemetry_text_bridge_node.py	102	parameter		"headwind_filter_tau_sec"	0.8
mpe_solarcar/telemetry_text_bridge_node.py	103	parameter		"max_abs_headwind_ms"	25.0
mpe_solarcar/telemetry_text_bridge_node.py	104	parameter		"timestamp_required"	True
mpe_solarcar/telemetry_text_bridge_node.py	105	parameter		"max_packet_age_sec"	5.0
mpe_solarcar/telemetry_text_bridge_node.py	106	parameter		"max_future_skew_sec"	2.0
mpe_solarcar/telemetry_text_bridge_node.py	107	parameter		"max_out_of_order_sec"	0.0
mpe_solarcar/telemetry_text_bridge_node.py	108	parameter		"solar_power_gain_to_pack"	1.0
mpe_solarcar/telemetry_text_bridge_node.py	143	publisher	String	"/telemetry/raw_solar"	

file	line	kind	type/period	topic/name	callback/default
mpe_solarcar/telemetry_text_bridge_node.py	144	publisher	String	"/telemetry/raw_chase"	
mpe_solarcar/telemetry_text_bridge_node.py	145	publisher	Float32	"/vehicle/speed_kmh"	
mpe_solarcar/telemetry_text_bridge_node.py	146	publisher	Float32	"/vehicle/batt_soc"	
mpe_solarcar/telemetry_text_bridge_node.py	147	publisher	Float32	"/vehicle/batt_temp_c"	
mpe_solarcar/telemetry_text_bridge_node.py	148	publisher	Float32	"/vehicle/batt_current_a"	
mpe_solarcar/telemetry_text_bridge_node.py	149	publisher	Float32	"/vehicle/batt_voltage_v"	
mpe_solarcar/telemetry_text_bridge_node.py	150	publisher	Float32	"/vehicle/solar_power_w"	
mpe_solarcar/telemetry_text_bridge_node.py	151	publisher	Float32	"/vehicle/altitude_m"	
mpe_solarcar/telemetry_text_bridge_node.py	152	publisher	Float32	"/vehicle/s_kmh"	
mpe_solarcar/telemetry_text_bridge_node.py	153	publisher	NavSatFix	"/vehicle/gps"	
mpe_solarcar/telemetry_text_bridge_node.py	154	publisher	NavSatFix	"/chase/gps"	
mpe_solarcar/telemetry_text_bridge_node.py	155	publisher	Float32	"/chase/altitude_m"	
mpe_solarcar/telemetry_text_bridge_node.py	156	publisher	Float32	"/weather/headwind_meas_ms"	
mpe_solarcar/telemetry_text_bridge_node.py	157	publisher	Float32	"/weather/wind_speed_ms"	
mpe_solarcar/telemetry_text_bridge_node.py	158	publisher	Float32	"/weather/wind_dir_deg"	
mpe_solarcar/telemetry_text_bridge_node.py	159	publisher	Float32	"/weather/course_deg"	
mpe_solarcar/telemetry_text_bridge_node.py	160	publisher	String	"/telemetry/bridge_status"	
mpe_solarcar/telemetry_text_bridge_node.py	161	publisher	Float64	"/telemetry/solar_source_ts_unix"	
mpe_solarcar/telemetry_text_bridge_node.py	162	publisher	Float64	"/telemetry/chase_source_ts_unix"	
mpe_solarcar/telemetry_text_bridge_node.py	184	subscription	Float32	"/planner/speed_cmd"	lambdamsg:self._set_outbound("speed_cmd_kmh",float(msg.data))
mpe_solarcar/telemetry_text_bridge_node.py	185	subscription	Float32	"/planner/upper_speed_cmd"	lambdamsg:self._set_outbound("upper_speed_cmd_kmh",float(msg.data))
mpe_solarcar/telemetry_text_bridge_node.py	186	subscription	String	"/planner/drive_mode"	lambdamsg:self._set_outbound("drive_mode",str(msg.data))
mpe_solarcar/telemetry_text_bridge_node.py	187	subscription	Float32	"/vehicle/speed_kmh"	lambdamsg:self._set_outbound("speed_meas_kmh",float(msg.data))
mpe_solarcar/telemetry_text_bridge_node.py	188	subscription	Float32	"/vehicle/batt_soc"	lambdamsg:self._set_outbound("soc",float(msg.data))
mpe_solarcar/telemetry_text_bridge_node.py	189	subscription	Float32	"/vehicle/s_kmh"	lambdamsg:self._set_outbound("s_kmh",float(msg.data))
mpe_solarcar/telemetry_text_bridge_node.py	191	timer	period[s]	0.05	self._poll_socket
mpe_solarcar/telemetry_text_bridge_node.py	192	timer	period[s]	self.publish_period_sec	self._send_outbound
mpe_solarcar/weather_fetch_node.py	18	parameter		"profile_yaml"	""
mpe_solarcar/weather_fetch_node.py	19	parameter		"output_csv"	""
mpe_solarcar/weather_fetch_node.py	20	parameter		"raw_forecast_csv"	""
mpe_solarcar/weather_fetch_node.py	21	parameter		"gps_topic"	"/chase/gps"
mpe_solarcar/weather_fetch_node.py	22	parameter		"fetch_period_sec"	3600.0
mpe_solarcar/weather_fetch_node.py	23	parameter		"forecast_days"	3
mpe_solarcar/weather_fetch_node.py	24	parameter		"step_minutes"	10
mpe_solarcar/weather_fetch_node.py	25	parameter		"timezone_name"	"Australia/Darwin"
mpe_solarcar/weather_fetch_node.py	26	parameter		"fallback_latitude"	-12.4634
mpe_solarcar/weather_fetch_node.py	27	parameter		"fallback_longitude"	130.8456
mpe_solarcar/weather_fetch_node.py	28	parameter		"tcell_gain"	0.03
mpe_solarcar/weather_fetch_node.py	98	publisher	String	"/weather/fetch_status"	
mpe_solarcar/weather_fetch_node.py	99	subscription	NavSatFix	self.gps_topic	self._on_gps
mpe_solarcar/weather_fetch_node.py	100	timer	period[s]	self.fetch_period_sec	self._fetch_once
mpe_solarcar/wind_correction_node.py	23	parameter		"forecast_csv"	""
mpe_solarcar/wind_correction_node.py	24	parameter		"corrected_forecast_csv"	"outputs/runtime/live_forecast_corrected.csv"
mpe_solarcar/wind_correction_node.py	25	parameter		"forecast_time_tz"	"Australia/Darwin"
mpe_solarcar/wind_correction_node.py	26	parameter		"publish_period_sec"	30.0
mpe_solarcar/wind_correction_node.py	27	parameter		"measurement_sigma_ms"	1.0
mpe_solarcar/wind_correction_node.py	28	parameter		"correlation_distance_kmh"	300.0
mpe_solarcar/wind_correction_node.py	29	parameter		"fallback_correlation_time_h"	3.0
mpe_solarcar/wind_correction_node.py	30	parameter		"forecast_sigma0_ms"	1.5
mpe_solarcar/wind_correction_node.py	31	parameter		"forecast_variance_growth_per_hour"	0.05
mpe_solarcar/wind_correction_node.py	32	parameter		"planning_quantile"	0.5
mpe_solarcar/wind_correction_node.py	33	parameter		"confidence_z"	1.96
mpe_solarcar/wind_correction_node.py	34	parameter		"min_sigma_ms"	0.2
mpe_solarcar/wind_correction_node.py	35	parameter		"preferred_source"	"auto"
mpe_solarcar/wind_correction_node.py	36	parameter		"use_exp_distance_decay"	True
mpe_solarcar/wind_correction_node.py	57	publisher	Float32	"/weather/headwind_corrected_ms"	
mpe_solarcar/wind_correction_node.py	58	publisher	String	"/weather/wind_correction_status"	
mpe_solarcar/wind_correction_node.py	59	subscription	Float32	"/weather/headwind_meas_ms"	self._on_headwind
mpe_solarcar/wind_correction_node.py	60	subscription	Float32	"/vehicle/s_kmh"	self._on_s
mpe_solarcar/wind_correction_node.py	61	timer	period[s]	max(1.0,float(self.get_parameter("publish_period_sec").value))	self._step

Chapter 17

全関数の入出力・呼出関係

17.1 launch/solar_race_live.launch.py

`_setup` [L10–L13]

```
signature: def _setup(context)
calls: LaunchConfiguration; build_live_nodes; load_profile; perform
self read: --
self write: --
```

`generate_launch_description` [L16–L22]

```
signature: def generate_launch_description()
calls: DeclareLaunchArgument; LaunchDescription; OpaqueFunction
self read: --
self write: --
```

17.2 launch/solar_race_live_wifi.launch.py

`_setup` [L12–L59]

```
signature: def _setup(context)
calls: LaunchConfiguration; Node; bool; build_live_nodes; float; get_section; live_cfg.get; live_forecast_paths; load_profile; model_cfg.get;
nodes.append; perform; str; wifi_cfg.get; wifi_cfg.items; wind_cfg.get; ... (+1)
self read: --
self write: --
```

`generate_launch_description` [L62–L68]

```
signature: def generate_launch_description()
calls: DeclareLaunchArgument; LaunchDescription; OpaqueFunction
self read: --
self write: --
```

17.3 mpc_solarcar/dashboard_node.py

`DashboardHandler.__init__` [L20–L22]

```
signature: def __init__(self, *args, node=None, directory=None, **kwargs)
calls: __init__; super
self read: --
self write: _node
```

`DashboardHandler.do_GET` [L24–L31]

```
signature: def do_GET(self)
calls: do_GET; self._node.get_prometheus_metrics; self._node.get_state; self._send_json; self._send_metrics; self.path.startswith; super
self read: _node; _send_json; _send_metrics; path
self write: --
```

DashboardHandler.log_message [L33–L35]

```
signature: def log_message(self, format, *args)
calls: --
self read: --
self write: --
```

DashboardHandler._send_json [L37–L43]

```
signature: def _send_json(self, data)
calls: encode; json.dumps; len; self.end_headers; self.send_header; self.send_response; self.wfile.write; str
self read: end_headers; send_header; send_response; wfile
self write: --
```

DashboardHandler._send_metrics [L45–L52]

```
signature: def _send_metrics(self, payload)
calls: len; payload.encode; self.end_headers; self.send_header; self.send_response; self.wfile.write; str
self read: end_headers; send_header; send_response; wfile
self write: --
```

DashboardNode.__init__ [L56–L154]

```
signature: def __init__(self)
calls: ThreadingHTTPServer; __init__; error; float; get_package_share_directory; info; int; os.path.abspath; os.path.dirname; os.path.join;
partial; self._init_dummy; self._server_thread.start; self.create_subscription; self.declare_parameter; self.get_logger; ... (+7)
self read: _init_dummy; _on_drive_mode; _on_env; _on_gps; _on_health; _on_ibatt; _on_lower_plan; _on_metrics; _on_mpc_state; _on_path; _on_s_km;
_on_soc; _on_speed_cmd; _on_speed_meas; _on_status; _on_sys_diag; ... (+10)
self write: _lock; _server; _server_thread; _soc_meas_monotonic; _state
```

DashboardNode.destroy_node [L156–L163]

```
signature: def destroy_node(self)
calls: destroy_node; self._server.server_close; self._server.shutdown; super
self read: _server
self write: --
```

DashboardNode.get_state [L165–L167]

```
signature: def get_state(self)
calls: dict
self read: _lock; _state
self write: --
```

DashboardNode.get_prometheus_metrics [L169–L234]

```
signature: def get_prometheus_metrics(self)
calls: dict; float; join; lines.extend; math.isfinite; max; metric_keys.items; replace; state.get; str; time.time
self read: _lock; _state
self write: --
```

DashboardNode._update [L236–L239]

```
signature: def _update(self, key, value)
calls: time.time
self read: _lock; _state
self write: --
```

DashboardNode._on_speed_cmd [L241–L242]

```
signature: def _on_speed_cmd(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_upper_speed_cmd [L244–L245]

```
signature: def _on_upper_speed_cmd(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_speed_meas [L247–L248]

```
signature: def _on_speed_meas(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_throttle_cmd [L250–L251]

```
signature: def _on_throttle_cmd(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_drive_mode [L253–L254]

```
signature: def _on_drive_mode(self, msg: String)
calls: self._update; str
self read: _update
self write: --
```

DashboardNode._on_soc [L256–L262]

```
signature: def _on_soc(self, msg: Float32)
calls: float; time.monotonic; time.time
self read: _lock; _state
self write: _soc_meas_monotonic
```

DashboardNode._set_estimated_soc [L264–L271]

```
signature: def _set_estimated_soc(self, value)
calls: float; time.monotonic
self read: _soc_meas_monotonic; _state
self write: --
```

DashboardNode._on_tb [L273–L274]

```
signature: def _on_tb(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_ibatt [L276–L277]

```
signature: def _on_ibatt(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_vbatt [L279–L280]

```
signature: def _on_vbatt(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_s_km [L282–L283]

```
signature: def _on_s_km(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_status [L285–L299]

```
signature: def _on_status(self, msg: Float32MultiArray)
calls: float; len; list; self._set_estimated_soc; time.time
self read: _lock; _set_estimated_soc; _state
self write: --
```

DashboardNode._on_env [L301–L311]

```
signature: def _on_env(self, msg: Float32MultiArray)
calls: float; len; list; time.time
self read: _lock; _state
self write: --
```

DashboardNode._on_metrics [L313–L331]

```
signature: def _on_metrics(self, msg: Float32MultiArray)
calls: float; len; list; self._set_estimated_soc; time.time
self read: _lock; _set_estimated_soc; _state
self write: --
```

DashboardNode._on_upper_plan [L333–L342]

```
signature: def _on_upper_plan(self, msg: Float32MultiArray)
calls: float; len; list; time.time
self read: _lock; _state
self write: --
```

DashboardNode._on_lower_plan [L344–L353]

```
signature: def _on_lower_plan(self, msg: Float32MultiArray)
calls: float; len; list; time.time
self read: _lock; _state
self write: --
```

DashboardNode._on_sys_state [L355–L356]

```
signature: def _on_sys_state(self, msg: String)
calls: self._update; str
self read: _update
self write: --
```

DashboardNode._on_sys_diag [L358–L359]

```
signature: def _on_sys_diag(self, msg: String)
calls: self._update; str
self read: _update
self write: --
```

DashboardNode._on_mpc_state [L361–L362]

```
signature: def _on_mpc_state(self, msg: String)
calls: self._update; str
self read: _update
self write: --
```

DashboardNode._on_health [L364–L365]

```
signature: def _on_health(self, msg: Float32)
calls: float; self._update
self read: _update
self write: --
```

DashboardNode._on_gps [L367–L371]

```
signature: def _on_gps(self, msg: NavSatFix)
calls: float; time.time
self read: _lock; _state
self write: --
```

DashboardNode._on_path [L373–L377]

```
signature: def _on_path(self, msg: Path)
calls: len; time.time
self read: _lock; _state
self write: --
```

DashboardNode._init_dummy [L379–L393]

```
signature: def _init_dummy(self, path: str, rate_hz: float)
calls: df.to_dict; error; info; len; max; pd.read_csv; self.create_timer; self.get_logger
self read: _tick_dummy; create_timer; get_logger
self write: _dummy_idx; _dummy_rows; _dummy_timer
```

DashboardNode._tick_dummy [L395–L406]

```
signature: def _tick_dummy(self)
calls: float; hasattr; len; row.items; time.time
self read: _dummy_rows; _lock; _state
self write: _dummy_idx
```

main [L409–L414]

```
signature: def main()
calls: DashboardNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.4 mpc_solarcar/distance_node.py**DistanceNode.__init__ [L10–L27]**

```
signature: def __init__(self)
calls: __init__; float; info; self.create_publisher; self.create_service; self.create_subscription; self.create_timer; self.declare_parameter;
self.get_logger; self.get_parameter; super
self read: _on_reset; _on_speed; _publish; create_publisher; create_service; create_subscription; create_timer; declare_parameter; get_logger;
get_parameter
self write: last_speed; last_time; max_dt_sec; pub_s; s_km; srv_reset; sub_speed; timer
```

DistanceNode._on_speed [L29–L38]

```
signature: def _on_speed(self, msg: Float32)
calls: float; math.isfinite; now; self.get_clock
self read: get_clock; max_dt_sec
self write: last_speed; last_time; s_km
```

DistanceNode._on_reset [L40–L46]

```
signature: def _on_reset(self, request, response)
calls: --
self read: --
self write: last_speed; last_time; s_km
```

DistanceNode._publish [L48–L51]

```
signature: def _publish(self)
calls: Float32; float; self.pub_s.publish
self read: pub_s; s_km
self write: --
```

main [L54–L59]

```
signature: def main()
calls: DistanceNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.5 mpc_solarcar/estimator.py**BatteryMHE.__init__ [L32–L52]**

```
signature: __init__(self, model, horizon_steps, w_soc, w_tb, w_i, w_v, w_prior, soc_bounds, tb_bounds)
calls: deque; float; int; max
self read: --
self write: model; samples; soc_bounds; tb_bounds; w_i; w_prior; w_soc; w_tb; w_v
```

BatteryMHE.push [L54–L55]

```
signature: def push(self, u: MheInput, y: MheMeas)
calls: self.samples.append
self read: samples
self write: --
```

BatteryMHE._simulate [L57–L83]

```
signature: def _simulate(self, z0: float, Tb0: float)
calls: float; outputs.append; self.model.electrical_balance; self.model.soc_step
self read: model; samples
self write: --
```

BatteryMHE.estimate [L85–L111]

```
signature: def estimate(self, z_init: float, Tb_init: float) -> Tuple[float, float]
calls: dict; float; len; math.isfinite; minimize; np.array; self._simulate; zip
self read: _simulate; samples; soc_bounds; tb_bounds; w_i; w_prior; w_soc; w_tb; w_v
self write: --
```

17.6 mpc_solarcar/forecast_grid.py**timestamp_ns [L23–L29]**

```
signature: def timestamp_ns(value) -> int
calls: int; pd.Timestamp; ts.tz_convert; ts.tz_localize
self read: --
self write: --
```

forecast_distance_column [L32–L39]

```
signature: def forecast_distance_column(df: pd.DataFrame) -> str
calls: isinstance
self read: --
self write: --
```

build_forecast_grid_payload [L42–L82]

```
signature: def build_forecast_grid_payload(df: pd.DataFrame) -> dict | None
calls: apply; bfill; df.copy; drop_duplicates; dropna; ffill; forecast_distance_column; interpolate; len; max; np.array; pd.Index; pd.to_numeric;
      pivot.to_numpy; reindex; sort_values; ... (+6)
self read: --
self write: --
```

interp_forecast_grid [L85–L130]

```
signature: interp_forecast_grid(payload, col, t_utc, s_km, default)
calls: float; get; int; len; max; np.clip; np.searchsorted; payload.get; timestamp_ns
self read: --
self write: --
```

17.7 mpc_solarcar/grade_node.py

GradeNode.__init__ [L11–L41]

```
signature: def __init__(self)
calls: __init__; float; info; self.create_publisher; self.create_subscription; self.create_timer; self.declare_parameter; self.get_logger; self.
      get_parameter; super; time.monotonic
self read: _on_altitude; _on_gps; _on_s_km; _on_speed; _step; create_publisher; create_subscription; create_timer; declare_parameter; get_logger;
      get_parameter
self write: alpha; altitude_ema; altitude_m; altitude_topic; gps_topic; grade; last_alt_ema; last_s_km; last_update; min_delta_s_km; min_speed_kmh
           ; pub_grade; s_km; speed_kmh; timer
```

GradeNode._on_s_km [L43–L44]

```
signature: def _on_s_km(self, msg: Float32)
calls: float
self read: --
self write: s_km
```

GradeNode._on_speed [L46–L47]

```
signature: def _on_speed(self, msg: Float32)
calls: float
self read: --
self write: speed_kmh
```

GradeNode._on_altitude [L49–L50]

```
signature: def _on_altitude(self, msg: Float32)
calls: float; self._update_altitude
self read: _update_altitude
self write: --
```

GradeNode._on_gps [L52–L54]

```
signature: def _on_gps(self, msg: NavSatFix)
calls: float; math.isfinite; self._update_altitude
self read: _update_altitude
self write: --
```

GradeNode._update_altitude [L56–L61]

```
signature: def _update_altitude(self, altitude_m: float)
calls: math.isfinite
self read: alpha
self write: altitude_ema; altitude_m
```

GradeNode._step [L63–L86]

```
signature: def _step(self)
calls: Float32; float; math.isfinite; self.pub_grade.publish
self read: altitude_ema; min_delta_s_kmh; min_speed_kmh; pub_grade; s_kmh; speed_kmh
self write: grade; last_alt_ema; last_s_kmh
```

main [L89–L94]

```
signature: def main()
calls: GradeNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.8 mpc_solarcar/live_launch.py

cfg_path [L11–L12]

```
signature: def cfg_path(profile_path, raw: str) -> str
calls: resolve_relative_path; str; strip
self read: --
self write: --
```

_drop_keys [L15–L17]

```
signature: def _drop_keys(payload: dict, *keys: str) -> dict
calls: items; set
self read: --
self write: --
```

live_forecast_paths [L20–L36]

```
signature: def live_forecast_paths(profile_path, cfg)
calls: Path; cfg_path; exists; get_path; get_section; shutil.copy2; str; target.exists; target.parent.mkdir; weather_cfg.get; wind_cfg.get
self read: --
self write: --
```

build_live_nodes [L39–L175]

```
signature: def build_live_nodes(profile_path, cfg, *, use_wifi: bool)
calls: Node; _drop_keys; autocal_cfg.get; bool; cfg_path; command_cfg.get; float; get; get_path; get_section; int; live_cfg.get;
live_forecast_paths; live_logging_cfg.get; logging_cfg.get; nodes.append; ... (+4)
self read: --
self write: --
```

17.9 mpc_solarcar/model.py

_is_symbolic [L10–L14]

```
signature: def _is_symbolic(x)
calls: hasattr; isinstance; x.is_symbolic
self read: --
self write: --
```

SolarCarModel.__init__ [L74–L122]

```
signature: __init__(self, drive_map_path, regen_map_path, Rint_map_path, params, panel_eff_map_path, mppt_eff_map_path, drive_map_eco_path,
drive_map_power_path, regen_map_eco_path, regen_map_power_path, ocv_soc_map_path)
calls: Params; read_id_map; read_Rint_map; read_eff_map; read_map; self._update_mode_limits
self read: _update_mode_limits
self write: Gg; Gm; Rmap; Tcg; Tg; Tm; Z_drv; Z_mppt; Z_panel; Z_reg; aux_power_override_w; drive_mode; drive_mode_default; drive_mode_tau_margin;
maps_drive; maps_regen; ... (+11)
```

SolarCarModel.eff_drive [L124–L135]

```
signature: def eff_drive(self, v_ms, tau_nm)
calls: _is_symbolic; abs; bilinear_interp; ca.fabs; ca.fmax; ca.fmin; ca.sqrt; float; np.clip; self._select_mode; self.maps_drive.get
self read: _select_mode; maps_drive; p
self write: --
```

SolarCarModel.eff_regen [L137–L148]

```
signature: def eff_regen(self, v_ms, tau_nm)
calls: _is_symbolic; abs; bilinear_interp; ca.fabs; ca.fmax; ca.fmin; float; np.clip; self._select_mode; self.maps_regen.get
self read: _select_mode; maps_regen; p
self write: --
```

SolarCarModel._update_mode_limits [L150–L156]

```
signature: def _update_mode_limits(self)
calls: float; max; self.maps_drive.items
self read: maps_drive
self write: tau_max
```

SolarCarModel._select_mode [L158–L167]

```
signature: def _select_mode(self, v_ms: float, tau_nm: float) -> str
calls: float; lower; self.tau_max.get; str
self read: drive_mode; drive_mode_tau_margin; maps_drive; tau_max
self write: --
```

SolarCarModel.select_drive_mode [L169–L170]

```
signature: def select_drive_mode(self, v_ms: float, tau_nm: float) -> str
calls: abs; self._select_mode
self read: _select_mode
self write: --
```

SolarCarModel.R_int [L172–L177]

```
signature: def R_int(self, T_C, z)
calls: _is_symbolic; bilinear_interp; float
self read: Rmap; Tg; p; zg
self write: --
```

SolarCarModel.pv_balance [L179–L220]

```
signature: def pv_balance(self, G_poa, T_cell_C)
calls: _is_symbolic; bilinear_interp; ca.fmax; ca.fmin; dict; float; max; min
self read: Gg; Gm; Tcg; Tm; Z_mppt; Z_panel; mppt_eff_map; p; panel_eff_map
self write: --
```

SolarCarModel.pv_power_mppt [L222–L223]

```
signature: def pv_power_mppt(self, G_poa, T_cell_C)
calls: self.pv_balance
self read: pv_balance
self write: --
```

SolarCarModel._scaled_slope_pct [L225–L226]

```
signature: def _scaled_slope_pct(self, slope_pct)
calls: float
self read: p
self write: --
```

SolarCarModel.charge_efficiency [L228–L233]

```
signature: def charge_efficiency(self, P_pack) -> float
calls: float
self read: p
self write: --
```

SolarCarModel.soc_step [L235–L260]

```
signature: soc_step(self, z, P_pack, dt_sec, *, current_a, Tbat_C)
calls: float; max; self.battery_iv; self.charge_efficiency
self read: battery_iv; charge_efficiency; p
self write: --
```

SolarCarModel.ocv_from_soc [L262–L269]

```
signature: def ocv_from_soc(self, z)
calls: _is_symbolic; ca.fmax; ca.fmin; float; np.clip; np.interp
self read: ocv_grid; ocv_soc_map; p; soc_grid
self write: --
```

SolarCarModel.load_ocv_map [L271–L278]

```
signature: def load_ocv_map(self, path: str) -> bool
calls: read_id_map
self read: --
self write: ocv_grid; ocv_soc_map; soc_grid
```

SolarCarModel.air_density [L280–L295]

```
signature: def air_density(self, ambient_temp_c=None, elevation_m=0.0)
calls: float; lower; max; np.clip; str
self read: p
self write: --
```

SolarCarModel.resistive_forces [L297–L342]

```
signature: resistive_forces(self, v_ms, slope_pct, headwind_ms, *, ambient_temp_c, elevation_m, air_density_kgm3)
calls: _is_symbolic; ca.atan; ca.cos; ca.fmax; ca.sin; dict; float; math.atan; math.cos; math.sin; max; self._scaled_slope_pct; self.air_density
self read: _scaled_slope_pct; air_density; p
self write: --
```

SolarCarModel.battery_iv [L344–L366]

```
signature: def battery_iv(self, P_pack, z, Tbat_C)
calls: _is_symbolic; ca.fmax; ca.fmin; ca.sqrt; dict; float; math.sqrt; max; np.clip; self.R_int; self.ocv_from_soc
self read: R_int; ocv_from_soc; p
self write: --
```

SolarCarModel.mech_power [L368–L387]

```
signature: mech_power(self, v_ms, slope_pct, headwind_ms, inertial_power_w, *, ambient_temp_c, elevation_m)
calls: self.resistive_forces
self read: resistive_forces
self write: --
```

SolarCarModel.auxiliary_power [L389–L394]

```
signature: def auxiliary_power(self, aux_power_w=None)
calls: float; max
self read: aux_power_override_w; p
self write: --
```

SolarCarModel.scheduled_auxiliary_power [L396–L403]

```
signature: def scheduled_auxiliary_power(self, *, is_driving: bool, irradiance_wm2: float) -> float
calls: float; max; self.auxiliary_power
self read: aux_power_override_w; auxiliary_power; p
self write: --
```

SolarCarModel.torque_from_mech [L405–L414]

```
signature: def torque_from_mech(self, P_mech, v_ms, wheel_radius=None, ratio=None)
calls: --
self read: p
self write: --
```

SolarCarModel.electrical_balance [L416–L477]

```
signature: electrical_balance(self, v_ms, slope_pct, z, Tbat_C, G_poa, Tcell_C, headwind_ms, aux_power_w, inertial_power_w, ambient_temp_c,
    elevation_m)
calls: _is_symbolic; ca.fmax; dict; float; max; np.clip; self.auxiliary_power; self.battery_iv; self.charge_efficiency; self.eff_drive; self.
    eff_regen; self.pv_balance; self.resistive_forces; self.torque_from_mech
self read: auxiliary_power; battery_iv; charge_efficiency; eff_drive; eff_regen; p; pv_balance; resistive_forces; torque_from_mech
self write: --
```

17.10 mpc_solarcar/mpc_node.py

MPCNode.__init__ [L40–L49]

```
signature: def __init__(self)
calls: MutuallyExclusiveCallbackGroup; __init__; self._init_solar; super
self read: _init_solar
self write: command_callback_group; lower_callback_group; params_cfg; telemetry_callback_group; upper_callback_group
```

MPCNode._load_stops [L52–L60]

```
signature: def _load_stops(self, stop_yaml: str)
calls: info; len; open; self.get_logger; y.get; yaml.safe_load
self read: get_logger
self write: stops
```

MPCNode._load_forecast_file [L62–L86]

```
signature: def _load_forecast_file(self, path: str)
calls: all; build_forecast_grid_payload; drop_duplicates; dropna; dt.tz_convert; getattr; isna; pd.Index; pd.read_csv; pd.to_datetime; self.
    get_logger; sort_values; str; t.dt.tz_convert; t.dt.tz_localize; tzname.upper; ... (+1)
self read: get_logger
self write: df; forecast_grid; forecast_times
```

MPCNode._apply_params_yaml [L88–L156]

```
signature: def _apply_params_yaml(self, path: str)
calls: Parameter; cfg.get; dict; float; hasattr; info; isinstance; len; model_cfg.items; motor_type.lower; mpc_cfg.items; mpc_cfg.update; open;
    params.append; resolve_path; runtime_section.get; ... (+10)
self read: get_logger; get_parameter; has_parameter; model; set_parameters
self write: params_cfg
```

MPCNode._maybe_reload_forecast [L158–L176]

```
signature: def _maybe_reload_forecast(self)
calls: info; os.path.getmtime; self._load_forecast_file; self.get_logger; time.monotonic; warn
self read: _load_forecast_file; forecast_path; forecast_reload_sec; get_logger
self write: forecast_mtime; forecast_reloaded; last_forecast_check
```

MPCNode._on_s_km_solar [L178–L192]

```
signature: def _on_s_km_solar(self, msg: Float32)
calls: bool; finite_float; float; math.isfinite; now; self.distance_meas_filter.update; self.get_clock; self.get_parameter
self read: distance_meas_filter; distance_meas_max_backtrack_km; get_clock; get_parameter
self write: s_km; s_meas; s_meas_time
```

MPCNode._on_speed_solar [L194–L200]

```
signature: def _on_speed_solar(self, msg: Float32)
calls: float; now; self.get_clock; self.speed_meas_filter.update
self read: get_clock; speed_meas_filter
self write: v_meas_time; v_now
```

MPCNode._on_soc_solar [L202–L209]

```
signature: def _on_soc_solar(self, msg: Float32)
calls: float; now; np.clip; self.get_clock; self.soc_meas_filter.update
self read: get_clock; model; soc_meas_filter
self write: soc_meas_time; solar_soc_meas; z
```

MPCNode._on_tb_solar [L211–L218]

```
signature: def _on_tb_solar(self, msg: Float32)
calls: float; now; np.clip; self.get_clock; self.tb_meas_filter.update
self read: get_clock; model; tb_meas_filter
self write: Tb; solar_tb_meas; tb_meas_time
```

MPCNode._on_i_solar [L220–L226]

```
signature: def _on_i_solar(self, msg: Float32)
calls: float; now; self.current_meas_filter.update; self.get_clock
self read: current_meas_filter; get_clock
self write: current_meas_time; solar_i_meas
```

MPCNode._on_v_solar [L228–L234]

```
signature: def _on_v_solar(self, msg: Float32)
calls: float; now; self.get_clock; self.voltage_meas_filter.update
self read: get_clock; voltage_meas_filter
self write: solar_v_meas; voltage_meas_time
```

MPCNode._measured_distance_km [L236–L240]

```
signature: def _measured_distance_km(self) -> float
calls: finite_float; fresh_enough; now; self.get_clock
self read: distance_meas_timeout_sec; get_clock; s_meas; s_meas_time
self write: --
```

MPCNode._sync_measured_state [L242–L254]

```
signature: def _sync_measured_state(self) -> None
calls: bool; float; fresh_enough; math.isfinite; now; np.clip; self._measured_distance_km; self.get_clock; self.get_parameter
self read: _measured_distance_km; battery_meas_timeout_sec; get_clock; get_parameter; model; soc_meas_time; solar_soc_meas; solar_tb_meas;
speed_meas_timeout_sec; tb_meas_time; v_meas_time
self write: Tb; s_km; v_now; z
```

MPCNode._on_calib_solar_gain [L256–L260]

```
signature: def _on_calib_solar_gain(self, msg: Float32)
calls: float; np.clip
self read: --
self write: solar_gain
```

MPCNode._on_calib_drive_power_gain [L262–L268]

```
signature: def _on_calib_drive_power_gain(self, msg: Float32)
calls: float; max; np.clip
self read: base_drive_eff_scale; base_regen_eff_scale; model
self write: --
```

MPCNode._on_calib_aux_power [L270–L274]

```
signature: def _on_calib_aux_power(self, msg: Float32)
calls: float; max
self read: model
self write: --
```

MPCNode._init_solar [L277–L749]

```
signature: def _init_solar(self)
calls: BatteryMHE; DriveSchedule.from_yaml; Params; RobustScalarFilter; SolarCarModel; abs; astimezone; bool; datetime.fromisoformat; datetime.now
; float; getattr; info; int; isinstance; len; ... (+24)
self read: _apply_params_yaml; _load_forecast_file; _load_stops; _on_calib_aux_power; _on_calib_drive_power_gain; _on_calib_solar_gain;
_on_i_solar; _on_s_km_solar; _on_soc_solar; _on_speed_solar; _on_tb_solar; _on_v_solar; _publish_lower_command_cycle; _step_lower;
_step_solar; command_callback_group; ... (+10)
self write: Np; Tb; _forecast_warned_out_of_range; _lower_solve_logged; _lower_upper_busy_logged; _upper_solve_in_progress; _upper_solve_logged;
_warned_lower_rate; active_control_stop_index; base_drive_eff_scale; base_regen_eff_scale; battery_meas_timeout_sec; completed_control_stops
; control_stop_end_monotonic; control_stop_hold; control_stop_position_initialized; ... (+120)
```

MPCNode._current_bin_index [L751–L799]

```
signature: def _current_bin_index(self) -> int
calls: datetime.now; getattr; int; len; lower; max; np.clip; np.datetime64; np.searchsorted; self.get_logger; str; timedelta; to_pydatetime;
total_seconds; warn
self read: df; dt; forecast_relative_wall_start; forecast_start_time; forecast_time_mode; forecast_time_offset; forecast_times; get_logger; k
self write: _forecast_warned_out_of_range; current_forecast_time_utc
```

MPCNode._horizon_data [L801–L841]

```
signature: def _horizon_data(self, k0: int)
calls: bool; data.append; float; getattr; int; len; lower; max; min; np.clip; np.datetime64; np.isfinite; np.searchsorted; range; row.get; self.
_forecast_at_time; ... (+6)
self read: Np; _forecast_at_time; current_forecast_time_utc; df; drive_schedule; dt; forecast_start_time; forecast_time_mode; forecast_times;
get_parameter; s_km; s_meas; v_upper_cmd
self write: --
```

MPCNode._forecast_at_time [L843–L917]

```
signature: _forecast_at_time(self, t_utc, s_km, drive, control_stop)
calls: all; dict; float; getattr; int; interp_forecast_grid; isna; len; max; np.clip; np.datetime64; np.searchsorted; row.get; self._route_value;
total_seconds
self read: _route_value; control_stop_tilt_fraction; df; dt; forecast_grid; forecast_start_time; poa_gain_drive; poa_gain_stop; solar_gain;
stop_tilt_fraction
self write: --
```

MPCNode._sample_plan_segments [L919–L928]

```
signature: def _sample_plan_segments(self, dt_sample: float)
calls: float; int; math.ceil; max; samples.extend
self read: v_plan_segments
self write: --
```

MPCNode._route_value [L930–L939]

```
signature: def _route_value(self, s_km: float, field: str, default: float) -> float
calls: float; interpolate_profile; np.isfinite
self read: route_profile
self write: --
```

MPCNode._route_average [L941–L948]

```
signature: def _route_average(self, s0_km: float, s1_km: float, field: str, default: float) -> float
calls: average_profile; float; np.isfinite
self read: route_profile
self write: --
```

MPCNode._speed_limit_at [L950–L956]

```
signature: def _speed_limit_at(self, s_km: float, default_kmh: float) -> float
calls: float; interpolate_profile
self read: speed_profile
self write: --
```

MPCNode._soc_guard_speed [L958–L1014]

```
signature: def _soc_guard_speed(self, v_kmh: float, s_km: float, d0: dict) -> float
calls: d0.get; float; lower; max; range; self._route_value; self.get_parameter; self.model.electrical_balance; self.model.soc_step; str;
      z_next_for
self read: Tb; _route_value; get_parameter; model; route_profile; z
self write: --
```

MPCNode._control_stop_catalog [L1016–L1034]

```
signature: def _control_stop_catalog(self)
calls: catalog.append; enumerate; float; int; lower; max; sorted; stop.get; str; strip
self read: stops
self write: --
```

MPCNode._update_live_control_stop_hold [L1036–L1084]

```
signature: def _update_live_control_stop_hold(self, s_km: float, v_kmh: float) -> bool
calls: float; getattr; info; int; max; self._control_stop_catalog; self.completed_control_stops.add; self.get_logger; self.get_parameter; time.
      monotonic
self read: _control_stop_catalog; completed_control_stops; get_logger; get_parameter
self write: active_control_stop_index; control_stop_end_monotonic; control_stop_position_initialized
```

MPCNode._dwell_penalty [L1086–L1096]

```
signature: def _dwell_penalty(self, s_km: float, v_ms: float) -> float
calls: abs; float; max; self.get_parameter; st.get
self read: get_parameter; stops
self write: --
```

MPCNode._mpc_solve_solar [L1098–L1260]

```
signature: def _mpc_solve_solar(self, data)
calls: abs; bool; d.get; dict; float; len; list; max; min; minimize; np.all; np.array; np.clip; np.isfinite; np.ones; np.zeros; ... (+15)
self read: Tb; _dwell_penalty; _route_value; _speed_limit_at; drive_schedule; get_logger; get_parameter; model; race_km; route_profile; s_km;
      s_meas; soc_band; soc_day_end_max; soc_finish_target; soc_finish_tol; ... (+10)
self write: last_upper_solve_ok
```

MPCNode._mpc_solve_solar_distance [L1262–L1778]

```
signature: def _mpc_solve_solar_distance(self, t0_utc: datetime, s0_km: float, x0=None)
calls: abs; apply_control_stop_at; bool; build_balance_seed; build_upper_distance_horizon; ctrl_s.copy; enumerate; env.get; env_leg.get;
      expand_ctrl; float; forces.get; getattr; hybrid_bounded_minimize; int; integrate_stationary_duration; ... (+50)
self read: Tb; _control_stop_catalog; _forecast_at_time; _route_average; _route_value; _speed_limit_at; drive_schedule; get_logger; get_parameter;
      initial_upper_policy; model; race_km; soc_day_end_max; soc_day_end_target; soc_finish_target; soc_finish_tol; ... (+15)
self write: last_upper_solve_ok; upper_plan_ctrl_s
```

MPCNode._publish_upper_plan [L1780–L1785]

```
signature: def _publish_upper_plan(self)
calls: Float32MultiArray; float; self.pub_plan.publish
self read: plan_dt_sec; pub_plan; v_plan_kmh
self write: --
```

MPCNode._publish_lower_plan [L1787–L1792]

```
signature: def _publish_lower_plan(self, v_seq_ms)
calls: Float32MultiArray; float; self.pub_lower_plan.publish
self read: lower_dt; pub_lower_plan
self write: --
```

MPCNode._publish_plan_path [L1794–L1820]

```
signature: def _publish_plan_path(self, data)
calls: Path; PoseStamped; bool; float; len; now; np.isfinite; path.poses.append; self.get_clock; self.get_parameter; self.pub_path.publish; to_msg
self read: get_clock; get_parameter; model; pub_path; s_kmh; s_meas; upper_plan_mode; v_plan_kmh; v_plan_segments; v_upper_cmd
self write: --
```

MPCNode._publish_metrics [L1822–L1859]

```
signature: def _publish_metrics(self, d0: dict, v_exec_kmh: float, s_for_profile: float)
calls: Float32MultiArray; abs; d0.get; float; out.get; self._route_value; self.model.electrical_balance; self.pub_metrics.publish
self read: Tb; _route_value; model; pub_metrics; route_profile; z
self write: --
```

MPCNode._publish_summary [L1861–L1891]

```
signature: def _publish_summary(self, _v_exec_kmh: float = math.nan) -> None
calls: Float32MultiArray; all; bool; datetime.now; float; isna; len; lower; max; pd.notna; self.pub_status.publish; str; time.monotonic; timedelta
; to_pydatetime; total_seconds
self read: Tb; completed_control_stops; control_stop_end_monotonic; control_stop_hold; df; forecast_start_time; forecast_time_mode;
forecast_time_offset; k; model; pub_status; s_kmh; z
self write: --
```

MPCNode._interp_upper_speed [L1893–L1914]

```
signature: def _interp_upper_speed(self, t_sec: float) -> float
calls: float; int; len; math.floor
self read: plan_dt_sec; upper_plan_mode; v_plan_kmh; v_plan_segments; v_upper_cmd
self write: --
```

MPCNode._distance_plan_speed [L1916–L1920]

```
signature: def _distance_plan_speed(self, s_kmh: float) -> float
calls: float; plan_segment_index
self read: v_plan_segments; v_upper_cmd
self write: --
```

MPCNode._tau_max_for_mode [L1922–L1927]

```
signature: def _tau_max_for_mode(self, maps, mode: str) -> float
calls: float; max
self read: --
self write: --
```

MPCNode._tau_limits [L1929–L1943]

```
signature: def _tau_limits(self)
calls: lower; self._tau_max_for_mode; str
self read: _tau_max_for_mode; model
self write: --
```

MPCNode._traction_force [L1945–L1950]

```
signature: def _traction_force(self, tau_nm: float) -> float
calls: float; int; max
self read: model
self write: --
```

MPCNode._pack_from_tau [L1952–L1974]

```
signature: def _pack_from_tau(self, v_ms: float, tau_nm: float, z: float, Tb: float, env: dict)
calls: dict; float; int; max; self.model.battery_iv; self.model.eff_drive; self.model.eff_regen; self.model.pv_power_mppt
self read: model
self write: --
```

MPCNode._build_lower_ref [L1976–L2001]

```
signature: def _build_lower_ref(self, base_time_utc, s_km: float, d0: dict)
calls: float; max; min; np.clip; range; ref.append; self._interp_upper_speed; self._soc_guard_speed; self._speed_limit_at; self.drive_schedule.
speed_limits; self.get_parameter; time.monotonic; timedelta
self read: _interp_upper_speed; _soc_guard_speed; _speed_limit_at; drive_schedule; get_parameter; lower_N; lower_dt; model; plan_start_monotonic;
v_upper_cmd; z
self write: --
```

MPCNode._lower_rollout [L2003–L2034]

```
signature: _lower_rollout(self, v0_ms, u_seq, env, z0, Tb0, tau_drive, tau_regen)
calls: env.get; float; max; self._pack_from_tau; self._traction_force; self.model.resistive_forces; self.model.soc_step; v_seq.append
self read: _pack_from_tau; _traction_force; lower_dt; model
self write: --
```

MPCNode._lower_mpc_solve [L2036–L2235]

```
signature: def _lower_mpc_solve(self, v0_ms: float, s0_km: float, z0: float, Tb0: float, env: dict, v_ref_seq)
calls: abs; bool; dict; env.get; float; info; int; len; max; min; minimize; np.all; np.array; np.asarray; np.clip; np.concatenate; ... (+16)
self read: _lower_rollout; _pack_from_tau; _tau_limits; _traction_force; _upper_solve_in_progress; get_logger; get_parameter; lower_N; lower_dt;
lower_last_u; lower_max_iter; lower_maxfun; lower_skip_optimization_during_upper_solve; model; throttle_rate_limit; w_throttle; ... (+1)
self write: _lower_upper_busy_logged; last_lower_solve_ok
```

MPCNode._step_solar [L2237–L2493]

```
signature: def _step_solar(self)
calls: Float32; Float32MultiArray; MheInput; MheMeas; bool; d0.get; datetime.now; float; fresh_enough; get; info; len; math.isfinite; max; now; np
.clip; ... (+33)
self read: _current_bin_index; _distance_plan_speed; _forecast_at_time; _horizon_data; _interp_upper_speed; _maybe_reload_forecast;
_measured_distance_km; _mpc_solve_solar; _mpc_solve_solar_distance; _publish_metrics; _publish_plan_path; _publish_summary;
_publish_upper_plan; _route_value; _sample_plan_segments; _soc_guard_speed; ... (+30)
self write: Tb; _upper_solve_in_progress; _upper_solve_logged; control_stop_hold; forecast_reloaded; k; last_bin; last_data; last_plan_time;
model_prev_speed_ms; plan_dt_sec; plan_start_monotonic; s_km; upper_plan_id; upper_plan_mode; upper_plan_s_km; ... (+7)
```

MPCNode._step_lower [L2495–L2544]

```
signature: def _step_lower(self)
calls: bool; d0.get; datetime.now; dict; float; info; len; list; np.isfinite; self._build_lower_ref; self._lower_mpc_solve; self.
_publish_lower_command_cycle; self._route_value; self.get_logger; self.get_parameter; str; ... (+1)
self read: Tb; _build_lower_ref; _lower_mpc_solve; _publish_lower_command_cycle; _route_value; get_logger; get_parameter; hierarchical; last_data;
route_profile; s_km; s_meas; timer_lower; v_now; v_upper_cmd; z
self write: _lower_solve_logged; lower_last_mode; lower_last_u; lower_last_v_pred; v_cmd; v_lower_cmd
```

MPCNode._publish_lower_command_cycle [L2546–L2561]

```
signature: def _publish_lower_command_cycle(self) -> None
calls: Float32; String; float; min; np.clip; self._publish_lower_plan; self._publish_summary; self._sync_measured_state; self.get_parameter; self.
pub_drive_mode.publish; self.pub_speed.publish; self.pub_throttle.publish; str
self read: _publish_lower_plan; _publish_summary; _sync_measured_state; control_stop_hold; get_parameter; hierarchical; lower_last_mode;
lower_last_u; lower_last_v_pred; pub_drive_mode; pub_speed; pub_throttle; timer_lower; v_lower_cmd
self write: --
```

main [L2563–L2573]

```
signature: def main()
calls: MPCNode; MultiThreadedExecutor; executor.add_node; executor.shutdown; executor.spin; node.destroy_node; rclpy.init; rclpy.shutdown
self read: --
self write: --
```

17.11 mpc_solarcar/path_utils.py

resolve_path [L14–L37]

```
signature: def resolve_path(path: str, default_subdir: str = '') -> str
calls: default_subdir.strip; get_package_share_directory; os.fspath; os.path.exists; os.path.expanduser; os.path.isabs; os.path.join; path.
      startswith; str
self read: --
self write: --
```

17.12 mpc_solarcar/route_utils.py

_interp_field [L6–L13]

```
signature: def _interp_field(d, y, s_km, default=0.0)
calls: float; len; np.clip; np.searchsorted
self read: --
self write: --
```

interpolate_route [L16–L22]

```
signature: def interpolate_route(route_df, s_km)
calls: _interp_field; float; len
self read: --
self write: --
```

interpolate_route_with_alt [L25–L33]

```
signature: def interpolate_route_with_alt(route_df, s_km)
calls: _interp_field; float; interpolate_route
self read: --
self write: --
```

interpolate_profile [L36–L40]

```
signature: def interpolate_profile(route_df, s_km, field: str, default: float = 0.0) -> float
calls: _interp_field; float
self read: --
self write: --
```

average_profile [L43–L64]

```
signature: def average_profile(route_df, s0_km, s1_km, field: str, default: float = 0.0) -> float
calls: average_profile_segments; float; np.asarray; np.isfinite
self read: --
self write: --
```

average_profile_segments [L67–L124]

```
signature: def average_profile_segments(route_df, starts_km, lengths_km, field: str, default: float = 0.0)
calls: ValueError; antiderivative; finite.sum; float; int; len; lengths.ravel; np.any; np.argsort; np.asarray; np.clip; np.concatenate; np.cumsum;
      np.diff; np.empty_like; np.full; ... (+10)
self read: --
self write: --
```

interpolate_route_heading [L127–L139]

```
signature: def interpolate_route_heading(route_df, s_km, span_km: float = 0.5) -> float
calls: float; interpolate_route; len; math.atan2; math.cos; math.degrees; math.radians; math.sin
self read: --
self write: --
```

17.13 mpc_solarcar/schedule_utils.py

`_parse_utc` [L14–L23]

```
signature: def _parse_utc(ts: str) -> Optional[datetime]
calls: datetime.fromisoformat; dt.astimezone; dt.replace; ts.endswith
self read: --
self write: --
```

`_parse_time_hhmm` [L26–L33]

```
signature: def _parse_time_hhmm(s: str) -> Optional[dttime]
calls: dttime; int; len; s.strip; split
self read: --
self write: --
```

`DriveWindow.contains` [L43–L44]

```
signature: def contains(self, t_utc: datetime) -> bool
calls: --
self read: end_utc; start_utc
self write: --
```

`DailyWindow.contains` [L56–L72]

```
signature: def contains(self, t_utc: datetime) -> bool
calls: ZoneInfo; local_dt.time; local_dt.weekday; t_utc.astimezone
self read: days; end_local; start_local; tz
self write: --
```

`DriveSchedule.__init__` [L76–L79]

```
signature: def __init__(self, windows: List[DriveWindow], daily: List[DailyWindow], deny_by_default: bool)
calls: --
self read: --
self write: daily; deny_by_default; windows
```

`DriveSchedule.from_yaml` [L82–L116]

```
signature: def from_yaml(cls, path: str) -> Optional['DriveSchedule']
calls: DailyWindow; DriveWindow; _parse_time_hhmm; _parse_utc; bool; cfg.get; cls; daily.append; float; int; open; os.path.exists; str; w.get;
windows.append; yaml.safe_load
self read: --
self write: --
```

`DriveSchedule.speed_limits` [L118–L132]

```
signature: def speed_limits(self, t_utc: datetime) -> Optional[Tuple[float, float]]
calls: limits.append; max; min; w.contains
self read: daily; deny_by_default; windows
self write: --
```

`DriveSchedule.is_drive_time` [L134–L138]

```
signature: def is_drive_time(self, t_utc: datetime) -> bool
calls: self.speed_limits
self read: deny_by_default; speed_limits
self write: --
```

`DriveSchedule.next_drive_start` [L140–L181]

```
signature: def next_drive_start(self, t_utc: datetime) -> datetime
calls: ZoneInfo; candidates.append; date; datetime.combine; local_dt.date; local_dt.time; local_dt.weekday; min; self.is_drive_time; start_local.
astimezone; t_utc.astimezone; timedelta; weekday
self read: daily; is_drive_time; windows
self write: --
```

DriveSchedule.current_drive_window [L183–L217]

```
signature: def current_drive_window(self, t_utc: datetime)
calls: ZoneInfo; datetime.combine; end_local.astimezone; local_dt.date; local_dt.time; local_dt.weekday; start_local.astimezone; t_utc.astimezone;
        timedelta; w.contains
self read: daily; windows
self write: --
```

17.14 mpc_solarcar/signal_utils.py

finite_float [L7–L12]

```
signature: def finite_float(value, default: float = math.nan) -> float
calls: float; math.isfinite
self read: --
self write: --
```

fresh_enough [L15–L20]

```
signature: def fresh_enough(last_time_sec: float | None, now_sec: float | None, timeout_sec: float) -> bool
calls: float
self read: --
self write: --
```

slew_limit [L23–L38]

```
signature: slew_limit(previous, target, dt_sec, rise_rate, fall_rate)
calls: finite_float; float; max; min
self read: --
self write: --
```

RobustScalarFilter.__init__ [L42–L58]

```
signature: __init__(self, *, tau_sec, initial_value, min_value, max_value, rise_rate, fall_rate)
calls: finite_float; max
self read: --
self write: fall_rate; last_time; max_value; min_value; rise_rate; tau_sec; value
```

RobustScalarFilter.reset [L60–L63]

```
signature: def reset(self, value: float, *, now: float | None = None) -> float
calls: finite_float; float; time.monotonic
self read: --
self write: last_time; value
```

RobustScalarFilter.update [L65–L84]

```
signature: def update(self, value: float, *, now: float | None = None) -> float
calls: finite_float; float; math.exp; math.isfinite; max; self._clamp; slew_limit; time.monotonic
self read: _clamp; fall_rate; rise_rate; tau_sec
self write: last_time; value
```

RobustScalarFilter._clamp [L86–L92]

```
signature: def _clamp(self, value: float) -> float
calls: finite_float; float; max; min
self read: max_value; min_value; value
self write: --
```

SmoothRateLimiter.__init__ [L96–L116]

```
signature: __init__(self, *, min_value, max_value, tau_sec, rise_rate, fall_rate, deadband, quantize_step, initial_value)
calls: finite_float; max; self._clamp
self read: _clamp
self write: deadband; fall_rate; last_time; max_value; min_value; quantize_step; rise_rate; tau_sec; value
```

SmoothRateLimiter.reset [L118–L121]

```
signature: def reset(self, value: float, *, now: float | None = None) -> float
calls: finite_float; float; self._clamp; time.monotonic
self read: _clamp
self write: last_time; value
```

SmoothRateLimiter.update [L123–L139]

```
signature: def update(self, target: float, *, now: float | None = None) -> float
calls: finite_float; float; math.exp; max; self._clamp; self._postprocess; slew_limit; time.monotonic
self read: _clamp; _postprocess; fall_rate; rise_rate; tau_sec
self write: last_time; value
```

SmoothRateLimiter._clamp [L141–L147]

```
signature: def _clamp(self, value: float) -> float
calls: finite_float; float; max; min
self read: max_value; min_value
self write: --
```

SmoothRateLimiter._postprocess [L149–L155]

```
signature: def _postprocess(self, value: float, *, reference: float) -> float
calls: abs; float; round; self._clamp
self read: _clamp; deadband; quantize_step
self write: --
```

17.15 mpc_solarcar/solar_autocal_logic.py

daytime_stationary_aux_estimate [L8–L24]

```
signature: daytime_stationary_aux_estimate(*, ghi_wm2, day_ghi_threshold_wm2, speed_kmh, stationary_speed_kmh, pack_power_w, solar_power_w)
calls: abs; all; float; math.isfinite; max
self read: --
self write: --
```

17.16 mpc_solarcar/solar_autocal_node.py

_clamp [L13–L14]

```
signature: def _clamp(value: float, lo: float, hi: float) -> float
calls: float; max; min
self read: --
self write: --
```

SolarAutocalNode.__init__ [L18–L72]

```
signature: def __init__(self) -> None
calls: __init__; _clamp; float; info; max; self.create_publisher; self.create_subscription; self.create_timer; self.declare_parameter; self.get_logger; self.get_parameter; super
self read: _on_current; _on_env; _on_metrics; _on_solar; _on_speed; _on_voltage; _step; create_publisher; create_subscription; create_timer; declare_parameter; get_logger; get_parameter
self write: _latest_current_a; _latest_voltage_v; alpha; aux_max; aux_min; aux_power; day_ghi; drive_gain; drive_gain_max; drive_gain_min; drive_speed; ghi; pack_power_w; period; pred_pack_w; pred_solar_w; ... (+11)
```

SolarAutocalNode._on_speed [L74–L75]

```
signature: def _on_speed(self, msg: Float32) -> None
calls: float
self read: --
self write: speed_kmh
```

SolarAutocalNode._on_current [L77–L80]

```
signature: def _on_current(self, msg: Float32) -> None
calls: float; math.isfinite
self read: _latest_voltage_v
self write: _latest_current_a; pack_power_w
```

SolarAutocalNode._on_voltage [L82–L85]

```
signature: def _on_voltage(self, msg: Float32) -> None
calls: float; math.isfinite
self read: _latest_current_a
self write: _latest_voltage_v; pack_power_w
```

SolarAutocalNode._on_solar [L87–L88]

```
signature: def _on_solar(self, msg: Float32) -> None
calls: float
self read: --
self write: solar_obs_w
```

SolarAutocalNode._on_env [L90–L93]

```
signature: def _on_env(self, msg: Float32MultiArray) -> None
calls: float; len; list
self read: --
self write: ghi
```

SolarAutocalNode._on_metrics [L95–L100]

```
signature: def _on_metrics(self, msg: Float32MultiArray) -> None
calls: float; len; list
self read: --
self write: pred_pack_w; pred_solar_w
```

SolarAutocalNode._ema [L102–L104]

```
signature: def _ema(self, current: float, estimate: float, lo: float, hi: float) -> float
calls: _clamp; float
self read: alpha
self write: --
```

SolarAutocalNode._step [L106–L150]

```
signature: def _step(self) -> None
calls: Float32; String; abs; daytime_stationary_aux_estimate; float; math.isfinite; max; self._ema; self.pub_aux.publish; self.pub_drive.publish;
self.pub_solar.publish; self.pub_status.publish
self read: _ema; aux_max; aux_min; day_ghi; drive_gain_max; drive_gain_min; drive_speed; ghi; pack_power_w; pred_pack_w; pred_solar_w; pub_aux;
pub_drive; pub_solar; pub_status; solar_gain_max; ... (+4)
self write: aux_power; drive_gain; solar_gain
```

main [L153–L158]

```
signature: def main() -> None
calls: SolarAutocalNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.17 mpc_solarcar/solar_logger_node.py

SolarLoggerNode.__init__ [L86–L137]

```
signature: def __init__(self)
calls: ARRAY_TOPICS.items; ARRAY_TOPICS.values; FLOAT64_TOPICS.items; FLOAT_TOPICS.items; STRING_TOPICS.items; __init__; csv.DictWriter; datetime.
now; float; info; int; list; max; open; os.fspath; os.makedirs; ... (+15)
self read: _set_array; _set_float; _set_string; _write_row; create_subscription; create_timer; declare_parameter; get_logger; get_parameter
self write: _topic_subscriptions; csv_file; fields; flush_every_rows; latest; log_path; rows_since_flush; timer; writer
```

SolarLoggerNode._set_float [L139–L146]

```
signature: def _set_float(self, field)
calls: float
self read: latest
self write: --
```

SolarLoggerNode._set_string [L148–L152]

```
signature: def _set_string(self, field)
calls: replace; str
self read: latest
self write: --
```

SolarLoggerNode._set_array [L154–L160]

```
signature: def _set_array(self, fields)
calls: enumerate; float; len; list
self read: latest
self write: --
```

SolarLoggerNode._clean_float [L163–L168]

```
signature: def _clean_float(value)
calls: float; math.isfinite
self read: --
self write: --
```

SolarLoggerNode._write_row [L170–L181]

```
signature: def _write_row(self)
calls: datetime.now; isoformat; now; self._clean_float; self.csv_file.flush; self.get_clock; self.latest.get; self.writer.writerow; str
self read: _clean_float; csv_file; fields; flush_every_rows; get_clock; latest; writer
self write: rows_since_flush
```

SolarLoggerNode.destroy_node [L183–L189]

```
signature: def destroy_node(self)
calls: destroy_node; self.csv_file.close; self.csv_file.flush; super
self read: csv_file
self write: --
```

main [L192–L199]

```
signature: def main()
calls: SolarLoggerNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.18 mpc_solarcar/solar_preflight_logic.py**evaluate_freshness [L19–L40]**

```
signature: evaluate_freshness(*, elapsed_sec, ages_sec, required, timeout_sec, startup_grace_sec)
calls: FreshnessResult; ages_sec.get; float; join
self read: --
self write: --
```

evaluate_command_gate [L43–L68]

```
signature: evaluate_command_gate(*, elapsed_sec, speed_input_age_sec, system_state, system_state_age_sec, startup_hold_sec, input_timeout_sec,
    system_state_timeout_sec, require_system_running)
calls: CommandGateResult; max; str; strip; upper
self read: --
self write: --
```

17.19 mpc_solarcar/solar_preflight_node.py

SolarPreflightNode.__init__ [L15–L39]

```
signature: def __init__(self)
calls: __init__; info; self._seen; self.create_publisher; self.create_subscription; self.create_timer; self.declare_parameter; self.get_logger;
super; time.monotonic
self read: _seen; _step; create_publisher; create_subscription; create_timer; declare_parameter; get_logger
self write: last_seen; pub_diag; pub_health; pub_state; start_mono; timer
```

SolarPreflightNode._seen [L41–L45]

```
signature: def _seen(self, key: str)
calls: time.monotonic
self read: last_seen
self write: --
```

SolarPreflightNode._required [L47–L57]

```
signature: def _required(self) -> tuple[str, ...]
calls: bool; required.append; required.extend; self.get_parameter; tuple
self read: get_parameter
self write: --
```

SolarPreflightNode._step [L59–L75]

```
signature: def _step(self)
calls: Float32; String; evaluate_freshness; float; self._required; self.get_parameter; self.pub_diag.publish; self.pub_health.publish; self.
pub_state.publish; time.monotonic
self read: _required; get_parameter; last_seen; pub_diag; pub_health; pub_state; start_mono
self write: --
```

main [L78–L85]

```
signature: def main()
calls: SolarPreflightNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.20 mpc_solarcar/solar_profile.py

require_csv_data_rows [L14–L34]

```
signature: require_csv_data_rows(path, *, label, required_columns)
calls: FileNotFoundError; Path; ValueError; csv.DictReader; expanduser; next; resolve; resolved.is_file; resolved.open; tuple
self read: --
self write: --
```

_resolve_profile_path [L37–L48]

```
signature: def _resolve_profile_path(path_like: str | Path) -> Path
calls: Path; Path.cwd; candidate.exists; expanduser; raw.is_absolute; raw.resolve; resolve; str; strip
self read: --
self write: --
```

resolve_relative_path [L51–L65]

```
signature: def resolve_relative_path(base_dir: str | Path, path_like: str | Path) -> str
calls: Path; Path.exists; expanduser; path.is_absolute; path.resolve; repo_candidate.exists; resolve; str; strip
self read: --
self write: --
```

load_profile [L68–L74]

```
signature: def load_profile(path_like: str | Path) -> tuple[Path, dict]
calls: ValueError; _resolve_profile_path; isinstance; profile_path.open; yaml.safe_load
self read: --
self write: --
```

merged_dict [L77–L90]

```
signature: def merged_dict(*payloads: dict | None) -> dict
calls: _merge; copy.deepcopy; dst.get; isinstance; items
self read: --
self write: --
```

get_section [L93–L105]

```
signature: def get_section(cfg: dict, key: str, default: Any = None) -> Any
calls: copy.deepcopy; current.get; isinstance; split; str
self read: --
self write: --
```

get_path [L108–L118]

```
signature: def get_path(cfg: dict, profile_path: str | Path, key: str, default: str = "") -> str
calls: Path; cfg.get; get_section; isinstance; paths_cfg.get; resolve; resolve_relative_path; str; strip
self read: --
self write: --
```

17.21 mpc_solarcar/speed_command_bridge_node.py

SpeedCommandBridgeNode.__init__ [L18–L87]

```
signature: def __init__(self) -> None
calls: SmoothRateLimiter; __init__; bool; float; info; int; max; self.create_publisher; self.create_subscription; self.create_timer; self.
declare_parameter; self.get_logger; self.get_parameter; self.limiter.reset; socket.socket; str; ... (+2)
self read: _on_drive_mode; _on_speed_cmd; _on_system_state; _on_upper_speed_cmd; _step; create_publisher; create_subscription; create_timer;
declare_parameter; get_logger; get_parameter
self write: applied_drive_mode; drive_mode_min_hold_sec; input_timeout_sec; last_drive_mode; last_drive_mode_change; last_gate_reason;
last_mode_input_time; last_speed_cmd; last_speed_input_time; last_system_state_time; last_udp_error_log_time; last_upper_speed_cmd; limiter;
mode_pub; output_drive_mode_topic; output_speed_topic; ... (+13)
```

SpeedCommandBridgeNode._on_speed_cmd [L89–L91]

```
signature: def _on_speed_cmd(self, msg: Float32) -> None
calls: float; time.monotonic
self read: --
self write: last_speed_cmd; last_speed_input_time
```

SpeedCommandBridgeNode._on_upper_speed_cmd [L93–L94]

```
signature: def _on_upper_speed_cmd(self, msg: Float32) -> None
calls: float
self read: --
self write: last_upper_speed_cmd
```

SpeedCommandBridgeNode._on_drive_mode [L96–L98]

```
signature: def _on_drive_mode(self, msg: String) -> None
calls: str; time.monotonic
self read: --
self write: last_drive_mode; last_mode_input_time
```

SpeedCommandBridgeNode._on_system_state [L100–L102]

```
signature: def _on_system_state(self, msg: String) -> None
calls: str; strip; time.monotonic; upper
self read: --
self write: last_system_state_time; system_state
```

SpeedCommandBridgeNode._step [L104–L159]

```
signature: def _step(self) -> None
calls: Float32; String; encode; error; evaluate_command_gate; float; json.dumps; self.get_logger; self.limiter.update; self.mode_pub.publish; self
.sock.sendto; self.speed_pub.publish; time.monotonic; time.time; utc_iso_now
self read: drive_mode_min_hold_sec; get_logger; input_timeout_sec; last_drive_mode; last_mode_input_time; last_speed_cmd; last_speed_input_time;
last_system_state_time; last_upper_speed_cmd; limiter; mode_pub; require_system_running; safe_speed_kmh; sock; speed_pub; start_time; ...
(+5)
self write: applied_drive_mode; last_drive_mode_change; last_gate_reason; last_udp_error_log_time
```

main [L162–L167]

```
signature: def main() -> None
calls: SpeedCommandBridgeNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.22 mpc_solarcar/telemetry_protocol.py

parse_source_timestamp [L18–L41]

```
signature: def parse_source_timestamp(payload: dict) -> float | None
calls: datetime.fromisoformat; float; math.isfinite; parsed.astimezone; payload.get; raw.endswith; str; strip; timestamp
self read: --
self write: --
```

validate_source_timestamp [L44–L72]

```
signature: validate_source_timestamp(payload, *, now_unix, last_source_unix, required, max_age_sec, max_future_skew_sec, max_out_of_order_sec)
calls: TimestampValidation; float; max; parse_source_timestamp
self read: --
self write: --
```

utc_iso_now [L75–L76]

```
signature: def utc_iso_now() -> str
calls: datetime.now; isoformat; replace
self read: --
self write: --
```

17.23 mpc_solarcar/telemetry_text_bridge_node.py

_safe_float [L19–L29]

```
signature: def _safe_float(payload: dict, *keys: str) -> float | None
calls: float; math.isfinite
self read: --
self write: --
```

_parse_key_value_text [L32–L50]

```
signature: def _parse_key_value_text(text: str) -> dict
calls: float; key.strip; replace; split; text.replace; token.split; token.strip; value.strip
self read: --
self write: --
```

parse_text_payload [L53–L63]

```
signature: def parse_text_payload(text: str) -> dict
calls: _parse_key_value_text; isinstance; json.loads; str; strip
self read: --
self write: --
```

headwind_component_ms [L66–L76]

```
signature: def headwind_component_ms(payload: dict) -> float | None
calls: _safe_float; float; math.cos; math.radians
self read: --
self write: --
```

TelemetryTextBridgeNode.__init__ [L80–L193]

```
signature: def __init__(self) -> None
calls: RobustScalarFilter; __init__; bool; float; info; int; max; self._set_outbound; self.create_publisher; self.create_subscription; self.create_timer; self.declare_parameter; self.get_logger; self.get_parameter; self.sock.bind; self.sock.setblocking; ... (+3)
self read: _poll_socket; _send_outbound; _set_outbound; create_publisher; create_subscription; create_timer; declare_parameter; get_logger; get_parameter
self write: chase_remote; enable_inbound; enable_outbound; headwind_filter; i_filter; last_source_unix; max_abs_headwind_ms; max_future_skew_sec; max_out_of_order_sec; max_packet_age_sec; outbound_state; prefer_direct_distance; pub_chase_alt; pub_chase_gps; pub_chase_source_time; pub_course_dir; ... (+31)
```

TelemetryTextBridgeNode._set_outbound [L195–L196]

```
signature: def _set_outbound(self, key: str, value) -> None
calls: --
self read: outbound_state
self write: --
```

TelemetryTextBridgeNode._poll_socket [L198–L236]

```
signature: def _poll_socket(self) -> None
calls: Float64; String; data.decode; lower; parse_text_payload; payload.get; pub.publish; self._handle_chase_payload; self._handle_vehicle_payload; self.pub_raw_chase.publish; self.pub_raw_solar.publish; self.pub_status.publish; self.sock.recvfrom; str; time.time; validate_source_timestamp
self read: _handle_chase_payload; _handle_vehicle_payload; last_source_unix; max_future_skew_sec; max_out_of_order_sec; max_packet_age_sec; pub_chase_source_time; pub_raw_chase; pub_raw_solar; pub_solar_source_time; pub_status; sock; timestamp_required
self write: --
```

TelemetryTextBridgeNode._publish_gps [L238–L251]

```
signature: def _publish_gps(self, pub, lat: float | None, lon: float | None, alt: float | None, source_unix: float | None) -> None
calls: NavSatFix; float; int; max; now; pub.publish; self.get_clock; to_msg
self read: get_clock
self write: --
```

TelemetryTextBridgeNode._handle_vehicle_payload [L253–L289]

```
signature: def _handle_vehicle_payload(self, payload: dict, source_unix: float | None) -> None
calls: Float32; _safe_float; float; headwind_component_ms; self._publish_gps; self._publish_weather; self.i_filter.update; self.pub_vehicle_alt.publish; self.pub_vehicle_i.publish; self.pub_vehicle_s.publish; self.pub_vehicle_soc.publish; self.pub_vehicle_solar.publish; self.pub_vehicle_speed.publish; self.pub_vehicle_tb.publish; self.pub_vehicle_v.publish; self.soc_filter.update; ... (+4)
self read: _publish_gps; _publish_weather; i_filter; prefer_direct_distance; pub_vehicle_alt; pub_vehicle_gps; pub_vehicle_i; pub_vehicle_s; pub_vehicle_soc; pub_vehicle_solar; pub_vehicle_speed; pub_vehicle_tb; pub_vehicle_v; soc_filter; solar_power_gain_to_pack; speed_filter; ... (+2)
self write: --
```

TelemetryTextBridgeNode._handle_chase_payload [L291–L303]

```
signature: def _handle_chase_payload(self, payload: dict, source_unix: float | None) -> None
calls: Float32; _safe_float; float; headwind_component_ms; self._publish_gps; self._publish_weather; self.pub_chase_alt.publish; time.monotonic
self read: _publish_gps; _publish_weather; pub_chase_alt; pub_chase_gps
self write: --
```

TelemetryTextBridgeNode._publish_weather [L305–L315]

```
signature: def _publish_weather(self, payload: dict, wind_speed, wind_dir, course_deg, headwind, now: float) -> None
calls: Float32; String; float; payload.get; self.headwind_filter.update; self.pub_course_dir.publish; self.pub_headwind.publish; self.pub_status.
      publish; self.pub_wind_dir.publish; self.pub_wind_speed.publish; self.wind_filter.update
self read: headwind_filter; pub_course_dir; pub_headwind; pub_status; pub_wind_dir; pub_wind_speed; wind_filter
self write: --
```

TelemetryTextBridgeNode._send_payload [L317–L323]

```
signature: def _send_payload(self, addr: tuple[str, int], payload: dict) -> None
calls: encode; json.dumps; self.tx_sock.sendto
self read: tx_sock
self write: --
```

TelemetryTextBridgeNode._send_outbound [L325–L346]

```
signature: def _send_outbound(self) -> None
calls: self._send_payload; time.time; utc_iso_now
self read: _send_payload; chase_remote; enable_outbound; outbound_state; send_to_chase; send_to_solar; solar_remote
self write: --
```

main [L349–L354]

```
signature: def main() -> None
calls: TelemetryTextBridgeNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.24 mpc_solarcar/upper_cost.py

UpperCostConfig.to_dict [L44–L45]

```
signature: def to_dict(self) -> Dict[str, Any]
calls: asdict
self read: --
self write: --
```

_cfg_value [L48–L51]

```
signature: def _cfg_value(cfg: Optional[dict], key: str, default)
calls: isinstance
self read: --
self write: --
```

load_upper_cost_config [L54–L107]

```
signature: load_upper_cost_config(mpc_cfg, *, legacy, default_drive_window)
calls: UpperCostConfig; _cfg_value; float; isinstance; lower; mpc_cfg.get; pick; str; strip
self read: --
self write: --
```

quad_penalty [L110–L115]

```
signature: def quad_penalty(x: float, cap: float = 1.0e3) -> float
calls: --
self read: --
self write: --
```

upper_stage_cost [L118–L260]

```
signature: upper_stage_cost(cfg, *, dt_wait, dt_travel, v_kmh, v_prev_kmh, vmax_local_kmh, drive_limits, dv_limit_kmhps, I_a, V_v, P_pv_w,
      P_pack_w, P_pack_prev_w, P_mech_wheel_w, losses_int_w, losses_line_w, F_aero_n, kinetic_step_wh, z_next, Tb_next_c, term_soc_min, soc_min,
      soc_max, temp_min_c, temp_max_c, day_end_soc_min, day_end_crossing, soc_day_end_max, soc_day_track_target, soc_day_track_tol, I_max,
      I_chg_min, V_min, V_max, time_ahead_h, progress_lag_km)
calls: abs; float; max; min; quad_penalty
self read: --
self write: --
```

upper_terminal_cost [L263–L280]

```
signature: upper_terminal_cost(cfg, *, z_terminal, term_soc_min, soc_finish_target, soc_finish_tol, progress_terminal_lag_km)
calls: float; max; quad_penalty
self read: --
self write: --
```

active_upper_cost_terms [L283–L288]

```
signature: def active_upper_cost_terms(cfg: UpperCostConfig, threshold: float = 1.0e-6) -> Dict[str, float]
calls: abs; cfg.to_dict; float; items; key.startswith
self read: --
self write: --
```

17.25 mpc_solarcar/upper_horizon.py

UpperDistanceHorizon.total_km [L20–L21]

```
signature: def total_km(self) -> float
calls: float; len; np.sum
self read: ds_seq_km
self write: --
```

_weighted_extra [L24–L30]

```
signature: def _weighted_extra(count: int, growth: float) -> np.ndarray
calls: abs; float; max; np.arange; np.ones; np.power
self read: --
self write: --
```

_adaptive_ds_sequence [L33–L79]

```
signature: _adaptive_ds_sequence(target_km, *, max_steps, min_ds_km, max_ds_km, growth)
calls: _weighted_extra; alloc_weights.sum; base.copy; base.sum; enumerate; float; int; math.ceil; max; min; np.any; np.array; np.flatnonzero; np.full; np.maximum; weights.sum
self read: --
self write: --
```

build_upper_distance_horizon [L82–L133]

```
signature: build_upper_distance_horizon(*, mode, s0_km, race_km, ds_km, horizon_km, max_steps, ctrl_km, adaptive_min_ds_km, adaptive_max_ds_km, adaptive_growth)
calls: UpperDistanceHorizon; _adaptive_ds_sequence; float; int; len; lower; math.ceil; max; min; np.append; np.arange; np.array; np.concatenate; np.cumsum; np.full; np.sum; ... (+2)
self read: --
self write: --
```

plan_segment_index [L136–L145]

```
signature: def plan_segment_index(plan_segments: Iterable[dict], s_km: float) -> int
calls: enumerate; float; len; list; seg.get
self read: --
self write: --
```

17.26 mpc_solarcar/upper_policy.py

load_upper_policy_csv [L9–L30]

```
signature: def load_upper_policy_csv(path: str | Path) -> pd.DataFrame
calls: Path; ValueError; copy; drop_duplicates; dropna; float; len; out.replace; pd.read_csv; pd.to_numeric; required.issubset; reset_index; sort_values; sorted
self read: --
self write: --
```

interpolate_upper_policy [L33–L49]

```
signature: interpolate_upper_policy(frame, control_s_kmh, *, minimum_speed_kmh, maximum_speed_kmh)
calls: ValueError; float; len; np.all; np.asarray; np.clip; np.interp; np.isfinite; pd.to_numeric; to_numpy
self read: --
self write: --
```

17.27 mpc_solarcar/upper_solver.py

_fork_finite_cost [L19–L22]

```
signature: def _fork_finite_cost(vec: np.ndarray) -> float
calls: finite_cost; float
self read: --
self write: --
```

clip_to_bounds [L25–L29]

```
signature: def clip_to_bounds(vec: np.ndarray, bounds: Bounds) -> np.ndarray
calls: copy; enumerate; float; np.asarray; np.clip
self read: --
self write: --
```

finite_cost [L32–L39]

```
signature: def finite_cost(cost_fn: Callable[[np.ndarray], float], vec: np.ndarray) -> float
calls: cost_fn; float; np.asarray; np.isfinite
self read: --
self write: --
```

default_seed_library [L42–L54]

```
signature: def default_seed_library(x0: np.ndarray, bounds: Bounds) -> List[tuple[str, np.ndarray]]
calls: clip_to_bounds; float; len; np.array; np.asarray; np.linspace; np.maximum; seeds.append
self read: --
self write: --
```

hybrid_bounded_minimize [L57–L473]

```
signature: hybrid_bounded_minimize(cost_fn, x0, bounds, *, maxiter, structured_seeds, cem_enabled, cem_mode, cem_generations, cem_population,
cem_elite, local_refine_topk, seed_library_mode, rng_seed, shgo_samples, shgo_iters, cert_grid_levels, cert_grid_values,
cert_max_evaluations, cert_workers, progress_callback, cert_progress_interval)
calls: ProcessPoolExecutor; ThreadPoolExecutor; abs; add_candidate; best.get; bool; clip_to_bounds; copy; default_seed_library; dict;
emit_progress; enumerate; evaluated.append; evaluated.extend; executor.map; finite_cost; ... (+51)
self read: --
self write: --
```

17.28 mpc_solarcar/utils_maps.py

bilinear_interp [L3–L12]

```
signature: def bilinear_interp(xg, yg, Z, x, y)
calls: len; np.asarray; np.clip; np.searchsorted
self read: --
self write: --
```

read_eff_map [L13–L15]

```
signature: def read_eff_map(path)
calls: df.columns.values.astype; df.index.values.astype; df.values.astype; pd.read_csv
self read: --
self write: --
```

read_Rint_map [L16–L18]

```
signature: def read_Rint_map(path)
calls: df.columns.values.astype; df.index.values.astype; df.values.astype; pd.read_csv
self read: --
self write: --
```

read_map [L20–L22]

```
signature: def read_map(path)
calls: df.columns.values.astype; df.index.values.astype; df.values.astype; pd.read_csv
self read: --
self write: --
```

read_1d_map [L24–L33]

```
signature: def read_1d_map(path)
calls: df.index.values.astype; pd.read_csv; values.astype
self read: --
self write: --
```

17.29 mpc_solarcar/weather_fetch_node.py

WeatherFetchNode.__init__ [L16–L102]

```
signature: def __init__(self) -> None
calls: Path; __init__; expanduser; float; get_path; get_section; info; int; live_cfg.get; load_profile; max; rclpy.parameter.Parameter;
runtime_cfg.get; self._fetch_once; self.create_publisher; self.create_subscription; ... (+9)
self read: _fetch_once; _on_gps; create_publisher; create_subscription; create_timer; declare_parameter; get_logger; get_parameter; set_parameters
self write: fallback_lat; fallback_lon; fetch_period_sec; forecast_days; gps_topic; latest_lat; latest_lon; output_csv; pub_status;
raw_forecast_csv; step_minutes; tcell_gain; timer; timezone_name
```

WeatherFetchNode._on_gps [L104–L109]

```
signature: def _on_gps(self, msg: NavSatFix) -> None
calls: float
self read: --
self write: latest_lat; latest_lon
```

WeatherFetchNode._fetch_once [L111–L135]

```
signature: def _fetch_once(self) -> None
calls: String; error; fetch_openmeteo_forecast; info; len; raw_path.is_absolute; resolve; self.get_logger; self.pub_status.publish;
write_forecast_csv
self read: fallback_lat; fallback_lon; forecast_days; get_logger; latest_lat; latest_lon; output_csv; pub_status; raw_forecast_csv; step_minutes;
tcell_gain; timezone_name
self write: --
```

main [L138–L143]

```
signature: def main() -> None
calls: WeatherFetchNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.30 mpc_solarcar/weather_utils.py

_fetch_json [L41–L44]

```
signature: def _fetch_json(url: str) -> dict
calls: json.load; urllib.request.Request; urllib.request.urlopen
self read: --
self write: --
```

ideal_tracking_poa_wm2 [L47–L64]

```
signature: def ideal_tracking_poa_wm2(ghi, dni, dhi, *, ground_albedo: float = 0.20)
calls: float; np.asarray; np.clip; np.divide; np.maximum; np.where; np.zeros_like
self read: --
self write: --
```

fetch_openmeteo_forecast [L67–L119]

```
signature: fetch_openmeteo_forecast(latitude, longitude, *, timezone_name, forecast_days, step_minutes, tcell_gain)
calls: _fetch_json; bfill; df.get; dropna; dt.strftime; ffill; fillna; float; ideal_tracking_poa_wm2; int; interpolate; join; max; payload.get; pd.DataFrame; pd.to_datetime; ... (+7)
self read: --
self write: --
```

write_forecast_csv [L122–L128]

```
signature: def write_forecast_csv(df: pd.DataFrame, out_csv: str | Path) -> None
calls: Path; df.copy; out_path.parent.mkdir; work.sort_values; work.to_csv
self read: --
self write: --
```

17.31 mpc_solarcar/wind_correction_node.py

_timestamp_ns [L16–L17]

```
signature: def _timestamp_ns(value) -> int
calls: int; pd.Timestamp
self read: --
self write: --
```

WindCorrectionNode.__init__ [L21–L62]

```
signature: def __init__(self) -> None
calls: NormalDist; Path; __init__; bool; expanduser; float; info; inv_cdf; max; min; self.create_publisher; self.create_subscription; self.create_timer; self.declare_parameter; self.get_logger; self.get_parameter; ... (+2)
self read: _on_headwind; _on_s; _step; create_publisher; create_subscription; create_timer; declare_parameter; get_logger; get_parameter
self write: base_df; base_mtime; confidence_z; corrected_csv; correlation_distance_km; correlation_time_h; forecast_csv; forecast_sigma0; forecast_time_tz; latest_bias; latest_headwind; latest_s_km; measurement_sigma; min_sigma; planning_quantile; pub_headwind; ... (+5)
```

WindCorrectionNode._on_headwind [L64–L65]

```
signature: def _on_headwind(self, msg: Float32) -> None
calls: float
self read: --
self write: latest_headwind
```

WindCorrectionNode._on_s [L67–L68]

```
signature: def _on_s(self, msg: Float32) -> None
calls: float
self read: --
self write: latest_s_km
```

WindCorrectionNode._load_base [L70–L89]

```
signature: def _load_base(self) -> None
calls: dt.tz_convert; getattr; os.path.getmtime; pd.read_csv; pd.to_datetime; t.dt.tz_convert; t.dt.tz_localize
self read: forecast_csv; forecast_time_tz
self write: base_df; base_mtime
```

WindCorrectionNode._interp_headwind_now [L91–L103]

```
signature: def _interp_headwind_now(self, now_utc: datetime) -> float
calls: any; float; int; len; max; min; notna; pd.Timestamp; row.get; searchsorted
self read: base_df
self write: --
```

WindCorrectionNode._step [L105–L151]

```
signature: def _step(self) -> None
calls: Float32; String; abs; astimezone; corrected_values.append; datetime.now; df.copy; df.get; df.itertuples; dt.strftime; dt.tz_convert; fillna
; float; getattr; len; math.exp; ... (+18)
self read: _interp_headwind_now; _load_base; base_df; confidence_z; corrected_csv; correlation_distance_km; correlation_time_h; forecast_sigma0;
forecast_time_tz; latest_headwind; latest_s_km; measurement_sigma; min_sigma; pub_headwind; pub_status; quantile_z; ... (+2)
self write: latest_bias
```

main [L154–L159]

```
signature: def main() -> None
calls: WindCorrectionNode; node.destroy_node; rclpy.init; rclpy.shutdown; rclpy.spin
self read: --
self write: --
```

17.32 setup.py

collect_data_files [L8–L21]

```
signature: def collect_data_files(src_dir, *, excluded_dirs=())
calls: collected.append; os.path.isdir; os.path.join; os.walk
self read: --
self write: --
```

Appendix A

live 到達 source 全文

A.1 SolarSim.ps1

```
1 param(
2     [ValidateSet('up', 'build', 'start', 'stop', 'restart', 'status', 'graph', 'simulate', 'forecast', 'identify', 'fit', 'learn', 'audit', '
3     package', 'blank', 'log', 'grafana', 'grafana-stop')]
4     [string]$Action = 'up',
5     [ValidateSet('sim', 'measure', 'live', 'live_wifi')]
6     [string]$Mode = 'sim',
7     [string]$Profile = 'config/solar/bwsc_2027_demo.yaml',
8     [string]$Distro = ''
9 )
10 $ErrorActionPreference = 'Stop'
11
12 function Normalize-WslText {
13     param([string]$Text)
14     if ($null -eq $Text) {
15         return ''
16     }
17     return ([string]$Text).Replace("`0", '').Trim()
18 }
19
20 function Resolve-WslDistro {
21     param([string]$Preferred)
22     if ($Preferred) {
23         return (Normalize-WslText $Preferred)
24     }
25     $installed = (& wsl.exe -l -q) | ForEach-Object { Normalize-WslText $_ } | Where-Object { $_ }
26     foreach ($candidate in @( 'Ubuntu-22.04', 'Ubuntu', 'Ubuntu-24.04' )) {
27         if ($installed -contains $candidate) {
28             return $candidate
29         }
30     }
31     if ($installed.Count -gt 0) {
32         return $installed[0]
33     }
34     throw 'WSL distro was not found.'
35 }
36
37 function Resolve-WslArgPath {
38     param(
39         [string]$TargetDistro,
40         [string]$PathValue
41     )
42     if ([string]::IsNullOrEmpty($PathValue)) {
43         return ''
44     }
45     if ([System.IO.Path]::IsPathRooted($PathValue)) {
46         return (& wsl.exe -d $TargetDistro wslpath -a $PathValue).Trim()
47     }
48     return ($PathValue -replace '\\', '/')
49 }
50
51 function Invoke-WslRepoCommand {
52     param(
53         [string]$RepoRoot,
54         [string]$TargetDistro,
55         [string]$Command,
56         [string]$TargetMode,
57         [string]$ProfilePath
58     )
59     $wslRepo = (& wsl.exe -d $TargetDistro wslpath -a $RepoRoot).Trim()
60     if (-not $wslRepo) {
61         throw 'Failed to resolve the workspace path in WSL.'
62     }
63     $wslProfile = Resolve-WslArgPath -TargetDistro $TargetDistro -PathValue $ProfilePath
64     & wsl.exe -d $TargetDistro bash -lc "cd '$wslRepo' && bash './scripts/solar_control.sh' '$Command' '$TargetMode' '$wslProfile'"
65 }
66
67 function Wait-ForDashboard {
68     param([int]$Port = 8080)
69     $uris = @(
70         "http://localhost:$Port/api/state",
71         "http://127.0.0.1:$Port/api/state",
72         "http://wsl.localhost:$Port/api/state"
73     )
74 }
```

```

74 $deadline = (Get-Date).AddSeconds(90)
75 while ((Get-Date) -lt $deadline) {
76     foreach ($uri in $uris) {
77         try {
78             $res = Invoke-RestMethod -Uri $uri -Method Get -TimeoutSec 2
79             if ($null -ne $res.ts) {
80                 return $uri
81             }
82         } catch {
83         }
84     }
85     Start-Sleep -Milliseconds 750
86 }
87 return $null
88 }
89
90 function Get-ProfileDashboardPort {
91     param([string]$ProfilePath)
92     $resolved = Join-Path $repoRoot $ProfilePath
93     if (-not (Test-Path $resolved)) {
94         return 8080
95     }
96     $inRuntime = $false
97     foreach ($line in Get-Content $resolved) {
98         if ($line -match '^s*runtime\s*:\s*$') {
99             $inRuntime = $true
100             continue
101         }
102         if ($inRuntime -and $line -match '^S') {
103             break
104         }
105         if ($inRuntime -and $line -match '^s*dashboard_port\s*:\s*([0-9]+)\s*$') {
106             return [int]$Matches[1]
107         }
108     }
109     return 8080
110 }
111
112 function Get-ProfileSimulationReportPath {
113     param([string]$ProfilePath)
114     $resolved = Join-Path $repoRoot $ProfilePath
115     $outputDir = 'outputs/prerace'
116     $outputPrefix = 'solar_prerace'
117     $reportHtml = ''
118     $latestManifest = ''
119     if (-not (Test-Path $resolved)) {
120         return (Join-Path $repoRoot (Join-Path $outputDir "$outputPrefix_report.html"))
121     }
122     $inSimulation = $false
123     foreach ($line in Get-Content $resolved) {
124         if ($line -match '^s*simulation\s*:\s*$') {
125             $inSimulation = $true
126             continue
127         }
128         if ($inSimulation -and $line -match '^S') {
129             break
130         }
131         if (-not $inSimulation) {
132             continue
133         }
134         if ($line -match '^s*output_dir\s*:\s*(.+)s*$') {
135             $outputDir = $Matches[1].Trim().Trim('""')
136             continue
137         }
138         if ($line -match '^s*output_prefix\s*:\s*(.+)s*$') {
139             $outputPrefix = $Matches[1].Trim().Trim('""')
140             continue
141         }
142         if ($line -match '^s*report_html\s*:\s*(.+)s*$') {
143             $reportHtml = $Matches[1].Trim().Trim('""')
144             continue
145         }
146         if ($line -match '^s*latest_manifest_json\s*:\s*(.+)s*$') {
147             $latestManifest = $Matches[1].Trim().Trim('""')
148         }
149     }
150     if ([string]::IsNullOrEmpty($latestManifest)) {
151         $latestManifest = Join-Path $outputDir 'latest_simulation_run.json'
152     }
153     if (-not [System.IO.Path]::IsPathRooted($latestManifest)) {
154         $latestManifest = Join-Path $repoRoot $latestManifest
155     }
156     if (Test-Path $latestManifest) {
157         try {
158             $manifest = Get-Content $latestManifest -Raw | ConvertFrom-Json
159             if ($null -ne $manifest.report_html -and -not [string]::IsNullOrEmpty([string]$manifest.report_html)) {
160                 $candidate = [string]$manifest.report_html
161                 if (-not [System.IO.Path]::IsPathRooted($candidate)) {
162                     $candidate = Join-Path $repoRoot $candidate
163                 }
164                 return $candidate
165             }
166         } catch {
167         }
168     }
169     if (-not [string]::IsNullOrEmpty($reportHtml)) {
170         if ([System.IO.Path]::IsPathRooted($reportHtml)) {
171             return $reportHtml
172         }
173         return (Join-Path $repoRoot $reportHtml)
174     }
175     return (Join-Path $repoRoot (Join-Path $outputDir "$outputPrefix_report.html"))
176 }

```

```

177
178 $repoRoot = Split-Path -Parent $MyInvocation.MyCommand.Path
179
180 if ($Action -in @('grafana', 'grafana-stop')) {
181     if (-not (Get-Command docker -ErrorAction SilentlyContinue)) {
182         throw 'Docker was not found. Install Docker Desktop (Windows) or Docker Engine with Compose (Ubuntu), then retry.'
183     }
184     $grafanaDir = Join-Path $repoRoot 'grafana'
185     Push-Location $grafanaDir
186     try {
187         if ($Action -eq 'grafana') {
188             & docker compose up -d
189             if ($LASTEXITCODE -ne 0) {
190                 throw "docker compose up failed with exit code $LASTEXITCODE"
191             }
192             Start-Process 'http://localhost:3000/d/solarcar-ems/yata-solarcar-ems' | Out-Null
193         } else {
194             & docker compose down
195             if ($LASTEXITCODE -ne 0) {
196                 throw "docker compose down failed with exit code $LASTEXITCODE"
197             }
198         }
199     } finally {
200         Pop-Location
201     }
202     exit 0
203 }
204
205 if ($Action -in @('package', 'blank')) {
206     $builder = if ($Action -eq 'package') { 'scripts/create_solarcar_only_package.py' } else { 'scripts/create_solarcar_blank_package.py' }
207     Push-Location $repoRoot
208     try {
209         & python $builder --force
210         if ($LASTEXITCODE -ne 0) {
211             throw "$builder failed with exit code $LASTEXITCODE"
212         }
213     } finally {
214         Pop-Location
215     }
216     exit 0
217 }
218
219 $resolvedDistro = Resolve-WslDistro -Preferred $Distro
220 $graphPath = Join-Path $repoRoot "rqt_graph_solar_$Mode.png"
221 $dashboardPort = Get-ProfileDashboardPort -ProfilePath $Profile
222
223 switch ($Action) {
224     'up' {
225         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'up' -TargetMode $Mode -ProfilePath $Profile
226         if ($Mode -in @('sim', 'measure', 'live', 'live_wifi')) {
227             $dashboardUri = Wait-ForDashboard -Port $dashboardPort
228             if (-not $dashboardUri) {
229                 throw "Dashboard did not become available on port $dashboardPort."
230             }
231             if (Get-Command docker -ErrorAction SilentlyContinue) {
232                 Push-Location (Join-Path $repoRoot 'grafana')
233                 try {
234                     & docker compose up -d
235                     if ($LASTEXITCODE -ne 0) {
236                         throw "docker compose up failed with exit code $LASTEXITCODE"
237                     }
238                 } finally {
239                     Pop-Location
240                 }
241                 Start-Process 'http://localhost:3000/d/solarcar-ems/yata-solarcar-ems' | Out-Null
242             } else {
243                 Write-Warning 'Docker was not found; opening the built-in fallback dashboard instead of Grafana.'
244                 Start-Process $dashboardUri | Out-Null
245             }
246             Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'graph' -TargetMode $Mode -ProfilePath $Profile
247             if (Test-Path $graphPath) {
248                 Start-Process $graphPath | Out-Null
249             }
250         }
251     }
252     'build' {
253         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'build' -TargetMode $Mode -ProfilePath $Profile
254     }
255     'start' {
256         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'start' -TargetMode $Mode -ProfilePath $Profile
257     }
258     'stop' {
259         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'stop' -TargetMode $Mode -ProfilePath $Profile
260     }
261     'restart' {
262         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'restart' -TargetMode $Mode -ProfilePath $Profile
263     }
264     'status' {
265         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'status' -TargetMode $Mode -ProfilePath $Profile
266     }
267     'graph' {
268         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'graph' -TargetMode $Mode -ProfilePath $Profile
269         if (Test-Path $graphPath) {
270             Start-Process $graphPath | Out-Null
271         }
272     }
273     'simulate' {
274         Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'simulate' -TargetMode 'sim' -ProfilePath $Profile
275         $reportPath = Get-ProfileSimulationReportPath -ProfilePath $Profile
276         if (Test-Path $reportPath) {
277             Start-Process $reportPath | Out-Null
278         }
279     }
280 }

```

```

280 'forecast' {
281     Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'forecast' -TargetMode 'sim' -ProfilePath $Profile
282 }
283 'identify' {
284     Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'identify' -TargetMode 'sim' -ProfilePath $Profile
285 }
286 'fit' {
287     Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'fit' -TargetMode 'sim' -ProfilePath $Profile
288 }
289 'learn' {
290     Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'learn' -TargetMode 'sim' -ProfilePath $Profile
291 }
292 'audit' {
293     Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'audit' -TargetMode 'sim' -ProfilePath $Profile
294 }
295 'log' {
296     Invoke-WslRepoCommand -RepoRoot $repoRoot -TargetDistro $resolvedDistro -Command 'log' -TargetMode $Mode -ProfilePath $Profile
297 }
298 }

```

A.2 scripts/solar_control.sh

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
5  RUN_DIR="${ROOT_DIR}/.run"
6  DISTRO_NAME="${ROS_DISTRO:-humble}"
7
8  mkdir -p "${RUN_DIR}"
9  export AMENT_TRACE_SETUP_FILES="${AMENT_TRACE_SETUP_FILES-}"
10
11 action="${1:-up}"
12 mode="${2:-sim}"
13 profile="${3:-config/solar/bwsc_2027_demo.yaml}"
14
15 case "${mode}" in
16     sim)
17         launch_file="solarcar_sim.launch.py"
18         ;;
19     measure)
20         launch_file="solar_measurement.launch.py"
21         ;;
22     live)
23         launch_file="solar_race_live.launch.py"
24         ;;
25     live_wifi)
26         launch_file="solar_race_live_wifi.launch.py"
27         ;;
28     *)
29         echo "unknown mode: ${mode}" >&2
30         exit 2
31         ;;
32 esac
33
34 PID_FILE="${RUN_DIR}/solar_${mode}.pid"
35 LOG_FILE="${RUN_DIR}/solar_${mode}.log"
36 GRAPH_BASE="${ROOT_DIR}/rqt_graph_solar_${mode}"
37
38 source_ros() {
39     set +u
40     # shellcheck disable=SC1091
41     source "/opt/ros/${DISTRO_NAME}/setup.bash"
42     if [[ -f "${ROOT_DIR}/install/setup.bash" ]]; then
43         # shellcheck disable=SC1091
44         source "${ROOT_DIR}/install/setup.bash"
45     fi
46     set -u
47 }
48
49 is_running() {
50     [[ -f "${PID_FILE}" ]] || return 1
51     local pid
52     pid="$(cat "${PID_FILE}")"
53     [[ -n "${pid}" ]] || return 1
54     ps -p "${pid}" -o pid= 2>/dev/null | grep -q .
55 }
56
57 find_matching_pids() {
58     ps -eo pid,args= | awk '
59     $0 ~ /\opt\/ros\/humble\/bin\/ros2 launch mpc_solarcar solar(car_sim|measurement|race_live|race_live_wifi)\.launch\.py/ || $0 ~ /install\/
60     mpc_solarcar\/lib\/mpc_solarcar\/(gps_sim_node|mpc_node|solar_state_node|dashboard_node|solar_logger_node|distance_node|grade_node|
61     weather_fetch_node|solar_autocal_node|speed_command_bridge_node|telemetry_text_bridge_node|wind_correction_node)/ {
62         print $1
63     }
64 '
65
66 has_matching_processes() {
67     [[ -n "$(find_matching_pids)" ]]
68 }
69
70 kill_matching_processes() {
71     local pid
72     while read -r pid; do
73         [[ -n "${pid}" ]] || continue
74         kill -TERM "${pid}" 2>/dev/null || true
75     done < <(find_matching_pids)

```

```

75 }
76
77 stop_launch() {
78   if is_running; then
79     local pid
80     pid="$(cat "${PID_FILE}")"
81     kill -TERM "${pid}" 2>/dev/null || true
82   fi
83   kill_matching_processes
84   for _ in $(seq 1 20); do
85     if ! has_matching_processes; then
86       rm -f "${RUN_DIR}"/solar_*.pid
87       echo "stopped"
88       return 0
89     fi
90     sleep 0.5
91   done
92   if [[ -f "${PID_FILE}" ]]; then
93     local pid
94     pid="$(cat "${PID_FILE}")"
95     kill -KILL "${pid}" 2>/dev/null || true
96   fi
97   while read -r pid; do
98     [[ -n "${pid}" ]] || continue
99     kill -KILL "${pid}" 2>/dev/null || true
100   done < <(find_matching_pids)
101   rm -f "${RUN_DIR}"/solar_*.pid
102   echo "stopped"
103 }
104
105 build_pkg() {
106   source_ros
107   cd "${ROOT_DIR}"
108   colcon build --packages-select mpc_solarcar
109 }
110
111 start_launch() {
112   source_ros
113   cd "${ROOT_DIR}"
114   if is_running; then
115     echo "already running"
116     return 0
117   fi
118   : > "${LOG_FILE}"
119   local root_q profile_q launch_cmd
120   printf -v root_q "%q" "${ROOT_DIR}"
121   printf -v profile_q "%q" "${profile}"
122   launch_cmd="export AMENT_TRACE_SETUP_FILES=\${AMENT_TRACE_SETUP_FILES-}; cd ${root_q} && set +u && source /opt/ros/${DISTR0_NAME}/setup.bash &&
123     source ${root_q}/install/setup.bash && set -u && exec ros2 launch mpc_solarcar ${launch_file} profile_yaml:=${profile_q}"
124   nohup setsid bash -lc "${launch_cmd}" >> "${LOG_FILE}" 2>&1 < /dev/null &
125   local pid=$!
126   echo "${pid}" > "${PID_FILE}"
127   for _ in $(seq 1 20); do
128     if ! ps -p "${pid}" -o pid= 2>/dev/null | grep -q .; then
129       echo "launch failed: ros2 launch exited immediately" >&2
130       [[ -f "${LOG_FILE}" ]] && tail -n 80 "${LOG_FILE}" >&2 || true
131       rm -f "${PID_FILE}"
132       return 1
133     fi
134     sleep 0.5
135   done
136   echo "started pid=${pid} mode=${mode}"
137 }
138
139 status_launch() {
140   if is_running || has_matching_processes; then
141     local pid
142     pid="$(cat "${PID_FILE}" 2>/dev/null || true)"
143     echo "running pid=${pid} mode=${mode}"
144     return 0
145   fi
146   echo "stopped"
147   return 1
148 }
149
150 graph_launch() {
151   source_ros
152   cd "${ROOT_DIR}"
153   python3 "${ROOT_DIR}/scripts/export_rqt_graph.py" --output-base "${GRAPH_BASE}" --wait-sec 20 --mode "${mode}"
154 }
155
156 simulate_offline() {
157   source_ros
158   cd "${ROOT_DIR}"
159   python3 "${ROOT_DIR}/scripts/solar_sim.py" --profile_yaml "${profile}"
160 }
161
162 fetch_weather() {
163   source_ros
164   cd "${ROOT_DIR}"
165   python3 "${ROOT_DIR}/scripts/fetch_weather_forecast.py" --profile_yaml "${profile}"
166 }
167
168 identify_pipeline() {
169   source_ros
170   cd "${ROOT_DIR}"
171   python3 "${ROOT_DIR}/scripts/run_identification_pipeline.py" --profile_yaml "${profile}"
172 }
173
174 fit_vehicle() {
175   source_ros
176   cd "${ROOT_DIR}"
177   # Fit quality, map reconstruction, and output tagging are profile-YAML settings.

```

```

177 python3 "${ROOT_DIR}/scripts/run_vehicle_identification.py" --profile "${profile}"
178 }
179
180 learn_upper_planner() {
181     source_ros
182     cd "${ROOT_DIR}"
183     python3 "${ROOT_DIR}/scripts/tune_upper_planner_weights.py" \
184         --profile_yaml "${profile}" --scenario-mode nominal
185 }
186
187 audit_package() {
188     cd "${ROOT_DIR}"
189     python3 "${ROOT_DIR}/scripts/generate_package_inventory.py"
190     python3 "${ROOT_DIR}/scripts/audit_solar_package.py"
191     python3 -m pytest -q
192 }
193
194 tail_log() {
195     if [[ -f "${LOG_FILE}" ]]; then
196         tail -n 80 "${LOG_FILE}"
197     fi
198 }
199
200 case "${action}" in
201     build)
202         build_pkg
203         ;;
204     start)
205         start_launch
206         ;;
207     stop)
208         stop_launch
209         ;;
210     restart)
211         stop_launch
212         start_launch
213         ;;
214     status)
215         status_launch
216         ;;
217     graph)
218         graph_launch
219         ;;
220     up)
221         build_pkg
222         stop_launch
223         start_launch
224         ;;
225     simulate)
226         build_pkg
227         simulate_offline
228         ;;
229     forecast)
230         build_pkg
231         fetch_weather
232         ;;
233     identify)
234         build_pkg
235         identify_pipeline
236         ;;
237     fit)
238         build_pkg
239         fit_vehicle
240         ;;
241     learn)
242         build_pkg
243         learn_upper_planner
244         ;;
245     audit)
246         audit_package
247         ;;
248     log)
249         tail_log
250         ;;
251 *)
252     echo "usage: $0 [up|build|start|stop|restart|status|graph|simulate|forecast|identify|fit|learn|audit|log] [sim|measure|live|live_wifi] [
253         profile_yaml]" >&2
254     exit 2
255 esac

```

A.3 launch/solar_race_live.launch.py

```

1 from __future__ import annotations
2
3 from launch import LaunchDescription
4 from launch.actions import DeclareLaunchArgument, OpaqueFunction
5 from launch.substitutions import LaunchConfiguration
6 from mpc_solarcar.live_launch import build_live_nodes
7 from mpc_solarcar.solar_profile import load_profile
8
9
10 def _setup(context):
11     profile_yaml = LaunchConfiguration("profile_yaml").perform(context)
12     profile_path, cfg = load_profile(profile_yaml)
13     return build_live_nodes(profile_path, cfg, use_wifi=False)
14
15

```

```

16 def generate_launch_description():
17     return LaunchDescription(
18         [
19             DeclareLaunchArgument("profile_yaml", default_value="config/solar/bwsc_2027_demo.yaml"),
20             OpaqueFunction(function=_setup),
21         ]
22     )

```

A.4 launch/solar_race_live_wifi.launch.py

```

1 from __future__ import annotations
2
3 from launch import LaunchDescription
4 from launch.actions import DeclareLaunchArgument, OpaqueFunction
5 from launch.substitutions import LaunchConfiguration
6 from launch_ros.actions import Node
7
8 from mpc_solarcar.live_launch import build_live_nodes, live_forecast_paths
9 from mpc_solarcar.solar_profile import get_section, load_profile
10
11
12 def _setup(context):
13     profile_yaml = LaunchConfiguration("profile_yaml").perform(context)
14     profile_path, cfg = load_profile(profile_yaml)
15     live_cfg = get_section(cfg, "live")
16     wifi_cfg = get_section(live_cfg, "wifi_bridge")
17     wind_cfg = get_section(live_cfg, "wind_model")
18     model_cfg = get_section(cfg, "model")
19
20     _base_csv, raw_forecast_csv, corrected_csv = live_forecast_paths(profile_path, cfg)
21
22     nodes = build_live_nodes(profile_path, cfg, use_wifi=True)
23     if bool(wifi_cfg.get("enabled", True)):
24         nodes.append(
25             Node(
26                 package="mpc_solarcar",
27                 executable="telemetry_text_bridge_node",
28                 name="telemetry_text_bridge_node",
29                 parameters=[
30                     {
31                         **{key: value for key, value in wifi_cfg.items() if key != "enabled"},
32                         "solar_power_gain_to_pack": float(
33                             model_cfg.get("solar_measurement_gain_to_pack", 1.0)
34                         ),
35                     },
36                 ],
37             )
38         )
39     if bool(wind_cfg.get("enabled", True)):
40         nodes.append(
41             Node(
42                 package="mpc_solarcar",
43                 executable="wind_correction_node",
44                 name="wind_correction_node",
45                 parameters=[
46                     {
47                         "forecast_csv": raw_forecast_csv,
48                         "corrected_forecast_csv": corrected_csv,
49                         "forecast_time_tz": str(live_cfg.get("forecast_time_tz", "Australia/Darwin")),
50                         **{
51                             key: value
52                             for key, value in wind_cfg.items()
53                             if key not in {"enabled", "corrected_forecast_csv"}
54                         },
55                     },
56                 ],
57             )
58         )
59     return nodes
60
61
62 def generate_launch_description():
63     return LaunchDescription(
64         [
65             DeclareLaunchArgument("profile_yaml", default_value="config/solar/bwsc_2027_demo.yaml"),
66             OpaqueFunction(function=_setup),
67         ]
68     )

```

A.5 mpc_solarcar/live_launch.py

```

1 from __future__ import annotations
2
3 import shutil
4 from pathlib import Path
5
6 from launch_ros.actions import Node
7
8 from .solar_profile import get_path, get_section, resolve_relative_path
9
10
11 def cfg_path(profile_path, raw: str) -> str:

```

```

12     return resolve_relative_path(profile_path.parent, raw) if str(raw or "").strip() else ""
13
14
15 def _drop_keys(payload: dict, *keys: str) -> dict:
16     blocked = set(keys)
17     return {key: value for key, value in (payload or {}).items() if key not in blocked}
18
19
20 def live_forecast_paths(profile_path, cfg):
21     live_cfg = get_section(cfg, "live")
22     weather_cfg = get_section(live_cfg, "weather")
23     wind_cfg = get_section(live_cfg, "wind_model")
24     base_path = get_path(cfg, profile_path, "forecast_csv")
25     raw_path = cfg_path(profile_path, str(weather_cfg.get("raw_forecast_csv", "")))
26     if not raw_path:
27         raw_path = cfg_path(profile_path, "outputs/runtime/live_forecast_raw.csv")
28     corrected_path = cfg_path(profile_path, str(wind_cfg.get("corrected_forecast_csv", "")))
29     if not corrected_path:
30         corrected_path = cfg_path(profile_path, "outputs/runtime/live_forecast_corrected.csv")
31     for runtime_path in (raw_path, corrected_path):
32         target = Path(runtime_path)
33         if not target.exists() and Path(base_path).exists():
34             target.parent.mkdir(parents=True, exist_ok=True)
35             shutil.copy2(base_path, target)
36     return base_path, raw_path, corrected_path
37
38
39 def build_live_nodes(profile_path, cfg, *, use_wifi: bool):
40     runtime_cfg = get_section(cfg, "runtime")
41     logging_cfg = get_section(cfg, "logging")
42     live_cfg = get_section(cfg, "live")
43     weather_cfg = get_section(live_cfg, "weather")
44     autocal_cfg = get_section(live_cfg, "autocal")
45     command_cfg = get_section(live_cfg, "command_bridge")
46     distance_cfg = get_section(live_cfg, "distance")
47     grade_cfg = get_section(live_cfg, "grade")
48     wind_cfg = get_section(live_cfg, "wind_model")
49     live_logging_cfg = get_section(live_cfg, "logging")
50     preflight_cfg = get_section(live_cfg, "preflight")
51
52     base_forecast_csv, raw_forecast_csv, corrected_forecast_csv = live_forecast_paths(profile_path, cfg)
53     mpc_forecast_csv = raw_forecast_csv
54     if use_wifi and bool(wind_cfg.get("enabled", True)):
55         mpc_forecast_csv = corrected_forecast_csv
56
57     nodes = [
58         Node(
59             package="mpc_solarcar",
60             executable="solar_preflight_node",
61             name="solar_preflight_node",
62             parameters=[
63                 {
64                     "require_speed": True,
65                     "require_distance": True,
66                     "require_battery": True,
67                     "require_planner": True,
68                     **preflight_cfg,
69                 }
70             ],
71         ),
72         Node(
73             package="mpc_solarcar",
74             executable="mpc_node",
75             name="mpc_node",
76             parameters=[
77                 {
78                     "forecast_csv": mpc_forecast_csv,
79                     "route_profile_csv": get_path(cfg, profile_path, "route_profile_csv"),
80                     "speed_profile_csv": get_path(cfg, profile_path, "speed_profile_csv"),
81                     "stop_yaml": get_path(cfg, profile_path, "stop_yaml"),
82                     "drive_schedule_yaml": get_path(cfg, profile_path, "drive_schedule_yaml"),
83                     "initial_upper_policy_csv": get_path(cfg, profile_path, "initial_upper_policy_csv"),
84                     "drive_eff_map": get_path(cfg, profile_path, "drive_eff_map"),
85                     "regen_eff_map": get_path(cfg, profile_path, "regen_eff_map"),
86                     "rint_map": get_path(cfg, profile_path, "rint_map"),
87                     "panel_eff_map": get_path(cfg, profile_path, "panel_eff_map"),
88                     "mppt_eff_map": get_path(cfg, profile_path, "mppt_eff_map"),
89                     "drive_map_eco": get_path(cfg, profile_path, "drive_map_eco"),
90                     "drive_map_power": get_path(cfg, profile_path, "drive_map_power"),
91                     "regen_map_eco": get_path(cfg, profile_path, "regen_map_eco"),
92                     "regen_map_power": get_path(cfg, profile_path, "regen_map_power"),
93                     "ocv_soc_map": get_path(cfg, profile_path, "ocv_soc_map"),
94                     "params_yaml": str(profile_path),
95                     "profile_runtime_mode": "live",
96                     "forecast_time_mode": str(live_cfg.get("forecast_time_mode", runtime_cfg.get("forecast_time_mode", "absolute"))),
97                     "forecast_time_tz": str(live_cfg.get("forecast_time_tz", runtime_cfg.get("forecast_time_tz", "Australia/Darwin"))),
98                     "forecast_start_time_utc": str(get_section(cfg, "simulation").get("forecast_start_time_utc", get_section(cfg, "simulation").get("start_utc", ""))),
99                 }
100             ],
101         ),
102         Node(
103             package="mpc_solarcar",
104             executable="dashboard_node",
105             name="dashboard_node",
106             parameters=[
107                 {
108                     "host": str(runtime_cfg.get("dashboard_host", "0.0.0.0")),
109                     "port": int(runtime_cfg.get("dashboard_port", 8080)),
110                 }
111             ],
112         ),
113     ]

```

```

114     package="mpc_solarcar",
115     executable="solar_logger_node",
116     name="solar_logger_node",
117     parameters=[
118         {
119             "log_dir": cfg_path(profile_path, str(logging_cfg.get("log_dir", "outputs/logs"))),
120             "file_prefix": str(live_logging_cfg.get("file_prefix", "solar_live")),
121             "log_rate_hz": float(logging_cfg.get("log_rate_hz", 2.0)),
122             "output_speed_topic": str(command_cfg.get("output_speed_topic", "/vehicle/speed_cmd_kmh")),
123             "output_drive_mode_topic": str(command_cfg.get("output_drive_mode_topic", "/vehicle/drive_mode_cmd")),
124         }
125     ],
126 ),
127 ]
128
129 if bool(command_cfg.get("enabled", True)):
130     nodes.append(
131         Node(
132             package="mpc_solarcar",
133             executable="speed_command_bridge_node",
134             name="speed_command_bridge_node",
135             parameters=[_drop_keys(command_cfg, "enabled")],
136         )
137     )
138
139 if bool(live_cfg.get("use_distance_node", True)):
140     nodes.append(Node(package="mpc_solarcar", executable="distance_node", name="distance_node", parameters=[distance_cfg]))
141 if bool(live_cfg.get("use_grade_node", True)):
142     nodes.append(Node(package="mpc_solarcar", executable="grade_node", name="grade_node", parameters=[grade_cfg]))
143 if bool(weather_cfg.get("enabled", True)):
144     nodes.append(
145         Node(
146             package="mpc_solarcar",
147             executable="weather_fetch_node",
148             name="weather_fetch_node",
149             parameters=[
150                 {
151                     "profile_yaml": str(profile_path),
152                     "output_csv": raw_forecast_csv,
153                     "raw_forecast_csv": "",
154                     "gps_topic": str(weather_cfg.get("gps_topic", "/chase/gps")),
155                     "fetch_period_sec": float(weather_cfg.get("fetch_period_sec", 3600.0)),
156                     "forecast_days": int(weather_cfg.get("forecast_days", 3)),
157                     "step_minutes": int(weather_cfg.get("step_minutes", 10)),
158                     "timezone_name": str(weather_cfg.get("timezone_name", live_cfg.get("forecast_time_tz", "Australia/Darwin"))),
159                     "fallback_latitude": float(weather_cfg.get("fallback_latitude", -12.4634)),
160                     "fallback_longitude": float(weather_cfg.get("fallback_longitude", 130.8456)),
161                     "tcell_gain": float(weather_cfg.get("tcell_gain", 0.03)),
162                 }
163             ],
164         )
165     )
166 if bool(autocal_cfg.get("enabled", True)):
167     nodes.append(
168         Node(
169             package="mpc_solarcar",
170             executable="solar_autocal_node",
171             name="solar_autocal_node",
172             parameters=[_drop_keys(autocal_cfg, "enabled")],
173         )
174     )
175 return nodes

```

A.6 mpc_solarcar/solar_profile.py

```

1 from __future__ import annotations
2
3 import copy
4 import csv
5 from pathlib import Path
6 from typing import Any
7
8 import yaml
9
10
11 ROOT = Path(__file__).resolve().parents[1]
12
13
14 def require_csv_data_rows(
15     path: str | Path,
16     *,
17     label: str,
18     required_columns: tuple[str, ...] = (),
19 ) -> Path:
20     resolved = Path(path).expanduser().resolve()
21     if not resolved.is_file():
22         raise FileNotFoundError(f"{label} CSV was not found: {resolved}")
23     with resolved.open("r", encoding="utf-8-sig", newline="") as stream:
24         reader = csv.DictReader(stream)
25         header = tuple(reader.fieldnames or ())
26         missing = [column for column in required_columns if column not in header]
27         if missing:
28             raise ValueError(f"{label} CSV is missing columns {missing}: {resolved}")
29         if next(reader, None) is None:
30             raise ValueError(
31                 f"{label} CSV has a header but no data rows: {resolved}. "
32                 "Fill the template or select an identified vehicle profile before launch."
33             )

```

```

34     return resolved
35
36
37 def _resolve_profile_path(path_like: str | Path) -> Path:
38     raw = Path(str(path_like or "").strip()).expanduser()
39     if raw.is_absolute():
40         return raw.resolve()
41     candidates = [
42         (Path.cwd() / raw).resolve(),
43         (ROOT / raw).resolve(),
44     ]
45     for candidate in candidates:
46         if candidate.exists():
47             return candidate
48     return candidates[-1]
49
50
51 def resolve_relative_path(base_dir: str | Path, path_like: str | Path) -> str:
52     raw = str(path_like or "").strip()
53     if not raw:
54         return ""
55     path = Path(raw).expanduser()
56     if path.is_absolute():
57         return str(path.resolve())
58     base = Path(base_dir).resolve()
59     candidate = (base / path).resolve()
60     if candidate.exists():
61         return str(candidate)
62     repo_candidate = (ROOT / path).resolve()
63     if repo_candidate.exists():
64         return str(repo_candidate)
65     return str(candidate)
66
67
68 def load_profile(path_like: str | Path) -> tuple[Path, dict]:
69     profile_path = _resolve_profile_path(path_like)
70     with profile_path.open("r", encoding="utf-8") as f:
71         cfg = yaml.safe_load(f) or {}
72     if not isinstance(cfg, dict):
73         raise ValueError(f"profile must be a mapping: {profile_path}")
74     return profile_path, cfg
75
76
77 def merged_dict(*payloads: dict | None) -> dict:
78     def _merge(dst: dict, src: dict) -> dict:
79         for key, value in (src or {}).items():
80             if isinstance(value, dict) and isinstance(dst.get(key), dict):
81                 dst[key] = _merge(copy.deepcopy(dst[key]), value)
82             else:
83                 dst[key] = copy.deepcopy(value)
84     return dst
85
86     out: dict[str, Any] = {}
87     for payload in payloads:
88         if isinstance(payload, dict):
89             out = _merge(out, payload)
90     return out
91
92
93 def get_section(cfg: dict, key: str, default: Any = None) -> Any:
94     current: Any = cfg
95     for part in str(key or "").split("."):
96         if not part:
97             continue
98         if not isinstance(current, dict):
99             return copy.deepcopy(default) if default is not None else {}
100         current = current.get(part)
101         if current is None:
102             return copy.deepcopy(default) if default is not None else {}
103     if current is None:
104         return copy.deepcopy(default) if default is not None else {}
105     return copy.deepcopy(current)
106
107
108 def get_path(cfg: dict, profile_path: str | Path, key: str, default: str = "") -> str:
109     profile_dir = Path(profile_path).resolve().parent
110     raw = ""
111     paths_cfg = cfg.get("paths", {}) if isinstance(cfg, dict) else {}
112     if isinstance(paths_cfg, dict):
113         raw = str(paths_cfg.get(key, "") or "").strip()
114     if not raw:
115         raw = str(get_section(cfg, key, default) or "").strip()
116     if not raw:
117         raw = str(default or "").strip()
118     return resolve_relative_path(profile_dir, raw)

```

A.7 mpc_solarcar/path_utils.py

```

1 import os
2 from pathlib import Path
3
4 try:
5     from ament_index_python.packages import get_package_share_directory # type: ignore
6 except Exception: # pragma: no cover - non-ROS fallback
7     get_package_share_directory = None
8
9
10 PKG_NAME = 'mpc_solarcar'

```

```

11 REPO_ROOT = Path(__file__).resolve().parents[1]
12
13
14 def resolve_path(path: str, default_subdir: str = '') -> str:
15     """Resolve a path relative to CWD or package share.
16
17     - If absolute, return as-is.
18     - If exists relative to CWD, return it.
19     - Otherwise, resolve under <pkg_share>/<default_subdir> (or directly under share).
20     """
21     if path is None:
22         return path
23     path = os.path.expanduser(str(path))
24     if os.path.isabs(path):
25         return path
26     if os.path.exists(path):
27         return path
28     if get_package_share_directory is not None:
29         pkg_share = get_package_share_directory(PKG_NAME)
30     else:
31         pkg_share = os.fspath(REPO_ROOT)
32     if default_subdir:
33         subdir = default_subdir.strip('/')
34         if path.startswith(subdir + os.sep) or path == subdir:
35             return os.path.join(pkg_share, path)
36         return os.path.join(pkg_share, subdir, path)
37     return os.path.join(pkg_share, path)

```

A.8 mpc_solarcar/mpc_node.py

```

1 # -*- coding: utf-8 -*-
2 import math
3 import os
4 import time
5 from collections import deque
6 from datetime import datetime, timezone, timedelta
7
8 import rclpy
9 from rclpy.callback_groups import MutuallyExclusiveCallbackGroup
10 from rclpy.executors import MultiThreadedExecutor
11 from rclpy.node import Node
12 from rclpy.parameter import Parameter
13
14 import yaml
15 import pandas as pd
16 import numpy as np
17
18 from std_msgs.msg import Bool, Float32, Float32MultiArray, String
19 from nav_msgs.msg import Path
20 from geometry_msgs.msg import PoseStamped
21
22 from scipy.optimize import minimize
23
24 from .model import SolarCarModel, Params
25 from .path_utils import resolve_path
26 from .route_utils import average_profile, interpolate_profile
27 from .schedule_utils import DriveSchedule
28 from .estimator import BatteryMHE, MheInput, MheMeas
29 from .forecast_grid import build_forecast_grid_payload, interp_forecast_grid
30 from .signal_utils import RobustScalarFilter, finite_float, fresh_enough, slew_limit
31 from .upper_cost import load_upper_cost_config, upper_stage_cost, upper_terminal_cost, quad_penalty
32 from .upper_horizon import build_upper_distance_horizon, plan_segment_index
33 from .upper_policy import interpolate_upper_policy, load_upper_policy_csv
34 from .upper_solver import hybrid_bounded_minimize
35
36
37 class MPCNode(Node):
38     """Forecast-driven solarcar MPC node."""
39
40     def __init__(self):
41         super().__init__('mpc_node')
42         self.params_cfg = {}
43         # A full-race upper solve can take longer than the 1 Hz control period.
44         # Keep telemetry and lower control responsive while that solve runs.
45         self.telemetry_callback_group = MutuallyExclusiveCallbackGroup()
46         self.upper_callback_group = MutuallyExclusiveCallbackGroup()
47         self.lower_callback_group = MutuallyExclusiveCallbackGroup()
48         self.command_callback_group = MutuallyExclusiveCallbackGroup()
49         self._init_solar()
50
51     # ----- common helpers -----
52     def _load_stops(self, stop_yaml: str):
53         self.stops = []
54         try:
55             with open(stop_yaml, 'r', encoding='utf-8') as f:
56                 y = yaml.safe_load(f) or {}
57                 self.stops = y.get('stops', [])
58                 self.get_logger().info(f'Loaded {len(self.stops)} stop points from {stop_yaml}')
59         except Exception:
60             self.get_logger().info('No stop_points.yaml provided. Running without dwell constraints.')
61
62     def _load_forecast_file(self, path: str):
63         self.df = pd.read_csv(path)
64         if 'time' in self.df.columns:
65             t = pd.to_datetime(self.df['time'], format='mixed', errors='coerce')
66             tzname = str(getattr(self, 'forecast_time_tz', 'UTC') or 'UTC')
67             if t.dt.tz is None:
68                 if tzname.upper() == 'UTC':

```

```

69         t = t.dt.tz_localize('UTC')
70     else:
71         try:
72             t = t.dt.tz_localize(tzname, ambiguous='NaT', nonexistent='NaT').dt.tz_convert('UTC')
73         except Exception:
74             t = t.dt.tz_localize('UTC')
75     else:
76         t = t.dt.tz_convert('UTC')
77     self.df['time'] = t
78     if self.df['time'].isna().all():
79         self.get_logger().warn("forecast 'time' column could not be parsed; falling back to index bins.")
80 else:
81     self.get_logger().warn("forecast CSV has no 'time' column; falling back to index bins.")
82 self.forecast_grid = build_forecast_grid_payload(self.df)
83 if 'time' in self.df.columns and not self.df['time'].isna().all():
84     self.forecast_times = pd.Index(self.df['time'].dropna().drop_duplicates().sort_values())
85 else:
86     self.forecast_times = pd.Index([])
87
88 def _apply_params_yaml(self, path: str):
89     try:
90         with open(path, 'r', encoding='utf-8') as f:
91             cfg = yaml.safe_load(f) or {}
92     except Exception as exc:
93         self.get_logger().warn(f'params_yaml load failed: {exc}')
94     return
95 self.params_cfg = cfg if isinstance(cfg, dict) else {}
96 model_cfg = cfg.get('model', {}) if isinstance(cfg, dict) else {}
97 if isinstance(model_cfg, dict):
98     motor_type = None
99     for k, v in model_cfg.items():
100         if hasattr(self.model.p, k):
101             try:
102                 setattr(self.model.p, k, float(v))
103             except Exception:
104                 try:
105                     setattr(self.model.p, k, v)
106                 except Exception:
107                     pass
108         if k == 'motor_type':
109             motor_type = str(v)
110         if k == 'drive_mode':
111             self.model.drive_mode = str(v)
112         if k == 'drive_mode_tau_margin':
113             try:
114                 self.model.drive_mode_tau_margin = float(v)
115             except Exception:
116                 pass
117         if k == 'ocv_soc_map':
118             try:
119                 v_str = str(v).strip()
120                 if v_str:
121                     self.model.load_ocv_map(resolve_path(v_str, 'maps'))
122             except Exception:
123                 pass
124         if motor_type:
125             mt = motor_type.lower()
126             if mt in ('inwheel', 'hub'):
127                 if not ('gear_ratio' in model_cfg):
128                     self.model.p.gear_ratio = 1.0
129                 if not ('gear_eta' in model_cfg):
130                     self.model.p.gear_eta = 1.0
131 mpc_cfg = dict(cfg.get('mpc', {}) or {})
132 runtime_mode = str(self.get_parameter('profile_runtime_mode').value or '').strip()
133 runtime_section = cfg.get(runtime_mode, {}) if runtime_mode else {}
134 runtime_overrides = (
135     runtime_section.get('mpc_overrides', {})
136     if isinstance(runtime_section, dict)
137     else {}
138 )
139 if isinstance(runtime_overrides, dict) and runtime_overrides:
140     mpc_cfg.update(runtime_overrides)
141     self.get_logger().info(
142         f'Applied {runtime_mode}.mpc_overrides from profile ({len(runtime_overrides)} values).'
143     )
144 if isinstance(mpc_cfg, dict):
145     params = []
146     for key, val in mpc_cfg.items():
147         # Nested blocks such as upper_cost are consumed directly from
148         # params_cfg; ROS parameters only support scalar/array values.
149         if isinstance(val, dict) or val is None or not self.has_parameter(key):
150             continue
151         try:
152             params.append(Parameter(key, value=val))
153         except (TypeError, ValueError):
154             self.get_logger().warn(f'Ignoring unsupported YAML parameter: mpc.{key}')
155     if params:
156         self.set_parameters(params)
157
158 def _maybe_reload_forecast(self):
159     if self.forecast_reload_sec <= 0:
160         return
161     now = time.monotonic()
162     if (now - self.last_forecast_check) < self.forecast_reload_sec:
163         return
164     self.last_forecast_check = now
165     try:
166         mtime = os.path.getmtime(self.forecast_path)
167     except Exception:
168         return
169     if self.forecast_mtime is None or mtime > self.forecast_mtime:
170         self.forecast_mtime = mtime
171     try:

```

```

172         self._load_forecast_file(self.forecast_path)
173         self.forecast_reloaded = True
174         self.get_logger().info('Forecast CSV reloaded.')
175     except Exception as exc:
176         self.get_logger().warn(f'Forecast reload failed: {exc}')
177
178 def _on_s_km_solar(self, msg: Float32):
179     try:
180         now_sec = self.get_clock().now().nanoseconds / 1e9
181         raw = finite_float(msg.data)
182         if not math.isfinite(raw):
183             return
184         current = float(self.distance_meas_filter.value)
185         if math.isfinite(current) and raw < current - self.distance_meas_max_backtrack_km:
186             return
187         self.s_meas = float(self.distance_meas_filter.update(raw, now=now_sec))
188         self.s_meas_time = now_sec
189         if bool(self.get_parameter('use_measured_s').value):
190             self.s_km = float(self.s_meas)
191     except Exception:
192         pass
193
194 def _on_speed_solar(self, msg: Float32):
195     try:
196         now_sec = self.get_clock().now().nanoseconds / 1e9
197         self.v_now = float(self.speed_meas_filter.update(msg.data, now=now_sec))
198         self.v_meas_time = now_sec
199     except Exception:
200         pass
201
202 def _on_soc_solar(self, msg: Float32):
203     try:
204         now_sec = self.get_clock().now().nanoseconds / 1e9
205         self.solar_soc_meas = float(self.soc_meas_filter.update(msg.data, now=now_sec))
206         self.soc_meas_time = now_sec
207         self.z = float(np.clip(self.solar_soc_meas, self.model.p.soc_min, self.model.p.soc_max))
208     except Exception:
209         pass
210
211 def _on_tb_solar(self, msg: Float32):
212     try:
213         now_sec = self.get_clock().now().nanoseconds / 1e9
214         self.solar_tb_meas = float(self.tb_meas_filter.update(msg.data, now=now_sec))
215         self.tb_meas_time = now_sec
216         self.Tb = float(np.clip(self.solar_tb_meas, self.model.p.T_min, self.model.p.T_max))
217     except Exception:
218         pass
219
220 def _on_i_solar(self, msg: Float32):
221     try:
222         now_sec = self.get_clock().now().nanoseconds / 1e9
223         self.solar_i_meas = float(self.current_meas_filter.update(msg.data, now=now_sec))
224         self.current_meas_time = now_sec
225     except Exception:
226         pass
227
228 def _on_v_solar(self, msg: Float32):
229     try:
230         now_sec = self.get_clock().now().nanoseconds / 1e9
231         self.solar_v_meas = float(self.voltage_meas_filter.update(msg.data, now=now_sec))
232         self.voltage_meas_time = now_sec
233     except Exception:
234         pass
235
236 def _measured_distance_km(self) -> float:
237     now_sec = self.get_clock().now().nanoseconds / 1e9
238     if fresh_enough(self.s_meas_time, now_sec, self.distance_meas_timeout_sec):
239         return finite_float(self.s_meas)
240     return math.nan
241
242 def _sync_measured_state(self) -> None:
243     now_sec = self.get_clock().now().nanoseconds / 1e9
244     if not fresh_enough(self.v_meas_time, now_sec, self.speed_meas_timeout_sec):
245         self.v_now = math.nan
246     distance_km = self._measured_distance_km()
247     if bool(self.get_parameter('use_measured_s').value) and math.isfinite(distance_km):
248         self.s_km = float(distance_km)
249     if fresh_enough(self.soc_meas_time, now_sec, self.battery_meas_timeout_sec):
250         if math.isfinite(self.solar_soc_meas):
251             self.z = float(np.clip(self.solar_soc_meas, self.model.p.soc_min, self.model.p.soc_max))
252     if fresh_enough(self.tb_meas_time, now_sec, self.battery_meas_timeout_sec):
253         if math.isfinite(self.solar_tb_meas):
254             self.Tb = float(np.clip(self.solar_tb_meas, self.model.p.T_min, self.model.p.T_max))
255
256 def _on_calib_solar_gain(self, msg: Float32):
257     try:
258         self.solar_gain = float(np.clip(float(msg.data), 0.2, 2.5))
259     except Exception:
260         return
261
262 def _on_calib_drive_power_gain(self, msg: Float32):
263     try:
264         gain = float(np.clip(float(msg.data), 0.4, 2.5))
265         self.model.p.drive_eff_scale = float(self.base_drive_eff_scale) / max(gain, 1.0e-3)
266         self.model.p.regen_eff_scale = float(self.base_regen_eff_scale) / max(gain, 1.0e-3)
267     except Exception:
268         return
269
270 def _on_calib_aux_power(self, msg: Float32):
271     try:
272         self.model.aux_power_override_w = max(0.0, float(msg.data))
273     except Exception:
274         return

```

```

275
276 # ----- solarcar mode -----
277 def _init_solar(self):
278     self.declare_parameter('forecast_csv', 'inputs/forecast_10min.csv')
279     self.declare_parameter('maps_dir', 'maps')
280     self.declare_parameter('drive_eff_map', '')
281     self.declare_parameter('regen_eff_map', '')
282     self.declare_parameter('rint_map', '')
283     self.declare_parameter('drive_map_eco', '')
284     self.declare_parameter('drive_map_power', '')
285     self.declare_parameter('regen_map_eco', '')
286     self.declare_parameter('regen_map_power', '')
287     self.declare_parameter('panel_eff_map', '')
288     self.declare_parameter('mppt_eff_map', '')
289     self.declare_parameter('ocv_soc_map', '')
290     self.declare_parameter('dt', 600.0) # 10 min [s]
291     self.declare_parameter('horizon_steps', 9)
292     self.declare_parameter('v_max_kmh', 110.0)
293     self.declare_parameter('terminal_soc_min', 0.10)
294     self.declare_parameter('stop_yaml', 'inputs/stop_points.yaml')
295     self.declare_parameter('forecast_time_mode', 'auto') # auto|absolute|relative|loop
296     self.declare_parameter('forecast_time_tz', 'UTC')
297     self.declare_parameter('forecast_start_time_utc', '')
298     self.declare_parameter('forecast_time_offset_sec', 0.0)
299     self.declare_parameter('forecast_reload_sec', 60.0)
300     self.declare_parameter('replan_on_forecast_reload', False)
301     self.declare_parameter('params_yaml', '')
302     self.declare_parameter('profile_runtime_mode', '')
303     self.declare_parameter('route_profile_csv', '')
304     self.declare_parameter('speed_profile_csv', '')
305     self.declare_parameter('drive_schedule_yaml', '')
306     self.declare_parameter('initial_upper_policy_csv', '')
307     self.declare_parameter('use_measured_s', True)
308     self.declare_parameter('use_measured_speed', True)
309     self.declare_parameter('soc0', 0.95)
310     self.declare_parameter('Tb0', 30.0)
311     self.declare_parameter('s0_kmh', 0.0)
312     self.declare_parameter('speed_meas_timeout_sec', 3.0)
313     self.declare_parameter('distance_meas_timeout_sec', 5.0)
314     self.declare_parameter('battery_meas_timeout_sec', 15.0)
315     self.declare_parameter('speed_meas_filter_tau_sec', 0.6)
316     self.declare_parameter('speed_meas_max_accel_kmhps', 12.0)
317     self.declare_parameter('speed_meas_max_decel_kmhps', 20.0)
318     self.declare_parameter('distance_meas_max_rate_kmhps', 0.06)
319     self.declare_parameter('distance_meas_max_backtrack_kmh', 0.02)
320     self.declare_parameter('battery_meas_filter_tau_sec', 1.0)
321     self.declare_parameter('w_dv', 0.05)
322     self.declare_parameter('w_dv_limit', 2.0)
323     self.declare_parameter('dv_max_kmhps', 5.0)
324     self.declare_parameter('w_T', 5.0)
325     self.declare_parameter('w_speed_limit', 50.0)
326     self.declare_parameter('w_drive_window', 1.0e5)
327     self.declare_parameter('w_current', 0.02)
328     self.declare_parameter('soc_target', 0.5)
329     self.declare_parameter('soc_band', 0.1)
330     self.declare_parameter('w_soc_target', 2.0)
331     self.declare_parameter('w_soc_band', 50.0)
332     self.declare_parameter('soc_day_end_max', -1.0)
333     self.declare_parameter('soc_day_end_target', -1.0)
334     self.declare_parameter('w_soc_day_max', 1.0e4)
335     self.declare_parameter('w_soc_day_track', 0.0)
336     self.declare_parameter('soc_day_end_tol', 0.02)
337     self.declare_parameter('soc_finish_target', -1.0)
338     self.declare_parameter('soc_finish_tol', 0.02)
339     self.declare_parameter('w_soc_progress', 1.0e5)
340     self.declare_parameter('w_soc_terminal', 1.0e5)
341     self.declare_parameter('race_kmh', 3035.5)
342     self.declare_parameter('upper_mode', 'time') # time|distance
343     self.declare_parameter('upper_ds_kmh', 20.0)
344     self.declare_parameter('upper_horizon_kmh', 1500.0)
345     self.declare_parameter('upper_max_steps', 200)
346     self.declare_parameter('upper_horizon_mode', 'fixed')
347     self.declare_parameter('upper_adaptive_min_ds_kmh', 10.0)
348     self.declare_parameter('upper_adaptive_max_ds_kmh', 200.0)
349     self.declare_parameter('upper_adaptive_growth', 1.18)
350     self.declare_parameter('upper_ctrl_kmh', 0.0)
351     self.declare_parameter('upper_vmin_kmh', 1.0)
352     self.declare_parameter('upper_replan_kmh', 0.0)
353     self.declare_parameter('upper_replan_sec', 0.0)
354     self.declare_parameter('upper_max_iter', 120)
355     self.declare_parameter('upper_global_search_enabled', True)
356     self.declare_parameter('upper_cem_generations', 'auto')
357     self.declare_parameter('upper_cem_generations', 4)
358     self.declare_parameter('upper_cem_population', 16)
359     self.declare_parameter('upper_cem_elite', 4)
360     self.declare_parameter('upper_local_refine_topk', 4)
361     self.declare_parameter('upper_seed_library_mode', 'full')
362     self.declare_parameter('upper_shgo_samples', 256)
363     self.declare_parameter('upper_shgo_iters', 2)
364     self.declare_parameter('upper_cert_grid_levels', 0)
365     self.declare_parameter('upper_cert_max_evaluations', 250000)
366     self.declare_parameter('upper_cert_workers', 1)
367     self.declare_parameter('upper_day_end_soc_min', 0.2)
368     self.declare_parameter('soc_guard_margin', 0.01)
369     self.declare_parameter('soc_guard_mode', 'stop') # stop|pv_only
370     self.declare_parameter('drive_mode', 'auto') # auto|eco|power
371     self.declare_parameter('drive_mode_tau_margin', 0.0)
372     self.declare_parameter('hierarchical', True)
373     self.declare_parameter('lower_dt', 1.0)
374     self.declare_parameter('lower_horizon_steps', 20)
375     self.declare_parameter('lower_rate_hz', 5.0)
376     # The bounded inverse-dynamics controller is the deterministic 1 Hz
377     # production path. Set this above zero only for optional offline

```

```

378 # L-BFGS refinement; it must never be required to publish a command.
379 self.declare_parameter('lower_max_iter', 0)
380 self.declare_parameter('lower_maxfun', 80)
381 self.declare_parameter('lower_skip_optimization_during_upper_solve', True)
382 self.declare_parameter('w_track', 5.0)
383 self.declare_parameter('w_throttle', 0.2)
384 self.declare_parameter('throttle_rate_limit', 30.0)
385 self.declare_parameter('mhe_enabled', True)
386 self.declare_parameter('mhe_horizon_steps', 12)
387 self.declare_parameter('mhe_w_soc', 50.0)
388 self.declare_parameter('mhe_w_tb', 5.0)
389 self.declare_parameter('mhe_w_i', 1.0)
390 self.declare_parameter('mhe_w_v', 1.0)
391 self.declare_parameter('mhe_w_prior', 5.0)
392 self.declare_parameter('soc_min', 0.05)
393 self.declare_parameter('soc_max', 0.98)
394 self.declare_parameter('T_min', -5.0)
395 self.declare_parameter('T_max', 55.0)
396 self.declare_parameter('solar_gain', 1.0)
397 self.declare_parameter('poa_gain_drive', 1.0)
398 self.declare_parameter('poa_gain_stop', 1.0)
399 self.declare_parameter('stop_tilt_fraction', 1.0)
400 self.declare_parameter('control_stop_tilt_fraction', 0.0)
401 self.declare_parameter('control_stop_arrival_tolerance_km', 0.20)
402 self.declare_parameter('control_stop_stationary_speed_kmh', 2.0)
403 self.declare_parameter('control_stop_brake_decel_kmhps', 3.5)
404 self.declare_parameter('control_stop_brake_margin_km', 0.03)
405
406 fcsv = resolve_path(self.get_parameter('forecast_csv').value, 'inputs')
407 maps_dir = resolve_path(self.get_parameter('maps_dir').value, 'maps')
408 self.dt = float(self.get_parameter('dt').value)
409 self.Np = int(self.get_parameter('horizon_steps').value)
410
411 # Load forecast
412 self.forecast_path = fcsv
413 self.forecast_reload_sec = float(self.get_parameter('forecast_reload_sec').value)
414 self.replan_on_forecast_reload = bool(self.get_parameter('replan_on_forecast_reload').value)
415 self._load_forecast_file(self.forecast_path)
416 try:
417     self.forecast_mtime = os.path.getmtime(self.forecast_path)
418 except Exception:
419     self.forecast_mtime = None
420 self.last_forecast_check = time.monotonic()
421 self._forecast_warned_out_of_range = False
422 self.forecast_reloaded = False
423
424 # Load stop points (optional)
425 stop_yaml = resolve_path(self.get_parameter('stop_yaml').value, 'inputs')
426 self._load_stops(stop_yaml)
427
428 # Optional route/speed profile
429 self.route_profile = None
430 route_profile_csv = self.get_parameter('route_profile_csv').value
431 if route_profile_csv:
432     try:
433         route_profile_csv = resolve_path(route_profile_csv, 'inputs')
434         self.route_profile = pd.read_csv(route_profile_csv)
435     except Exception as exc:
436         self.get_logger().warn(f'Failed to load route_profile_csv: {exc}')
437
438 self.speed_profile = None
439 speed_profile_csv = self.get_parameter('speed_profile_csv').value
440 if speed_profile_csv:
441     try:
442         speed_profile_csv = resolve_path(speed_profile_csv, 'inputs')
443         self.speed_profile = pd.read_csv(speed_profile_csv)
444     except Exception as exc:
445         self.get_logger().warn(f'Failed to load speed_profile_csv: {exc}')
446
447 # A long offline CUDA search supplies only a warm start. Live MPC keeps
448 # authority and re-optimizes it against current SoC, weather, and route state.
449 self.initial_upper_policy = None
450 initial_policy_csv = str(self.get_parameter('initial_upper_policy_csv').value or '').strip()
451 if initial_policy_csv:
452     try:
453         initial_policy_csv = resolve_path(initial_policy_csv, 'inputs')
454         self.initial_upper_policy = load_upper_policy_csv(initial_policy_csv)
455         self.get_logger().info(
456             f'Loaded learned upper-policy warm start ({len(self.initial_upper_policy)} points): '
457             f'{initial_policy_csv}'
458         )
459     except Exception as exc:
460         self.get_logger().warn(f'Failed to load initial_upper_policy_csv: {exc}')
461
462 # Optional driving schedule
463 self.drive_schedule = None
464 drive_schedule_yaml = self.get_parameter('drive_schedule_yaml').value
465 if drive_schedule_yaml:
466     drive_schedule_yaml = resolve_path(drive_schedule_yaml, 'inputs')
467     self.drive_schedule = DriveSchedule.from_yaml(drive_schedule_yaml)
468     if self.drive_schedule is None:
469         self.get_logger().warn('drive_schedule_yaml could not be loaded.')
470
471 # Model
472 panel_map = self.get_parameter('panel_eff_map').value
473 mppt_map = self.get_parameter('mppt_eff_map').value
474 ocv_map = self.get_parameter('ocv_soc_map').value
475 drive_map_eco = self.get_parameter('drive_map_eco').value
476 drive_map_power = self.get_parameter('drive_map_power').value
477 regen_map_eco = self.get_parameter('regen_map_eco').value
478 regen_map_power = self.get_parameter('regen_map_power').value
479 panel_map = resolve_path(panel_map, 'maps') if panel_map else None
480 mppt_map = resolve_path(mppt_map, 'maps') if mppt_map else None

```

```

481 ocv_map = resolve_path(ocv_map, 'maps') if ocv_map else None
482 drive_map_eco = resolve_path(drive_map_eco, 'maps') if drive_map_eco else None
483 drive_map_power = resolve_path(drive_map_power, 'maps') if drive_map_power else None
484 regen_map_eco = resolve_path(regen_map_eco, 'maps') if regen_map_eco else None
485 regen_map_power = resolve_path(regen_map_power, 'maps') if regen_map_power else None
486 drive_map = self.get_parameter('drive_eff_map').value
487 regen_map = self.get_parameter('regen_eff_map').value
488 rint_map = self.get_parameter('rint_map').value
489 drive_map = resolve_path(drive_map, 'maps') if drive_map else f'{maps_dir}/drive_eff_map.csv'
490 regen_map = resolve_path(regen_map, 'maps') if regen_map else f'{maps_dir}/regen_eff_map.csv'
491 rint_map = resolve_path(rint_map, 'maps') if rint_map else f'{maps_dir}/Rint_T_by_soc.csv'
492 self.model = SolarCarModel(
493     drive_map,
494     regen_map,
495     rint_map,
496     params=Params(dt=self.dt),
497     panel_eff_map_path=panel_map,
498     mppt_eff_map_path=mppt_map,
499     drive_map_eco_path=drive_map_eco,
500     drive_map_power_path=drive_map_power,
501     regen_map_eco_path=regen_map_eco,
502     regen_map_power_path=regen_map_power,
503     ocv_soc_map_path=ocv_map,
504 )
505 params_yaml = self.get_parameter('params_yaml').value
506 if params_yaml:
507     params_yaml = resolve_path(params_yaml, 'inputs')
508     self.apply_params_yaml(params_yaml)
509 self.model.p.soc_min = float(self.get_parameter('soc_min').value)
510 self.model.p.soc_max = float(self.get_parameter('soc_max').value)
511 self.model.p.T_min = float(self.get_parameter('T_min').value)
512 self.model.p.T_max = float(self.get_parameter('T_max').value)
513 self.soc_target = float(self.get_parameter('soc_target').value)
514 self.soc_band = float(self.get_parameter('soc_band').value)
515 self.w_soc_target = float(self.get_parameter('w_soc_target').value)
516 self.w_soc_band = float(self.get_parameter('w_soc_band').value)
517 self.soc_day_end_max = float(self.get_parameter('soc_day_end_max').value)
518 self.soc_day_end_target = float(self.get_parameter('soc_day_end_target').value)
519 self.w_soc_day_max = float(self.get_parameter('w_soc_day_max').value)
520 self.soc_finish_target = float(self.get_parameter('soc_finish_target').value)
521 self.soc_finish_tol = float(self.get_parameter('soc_finish_tol').value)
522 self.w_soc_progress = float(self.get_parameter('w_soc_progress').value)
523 self.w_soc_terminal = float(self.get_parameter('w_soc_terminal').value)
524 self.race_km = float(self.get_parameter('race_km').value)
525 self.soc_target = float(np.clip(self.soc_target, self.model.p.soc_min, self.model.p.soc_max))
526 self.soc_band = max(0.0, float(self.soc_band))
527 if self.soc_day_end_max > 0.0:
528     self.soc_day_end_max = float(np.clip(self.soc_day_end_max, self.model.p.soc_min, self.model.p.soc_max))
529 if self.soc_day_end_target > 0.0:
530     self.soc_day_end_target = float(np.clip(
531         self.soc_day_end_target, self.model.p.soc_min, self.model.p.soc_max
532     ))
533 if self.soc_finish_target > 0.0:
534     self.soc_finish_target = float(np.clip(self.soc_finish_target, self.model.p.soc_min, self.model.p.soc_max))
535 self.soc_finish_tol = max(0.0, float(self.soc_finish_tol))
536 self.race_km = max(1.0, float(self.race_km))
537 self.model.drive_mode = str(self.get_parameter('drive_mode').value)
538 self.model.drive_mode_tau_margin = float(self.get_parameter('drive_mode_tau_margin').value)
539 self.base_drive_eff_scale = float(getattr(self.model.p, 'drive_eff_scale', 1.0))
540 self.base_regen_eff_scale = float(getattr(self.model.p, 'regen_eff_scale', self.base_drive_eff_scale))
541 self.solar_gain = float(self.get_parameter('solar_gain').value)
542 self.poa_gain_drive = float(self.get_parameter('poa_gain_drive').value)
543 self.poa_gain_stop = float(self.get_parameter('poa_gain_stop').value)
544 self.stop_tilt_fraction = float(np.clip(
545     self.get_parameter('stop_tilt_fraction').value, 0.0, 1.0
546 ))
547 self.control_stop_tilt_fraction = float(np.clip(
548     self.get_parameter('control_stop_tilt_fraction').value, 0.0, 1.0
549 ))
550 self.upper_mode = str(self.get_parameter('upper_mode').value).lower()
551 self.upper_ds_km = float(self.get_parameter('upper_ds_km').value)
552 self.upper_horizon_km = float(self.get_parameter('upper_horizon_km').value)
553 self.upper_max_steps = int(self.get_parameter('upper_max_steps').value)
554 self.upper_horizon_mode = str(self.get_parameter('upper_horizon_mode').value).lower()
555 self.upper_adaptive_min_ds_km = float(self.get_parameter('upper_adaptive_min_ds_km').value)
556 self.upper_adaptive_max_ds_km = float(self.get_parameter('upper_adaptive_max_ds_km').value)
557 self.upper_adaptive_growth = float(self.get_parameter('upper_adaptive_growth').value)
558 self.upper_ctrl_km = float(self.get_parameter('upper_ctrl_km').value)
559 self.upper_vmin_kmh = float(self.get_parameter('upper_vmin_kmh').value)
560 self.upper_replan_km = float(self.get_parameter('upper_replan_km').value)
561 self.upper_replan_sec = float(self.get_parameter('upper_replan_sec').value)
562 self.upper_max_iter = int(self.get_parameter('upper_max_iter').value)
563 self.upper_day_end_soc_min = float(self.get_parameter('upper_day_end_soc_min').value)
564 self.upper_cost_cfg = load_upper_cost_config(
565     self.params_cfg.get('mpc', {}) if isinstance(self.params_cfg, dict) else {},
566     legacy={
567         'w_dv': float(self.get_parameter('w_dv').value),
568         'w_dv_limit': float(self.get_parameter('w_dv_limit').value),
569         'w_T': float(self.get_parameter('w_T').value),
570         'w_speed_limit': float(self.get_parameter('w_speed_limit').value),
571         'w_drive_window': float(self.get_parameter('w_drive_window').value),
572         'w_current': float(self.get_parameter('w_current').value),
573         'w_soc_day_max': float(self.w_soc_day_max),
574         'w_soc_day_track': float(self.get_parameter('w_soc_day_track').value),
575         'w_soc_terminal': float(self.w_soc_terminal),
576     },
577 )
578
579 # States
580 self.k = 0
581 self.last_bin = None
582 self.s_km = max(0.0, float(self.get_parameter('s0_km').value))
583 self.z = float(np.clip(

```

```

584         float(self.get_parameter('soc0').value), self.model.p.soc_min, self.model.p.soc_max
585     ))
586     self.Tb = float(np.clip(
587         float(self.get_parameter('Tb0').value), self.model.p.T_min, self.model.p.T_max
588     ))
589     self.v_cmd = 40.0
590     self.v_now = math.nan
591     self.model_prev_speed_ms = None
592     self.s_meas = math.nan
593     self.s_meas_time = None
594     self.v_meas_time = None
595     self.solar_soc_meas = math.nan
596     self.soc_meas_time = None
597     self.solar_tb_meas = math.nan
598     self.tb_meas_time = None
599     self.solar_i_meas = math.nan
600     self.current_meas_time = None
601     self.solar_v_meas = math.nan
602     self.voltage_meas_time = None
603     self.speed_meas_timeout_sec = max(0.0, float(self.get_parameter('speed_meas_timeout_sec').value))
604     self.distance_meas_timeout_sec = max(0.0, float(self.get_parameter('distance_meas_timeout_sec').value))
605     self.battery_meas_timeout_sec = max(0.0, float(self.get_parameter('battery_meas_timeout_sec').value))
606     battery_tau = max(0.0, float(self.get_parameter('battery_meas_filter_tau_sec').value))
607     self.distance_meas_max_backtrack_km = max(
608         0.0, float(self.get_parameter('distance_meas_max_backtrack_km').value)
609     )
610     self.speed_meas_filter = RobustScalarFilter(
611         tau_sec=float(self.get_parameter('speed_meas_filter_tau_sec').value),
612         min_value=0.0,
613         max_value=float(self.get_parameter('v_max_kmh').value),
614         rise_rate=float(self.get_parameter('speed_meas_max_accel_kmhps').value),
615         fall_rate=float(self.get_parameter('speed_meas_max_decel_kmhps').value),
616     )
617     self.distance_meas_filter = RobustScalarFilter(
618         tau_sec=0.0,
619         min_value=0.0,
620         rise_rate=float(self.get_parameter('distance_meas_max_rate_kmps').value),
621     )
622     self.soc_meas_filter = RobustScalarFilter(
623         tau_sec=battery_tau, min_value=self.model.p.soc_min, max_value=self.model.p.soc_max
624     )
625     self.tb_meas_filter = RobustScalarFilter(
626         tau_sec=battery_tau, min_value=self.model.p.T_min, max_value=self.model.p.T_max
627     )
628     self.current_meas_filter = RobustScalarFilter(tau_sec=battery_tau)
629     self.voltage_meas_filter = RobustScalarFilter(tau_sec=battery_tau, min_value=0.0)
630     self.v_upper_cmd = self.v_cmd
631     self.v_lower_cmd = self.v_cmd
632     self.v_plan_kmh = None
633     self.v_plan_segments = None
634     self.upper_plan_seq = None
635     self.upper_plan_ctrl_s = None
636     self.upper_plan_s_km = None
637     self.upper_plan_time = None
638     self.upper_plan_id = 0
639     self.upper_plan_mode = 'time'
640     self.plan_dt_sec = float(self.model.p.dt)
641     self.last_data = []
642     self.last_upper_solve_ok = False
643     self.last_lower_solve_ok = False
644     self._upper_solve_in_progress = False
645     self._upper_solve_logged = False
646     self._lower_solve_logged = False
647     self._lower_upper_busy_logged = False
648     self.plan_start_monotonic = None
649     self.last_plan_time = None
650     self.lower_last_u = 0.0
651     self.lower_last_mode = 'eco'
652     self.lower_last_v_pred = [self.v_lower_cmd / 3.6] * max(1, int(self.get_parameter('lower_horizon_steps').value))
653     self.completed_control_stops = set()
654     self.active_control_stop_index = None
655     self.control_stop_end_monotonic = None
656     self.control_stop_position_initialized = False
657     self.control_stop_hold = False
658
659     # Hierarchical MPC settings
660     self.hierarchical = bool(self.get_parameter('hierarchical').value)
661     self.lower_dt = float(self.get_parameter('lower_dt').value)
662     self.lower_N = int(self.get_parameter('lower_horizon_steps').value)
663     self.lower_rate_hz = float(self.get_parameter('lower_rate_hz').value)
664     self.lower_max_iter = max(0, int(self.get_parameter('lower_max_iter').value))
665     self.lower_max_fun = max(1, int(self.get_parameter('lower_maxfun').value))
666     self.lower_skip_optimization_during_upper_solve = bool(
667         self.get_parameter('lower_skip_optimization_during_upper_solve').value
668     )
669     self.w_track = float(self.get_parameter('w_track').value)
670     self.w_throttle = float(self.get_parameter('w_throttle').value)
671     self.throttle_rate_limit = float(self.get_parameter('throttle_rate_limit').value)
672     self._warned_lower_rate = False
673
674     # Forecast clock setup
675     self.forecast_time_mode = str(self.get_parameter('forecast_time_mode').value)
676     self.forecast_time_tz = str(self.get_parameter('forecast_time_tz').value)
677     self.forecast_time_offset = float(self.get_parameter('forecast_time_offset_sec').value)
678     self.forecast_start_time = None
679     start_time_utc = str(self.get_parameter('forecast_start_time_utc').value).strip()
680     if start_time_utc:
681         try:
682             if start_time_utc.endswith('Z'):
683                 start_time_utc = start_time_utc[:-1] + '+00:00'
684             self.forecast_start_time = datetime.fromisoformat(start_time_utc).astimezone(timezone.utc)
685         except Exception:
686             self.get_logger().warn('forecast_start_time_utc parse failed; using node start time.')

```

```

687     if self.forecast_start_time is None:
688         self.forecast_start_time = datetime.now(timezone.utc)
689     self.forecast_relative_wall_start = datetime.now(timezone.utc)
690     self.current_forecast_time_utc = self.forecast_start_time
691
692     # MHE
693     self.mhe = None
694     if bool(self.get_parameter('mhe_enabled').value):
695         self.mhe = BatteryMHE(
696             self.model,
697             horizon_steps=int(self.get_parameter('mhe_horizon_steps').value),
698             w_soc=float(self.get_parameter('mhe_w_soc').value),
699             w_tb=float(self.get_parameter('mhe_w_tb').value),
700             w_i=float(self.get_parameter('mhe_w_i').value),
701             w_v=float(self.get_parameter('mhe_w_v').value),
702             w_prior=float(self.get_parameter('mhe_w_prior').value),
703             soc_bounds=(self.model.p.soc_min, self.model.p.soc_max),
704             tb_bounds=(self.model.p.T_min, self.model.p.T_max),
705         )
706
707     # Pubs
708     self.pub_speed = self.create_publisher(Float32, '/planner/speed_cmd', 10)
709     self.pub_upper_speed = self.create_publisher(Float32, '/planner/upper_speed_cmd', 10)
710     self.pub_throttle = self.create_publisher(Float32, '/planner/throttle_cmd_pct', 10)
711     self.pub_drive_mode = self.create_publisher(String, '/planner/drive_mode', 10)
712     self.pub_path = self.create_publisher(Path, '/planner/trajectory', 10)
713     self.pub_plan = self.create_publisher(Float32MultiArray, '/planner/upper_plan', 10)
714     self.pub_lower_plan = self.create_publisher(Float32MultiArray, '/planner/lower_plan', 10)
715     self.pub_env = self.create_publisher(Float32MultiArray, '/planner/env', 10)
716     self.pub_metrics = self.create_publisher(Float32MultiArray, '/planner/metrics', 10)
717     self.pub_status = self.create_publisher(Float32MultiArray, '/planner/status', 10)
718
719     # Subs (optional measurements)
720     self.create_subscription(Float32, '/vehicle/s_km', self.on_s_km_solar, 10, callback_group=self.telemetry_callback_group)
721     self.create_subscription(Float32, '/vehicle/speed_kmh', self.on_speed_solar, 10, callback_group=self.telemetry_callback_group)
722     self.create_subscription(Float32, '/vehicle/batt_soc', self.on_soc_solar, 10, callback_group=self.telemetry_callback_group)
723     self.create_subscription(Float32, '/vehicle/batt_temp_c', self.on_tb_solar, 10, callback_group=self.telemetry_callback_group)
724     self.create_subscription(Float32, '/vehicle/batt_current_a', self.on_i_solar, 10, callback_group=self.telemetry_callback_group)
725     self.create_subscription(Float32, '/vehicle/batt_voltage_v', self.on_v_solar, 10, callback_group=self.telemetry_callback_group)
726     self.create_subscription(Float32, '/calib/solar_gain', self.on_calib_solar_gain, 10, callback_group=self.telemetry_callback_group)
727     self.create_subscription(Float32, '/calib/drive_power_gain', self.on_calib_drive_power_gain, 10, callback_group=self.telemetry_callback_group)
728     self.create_subscription(Float32, '/calib/aux_power_w', self.on_calib_aux_power, 10, callback_group=self.telemetry_callback_group)
729
730     # Timer @1Hz (upper layer)
731     self.timer = self.create_timer(1.0, self.step_solar, callback_group=self.upper_callback_group)
732     # Lower layer timer
733     if self.hierarchical and self.lower_rate_hz > 0.0:
734         expected_dt = 1.0 / self.lower_rate_hz
735         if abs(expected_dt - self.lower_dt) > 1.0e-3 and not self._warned_lower_rate:
736             self.get_logger().warn('lower_dt and lower_rate_hz mismatch; using lower_dt for prediction.')
737             self._warned_lower_rate = True
738             self.timer_lower = self.create_timer(
739                 1.0 / self.lower_rate_hz, self.step_lower, callback_group=self.lower_callback_group
740             )
741             self.timer_command = self.create_timer(
742                 1.0 / self.lower_rate_hz,
743                 self.publish_lower_command_cycle,
744                 callback_group=self.command_callback_group,
745             )
746         else:
747             self.timer_lower = None
748             self.timer_command = None
749         self.get_logger().info('MPCNode started (solarcar mode).')
750
751     def current_bin_index(self) -> int:
752         mode = str(self.forecast_time_mode).lower()
753         has_time = len(getattr(self, 'forecast_times', [])) > 0
754         if mode == 'auto':
755             mode = 'absolute' if has_time else 'relative'
756
757         now = datetime.now(timezone.utc) + timedelta(seconds=self.forecast_time_offset)
758         lookup_time = now
759
760         if mode == 'absolute' and has_time:
761             t_series = self.forecast_times.values
762             t_min = self.forecast_times[0]
763             t_max = self.forecast_times[-1]
764             if (now < t_min) or (now > t_max):
765                 if not getattr(self, '_forecast_warned_out_of_range', False):
766                     self.get_logger().warn('Forecast time out of range; switching to relative indexing.')
767                     self._forecast_warned_out_of_range = True
768                 mode = 'relative'
769             else:
770                 idx = int(np.searchsorted(t_series, np.datetime64(now)) - 1)
771                 self.current_forecast_time_utc = now
772                 return int(np.clip(idx, 0, len(self.forecast_times) - 1))
773
774         if mode in ('relative', 'loop') or not has_time:
775             elapsed = (now - self.forecast_relative_wall_start).total_seconds()
776             elapsed = max(0.0, elapsed)
777             bin_count = len(self.forecast_times) if has_time else len(self.df)
778             if bin_count == 0:
779                 return 0
780             if has_time:
781                 lookup_time = self.forecast_start_time + timedelta(seconds=elapsed)
782                 if mode == 'loop' and len(self.forecast_times) > 1:
783                     t_first = self.forecast_times[0].to_pydatetime()
784                     t_last = self.forecast_times[-1].to_pydatetime()
785                     loop_sec = max(1.0, (t_last - t_first).total_seconds())
786                     lookup_time = t_first + timedelta(
787                         seconds=(lookup_time - t_first).total_seconds() % loop_sec
788                     )

```

```

789         self.current_forecast_time_utc = lookup_time
790         idx = int(np.searchsorted(
791             self.forecast_times.values, np.datetime64(lookup_time)
792         ) - 1)
793         return int(np.clip(idx, 0, bin_count - 1))
794     idx = int(elapsed / max(self.dt, 1.0e-3))
795     self.current_forecast_time_utc = self.forecast_start_time + timedelta(seconds=elapsed)
796     return int(idx % bin_count) if mode == 'loop' else int(np.clip(idx, 0, bin_count - 1))
797
798     bin_count = len(self.forecast_times) if has_time else len(self.df)
799     return int(np.clip(self.k, 0, max(0, bin_count - 1)))
800
801 def _horizon_data(self, k0: int):
802     data = []
803     has_time = len(getattr(self, 'forecast_times', [])) > 0
804     N = len(self.forecast_times) if has_time else len(self.df)
805     mode = str(self.forecast_time_mode).lower()
806     if mode == 'auto':
807         mode = 'absolute' if has_time else 'relative'
808     if N <= 0:
809         return data
810     Np = max(1, self.Np)
811     s0_km = float(self.s_km)
812     if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
813         s0_km = float(self.s_meas)
814     speed_guess_kmh = max(0.0, float(self.v_upper_cmd))
815     for i in range(Np):
816         if has_time:
817             t_utc = self.current_forecast_time_utc + timedelta(seconds=self.dt * i)
818             if mode == 'loop' and len(self.forecast_times) > 1:
819                 t_first = self.forecast_times[0].to_pydatetime()
820                 t_last = self.forecast_times[-1].to_pydatetime()
821                 loop_sec = max(1.0, (t_last - t_first).total_seconds())
822                 t_utc = t_first + timedelta(seconds=(t_utc - t_first).total_seconds() % loop_sec)
823             j = int(np.searchsorted(self.forecast_times.values, np.datetime64(t_utc)) - 1)
824             j = int(np.clip(j, 0, N - 1))
825             row_match = self.df.loc[self.df['time'] == self.forecast_times[j]]
826             row = row_match.iloc[0] if not row_match.empty else self.df.iloc[0]
827         else:
828             j = (k0 + i) % N if mode == 'loop' else min(k0 + i, N - 1)
829             row = self.df.iloc[j]
830             t_utc = self.forecast_start_time + timedelta(seconds=self.dt * (k0 + i))
831             s_query_km = s0_km + speed_guess_kmh * self.dt * i / 3600.0
832             is_drive = True
833             if self.drive_schedule is not None:
834                 limits = self.drive_schedule.speed_limits(t_utc)
835                 if limits is not None and limits[1] <= 0.0:
836                     is_drive = False
837             env = self._forecast_at_time(t_utc, s_query_km, drive=is_drive)
838             env['slope_pct'] = float(row.get('slope_pct', 0.0))
839             env['t_utc'] = t_utc
840             data.append(env)
841     return data
842
843 def _forecast_at_time(
844     self,
845     t_utc: datetime,
846     s_km: float | None = None,
847     drive: bool = True,
848     control_stop: bool = False,
849 ) -> dict:
850     if len(self.df) == 0:
851         return dict(
852             G_poa=0.0, Tcell_C=40.0, Tamb_C=30.0, headwind_ms=0.0,
853             elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0),
854         )
855     if getattr(self, 'forecast_grid', None):
856         ghi = interp_forecast_grid(self.forecast_grid, 'GHI', t_utc, s_km, 0.0)
857         poa_drive = interp_forecast_grid(self.forecast_grid, 'POA_drive', t_utc, s_km, ghi)
858         poa_stop_ideal = interp_forecast_grid(
859             self.forecast_grid, 'POA_stop_ideal', t_utc, s_km, poa_drive
860         )
861         tamb = interp_forecast_grid(self.forecast_grid, 'Tamb_C', t_utc, s_km, 30.0)
862         tcell_drive = interp_forecast_grid(
863             self.forecast_grid,
864             'Tcell_drive_C',
865             t_utc,
866             s_km,
867             interp_forecast_grid(self.forecast_grid, 'Tcell_C', t_utc, s_km, tamb),
868         )
869         tcell_stop_ideal = interp_forecast_grid(
870             self.forecast_grid, 'Tcell_stop_ideal_C', t_utc, s_km, tcell_drive
871         )
872         headwind = interp_forecast_grid(self.forecast_grid, 'headwind_ms', t_utc, s_km, 0.0)
873         if drive:
874             tilt_fraction = 0.0
875             gain = self.poa_gain_drive
876             stop_mode = 'drive'
877         else:
878             tilt_fraction = (
879                 self.control_stop_tilt_fraction if control_stop else self.stop_tilt_fraction
880             )
881             gain = self.poa_gain_stop
882             stop_mode = 'control_stop' if control_stop else 'camp_or_strategy'
883         return dict(
884             G_poa=(poa_drive + tilt_fraction * (poa_stop_ideal - poa_drive))
885             * self.solar_gain * gain,
886             Tcell_C=tcell_drive + tilt_fraction * (tcell_stop_ideal - tcell_drive),
887             Tamb_C=tamb,
888             headwind_ms=headwind,
889             panel_stop_mode=stop_mode,
890             elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0),
891         )

```

```

892     has_time = ('time' in self.df.columns) and (not self.df['time'].isna().all())
893     if has_time:
894         t_series = self.df['time'].values
895         idx = int(np.searchsorted(t_series, np.datetime64(t_utc)) - 1)
896         idx = int(np.clip(idx, 0, len(self.df) - 1))
897     else:
898         elapsed = (t_utc - self.forecast_start_time).total_seconds()
899         idx = int(np.clip(elapsed / max(self.dt, 1e-3), 0, len(self.df) - 1))
900     row = self.df.iloc[idx]
901     gain = self.poa_gain_drive if drive else self.poa_gain_stop
902     poa_drive = float(row.get('POA_drive', row.get('GHI', 0.0)))
903     poa_stop_ideal = float(row.get('POA_stop_ideal', poa_drive))
904     tilt_fraction = 0.0 if drive else (
905         self.control_stop_tilt_fraction if control_stop else self.stop_tilt_fraction
906     )
907     tcell_drive = float(row.get('Tcell_drive_C', row.get('Tcell_C', 40.0)))
908     tcell_stop_ideal = float(row.get('Tcell_stop_ideal_C', tcell_drive))
909     return dict(
910         G_poa=(poa_drive + tilt_fraction * (poa_stop_ideal - poa_drive))
911         * self.solar_gain * gain,
912         Tcell_C=tcell_drive + tilt_fraction * (tcell_stop_ideal - tcell_drive),
913         Tamb_C=float(row.get('Tamb_C', 30.0)),
914         headwind_ms=float(row.get('headwind_ms', 0.0)) if 'headwind_ms' in row else 0.0,
915         panel_stop_mode='drive' if drive else ('control_stop' if control_stop else 'camp_or_strategy'),
916         elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0),
917     )
918
919 def _sample_plan_segments(self, dt_sample: float):
920     if not self.v_plan_segments:
921         return []
922     if dt_sample <= 0.0:
923         return [float(seg['v_kmh']) for seg in self.v_plan_segments]
924     samples = []
925     for seg in self.v_plan_segments:
926         n = max(1, int(math.ceil(seg['dt_sec'] / dt_sample)))
927         samples.extend([float(seg['v_kmh'])] * n)
928     return samples
929
930 def _route_value(self, s_km: float, field: str, default: float) -> float:
931     if self.route_profile is None:
932         return float(default)
933     try:
934         val = float(interpolate_profile(self.route_profile, s_km, field, default))
935         if not np.isfinite(val):
936             return float(default)
937         return float(val)
938     except Exception:
939         return float(default)
940
941 def _route_average(self, s0_km: float, s1_km: float, field: str, default: float) -> float:
942     if self.route_profile is None:
943         return float(default)
944     try:
945         value = float(average_profile(self.route_profile, s0_km, s1_km, field, default))
946         return value if np.isfinite(value) else float(default)
947     except Exception:
948         return float(default)
949
950 def _speed_limit_at(self, s_km: float, default_kmh: float) -> float:
951     if self.speed_profile is None:
952         return float(default_kmh)
953     try:
954         return float(interpolate_profile(self.speed_profile, s_km, 'v_max_kmh', default_kmh))
955     except Exception:
956         return float(default_kmh)
957
958 def _soc_guard_speed(self, v_kmh: float, s_km: float, d0: dict) -> float:
959     mode = str(self.get_parameter('soc_guard_mode').value).lower()
960     soc_guard = float(self.get_parameter('soc_guard_margin').value)
961     target = self.model.p.soc_min + soc_guard
962
963     slope_pct = d0['slope_pct']
964     if self.route_profile is not None:
965         slope_pct = self._route_value(s_km, 'slope_pct', slope_pct)
966     headwind_ms = d0.get('headwind_ms', 0.0)
967     if self.route_profile is not None:
968         headwind_ms = self._route_value(s_km, 'headwind_ms', headwind_ms)
969
970 def _z_next_for(v_kmh_local: float) -> float:
971     out = self.model.electrical_balance(
972         v_kmh_local / 3.6, slope_pct, self.z, self.Tb,
973         d0['G_poa'], d0['Tcell_C'], headwind_ms=headwind_ms,
974         ambient_temp_c=d0.get('Tamb_C'),
975         elevation_m=self._route_value(s_km, 'elev_m', d0.get('elevation_m', 0.0)),
976     )
977     P_pack = float(out['P_pack'])
978     return self.model.soc_step(
979         self.z,
980         P_pack,
981         self.model.p.dt,
982         current_a=float(out['I']),
983         Tbat_C=self.Tb,
984     )
985
986 # If already below target, apply guard mode
987 if self.z <= target:
988     if mode == 'stop':
989         return 0.0
990     if mode != 'pv_only':
991         return v_kmh
992     # find max speed such that P_pack <= 0
993     lo = 0.0
994     hi = max(0.0, float(v_kmh))

```

```

995         for _ in range(20):
996             mid = 0.5 * (lo + hi)
997             if z_next_for(mid) < self.z:
998                 hi = mid
999             else:
1000                 lo = mid
1001         return float(lo)
1002
1003     # If next step would violate, limit speed to keep z_next >= target
1004     if z_next_for(v_kmh) >= target:
1005         return v_kmh
1006     lo = 0.0
1007     hi = max(0.0, float(v_kmh))
1008     for _ in range(25):
1009         mid = 0.5 * (lo + hi)
1010         if z_next_for(mid) < target:
1011             hi = mid
1012         else:
1013             lo = mid
1014     return float(lo)
1015
1016 def _control_stop_catalog(self):
1017     catalog = []
1018     for source_index, stop in enumerate(self.stops):
1019         kind = str(stop.get('kind', '')).strip().lower()
1020         explicit = stop.get('is_control_stop', None)
1021         if explicit is False and kind != 'control_stop':
1022             continue
1023         if explicit is None and kind in {'trouble_stop', 'unscheduled_stop', 'strategy_stop'}:
1024             continue
1025         dwell_sec = max(0.0, float(stop.get('dwell_sec', stop.get('dwell_s', 0.0))))
1026         if dwell_sec <= 0.0:
1027             continue
1028         catalog.append({
1029             'source_index': int(source_index),
1030             's_km': float(stop.get('s_km', 0.0)),
1031             'dwell_sec': dwell_sec,
1032             'label': str(stop.get('label', f'control_stop_{source_index + 1}')),
1033         })
1034     return sorted(catalog, key=lambda item: item['s_km'])
1035
1036 def _update_live_control_stop_hold(self, s_km: float, v_kmh: float) -> bool:
1037     """Enforce approach, standstill detection, and timed hold at declared control stops."""
1038     catalog = self._control_stop_catalog()
1039     if not catalog:
1040         return False
1041     now_mono = time.monotonic()
1042     if self.active_control_stop_index is not None:
1043         if self.control_stop_end_monotonic is None or now_mono < self.control_stop_end_monotonic:
1044             return True
1045         self.completed_control_stops.add(int(self.active_control_stop_index))
1046         self.get_logger().info('Control-stop dwell completed; speed command released.')
1047         self.active_control_stop_index = None
1048         self.control_stop_end_monotonic = None
1049
1050     tolerance_kmh = max(
1051         0.01, float(self.get_parameter('control_stop_arrival_tolerance_kmh').value)
1052     )
1053     if not getattr(self, 'control_stop_position_initialized', False):
1054         for stop in catalog:
1055             if stop['s_km'] < float(s_km) - tolerance_kmh:
1056                 self.completed_control_stops.add(int(stop['source_index']))
1057             self.control_stop_position_initialized = True
1058
1059     stationary_kmh = max(
1060         0.0, float(self.get_parameter('control_stop_stationary_speed_kmh').value)
1061     )
1062     decel_ms2 = max(
1063         0.1,
1064         float(self.get_parameter('control_stop_brake_decel_kmhps').value) / 3.6,
1065     )
1066     v_ms = max(0.0, float(v_kmh)) / 3.6
1067     braking_kmh = v_ms * v_ms / (2.0 * decel_ms2) / 1000.0
1068     braking_kmh += max(0.0, float(self.get_parameter('control_stop_brake_margin_kmh').value))
1069
1070     for stop in catalog:
1071         stop_index = int(stop['source_index'])
1072         if stop_index in self.completed_control_stops:
1073             continue
1074         distance_to_stop_kmh = float(stop['s_km']) - float(s_km)
1075         if distance_to_stop_kmh > braking_kmh:
1076             return False
1077         if distance_to_stop_kmh <= tolerance_kmh and float(v_kmh) <= stationary_kmh:
1078             self.active_control_stop_index = stop_index
1079             self.control_stop_end_monotonic = now_mono + float(stop['dwell_sec'])
1080             self.get_logger().info(
1081                 f"Control-stop dwell started: {stop['label']} ({stop['dwell_sec']:.0f} s)."
1082             )
1083             return True
1084     return False
1085
1086 def _dwell_penalty(self, s_km: float, v_ms: float) -> float:
1087     vmax_kmh = float(self.get_parameter('v_max_kmh').value)
1088     vmax_ms = vmax_kmh / 3.6
1089     pen = 0.0
1090     for st in self.stops:
1091         s_stop = float(st.get('s_km', 0.0))
1092         dwell_s = float(st.get('dwell_sec', st.get('dwell_s', 0.0)))
1093         width_kmh = max(0.05, (dwell_s * vmax_ms) / 1000.0 * 0.5)
1094         if abs(s_km - s_stop) <= width_kmh:
1095             pen += 1.0e5 * (v_ms ** 2)
1096     return pen
1097

```

```

1098 def _mpc_solve_solar(self, data):
1099     p = self.model.p
1100     Np = len(data)
1101     if Np <= 0:
1102         return self.v_cmd, [self.v_cmd]
1103
1104     v0_guess = self.v_cmd / 3.6
1105     if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
1106         v0_guess = float(self.v_now) / 3.6
1107     x0 = np.array([v0_guess] * Np, dtype=float) # m/s
1108     lb = np.zeros(Np, dtype=float)
1109     v_max_kmh = float(self.get_parameter('v_max_kmh').value)
1110     v_min_solver = max(0.1, float(self.upper_vmin_kmh))
1111     ub = np.ones(Np, dtype=float) * (v_max_kmh / 3.6)
1112
1113     term_soc_min = float(self.get_parameter('terminal_soc_min').value)
1114     w_dv = float(self.get_parameter('w_dv').value)
1115     w_dv_limit = float(self.get_parameter('w_dv_limit').value)
1116     dv_max_kmhps = float(self.get_parameter('dv_max_kmhps').value)
1117     dv_max_msps = dv_max_kmhps / 3.6
1118     w_T = float(self.get_parameter('w_T').value)
1119     w_speed_limit = float(self.get_parameter('w_speed_limit').value)
1120     w_drive_window = float(self.get_parameter('w_drive_window').value)
1121     w_current = float(self.get_parameter('w_current').value)
1122     soc_target = float(self.soc_target)
1123     soc_band = float(self.soc_band)
1124     w_soc_target = float(self.w_soc_target)
1125     w_soc_band = float(self.w_soc_band)
1126     soc_day_end_max = float(self.soc_day_end_max)
1127     w_soc_day_max = float(self.w_soc_day_max)
1128     soc_finish_target = float(self.soc_finish_target)
1129     soc_finish_tol = float(self.soc_finish_tol)
1130     w_soc_progress = float(self.w_soc_progress)
1131     w_soc_terminal = float(self.w_soc_terminal)
1132     race_km = float(self.race_km)
1133     z_start = float(self.z)
1134
1135     def quad_penalty(x, cap=1.0e3):
1136         if x <= 0.0:
1137             return 0.0
1138         if x > cap:
1139             x = cap
1140         return x * x
1141
1142     def cost(v):
1143         z = float(self.z)
1144         Tb = float(self.Tb)
1145         s_km = float(self.s_km)
1146         if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
1147             s_km = float(self.s_meas)
1148         v_prev = float(v0_guess)
1149         J = 0.0
1150
1151         for k in range(Np):
1152             d = data[k]
1153             v_k = float(v[k])
1154             slope_pct = d['slope_pct']
1155             if self.route_profile is not None:
1156                 slope_pct = self._route_value(s_km, 'slope_pct', slope_pct)
1157             headwind_ms = d.get('headwind_ms', 0.0)
1158             if self.route_profile is not None:
1159                 headwind_ms = self._route_value(s_km, 'headwind_ms', headwind_ms)
1160             aux_power_w = None
1161             if self.drive_schedule is not None and 't_utc' in d and not self.drive_schedule.is_drive_time(d['t_utc']):
1162                 aux_power_w = self.model.scheduled_auxiliary_power(
1163                     is_driving=False,
1164                     irradiance_wm2=d.get('G_poa', 0.0),
1165                 )
1166             inertial_power_w = 0.5 * p.m * (v_k * v_k - v_prev * v_prev) / max(p.dt, 1.0e-9)
1167             out = self.model.electrical_balance(
1168                 v_k,
1169                 slope_pct,
1170                 z,
1171                 Tb,
1172                 d['G_poa'],
1173                 d['Tcell_C'],
1174                 headwind_ms=headwind_ms,
1175                 aux_power_w=aux_power_w,
1176                 inertial_power_w=inertial_power_w,
1177                 ambient_temp_c=d.get('Tamb_C'),
1178                 elevation_m=d.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)),
1179             )
1180             I = float(out['I'])
1181             V = float(out['V'])
1182             P_pack = float(out['P_pack'])
1183             loss_int = float(out['losses_int'])
1184
1185             # dynamics
1186             z_next = self.model.soc_step(
1187                 z,
1188                 P_pack,
1189                 p.dt,
1190                 current_a=I,
1191                 Tbat_C=Tb,
1192             )
1193             Tb_next = Tb + (p.dt / 1800.0) * (d['Tamb_C'] - Tb) + (loss_int * p.dt) / 50000.0
1194             s_km = s_km + v_k * (p.dt / 1000.0)
1195
1196             # objective
1197             J += -1.0 * v_k * p.dt
1198             J += 30.0 * quad_penalty(term_soc_min - z_next)
1199             if self.drive_schedule is not None and soc_day_end_max > 0.0 and 't_utc' in d:
1200                 t_next = d['t_utc'] + timedelta(seconds=p.dt)

```

```

1201         if self.drive_schedule.is_drive_time(d['t_utc']) and not self.drive_schedule.is_drive_time(t_next):
1202             J += w_soc_day_max * quad_penalty(z_next - soc_day_end_max)
1203     if soc_finish_target > 0.0:
1204         prog = max(0.0, min(1.0, s_km / max(race_km, 1.0)))
1205         soc_line = z_start + (soc_finish_target - z_start) * prog
1206         if z_next > (soc_line + soc_finish_tol):
1207             J += w_soc_progress * quad_penalty(z_next - (soc_line + soc_finish_tol))
1208         if z_next < (soc_line - soc_finish_tol):
1209             J += w_soc_progress * quad_penalty((soc_line - soc_finish_tol) - z_next)
1210
1211     dv = (v_k - v_prev) / max(p.dt, 1.0e-3)
1212     if dv_max_msp > 0.0:
1213         J += w_dv_limit * quad_penalty(abs(dv) - dv_max_msp)
1214
1215     if self.drive_schedule is not None and 't_utc' in d:
1216         limits = self.drive_schedule.speed_limits(d['t_utc'])
1217         if limits is not None:
1218             vmin_kmh, vmax_kmh = limits
1219             vmin_ms = vmin_kmh / 3.6
1220             vmax_ms = vmax_kmh / 3.6
1221             J += w_drive_window * quad_penalty(vmin_ms - v_k)
1222             J += w_drive_window * quad_penalty(v_k - vmax_ms)
1223
1224     vmax_local = self._speed_limit_at(s_km, self.get_parameter('v_max_kmh').value)
1225     if vmax_local < self.get_parameter('v_max_kmh').value:
1226         J += w_speed_limit * quad_penalty(v_k * 3.6 - vmax_local)
1227
1228     # inequalities (soft)
1229     J += 1e4 * quad_penalty(I - p.I_max)
1230     J += 1e4 * quad_penalty(p.I_chg_min - I)
1231     J += 1e4 * quad_penalty(p.V_min - V)
1232     J += 1e4 * quad_penalty(V - p.V_max)
1233     J += w_T * quad_penalty(Tb_next - p.T_max)
1234     J += w_T * quad_penalty(p.T_min - Tb_next)
1235     J += 1e4 * quad_penalty(p.soc_min - z_next)
1236     J += 1e4 * quad_penalty(z_next - p.soc_max)
1237
1238     J += self._dwell_penalty(s_km, v_k)
1239
1240     z, Tb = z_next, Tb_next
1241     v_prev = v_k
1242
1243     J += 1e4 * quad_penalty(term_soc_min - z)
1244     if soc_finish_target > 0.0:
1245         J += w_soc_terminal * quad_penalty(z - (soc_finish_target + soc_finish_tol))
1246     return J
1247
1248 bounds = list(zip(lb, ub))
1249 res = minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=150))
1250 if np.all(np.isfinite(res.x)):
1251     self.last_upper_solve_ok = bool(res.success)
1252     v_seq = res.x
1253     if not res.success:
1254         self.get_logger().warn('Upper MPC reached an early stop; using best finite iterate.')
1255 else:
1256     self.last_upper_solve_ok = False
1257     self.get_logger().warn('Upper MPC solve failed; reusing warm-start plan.')
1258     v_seq = x0
1259 v_seq_kmh = np.clip(v_seq * 3.6, 0.0, v_max_kmh)
1260 return float(v_seq_kmh[0]), [float(v) for v in v_seq_kmh]
1261
1262 def mpc_solve_solar_distance(self, t0_utc: datetime, s0_km: float, x0=None):
1263     p = self.model.p
1264     horizon = build_upper_distance_horizon(
1265         mode=self.upper_horizon_mode,
1266         s0_km=s0_km,
1267         race_km=self.race_km,
1268         ds_km=self.upper_ds_km,
1269         horizon_km=self.upper_horizon_km,
1270         max_steps=self.upper_max_steps,
1271         ctrl_km=self.upper_ctrl_km,
1272         adaptive_min_ds_km=self.upper_adaptive_min_ds_km,
1273         adaptive_max_ds_km=self.upper_adaptive_max_ds_km,
1274         adaptive_growth=self.upper_adaptive_growth,
1275     )
1276     ds_seq = np.array(horizon.ds_seq_km, dtype=float)
1277     seg_s = np.array(horizon.seg_s_km, dtype=float)
1278     Np = int(len(ds_seq))
1279     if Np <= 0:
1280         return self.v_cmd, [{'v_kmh': float(self.v_cmd), 'dt_sec': float(p.dt)}, [float(self.v_cmd)]]
1281
1282     # Split the integration mesh at every declared control stop so arrival
1283     # time, dwell charging, and the next departure are evaluated exactly at
1284     # the stop coordinate rather than at the end of an arbitrary route bin.
1285     control_stops = self._control_stop_catalog()
1286     horizon_end_km = float(s0_km + np.sum(ds_seq))
1287     extra_edges = [
1288         float(stop['s_km'])
1289         for stop in control_stops
1290         if float(s0_km) + 1.0e-9 < float(stop['s_km']) < horizon_end_km - 1.0e-9
1291     ]
1292     if extra_edges:
1293         base_edges = np.concatenate(([float(s0_km)], float(s0_km) + np.cumsum(ds_seq)))
1294         route_edges = np.array(sorted(set(np.round(np.concatenate((base_edges, extra_edges)), 9))), dtype=float)
1295         ds_seq = np.diff(route_edges)
1296         seg_s = route_edges[:-1]
1297         Np = int(len(ds_seq))
1298
1299     v_max_kmh = float(self.get_parameter('v_max_kmh').value)
1300     v_min_solver = max(0.1, float(self.upper_vmin_kmh))
1301     v0 = float(self.v_cmd)
1302     if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
1303         v0 = float(self.v_now)

```

```

3305 ctrl_s = np.array(horizon.ctrl_s_km, dtype=float)
3306 Nc = int(len(ctrl_s))
3307
3308 previous_ctrl_s = getattr(self, 'upper_plan_ctrl_s', None)
3309 if (
3310     x0 is not None
3311     and previous_ctrl_s is not None
3312     and len(x0) == len(previous_ctrl_s)
3313     and len(previous_ctrl_s) >= 2
3314 ):
3315     x0 = np.interp(ctrl_s, np.asarray(previous_ctrl_s, dtype=float), np.asarray(x0, dtype=float))
3316 elif x0 is not None and len(x0) == Nc:
3317     x0 = np.array(x0, dtype=float)
3318 elif self.initial_upper_policy is not None:
3319     x0 = interpolate_upper_policy(
3320         self.initial_upper_policy,
3321         ctrl_s,
3322         minimum_speed_kmh=v_min_solver,
3323         maximum_speed_kmh=v_max_kmh,
3324     )
3325 else:
3326     # The first receding-horizon solve has no previous plan. Starting
3327     # every control knot at the legal maximum biases a reduced live
3328     # candidate library toward an unsafe flat-out policy. The measured
3329     # (or current commanded) speed is the physically continuous warm
3330     # start; the separate balance seed can still increase speed when
3331     # the full-race energy objective supports it.
3332     x0 = np.full(Nc, float(np.clip(v0, v_min_solver, v_max_kmh)), dtype=float)
3333 x0 = np.clip(np.asarray(x0, dtype=float), v_min_solver, v_max_kmh)
3334 self.upper_plan_ctrl_s = ctrl_s.copy()
3335 bounds = [(v_min_solver, v_max_kmh)] * Nc
3336
3337 idx = np.searchsorted(ctrl_s, seg_s, side='right') - 1
3338 idx = np.clip(idx, 0, Nc - 1)
3339 idx_next = np.clip(idx + 1, 0, Nc - 1)
3340 denom = np.maximum(ctrl_s[idx_next] - ctrl_s[idx], 1.0e-6)
3341 alpha = (seg_s - ctrl_s[idx]) / denom
3342
3343 def expand_ctrl(u_vec):
3344     return (1.0 - alpha) * u_vec[idx] + alpha * u_vec[idx_next]
3345
3346 def build_balance_seed():
3347     u_seed = np.array(x0, dtype=float)
3348     z_seed = float(self.z)
3349     Tb_seed = float(self.Tb)
3350     s_seed = float(s0_kmh)
3351     t_seed = t0_utc
3352     v_prev_seed = float(v0)
3353     for idx_ctrl in range(Nc):
3354         t_seed, z_seed, Tb_seed, _, _ = step_wait(t_seed, z_seed, Tb_seed, s_seed)
3355         vmax_local = self._speed_limit_at(s_seed, v_max_kmh)
3356         vmin_local = v_min_solver
3357         if self.drive_schedule is not None:
3358             limits = self.drive_schedule.speed_limits(t_seed)
3359             if limits is not None:
3360                 vmin_local = max(vmin_local, float(limits[0]))
3361                 vmax_local = min(vmax_local, float(limits[1]))
3362         if vmax_local < vmin_local:
3363             u_seed[idx_ctrl] = max(0.0, vmax_local)
3364             continue
3365         candidate_grid = np.linspace(vmin_local, vmax_local, num=max(5, min(13, int((vmax_local - vmin_local) / 4.0) + 2)))
3366         best_v = float(np.clip(v_prev_seed, vmin_local, vmax_local))
3367         best_score = float("inf")
3368         env = self._forecast_at_time(t_seed, s_seed, drive=True)
3369         slope_pct = self._route_value(s_seed, 'slope_pct', 0.0)
3370         headwind_ms = self._route_value(s_seed, 'headwind_ms', env.get('headwind_ms', 0.0))
3371         for v_test in candidate_grid:
3372             out = self.model.electrical_balance(
3373                 v_test / 3.6, slope_pct, z_seed, Tb_seed, env['G_poa'], env['Tcell_C'],
3374                 headwind_ms=headwind_ms, ambient_temp_c=env.get('Tamb_C'),
3375                 elevation_m=env.get('elevation_m', self._route_value(s_seed, 'elev_m', 0.0)),
3376             )
3377             p_pack = float(out['P_pack'])
3378             score = abs(p_pack) + 0.35 * ((float(v_test) - float(v_prev_seed)) ** 2)
3379             if score < best_score:
3380                 best_score = score
3381                 best_v = float(v_test)
3382         u_seed[idx_ctrl] = best_v
3383     if idx_ctrl + 1 < len(ctrl_s):
3384         ds_ctrl = max(0.0, float(ctrl_s[idx_ctrl + 1] - ctrl_s[idx_ctrl]))
3385     else:
3386         ds_ctrl = max(float(ds_seq[-1]), float(self.upper_ds_kmh))
3387     target_s_kmh = s_seed + ds_ctrl
3388     stop_edges = [
3389         float(stop['s_kmh'])
3390         for stop in control_stops
3391         if s_seed + 1.0e-9 < float(stop['s_kmh']) <= target_s_kmh + 1.0e-9
3392     ]
3393     seed_edges = sorted(set([*stop_edges, float(target_s_kmh)]))
3394     stopped_in_chunk = False
3395     for edge_s_kmh in seed_edges:
3396         ds_leg_kmh = max(0.0, float(edge_s_kmh) - s_seed)
3397         if ds_leg_kmh > 1.0e-9:
3398             dt_seed = ds_leg_kmh / max(best_v, 1.0e-3) * 3600.0
3399             env_leg = self._forecast_at_time(t_seed, s_seed, drive=True)
3400             slope_leg = self._route_average(
3401                 s_seed, edge_s_kmh, 'slope_pct', 0.0
3402             )
3403             headwind_leg = self._route_value(
3404                 s_seed, 'headwind_ms', env_leg.get('headwind_ms', 0.0)
3405             )
3406             out = self.model.electrical_balance(

```

```

1407         best_v / 3.6,
1408         slope_leg,
1409         z_seed,
1410         Tb_seed,
1411         env_leg['G_poa'],
1412         env_leg['Tcell_C'],
1413         headwind_ms=headwind_leg,
1414         ambient_temp_c=env_leg.get('Tamb_C'),
1415         elevation_m=env_leg.get('elevation_m', self._route_value(s_seed, 'elev_m', 0.0)),
1416     )
1417     z_seed = float(np.clip(self.model.soc_step(
1418         z_seed,
1419         float(out['P_pack']),
1420         dt_seed,
1421         current_a=float(out['I']),
1422         Tbat_C=Tb_seed,
1423     ), p.soc_min, p.soc_max))
1424     Tb_seed = float(np.clip(
1425         Tb_seed + (dt_seed / 1800.0) * (env_leg['Tamb_C'] - Tb_seed)
1426         + (float(out['losses_int']) * dt_seed) / 50000.0,
1427         p.T_min,
1428         p.T_max,
1429     ))
1430     t_seed += timedelta(seconds=dt_seed)
1431     s_seed = float(edge_s_km)
1432     t_seed, z_seed, Tb_seed, control_wait, _ = apply_control_stop_at(
1433         t_seed, z_seed, Tb_seed, s_seed
1434     )
1435     stopped_in_chunk = stopped_in_chunk or control_wait > 1.0e-9
1436     v_prev_seed = 0.0 if stopped_in_chunk else best_v
1437     return np.array(u_seed, dtype=float)
1438
1439     w_dv_limit = float(self.get_parameter('w_dv_limit').value)
1440     dv_max_kmhps = float(self.get_parameter('dv_max_kmhps').value)
1441     term_soc_min = float(self.get_parameter('terminal_soc_min').value)
1442     day_end_soc_min = float(self.upper_day_end_soc_min)
1443     soc_day_end_max = float(self.soc_day_end_max)
1444     soc_finish_target = float(self.soc_finish_target)
1445     soc_day_end_tol = float(self.get_parameter('soc_day_end_tol').value)
1446
1447     def integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, *, control_stop):
1448         dt_wait = max(0.0, float(dt_wait))
1449         if dt_wait <= 0.0:
1450             return t_utc, float(z), float(Tb), 0.0, 0.0
1451         slope_pct = self._route_value(s_km, 'slope_pct', 0.0)
1452         t_cursor = t_utc
1453         remaining = dt_wait
1454         dt_sample = min(600.0, max(60.0, float(p.dt)))
1455         wait_cost = float(self.upper_cost_cfg.w_wait) * dt_wait
1456         while remaining > 1.0e-9:
1457             dt_local = min(dt_sample, remaining)
1458             env = self._forecast_at_time(
1459                 t_cursor, s_km, drive=False, control_stop=control_stop
1460             )
1461             headwind_ms = self._route_value(s_km, 'headwind_ms', env.get('headwind_ms', 0.0))
1462             out = self.model.electrical_balance(
1463                 0.0,
1464                 slope_pct,
1465                 z,
1466                 Tb,
1467                 env['G_poa'],
1468                 env['Tcell_C'],
1469                 headwind_ms=headwind_ms,
1470                 aux_power_w=self.model.scheduled_auxiliary_power(
1471                     is_driving=False,
1472                     irradiance_wm2=env['G_poa'],
1473                 ),
1474                 ambient_temp_c=env.get('Tamb_C'),
1475                 elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)),
1476             )
1477             P_pack = float(out['P_pack'])
1478             loss_int = float(out['losses_int'])
1479             I = float(out['I'])
1480             V = float(out['V'])
1481             z = float(np.clip(self.model.soc_step(
1482                 z,
1483                 P_pack,
1484                 dt_local,
1485                 current_a=I,
1486                 Tbat_C=Tb,
1487             ), p.soc_min, p.soc_max))
1488             Tb = float(np.clip(Tb + (dt_local / 1800.0) * (env['Tamb_C'] - Tb) + (loss_int * dt_local) / 50000.0, p.T_min, p.T_max))
1489             constraint = float(self.upper_cost_cfg.constraint_penalty)
1490             wait_cost += constraint * (
1491                 quad_penalty(I - p.I_max)
1492                 + quad_penalty(p.I_chg_min - I)
1493                 + quad_penalty(p.V_min - V)
1494                 + quad_penalty(V - p.V_max)
1495                 + quad_penalty(p.soc_min - z)
1496                 + quad_penalty(z - p.soc_max)
1497             )
1498             t_cursor += timedelta(seconds=dt_local)
1499             remaining -= dt_local
1500         return t_cursor, float(z), float(Tb), dt_wait, float(wait_cost)
1501
1502     def step_wait(t_utc, z, Tb, s_km):
1503         if self.drive_schedule is None or self.drive_schedule.is_drive_time(t_utc):
1504             return t_utc, float(z), float(Tb), 0.0, 0.0
1505         t_start = self.drive_schedule.next_drive_start(t_utc)
1506         dt_wait = max(0.0, (t_start - t_utc).total_seconds())
1507         return integrate_stationary_duration(
1508             t_utc, z, Tb, s_km, dt_wait, control_stop=False
1509         )

```

```

1510
1511 def apply_control_stop_at(t_utc, z, Tb, s_km):
1512     total_wait = 0.0
1513     total_cost = 0.0
1514     for stop in control_stops:
1515         if abs(float(stop['s_km']) - float(s_km)) > 1.0e-7:
1516             continue
1517         t_utc, z, Tb, dt_stop, stop_cost = integrate_stationary_duration(
1518             t_utc,
1519             z,
1520             Tb,
1521             float(stop['s_km']),
1522             float(stop['dwell_sec']),
1523             control_stop=True,
1524         )
1525         total_wait += dt_stop
1526         total_cost += stop_cost
1527     return t_utc, float(z), float(Tb), total_wait, total_cost
1528
1529 def cost(u_vec):
1530     z = float(self.z)
1531     Tb = float(self.Tb)
1532     s_km = float(s0_km)
1533     t_utc = t0_utc
1534     v_prev = v0
1535     p_pack_prev = None
1536     elapsed_plan_sec = 0.0
1537     J = 0.0
1538     v_seq = expand_ctrl(u_vec)
1539     for k in range(Np):
1540         t_utc, z, Tb, dt_wait, wait_cost = step_wait(t_utc, z, Tb, s_km)
1541         J += wait_cost
1542         if dt_wait > 1.0e-9:
1543             v_prev = 0.0
1544             v_k = float(v_seq[k])
1545             ds_step_km = float(ds_seq[k])
1546             vmax_local = self._speed_limit_at(s_km, v_max_kmh)
1547             if vmax_local >= v_min_solver:
1548                 v_k = max(v_min_solver, min(v_k, vmax_local))
1549             else:
1550                 v_k = max(0.0, min(v_k, vmax_local))
1551             limits = None
1552             if self.drive_schedule is not None:
1553                 limits = self.drive_schedule.speed_limits(t_utc)
1554
1555             dt_travel = ds_step_km / max(v_k, 1.0e-3) * 3600.0
1556             env = self._forecast_at_time(t_utc, s_km, drive=True)
1557             slope_pct = self._route_average(
1558                 s_km, s_km + ds_step_km, 'slope_pct', 0.0
1559             )
1560             headwind_ms = self._route_value(s_km, 'headwind_ms', env.get('headwind_ms', 0.0))
1561             kinetic_delta_wh = 0.5 * p.m * (
1562                 (v_k / 3.6) ** 2 - (v_prev / 3.6) ** 2
1563             ) / 3600.0
1564             inertial_power_w = kinetic_delta_wh * 3600.0 / max(dt_travel, 1.0e-9)
1565             out = self.model.electrical_balance(
1566                 v_k / 3.6,
1567                 slope_pct,
1568                 z,
1569                 Tb,
1570                 env['G_poa'],
1571                 env['Tcell_C'],
1572                 headwind_ms=headwind_ms,
1573                 inertial_power_w=inertial_power_w,
1574                 ambient_temp_c=env.get('Tamb_C'),
1575                 elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)),
1576             )
1577             I = float(out['I'])
1578             V = float(out['V'])
1579             P_pv = float(out.get('P_pv', 0.0))
1580             P_pack = float(out['P_pack'])
1581             loss_int = float(out['losses_int'])
1582             loss_line = float(out.get('losses_line', 0.0))
1583             P_mech_wheel = float(out.get('P_mech_wheel', 0.0))
1584             kinetic_step_wh = max(0.0, kinetic_delta_wh)
1585
1586             z_next = self.model.soc_step(
1587                 z,
1588                 P_pack,
1589                 dt_travel,
1590                 current_a=I,
1591                 Tbat_C=Tb,
1592             )
1593             Tb_next = Tb + (dt_travel / 1800.0) * (env['Tamb_C'] - Tb) + (loss_int * dt_travel) / 50000.0
1594             forces = self.model.resistive_forces(
1595                 v_k / 3.6, slope_pct, headwind_ms=headwind_ms,
1596                 ambient_temp_c=env.get('Tamb_C'),
1597                 elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)),
1598             )
1599
1600             soc_line = None
1601             if self.drive_schedule is not None and self.soc_day_end_target > 0.0:
1602                 win = self.drive_schedule.current_drive_window(t_utc)
1603                 if win is not None:
1604                     t_start, t_end = win
1605                     if t_end > t_start:
1606                         prog = (t_utc - t_start).total_seconds() / (t_end - t_start).total_seconds()
1607                         prog = max(0.0, min(1.0, prog))
1608                         soc_line = float(self.z) + (self.soc_day_end_target - float(self.z)) * prog
1609
1610             t_next = t_utc + timedelta(seconds=dt_travel)
1611             day_end_crossing = bool(
1612                 self.drive_schedule is not None

```

```

1613         and self.drive_schedule.is_drive_time(t_utc)
1614         and not self.drive_schedule.is_drive_time(t_next)
1615     )
1616     elapsed_plan_sec += dt_wait + dt_travel
1617     J += upper_stage_cost(
1618         self.upper_cost_cfg,
1619         dt_wait=0.0,
1620         dt_travel=dt_travel,
1621         v_kmh=v_k,
1622         v_prev_kmh=v_prev,
1623         vmax_local_kmh=vmax_local,
1624         drive_limits=limits,
1625         dv_limit_kmhps=dv_max_kmhps,
1626         I_a=I,
1627         V_v=V,
1628         P_pv_w=P_pv,
1629         P_pack_w=P_pack,
1630         P_pack_prev_w=p_pack_prev,
1631         P_mech_wheel_w=P_mech_wheel,
1632         losses_int_w=loss_int,
1633         losses_line_w=loss_line,
1634         F_aero_n=float(forces.get('F_aero', 0.0)),
1635         kinetic_step_wh=kinetic_step_wh,
1636         z_next=z_next,
1637         Tb_next_c=Tb_next,
1638         term_soc_min=term_soc_min,
1639         soc_min=p.soc_min,
1640         soc_max=p.soc_max,
1641         temp_min_c=p.T_min,
1642         temp_max_c=p.T_max,
1643         day_end_soc_min=day_end_soc_min,
1644         day_end_crossing=day_end_crossing,
1645         soc_day_end_max=soc_day_end_max,
1646         soc_day_track_target=soc_line,
1647         soc_day_track_tol=soc_day_end_tol,
1648         I_max=p.I_max,
1649         I_chg_min=p.I_chg_min,
1650         V_min=p.V_min,
1651         V_max=p.V_max,
1652         time_ahead_h=elapsed_plan_sec / 3600.0,
1653     )
1654
1655     s_next_km = s_km + ds_step_km
1656     t_after, z_after, Tb_after, stop_wait, stop_cost = apply_control_stop_at(
1657         t_next, z_next, Tb_next, s_next_km
1658     )
1659     J += stop_cost
1660     elapsed_plan_sec += stop_wait
1661     t_utc = t_after
1662     s_km = s_next_km
1663     z, Tb = z_after, Tb_after
1664     if stop_wait > 1.0e-9:
1665         v_prev = 0.0
1666         p_pack_prev = None
1667     else:
1668         v_prev = v_k
1669         p_pack_prev = P_pack
1670
1671     J += upper_terminal_cost(
1672         self.upper_cost_cfg,
1673         z_terminal=z,
1674         term_soc_min=term_soc_min,
1675         soc_finish_target=soc_finish_target,
1676         soc_finish_tol=float(self.soc_finish_tol),
1677     )
1678     return J
1679
1680 structured_seeds = [("balance_seed", build_balance_seed())]
1681 global_search_enabled = bool(self.get_parameter('upper_global_search_enabled').value)
1682 global_search_mode = str(self.get_parameter('upper_global_search_mode').value or 'auto').strip().lower()
1683 u_seq, solve_info = hybrid_bounded_minimize(
1684     cost,
1685     x0,
1686     bounds,
1687     maxiter=int(self.upper_max_iter),
1688     structured_seeds=structured_seeds,
1689     cem_enabled=global_search_enabled,
1690     cem_mode=global_search_mode,
1691     cem_generations=int(self.get_parameter('upper_cem_generations').value),
1692     cem_population=int(self.get_parameter('upper_cem_population').value),
1693     cem_elite=int(self.get_parameter('upper_cem_elite').value),
1694     local_refine_topk=int(self.get_parameter('upper_local_refine_topk').value),
1695     seed_library_mode=str(self.get_parameter('upper_seed_library_mode').value),
1696     rng_seed=int(max(0.0, round(s0_km * 10.0))),
1697     shgo_samples=int(self.get_parameter('upper_shgo_samples').value),
1698     shgo_iters=int(self.get_parameter('upper_shgo_iters').value),
1699     cert_grid_levels=int(self.get_parameter('upper_cert_grid_levels').value),
1700     cert_max_evaluations=int(self.get_parameter('upper_cert_max_evaluations').value),
1701     cert_workers=int(self.get_parameter('upper_cert_workers').value),
1702 )
1703 self.last_upper_solve_ok = bool(solve_info.get('success', False))
1704 if not self.last_upper_solve_ok:
1705     self.get_logger().warn(
1706         'Upper distance-MPC returned a fallback candidate; '
1707         f"label={solve_info.get('label', '')} candidates={solve_info.get('candidates_evaluated', 0)}"
1708     )
1709 v_seq = expand_ctrl(u_seq)
1710
1711 # Build segments with variable time
1712 segments = []
1713 t_utc = t0_utc
1714 s_km = float(s0_km)
1715 z = float(self.z)

```

```

1716     Tb = float(self.Tb)
1717     v_prev_seg = float(v0)
1718     for idx_seg, v_k in enumerate(v_seq):
1719         t_utc, z, Tb, schedule_wait, _ = step_wait(t_utc, z, Tb, s_km)
1720         if schedule_wait > 1.0e-9:
1721             v_prev_seg = 0.0
1722             ds_step_km = float(ds_seq[idx_seg])
1723             v_k = float(np.clip(v_k, 0.0, v_max_kmh))
1724             vmax_local = self._speed_limit_at(s_km, v_max_kmh)
1725             if vmax_local >= v_min_solver:
1726                 v_k = max(v_min_solver, min(v_k, vmax_local))
1727             else:
1728                 v_k = max(0.0, min(v_k, vmax_local))
1729             dt_travel = ds_step_km / max(v_k, 1.0e-3) * 3600.0
1730             env = self._forecast_at_time(t_utc, s_km, drive=True)
1731             slope_pct = self._route_value(s_km, 'slope_pct', 0.0)
1732             headwind_ms = self._route_value(s_km, 'headwind_ms', env.get('headwind_ms', 0.0))
1733             kinetic_delta_wh = 0.5 * p.m * (
1734                 (v_k / 3.6) ** 2 - (v_prev_seg / 3.6) ** 2
1735             ) / 3600.0
1736             out = self.model.electrical_balance(
1737                 v_k / 3.6,
1738                 slope_pct,
1739                 z,
1740                 Tb,
1741                 env['G_poa'],
1742                 env['Tcell_C'],
1743                 headwind_ms=headwind_ms,
1744                 inertial_power_w=kinetic_delta_wh * 3600.0 / max(dt_travel, 1.0e-9),
1745                 ambient_temp_c=env.get('Tamb_C'),
1746                 elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)),
1747             )
1748             z = float(np.clip(self.model.soc_step(
1749                 z,
1750                 float(out['P_pack']),
1751                 dt_travel,
1752                 current_a=float(out['I']),
1753                 Tbat_C=Tb,
1754             ), p.soc_min, p.soc_max))
1755             Tb = float(np.clip(
1756                 Tb + (dt_travel / 1800.0) * (env['Tamb_C'] - Tb)
1757                 + (float(out['losses_int']) * dt_travel) / 50000.0,
1758                 p.T_min,
1759                 p.T_max,
1760             ))
1761             segments.append(
1762                 {
1763                     'v_kmh': v_k,
1764                     'dt_sec': float(dt_travel),
1765                     'ds_km': ds_step_km,
1766                     's_start_km': float(s_km),
1767                     's_end_km': float(s_km + ds_step_km),
1768                     'schedule_wait_before_sec': float(schedule_wait),
1769                 }
1770             )
1771             t_utc = t_utc + timedelta(seconds=dt_travel)
1772             s_km += ds_step_km
1773             t_utc, z, Tb, control_wait, _ = apply_control_stop_at(t_utc, z, Tb, s_km)
1774             segments[-1]['control_stop_dwell_after_sec'] = float(control_wait)
1775             v_prev_seg = 0.0 if control_wait > 1.0e-9 else v_k
1776
1777     v0_kmh = float(segments[0]['v_kmh']) if segments else float(self.v_cmd)
1778     return v0_kmh, segments, u_seq
1779
1780 def _publish_upper_plan(self):
1781     if not self.v_plan_kmh:
1782         return
1783     msg = Float32MultiArray()
1784     msg.data = [float(self.plan_dt_sec)] + [float(v) for v in self.v_plan_kmh]
1785     self.pub_plan.publish(msg)
1786
1787 def _publish_lower_plan(self, v_seq_ms):
1788     if not v_seq_ms:
1789         return
1790     msg = Float32MultiArray()
1791     msg.data = [float(self.lower_dt)] + [float(v * 3.6) for v in v_seq_ms]
1792     self.pub_lower_plan.publish(msg)
1793
1794 def _publish_plan_path(self, data):
1795     if len(data) == 0:
1796         return
1797     path = Path()
1798     path.header.stamp = self.get_clock().now().to_msg()
1799     path.header.frame_id = 'map'
1800     s_tmp = float(self.s_km)
1801     if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
1802         s_tmp = float(self.s_meas)
1803     if self.upper_plan_mode == 'distance' and self.v_plan_segments:
1804         for seg in self.v_plan_segments:
1805             s_tmp += float(seg['v_kmh']) * (seg['dt_sec'] / 3600.0)
1806             pose = PoseStamped()
1807             pose.header = path.header
1808             pose.pose.position.x = s_tmp
1809             pose.pose.position.y = 0.0
1810             path.poses.append(pose)
1811     else:
1812         v_list = self.v_plan_kmh if self.v_plan_kmh else [float(self.v_upper_cmd)] * len(data)
1813         for v_kmh in v_list[:len(data)]:
1814             s_tmp += float(v_kmh) * (self.model.p.dt / 3600.0)
1815             pose = PoseStamped()
1816             pose.header = path.header
1817             pose.pose.position.x = s_tmp
1818             pose.pose.position.y = 0.0

```

```

1819         path.poses.append(pose)
1820     self.pub_path.publish(path)
1821
1822 def _publish_metrics(self, d0: dict, v_exec_kmh: float, s_for_profile: float):
1823     slope_pct = d0.get('slope_pct', 0.0)
1824     if self.route_profile is not None:
1825         slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
1826     headwind_ms = d0.get('headwind_ms', 0.0)
1827     if self.route_profile is not None:
1828         headwind_ms = self._route_value(s_for_profile, 'headwind_ms', headwind_ms)
1829     out = self.model.electrical_balance(
1830         v_exec_kmh / 3.6,
1831         slope_pct,
1832         self.z,
1833         self.Tb,
1834         d0.get('G_poa', 0.0),
1835         d0.get('Tcell_C', 40.0),
1836         headwind_ms=headwind_ms,
1837         ambient_temp_c=d0.get('Tamb_C'),
1838         elevation_m=d0.get('elevation_m', self._route_value(s_for_profile, 'elev_m', 0.0)),
1839     )
1840     V = float(out['V'])
1841     I = float(out['I'])
1842     P_pv = float(out['P_pv'])
1843     P_pack = float(out['P_pack'])
1844     P_mech_wheel = float(out.get('P_mech_wheel', out.get('P_mech', 0.0)))
1845     P_motor_elec = float(out.get('P_dc_to_drv', 0.0)) - float(out.get('P_reg_to_dc', 0.0))
1846     I_motor = P_motor_elec / V if abs(V) > 1e-3 else 0.0
1847     msg = Float32MultiArray()
1848     msg.data = [
1849         float(V),
1850         float(I),
1851         float(self.z),
1852         float(P_motor_elec),
1853         float(I_motor),
1854         float(P_pv),
1855         float(v_exec_kmh),
1856         float(P_mech_wheel),
1857         float(P_pack),
1858     ]
1859     self.pub_metrics.publish(msg)
1860
1861 def _publish_summary(self, _v_exec_kmh: float = math.nan) -> None:
1862     """Publish current state even while a long upper solve is in progress."""
1863     mode = str(self.forecast_time_mode).lower()
1864     has_time = ('time' in self.df.columns) and (not self.df['time'].isna().all())
1865     if mode == 'auto':
1866         mode = 'absolute' if has_time else 'relative'
1867     if has_time and mode == 'absolute':
1868         if self.k + 1 < len(self.df) and pd.notna(self.df['time'].iloc[self.k + 1]):
1869             t_next = self.df['time'].iloc[self.k + 1].to_pydatetime()
1870             sec_to_next = max(0.0, (t_next - datetime.now(timezone.utc)).total_seconds())
1871         else:
1872             sec_to_next = 0.0
1873     else:
1874         now = datetime.now(timezone.utc) + timedelta(seconds=self.forecast_time_offset)
1875         elapsed = max(0.0, (now - self.forecast_start_time).total_seconds())
1876         sec_to_next = max(0.0, self.model.p.dt - (elapsed % self.model.p.dt))
1877     stop_remaining_sec = 0.0
1878     if self.control_stop_end_monotonic is not None:
1879         stop_remaining_sec = max(0.0, self.control_stop_end_monotonic - time.monotonic())
1880     st = Float32MultiArray()
1881     st.data = [
1882         float(self.z),
1883         float(self.Tb),
1884         float(self.s_kmh),
1885         float(self.k),
1886         float(sec_to_next),
1887         float(bool(self.control_stop_hold)),
1888         float(stop_remaining_sec),
1889         float(len(self.completed_control_stops)),
1890     ]
1891     self.pub_status.publish(st)
1892
1893 def _interp_upper_speed(self, t_sec: float) -> float:
1894     if self.upper_plan_mode == 'distance' and self.v_plan_segments:
1895         acc = 0.0
1896         for seg in self.v_plan_segments:
1897             acc_next = acc + seg['dt_sec']
1898             if t_sec <= acc_next:
1899                 return float(seg['v_kmh'])
1900             acc = acc_next
1901         return float(self.v_plan_segments[-1]['v_kmh'])
1902     if not self.v_plan_kmh:
1903         return float(self.v_upper_cmd)
1904     dt = float(self.plan_dt_sec)
1905     if dt <= 0.0:
1906         return float(self.v_plan_kmh[0])
1907     idx = t_sec / dt
1908     i = int(math.floor(idx))
1909     if i <= 0:
1910         return float(self.v_plan_kmh[0])
1911     if i >= len(self.v_plan_kmh) - 1:
1912         return float(self.v_plan_kmh[-1])
1913     alpha = idx - i
1914     return float((1.0 - alpha) * self.v_plan_kmh[i] + alpha * self.v_plan_kmh[i + 1])
1915
1916 def _distance_plan_speed(self, s_kmh: float) -> float:
1917     idx = plan_segment_index(self.v_plan_segments or [], s_kmh)
1918     if idx < 0:
1919         return float(self.v_upper_cmd)
1920     return float(self.v_plan_segments[idx]['v_kmh'])
1921

```

```

922 def _tau_max_for_mode(self, maps, mode: str) -> float:
923     key = mode if mode in maps else 'default'
924     try:
925         return float(max(maps[key][1]))
926     except Exception:
927         return 0.0
928
929 def _tau_limits(self):
930     mode = str(self.model.drive_mode or 'default').lower()
931     if mode in ('eco', 'power'):
932         tau_drive = self._tau_max_for_mode(self.model.maps_drive, mode)
933         tau_regen = self._tau_max_for_mode(self.model.maps_regen, mode)
934     else:
935         tau_drive = self._tau_max_for_mode(self.model.maps_drive, 'power') if 'power' in self.model.maps_drive else \
936             self._tau_max_for_mode(self.model.maps_drive, 'eco') if 'eco' in self.model.maps_drive else \
937             self._tau_max_for_mode(self.model.maps_drive, 'default')
938         tau_regen = self._tau_max_for_mode(self.model.maps_regen, 'power') if 'power' in self.model.maps_regen else \
939             self._tau_max_for_mode(self.model.maps_regen, 'eco') if 'eco' in self.model.maps_regen else \
940             self._tau_max_for_mode(self.model.maps_regen, 'default')
941     if tau_regen <= 0.0:
942         tau_regen = tau_drive
943     return tau_drive, tau_regen
944
945 def _traction_force(self, tau_nm: float) -> float:
946     p = self.model.p
947     wheel_r = max(1e-3, float(p.wheel_radius))
948     motor_count = int(p.motor_count) if int(p.motor_count) > 0 else int(p.driven_wheel_count or 1)
949     motor_count = max(1, motor_count)
950     return float(tau_nm) * float(p.gear_ratio) * float(p.gear_eta) * motor_count / wheel_r
951
952 def _pack_from_tau(self, v_ms: float, tau_nm: float, z: float, Tb: float, env: dict):
953     p = self.model.p
954     wheel_r = max(1e-3, float(p.wheel_radius))
955     motor_count = int(p.motor_count) if int(p.motor_count) > 0 else int(p.driven_wheel_count or 1)
956     motor_count = max(1, motor_count)
957     omega_m = float(v_ms) / wheel_r * float(p.gear_ratio)
958     P_mech_motor = float(tau_nm) * omega_m * motor_count
959     if tau_nm >= 0.0:
960         eff = float(self.model.eff_drive(v_ms, tau_nm))
961         eff = max(1.0e-3, eff)
962         P_elec = P_mech_motor / (eff * float(p.inverter_eta))
963     else:
964         eff = float(self.model.eff_regen(v_ms, tau_nm))
965         eff = max(1.0e-3, eff)
966         P_elec = P_mech_motor * eff * float(p.inverter_eta)
967     P_pv = float(self.model.pv_power_mpp(env['G_poa'], env['Tcell_C']))
968     P_pack = P_elec + float(p.P_aux) - P_pv
969     iv = self.model.battery_iv(P_pack, z, Tb)
970     I = float(iv['I'])
971     V = float(iv['V'])
972     Rint = float(iv['Rint'])
973     loss_int = I * I * Rint
974     return dict(P_pack=P_pack, I=I, V=V, loss_int=loss_int, eff=eff, P_pv=P_pv, P_elec=P_elec)
975
976 def _build_lower_ref(self, base_time_utc, s_km: float, d0: dict):
977     ref = []
978     s_tmp = float(s_km)
979     offset = 0.0
980     if self.plan_start_monotonic is not None:
981         offset = max(0.0, time.monotonic() - self.plan_start_monotonic)
982     soc_guard = float(self.get_parameter('soc_guard_margin').value)
983     guard_speed = None
984     if self.z <= (self.model.p.soc_min + soc_guard):
985         guard_speed = self._soc_guard_speed(self.v_upper_cmd, s_tmp, d0)
986     for i in range(max(1, self.lower_N)):
987         t_sec = offset + i * self.lower_dt
988         v_ref = self._interp_upper_speed(t_sec)
989         if guard_speed is not None:
990             v_ref = min(v_ref, guard_speed)
991         vmax_local = self._speed_limit_at(s_tmp, self.get_parameter('v_max_kmh').value)
992         v_ref = min(v_ref, vmax_local)
993         if self.drive_schedule is not None and base_time_utc is not None:
994             t_utc = base_time_utc + timedelta(seconds=t_sec)
995             limits = self.drive_schedule.speed_limits(t_utc)
996             if limits is not None:
997                 vmin_kmh, vmax_kmh = limits
998                 v_ref = float(np.clip(v_ref, vmin_kmh, vmax_kmh))
999             ref.append(float(v_ref) / 3.6)
1000         s_tmp += float(v_ref) * (self.lower_dt / 3600.0)
1001     return ref
1002
1003 def _lower_rollout(self, v0_ms: float, u_seq: np.ndarray, env: dict, z0: float, Tb0: float,
1004                   tau_drive: float, tau_regen: float):
1005     p = self.model.p
1006     v = float(v0_ms)
1007     z = float(z0)
1008     Tb = float(Tb0)
1009     v_seq = []
1010     for u in u_seq:
1011         u = float(u)
1012         if u >= 0.0:
1013             tau = u * tau_drive
1014         else:
1015             tau = u * tau_regen
1016         forces = self.model.resistive_forces(
1017             v, env['slope_pct'], env['headwind_ms'],
1018             ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0),
1019         )
1020         F_res = float(forces['F_total'])
1021         F_trac = self._traction_force(tau)
1022         a = (F_trac - F_res) / float(p.m)
1023         v = max(0.0, v + a * self.lower_dt)
1024         pack = self._pack_from_tau(v, tau, z, Tb, env)

```

```

2025         z = self.model.soc_step(
2026             z,
2027             float(pack['P_pack']),
2028             self.lower_dt,
2029             current_a=float(pack['I']),
2030             Tbat_C=Tb,
2031         )
2032         Tb = Tb + (self.lower_dt / 1800.0) * (env['Tamb_C'] - Tb) + (float(pack['loss_int']) * self.lower_dt) / 50000.0
2033         v_seq.append(float(v))
2034     return v_seq
2035
2036 def _lower_mpc_solve(self, v0_ms: float, s0_km: float, z0: float, Tb0: float, env: dict, v_ref_seq):
2037     N = max(1, min(self.lower_N, len(v_ref_seq)))
2038     if N <= 0:
2039         return np.zeros(1), [v0_ms], 'eco'
2040     v_ref_seq = v_ref_seq[:N]
2041     tau_drive, tau_regen = self._tau_limits()
2042     p = self.model.p
2043     w_track = float(self.w_track)
2044     w_throttle = float(self.w_throttle)
2045     w_current = float(self.get_parameter('w_current').value)
2046     w_T = float(self.get_parameter('w_T').value)
2047     rate_lim = float(self.throttle_rate_limit) / 100.0 if self.throttle_rate_limit > 0.0 else 0.0
2048     u0 = float(np.clip(self.lower_last_u, -1.0, 1.0))
2049     bounds = [(-1.0, 1.0)] * N
2050
2051     # Inverse dynamics gives a physically meaningful, bounded horizon seed in
2052     # deterministic time. It also remains the safe lower policy while the
2053     # hourly full-race optimizer is using the other executor thread.
2054     seed = []
2055     v_seed = float(v0_ms)
2056     z_seed = float(z0)
2057     tb_seed = float(Tb0)
2058     u_prev_seed = u0
2059     motor_count = int(p.motor_count) if int(p.motor_count) > 0 else int(p.driven_wheel_count or 1)
2060     motor_count = max(1, motor_count)
2061     torque_factor = (
2062         max(1.0e-3, float(p.wheel_radius))
2063         / max(1.0e-6, float(p.gear_ratio) * float(p.gear_eta) * motor_count)
2064     )
2065     max_du = rate_lim * max(self.lower_dt, 1.0e-3) if rate_lim > 0.0 else 2.0
2066     for v_ref in v_ref_seq:
2067         forces = self.model.resistive_forces(
2068             v_seed, env['slope_pct'], env['headwind_ms'],
2069             ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0),
2070         )
2071         desired_accel = (float(v_ref) - v_seed) / max(self.lower_dt, 1.0e-3)
2072         tau_request = (float(p.m) * desired_accel + float(forces['F_total'])) * torque_factor
2073         tau_limit = tau_drive if tau_request >= 0.0 else tau_regen
2074         u_request = tau_request / max(abs(float(tau_limit)), 1.0e-6)
2075         u_request = float(np.clip(u_request, u_prev_seed - max_du, u_prev_seed + max_du))
2076         u_request = float(np.clip(u_request, -1.0, 1.0))
2077
2078         # Search the slew-limited scalar control interval deterministically.
2079         # Merely shrinking toward zero is unsafe at high SoC: if PV causes
2080         # over-voltage, *more* positive traction power is needed to create
2081         # headroom. The grid handles both discharge and charge violations.
2082         u_min = max(-1.0, u_prev_seed - max_du)
2083         u_max = min(1.0, u_prev_seed + max_du)
2084         candidate_controls = np.unique(np.concatenate((
2085             np.linspace(u_min, u_max, 25, dtype=float),
2086             np.array([u_request, np.clip(0.0, u_min, u_max)], dtype=float),
2087         )))
2088         best_feasible = None
2089         best_feasible_score = float('inf')
2090         best_fallback = float(np.clip(u_request, u_min, u_max))
2091         best_violation_score = float('inf')
2092         for candidate_u in candidate_controls:
2093             candidate_u = float(np.clip(candidate_u, u_min, u_max))
2094             tau = candidate_u * (tau_drive if candidate_u >= 0.0 else tau_regen)
2095             F_trac = self._traction_force(tau)
2096             v_trial = max(
2097                 0.0,
2098                 v_seed
2099                 + ((F_trac - float(forces['F_total'])) / float(p.m)) * self.lower_dt,
2100             )
2101             pack = self._pack_from_tau(v_trial, tau, z_seed, tb_seed, env)
2102             current = float(pack['I'])
2103             voltage = float(pack['V'])
2104             tracking_score = (
2105                 (v_trial - float(v_ref)) ** 2
2106                 + 0.01 * (candidate_u - u_request) ** 2
2107             )
2108             feasible = (
2109                 float(p.I_chg_min) <= current <= float(p.I_max)
2110                 and float(p.V_min) <= voltage <= float(p.V_max)
2111             )
2112             if feasible and tracking_score < best_feasible_score:
2113                 best_feasible = candidate_u
2114                 best_feasible_score = tracking_score
2115             violation_score = (
2116                 (max(0.0, current - float(p.I_max)) / max(abs(float(p.I_max)), 1.0)) ** 2
2117                 + (max(0.0, float(p.I_chg_min) - current) / max(abs(float(p.I_chg_min)), 1.0)) ** 2
2118                 + (max(0.0, float(p.V_min) - voltage) / max(abs(float(p.V_min)), 1.0)) ** 2
2119                 + (max(0.0, voltage - float(p.V_max)) / max(abs(float(p.V_max)), 1.0)) ** 2
2120                 + 1.0e-3 * tracking_score
2121             )
2122             if violation_score < best_violation_score:
2123                 best_violation_score = violation_score
2124                 best_fallback = candidate_u
2125         u_request = float(best_feasible if best_feasible is not None else best_fallback)
2126
2127     seed.append(u_request)

```

```

1128     tau = u_request * (tau_drive if u_request >= 0.0 else tau_regen)
1129     F_trac = self._traction_force(tau)
1130     v_seed = max(
1131         0.0,
1132         v_seed + ((F_trac - float(forces['F_total'])) / float(p.m)) * self.lower_dt,
1133     )
1134     pack = self._pack_from_tau(v_seed, tau, z_seed, tb_seed, env)
1135     z_seed = self.model.soc_step(
1136         z_seed,
1137         float(pack['P_pack']),
1138         self.lower_dt,
1139         current_a=float(pack['I']),
1140         Tbat_C=tb_seed,
1141     )
1142     tb_seed += (
1143         (self.lower_dt / 1800.0) * (env['Tamb_C'] - tb_seed)
1144         + (float(pack['loss_int']) * self.lower_dt) / 50000.0
1145     )
1146     u_prev_seed = u_request
1147     x0 = np.asarray(seed, dtype=float)
1148
1149     def cost(u_vec):
1150         v = float(v0_ms)
1151         z = float(z0)
1152         Tb = float(Tb0)
1153         u_prev = u0
1154         J = 0.0
1155         for i in range(N):
1156             u = float(u_vec[i])
1157             if u >= 0.0:
1158                 tau = u * tau_drive
1159             else:
1160                 tau = u * tau_regen
1161             forces = self.model.resistive_forces(
1162                 v, env['slope_pct'], env['headwind_ms'],
1163                 ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0),
1164             )
1165             F_res = float(forces['F_total'])
1166             F_trac = self._traction_force(tau)
1167             a = (F_trac - F_res) / float(p.m)
1168             v = max(0.0, v + a * self.lower_dt)
1169             pack = self._pack_from_tau(v, tau, z, Tb, env)
1170             P_pack = float(pack['P_pack'])
1171             I = float(pack['I'])
1172             V = float(pack['V'])
1173             loss_int = float(pack['loss_int'])
1174             z = self.model.soc_step(
1175                 z,
1176                 P_pack,
1177                 self.lower_dt,
1178                 current_a=I,
1179                 Tbat_C=Tb,
1180             )
1181             Tb = Tb + (self.lower_dt / 1800.0) * (env['Tamb_C'] - Tb) + (loss_int * self.lower_dt) / 50000.0
1182
1183             v_ref = float(v_ref_seq[i])
1184             J += w_track * (v - v_ref) ** 2
1185             J += w_throttle * (u ** 2)
1186             du = (u - u_prev) / max(self.lower_dt, 1.0e-3)
1187             if rate_lim > 0.0:
1188                 J += w_throttle * max(0.0, abs(du) - rate_lim) ** 2
1189             J += w_current * (I ** 2) * self.lower_dt
1190
1191             # soft constraints
1192             J += 1e4 * max(0.0, I - p.I_max) ** 2
1193             J += 1e4 * max(0.0, p.I_chg_min - I) ** 2
1194             J += 1e4 * max(0.0, p.V_min - V) ** 2
1195             J += 1e4 * max(0.0, V - p.V_max) ** 2
1196             J += w_T * max(0.0, Tb - p.T_max) ** 2
1197             J += w_T * max(0.0, p.T_min - Tb) ** 2
1198             J += 1e4 * max(0.0, p.soc_min - z) ** 2
1199             J += 1e4 * max(0.0, z - p.soc_max) ** 2
1200             u_prev = u
1201         return J
1202
1203     skip_refine = bool(
1204         self._upper_solve_in_progress and self.lower_skip_optimization_during_upper_solve
1205     )
1206     if skip_refine or self.lower_max_iter <= 0:
1207         self.last_lower_solve_ok = True
1208         u_seq = x0
1209         if skip_refine and not self._lower_upper_busy_logged:
1210             self.get_logger().info(
1211                 'Upper MPC is solving; lower MPC is using its bounded inverse-dynamics horizon.'
1212             )
1213             self._lower_upper_busy_logged = True
1214     else:
1215         res = minimize(
1216             cost,
1217             x0,
1218             method='L-BFGS-B',
1219             bounds=bounds,
1220             options=dict(maxiter=self.lower_max_iter, maxfun=self.lower_maxfun, ftol=1.0e-6),
1221         )
1222         if np.all(np.isfinite(res.x)):
1223             self.last_lower_solve_ok = bool(res.success)
1224             u_seq = res.x
1225             if not res.success:
1226                 self.get_logger().warn('Lower MPC reached an early stop; using best finite iterate.')
1227         else:
1228             self.last_lower_solve_ok = False
1229             self.get_logger().warn('Lower MPC solve failed; using inverse-dynamics horizon.')
1230             u_seq = x0

```

```

2231     v_pred = self._lower_rollout(v0_ms, u_seq, env, z0, Tb0, tau_drive, tau_regen)
2232     u0_cmd = float(u_seq[0]) if len(u_seq) > 0 else 0.0
2233     tau0 = u0_cmd * (tau_drive if u0_cmd >= 0.0 else tau_regen)
2234     mode = self.model.select_drive_mode(v0_ms, tau0)
2235     return u_seq, v_pred, mode
2236
2237 def _step_solar(self):
2238     self._maybe_reload_forecast()
2239     self._sync_measured_state()
2240     s_for_stop = float(self.s_km)
2241     if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2242         s_for_stop = float(self.s_meas)
2243     v_for_stop = float(self.v_upper_cmd)
2244     if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
2245         v_for_stop = float(self.v_now)
2246     self.control_stop_hold = self._update_live_control_stop_hold(s_for_stop, v_for_stop)
2247     k_now = self._current_bin_index()
2248     moved_to_new_bin = (self.last_bin is None) or (k_now != self.last_bin)
2249     self.k = k_now
2250
2251     data = self._horizon_data(self.k)
2252     if self.control_stop_hold and data:
2253         env_stop = self._forecast_at_time(
2254             data[0].get('t_utc', datetime.now(timezone.utc)),
2255             s_for_stop,
2256             drive=False,
2257             control_stop=True,
2258         )
2259         env_stop['slope_pct'] = self._route_value(s_for_stop, 'slope_pct', 0.0)
2260         env_stop['t_utc'] = data[0].get('t_utc', datetime.now(timezone.utc))
2261         data[0] = env_stop
2262     self.last_data = data
2263     if len(data) > 0:
2264         d0 = data[0]
2265         s_for_profile = self.s_km
2266         if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2267             s_for_profile = float(self.s_meas)
2268         slope_pct = d0.get('slope_pct', 0.0)
2269         if self.route_profile is not None:
2270             slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
2271         headwind_ms = d0.get('headwind_ms', 0.0)
2272         if self.route_profile is not None:
2273             headwind_ms = self._route_value(s_for_profile, 'headwind_ms', headwind_ms)
2274         env_msg = Float32MultiArray()
2275         env_msg.data = [
2276             float(d0.get('G_poa', 0.0)),
2277             float(d0.get('Tcell_C', 40.0)),
2278             float(d0.get('Tamb_C', 30.0)),
2279             float(slope_pct),
2280             float(headwind_ms),
2281         ]
2282         self.pub_env.publish(env_msg)
2283         v_exec_kmh = self.v_upper_cmd
2284         if self.hierarchical and self.timer_lower is not None:
2285             v_exec_kmh = self.v_lower_cmd
2286         if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
2287             v_exec_kmh = float(self.v_now)
2288         self._publish_metrics(d0, v_exec_kmh, s_for_profile)
2289         self._publish_summary(v_exec_kmh)
2290         need_plan = (self.v_plan_kmh is None) or (self.forecast_reloaded and self.replan_on_forecast_reload)
2291         if self.upper_mode == 'distance' and self.v_plan_segments is None:
2292             need_plan = True
2293         if self.upper_mode == 'distance' and self.v_plan_segments:
2294             plan_end_kmh = float(self.v_plan_segments[-1].get('s_end_kmh', self.s_km))
2295             if self.s_km >= plan_end_kmh - 1.0e-6:
2296                 need_plan = True
2297         if self.upper_replan_kmh > 0.0 and self.upper_plan_s_kmh is not None:
2298             if (self.s_km - self.upper_plan_s_kmh) >= self.upper_replan_kmh:
2299                 need_plan = True
2300         if self.upper_replan_sec > 0.0 and self.upper_plan_time is not None:
2301             if data and data[0].get('t_utc') and (data[0]['t_utc'] - self.upper_plan_time).total_seconds() >= self.upper_replan_sec:
2302                 need_plan = True
2303         if (self.upper_replan_kmh <= 0.0) and (self.upper_replan_sec <= 0.0):
2304             need_plan = need_plan or moved_to_new_bin
2305         if (
2306             need_plan
2307             and self.upper_mode == 'distance'
2308             and self.v_plan_segments
2309             and self.drive_schedule is not None
2310             and len(data) > 0
2311             and data[0].get('t_utc')
2312             and not self.drive_schedule.is_drive_time(data[0]['t_utc'])
2313         ):
2314             plan_end_kmh = float(self.v_plan_segments[-1].get('s_end_kmh', self.s_km))
2315             if self.s_km < plan_end_kmh - 1.0e-6:
2316                 need_plan = False
2317         if need_plan and len(data) > 0:
2318             s_for_profile = self.s_kmh
2319             if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2320                 s_for_profile = float(self.s_meas)
2321             t0 = data[0].get('t_utc', datetime.now(timezone.utc))
2322             upper_started = time.monotonic()
2323             self._upper_solve_in_progress = True
2324             try:
2325                 if self.upper_mode == 'distance':
2326                     self.v_upper_cmd, self.v_plan_segments, self.upper_plan_seq = self._mpc_solve_solar_distance(
2327                         t0, s_for_profile, self.upper_plan_seq
2328                     )
2329                 self.upper_plan_mode = 'distance'
2330                 self.plan_dt_sec = float(self.model.p.dt)
2331                 self.v_plan_kmh = self._sample_plan_segments(self.plan_dt_sec)
2332                 self.upper_plan_s_kmh = float(s_for_profile)
2333                 self.upper_plan_time = t0

```

```

2334         self.upper_plan_id += 1
2335     else:
2336         self.v_upper_cmd, self.v_plan_kmh = self._mpc_solve_solar(data)
2337         self.v_plan_segments = None
2338         self.upper_plan_mode = 'time'
2339         self.plan_dt_sec = float(self.model.p.dt)
2340     finally:
2341         self._upper_solve_in_progress = False
2342     if not self._upper_solve_logged:
2343         self.get_logger().info(
2344             f'Upper MPC first solve completed in {time.monotonic() - upper_started:.3f}s.'
2345         )
2346         self._upper_solve_logged = True
2347         self.plan_start_monotonic = time.monotonic()
2348         self.last_plan_time = time.monotonic()
2349         self.forecast_reloaded = False
2350 elif self.v_plan_kmh is None and len(data) > 0:
2351     self.v_plan_kmh = [float(self.v_upper_cmd)] * len(data)
2352
2353 if self.upper_plan_mode == 'distance' and self.v_plan_segments:
2354     s_for_plan = self.s_km
2355     distance_meas_kmh = self._measured_distance_kmh()
2356     if bool(self.get_parameter('use_measured_s').value) and math.isfinite(distance_meas_kmh):
2357         s_for_plan = float(distance_meas_kmh)
2358     self.v_upper_cmd = self._distance_plan_speed(s_for_plan)
2359 elif self.upper_plan_mode == 'time' and self.plan_start_monotonic is not None:
2360     t_sec = max(0.0, time.monotonic() - self.plan_start_monotonic)
2361     self.v_upper_cmd = self._interp_upper_speed(t_sec)
2362
2363 # hard schedule enforcement (current step)
2364 hard_stop = bool(self.control_stop_hold)
2365 if self.drive_schedule is not None and len(data) > 0 and 't_utc' in data[0]:
2366     limits = self.drive_schedule.speed_limits(data[0]['t_utc'])
2367     if limits is not None:
2368         vmin_kmh, vmax_kmh = limits
2369         self.v_upper_cmd = float(np.clip(self.v_upper_cmd, vmin_kmh, vmax_kmh))
2370         if vmax_kmh <= 0.0:
2371             hard_stop = True
2372
2373 # SoC guard
2374 if len(data) > 0:
2375     soc_guard = float(self.get_parameter('soc_guard_margin').value)
2376     if self.z <= (self.model.p.soc_min + soc_guard):
2377         s_for_profile = self.s_km
2378         if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2379             s_for_profile = float(self.s_meas)
2380         self.v_upper_cmd = self._soc_guard_speed(self.v_upper_cmd, s_for_profile, data[0])
2381 if hard_stop:
2382     self.v_upper_cmd = 0.0
2383
2384 if self.v_plan_kmh:
2385     self.v_plan_kmh[0] = float(self.v_upper_cmd)
2386
2387 self.pub_upper_speed.publish(Float32(data=float(self.v_upper_cmd)))
2388 self._publish_upper_plan()
2389
2390 if not self.hierarchical or self.timer_lower is None:
2391     self.v_cmd = float(self.v_upper_cmd)
2392     self.pub_speed.publish(Float32(data=self.v_cmd))
2393 else:
2394     self.v_cmd = float(self.v_lower_cmd)
2395
2396 self._publish_plan_path(data)
2397
2398 if moved_to_new_bin and len(data) > 0:
2399     # The upper solve may have occupied this callback for many seconds.
2400     # Refresh state from telemetry received concurrently before the
2401     # estimator/model update and before publishing planner status.
2402     self._sync_measured_state()
2403     meas_now_sec = self.get_clock().now().nanoseconds / 1e9
2404     soc_meas_fresh = fresh_enough(
2405         self.soc_meas_time, meas_now_sec, self.battery_meas_timeout_sec
2406     )
2407     tb_meas_fresh = fresh_enough(
2408         self.tb_meas_time, meas_now_sec, self.battery_meas_timeout_sec
2409     )
2410     current_meas_fresh = fresh_enough(
2411         self.current_meas_time, meas_now_sec, self.battery_meas_timeout_sec
2412     )
2413     voltage_meas_fresh = fresh_enough(
2414         self.voltage_meas_time, meas_now_sec, self.battery_meas_timeout_sec
2415     )
2416     d0 = data[0]
2417     s_for_profile = self.s_km
2418     if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2419         s_for_profile = float(self.s_meas)
2420     slope_pct = d0['slope_pct']
2421     if self.route_profile is not None:
2422         slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
2423     headwind_ms = d0.get('headwind_ms', 0.0)
2424     if self.route_profile is not None:
2425         headwind_ms = self._route_value(s_for_profile, 'headwind_ms', headwind_ms)
2426     v_exec_kmh = self.v_upper_cmd
2427     if self.hierarchical and self.timer_lower is not None:
2428         v_exec_kmh = self.v_lower_cmd
2429     if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
2430         v_exec_kmh = float(self.v_now)
2431     v_exec_ms = v_exec_kmh / 3.6
2432     inertial_power_w = 0.0
2433     if self.model_prev_speed_ms is not None:
2434         inertial_power_w = 0.5 * self.model.p.m * (
2435             v_exec_ms * v_exec_ms - self.model_prev_speed_ms * self.model_prev_speed_ms
2436         ) / max(self.model.p.dt, 1.0e-9)

```

```

2437         self.model_prev_speed_ms = float(v_exec_ms)
2438
2439         u = MheInput(
2440             v_ms=v_exec_ms,
2441             slope_pct=slope_pct,
2442             G_poa=d0['G_poa'],
2443             Tcell_C=d0['Tcell_C'],
2444             Tamb_C=d0['Tamb_C'],
2445             headwind_ms=headwind_ms,
2446             dt=self.model.p.dt,
2447             inertial_power_w=inertial_power_w,
2448             elevation_m=d0.get('elevation_m', self._route_value(s_for_profile, 'elev_m', 0.0)),
2449         )
2450         meas = MheMeas(
2451             soc=self.solar_soc_meas if soc_meas_fresh and np.isfinite(self.solar_soc_meas) else None,
2452             Tb=self.solar_tb_meas if tb_meas_fresh and np.isfinite(self.solar_tb_meas) else None,
2453             I=self.solar_i_meas if current_meas_fresh and np.isfinite(self.solar_i_meas) else None,
2454             V=self.solar_v_meas if voltage_meas_fresh and np.isfinite(self.solar_v_meas) else None,
2455         )
2456         if self.mhe is not None:
2457             self.mhe.push(u, meas)
2458             self.z, self.Tb = self.mhe.estimate(self.z, self.Tb)
2459         else:
2460             out = self.model.electrical_balance(
2461                 u.v_ms,
2462                 u.slope_pct,
2463                 self.z,
2464                 self.Tb,
2465                 u.G_poa,
2466                 u.Tcell_C,
2467                 headwind_ms=u.headwind_ms,
2468                 inertial_power_w=u.inertial_power_w,
2469                 ambient_temp_c=u.Tamb_C,
2470                 elevation_m=d0.get('elevation_m', self._route_value(s_for_profile, 'elev_m', 0.0)),
2471             )
2472             P_pack = float(out['P_pack'])
2473             loss_int = float(out['losses_int'])
2474             if not soc_meas_fresh:
2475                 self.z = self.model.soc_step(
2476                     self.z,
2477                     P_pack,
2478                     self.model.p.dt,
2479                     current_a=float(out['I']),
2480                     Tbat_C=self.Tb,
2481                 )
2482             if not tb_meas_fresh:
2483                 self.Tb = self.Tb + (self.model.p.dt / 1800.0) * (u.Tamb_C - self.Tb) + (loss_int * self.model.p.dt) / 50000.0
2484             self.z = float(np.clip(self.z, self.model.p.soc_min, self.model.p.soc_max))
2485             self.Tb = float(np.clip(self.Tb, self.model.p.T_min, self.model.p.T_max))
2486
2487             if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2488                 self.s_km = float(self.s_meas)
2489             else:
2490                 self.s_km += float(v_exec_kmh) * (self.model.p.dt / 3600.0)
2491             self.last_bin = k_now
2492
2493         self._publish_summary()
2494
2495     def _step_lower(self):
2496         if not self.hierarchical or self.timer_lower is None:
2497             return
2498         self._publish_lower_command_cycle()
2499         if not self.last_data:
2500             return
2501         d0 = self.last_data[0]
2502         s_for_profile = self.s_km
2503         if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
2504             s_for_profile = float(self.s_meas)
2505         slope_pct = d0.get('slope_pct', 0.0)
2506         if self.route_profile is not None:
2507             slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
2508         headwind_ms = d0.get('headwind_ms', 0.0)
2509         if self.route_profile is not None:
2510             headwind_ms = self._route_value(s_for_profile, 'headwind_ms', headwind_ms)
2511         env = dict(
2512             slope_pct=float(slope_pct),
2513             headwind_ms=float(headwind_ms),
2514             G_poa=float(d0.get('G_poa', 0.0)),
2515             Tcell_C=float(d0.get('Tcell_C', 40.0)),
2516             Tamb_C=float(d0.get('Tamb_C', 30.0)),
2517         )
2518         base_time = d0.get('t_utc', None)
2519         if base_time is None:
2520             base_time = datetime.now(timezone.utc)
2521         v_ref_seq = self._build_lower_ref(base_time, s_for_profile, d0)
2522
2523         if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
2524             v0_ms = float(self.v_now) / 3.6
2525         else:
2526             v0_ms = float(self.v_lower_cmd) / 3.6
2527             if not np.isfinite(v0_ms) or v0_ms <= 0.0:
2528                 v0_ms = float(self.v_upper_cmd) / 3.6
2529
2530         solve_started = time.monotonic()
2531         u_seq, v_pred, mode = self._lower_mpc_solve(v0_ms, s_for_profile, self.z, self.Tb, env, v_ref_seq)
2532         if not self._lower_solve_logged:
2533             self.get_logger().info(
2534                 f'Lower MPC first solve completed in {time.monotonic() - solve_started:.3f}s.'
2535             )
2536         self._lower_solve_logged = True
2537         v_cmd_ms = v_pred[0] if v_pred else v0_ms
2538         self.v_lower_cmd = float(v_cmd_ms * 3.6)
2539         self.v_cmd = float(self.v_lower_cmd)

```

```

2540         if len(u_seq) > 0:
2541             self.lower_last_u = float(u_seq[0])
2542             self.lower_last_mode = str(mode)
2543             self.lower_last_v_pred = list(v_pred) if v_pred else [self.v_lower_cmd / 3.6]
2544             self._publish_lower_command_cycle()
2545
2546     def _publish_lower_command_cycle(self) -> None:
2547         """Non-blocking 1 Hz output path, independent of both optimizers."""
2548         if not self.hierarchical or self.timer_lower is None:
2549             return
2550         self._sync_measured_state()
2551         speed_kmh = float(np.clip(self.v_lower_cmd, 0.0, self.get_parameter('v_max_kmh').value))
2552         if self.control_stop_hold:
2553             speed_kmh = 0.0
2554         self.pub_speed.publish(Float32(data=speed_kmh))
2555         throttle_pct = float(np.clip(self.lower_last_u * 100.0, -100.0, 100.0))
2556         if self.control_stop_hold:
2557             throttle_pct = min(0.0, throttle_pct)
2558         self.pub_throttle.publish(Float32(data=throttle_pct))
2559         self.pub_drive_mode.publish(String(data=str(self.lower_last_mode)))
2560         self._publish_lower_plan(self.lower_last_v_pred)
2561         self._publish_summary()
2562
2563     def main():
2564         rclpy.init()
2565         node = MPCNode()
2566         executor = MultiThreadedExecutor(num_threads=4)
2567         executor.add_node(node)
2568         try:
2569             executor.spin()
2570         finally:
2571             executor.shutdown()
2572             node.destroy_node()
2573             rclpy.shutdown()

```

A.9 mpc_solarcar/model.py

```

1 import math
2 import numpy as np
3 try:
4     import casadi as ca
5 except ImportError: # Numeric ROS/offline paths do not require CasADi.
6     ca = None
7 from dataclasses import dataclass
8 from .utils_maps import read_eff_map, read_Rint_map, read_map, bilinear_interp, read_1d_map
9
10 def _is_symbolic(x):
11     return ca is not None and (
12         isinstance(x, (ca.SX, ca.MX))
13         or (hasattr(x, 'is_symbolic') and x.is_symbolic())
14     )
15
16 @dataclass
17 class Params:
18     dt: float=600.0
19     rho: float=1.18
20     air_density_mode: str='constant'
21     air_density_reference_pressure_pa: float=101325.0
22     CdA: float=0.13
23     Crr: float=0.002
24     Crr_per_wheel: float=0.0
25     m: float=250.0
26     g: float=9.80665
27     P_aux: float=60.0
28     P_aux_stopped: float=60.0
29     P_aux_night: float=0.0
30     aux_night_ghi_threshold_wm2: float=20.0
31     gear_eta: float=0.98
32     gear_ratio: float=6.0
33     wheel_radius: float=0.28
34     wheel_count: int=4
35     driven_wheel_count: int=2
36     motor_count: int=1
37     motor_type: str='generic'
38     inverter_eta: float=1.0
39     pv_area: float=4.0
40     pv_eta_ref: float=0.23
41     pv_mu_p: float=-0.0045
42     mppt_eta: float=0.95
43     panel_gain: float=1.0
44     # Calibration from the vehicle telemetry solar-power channel to actual
45     # battery-side solar power. It is applied only at telemetry ingestion.
46     solar_measurement_gain_to_pack: float=1.0
47     # Aggregate battery-side limit of the installed MPPT channels. A value
48     # at or below zero disables clipping for vehicles without declared data.
49     pv_power_limit_w: float=0.0
50     E_nom_Wh: float=3055.0
51     Q_nom_Ah: float=0.0
52     V_min: float=260.0
53     V_max: float=400.0
54     I_max: float=120.0
55     I_chg_min: float=-90.0
56     T_max: float=55.0
57     T_min: float=-5.0
58     soc_min: float=0.05
59     soc_max: float=0.98
60     grade_scale: float=1.0
61     drive_eff_scale: float=1.0

```

```

62     regen_eff_scale: float=1.0
63     # Conversion efficiency and commanded/available regeneration are distinct.
64     # Values below one represent coasting or friction braking that bypasses the
65     # electrical regeneration path.
66     regen_utilization: float=1.0
67     rint_scale: float=1.0
68     r_line_ohm: float=0.01
69     r_polarization_ohm: float=0.0
70     polarization_tau_sec: float=60.0
71     eta_charge: float=1.0
72
73 class SolarCarModel:
74     def __init__(self, drive_map_path, regen_map_path, Rint_map_path,
75                  params=None, panel_eff_map_path=None, mppt_eff_map_path=None,
76                  drive_map_eco_path=None, drive_map_power_path=None,
77                  regen_map_eco_path=None, regen_map_power_path=None,
78                  ocv_soc_map_path=None):
79         self.p = params or Params()
80         self.aux_power_override_w = None
81         self.v_grid, self.tau_grid, self.Z_drv = read_eff_map(drive_map_path)
82         self.v_gridR, self.tau_gridR, self.Z_reg = read_eff_map(regen_map_path)
83         self.drive_mode = 'auto'
84         self.drive_mode_default = 'eco'
85         self.drive_mode_tau_margin = 0.0
86         self.maps_drive = {
87             'default': (self.v_grid, self.tau_grid, self.Z_drv),
88         }
89         self.maps_regen = {
90             'default': (self.v_gridR, self.tau_gridR, self.Z_reg),
91         }
92         if drive_map_eco_path:
93             self.maps_drive['eco'] = read_eff_map(drive_map_eco_path)
94         if drive_map_power_path:
95             self.maps_drive['power'] = read_eff_map(drive_map_power_path)
96         if regen_map_eco_path:
97             self.maps_regen['eco'] = read_eff_map(regen_map_eco_path)
98         if regen_map_power_path:
99             self.maps_regen['power'] = read_eff_map(regen_map_power_path)
100         self._update_mode_limits()
101         self.Tg, self.zg, self.Rmap = read_Rint_map(Rint_map_path)
102         self.panel_eff_map = None
103         self.mppt_eff_map = None
104         if panel_eff_map_path:
105             try:
106                 self.Gg, self.Tcg, self.Z_panel = read_map(panel_eff_map_path)
107                 self.panel_eff_map = True
108             except Exception:
109                 self.panel_eff_map = None
110         if mppt_eff_map_path:
111             try:
112                 self.Gm, self.Tm, self.Z_mppt = read_map(mppt_eff_map_path)
113                 self.mppt_eff_map = True
114             except Exception:
115                 self.mppt_eff_map = None
116         self.ocv_soc_map = None
117         if ocv_soc_map_path:
118             try:
119                 self.soc_grid, self.ocv_grid = read_1d_map(ocv_soc_map_path)
120                 self.ocv_soc_map = True
121             except Exception:
122                 self.ocv_soc_map = None
123
124     def eff_drive(self, v_ms, tau_nm):
125         if _is_symbolic(v_ms) or _is_symbolic(tau_nm):
126             v = ca.fabs(v_ms); t = ca.fabs(tau_nm)
127             vN = v/35.0; tN = t/60.0
128             eff = 0.92 - 0.08*vN*vN - 0.06*ca.sqrt(tN+1e-9)
129             eff = eff * float(self.p.drive_eff_scale)
130             return ca.fmin(0.99, ca.fmax(0.55, eff))
131         mode = self._select_mode(float(v_ms), float(abs(tau_nm)))
132         v_grid, t_grid, Z = self.maps_drive.get(mode, self.maps_drive['default'])
133         eff = float(bilinear_interp(v_grid, t_grid, Z, float(v_ms), float(abs(tau_nm))))
134         eff *= float(self.p.drive_eff_scale)
135         return float(np.clip(eff, 0.55, 0.99))
136
137     def eff_regen(self, v_ms, tau_nm):
138         if _is_symbolic(v_ms) or _is_symbolic(tau_nm):
139             v = ca.fabs(v_ms); t = ca.fabs(tau_nm)
140             vN = v/35.0; tN = t/60.0
141             eff = 0.70 + 0.12*vN - 0.05*(tN-0.3)*(tN-0.3)
142             eff = eff * float(self.p.regen_eff_scale or self.p.drive_eff_scale)
143             return ca.fmin(0.95, ca.fmax(0.40, eff))
144         mode = self._select_mode(float(v_ms), float(abs(tau_nm)))
145         v_grid, t_grid, Z = self.maps_regen.get(mode, self.maps_regen['default'])
146         eff = float(bilinear_interp(v_grid, t_grid, Z, float(v_ms), float(abs(tau_nm))))
147         eff *= float(self.p.regen_eff_scale or self.p.drive_eff_scale)
148         return float(np.clip(eff, 0.40, 0.95))
149
150     def _update_mode_limits(self):
151         self.tau_max = {}
152         for k, (_, t_grid, _) in self.maps_drive.items():
153             try:
154                 self.tau_max[k] = float(max(t_grid))
155             except Exception:
156                 self.tau_max[k] = 0.0
157
158     def _select_mode(self, v_ms: float, tau_nm: float) -> str:
159         mode = str(self.drive_mode or 'default').lower()
160         if mode in ('eco', 'power'):
161             return mode
162         # auto
163         eco_max = self.tau_max.get('eco', self.tau_max.get('default', 0.0))
164         margin = float(self.drive_mode_tau_margin or 0.0)

```

```

165         if tau_nm > (eco_max + margin):
166             return 'power' if 'power' in self.maps_drive else 'default'
167         return 'eco' if 'eco' in self.maps_drive else 'default'
168
169     def select_drive_mode(self, v_ms: float, tau_nm: float) -> str:
170         return self._select_mode(v_ms, abs(tau_nm))
171
172     def R_int(self, T_C, z):
173         if _is_symbolic(T_C) or _is_symbolic(z):
174             R0=0.015; R_T=0.0002*(25.0-T_C); R_z=0.01*(1.0-z)
175             return (R0+R_T+R_z) * float(self.p.rint_scale)
176         else:
177             return float(self.p.rint_scale) * float(bilinear_interp(self.Tg, self.zg, self.Rmap, T_C, z))
178
179     def pv_balance(self, G_poa, T_cell_C):
180         symbolic = _is_symbolic(G_poa) or _is_symbolic(T_cell_C)
181         if symbolic:
182             # Map interpolation is numeric. Keep the symbolic optimization
183             # path differentiable by using the map's calibrated reference
184             # model, as is done for the symbolic drive-efficiency path.
185             eta_panel = self.p.pv_eta_ref * (
186                 1.0 + self.p.pv_mu_p * (T_cell_C - 25.0)
187             )
188             eta_panel = ca.fmax(0.0, eta_panel)
189         elif self.panel_eff_map:
190             eta_panel = bilinear_interp(self.Gg, self.Tcg, self.Z_panel, float(G_poa), float(T_cell_C))
191             eta_panel = max(0.0, float(eta_panel))
192         else:
193             eta_panel = self.p.pv_eta_ref*(1.0+self.p.pv_mu_p*(T_cell_C-25.0))
194             eta_panel = max(0.0, float(eta_panel))
195         eta_panel *= float(self.p.panel_gain)
196         P_pv_raw = eta_panel*self.p.pv_area*G_poa
197         if symbolic:
198             eta_mppt = float(self.p.mppt_eta)
199         elif self.mppt_eff_map:
200             eta_mppt = bilinear_interp(self.Gm, self.Tm, self.Z_mppt, float(G_poa), float(T_cell_C))
201             eta_mppt = max(0.0, float(eta_mppt))
202         else:
203             eta_mppt = float(self.p.mppt_eta)
204         P_pv_unlimited = eta_mppt*P_pv_raw
205         power_limit_w = max(0.0, float(self.p.pv_power_limit_w))
206         if power_limit_w > 0.0:
207             if symbolic:
208                 P_pv = ca.fmin(P_pv_unlimited, power_limit_w)
209             else:
210                 P_pv = min(float(P_pv_unlimited), power_limit_w)
211         else:
212             P_pv = P_pv_unlimited
213         return dict(
214             eta_panel=eta_panel,
215             eta_mppt=eta_mppt,
216             P_pv_raw=P_pv_raw,
217             P_pv_unlimited=P_pv_unlimited,
218             P_pv_limit_loss=P_pv_unlimited-P_pv,
219             P_pv=P_pv,
220         )
221
222     def pv_power_mppt(self, G_poa, T_cell_C):
223         return self.pv_balance(G_poa, T_cell_C)['P_pv']
224
225     def _scaled_slope_pct(self, slope_pct):
226         return slope_pct * float(self.p.grade_scale)
227
228     def charge_efficiency(self, P_pack) -> float:
229         try:
230             p_pack = float(P_pack)
231         except Exception:
232             return 1.0
233         return float(self.p.eta_charge) if p_pack < 0.0 else 1.0
234
235     def soc_step(
236         self,
237         z: float,
238         P_pack: float,
239         dt_sec: float,
240         *,
241         current_a: float | None = None,
242         Tbat_C: float = 25.0,
243     ) -> float:
244         """Advance SoC using the profile's declared state definition.
245
246         A positive ``Q_nom_Ah`` selects charge SoC and therefore coulomb
247         counting, which is consistent with an OCV-vs-SoC map. Legacy
248         profiles that do not declare pack capacity retain energy integration
249         so existing non-solar profiles are not silently reinterpreted.
250         """
251         eta = self.charge_efficiency(P_pack)
252         q_nom_ah = float(self.p.Q_nom_Ah)
253         if q_nom_ah > 0.0:
254             current = (
255                 float(current_a)
256                 if current_a is not None
257                 else float(self.battery_iv(P_pack, z, Tbat_C)["I"])
258             )
259             return float(z) - eta * current * float(dt_sec) / (3600.0 * q_nom_ah)
260         return float(z) - eta * (float(P_pack) * float(dt_sec) / 3600.0) / max(float(self.p.E_nom_Wh), 1.0e-6)
261
262     def ocv_from_soc(self, z):
263         if _is_symbolic(z):
264             z_clamped = ca.fmin(self.p.soc_max, ca.fmax(self.p.soc_min, z))
265             return self.p.V_min + (self.p.V_max - self.p.V_min) * z_clamped
266         zc = float(np.clip(z, self.p.soc_min, self.p.soc_max))
267         if not self.ocv_soc_map:

```

```

268         return self.p.V_min + (self.p.V_max - self.p.V_min) * zc
269     return float(np.interp(zc, self.soc_grid, self.ocv_grid))
270
271 def load_ocv_map(self, path: str) -> bool:
272     try:
273         self.soc_grid, self.ocv_grid = read_1d_map(path)
274         self.ocv_soc_map = True
275         return True
276     except Exception:
277         self.ocv_soc_map = None
278         return False
279
280 def air_density(self, ambient_temp_c=None, elevation_m=0.0):
281     """Return dry-air density from the configured atmosphere model.
282
283     The ideal-gas-altitude mode uses the ISA tropospheric pressure law and
284     measured ambient temperature. ``rho`` remains the explicit fallback so
285     old profiles retain bit-for-bit constant-density behavior.
286     """
287     if str(self.p.air_density_mode or 'constant').lower() != 'ideal_gas_altitude':
288         return float(self.p.rho)
289     if ambient_temp_c is None:
290         return float(self.p.rho)
291     temperature_k = max(180.0, float(ambient_temp_c) + 273.15)
292     altitude = float(np.clip(elevation_m, -500.0, 11000.0))
293     pressure_ratio = max(0.05, 1.0 - 0.0065 * altitude / 288.15) ** 5.255877
294     pressure_pa = max(1000.0, float(self.p.air_density_reference_pressure_pa)) * pressure_ratio
295     return float(pressure_pa / (287.05 * temperature_k))
296
297 def resistive_forces(
298     self,
299     v_ms,
300     slope_pct,
301     headwind_ms=0.0,
302     *,
303     ambient_temp_c=None,
304     elevation_m=0.0,
305     air_density_kgm3=None,
306 ):
307     rho = (
308         self.air_density(ambient_temp_c, elevation_m)
309         if air_density_kgm3 is None
310         else air_density_kgm3
311     )
312     if _is_symbolic(v_ms) or _is_symbolic(slope_pct) or _is_symbolic(headwind_ms):
313         v_rel = ca.fmax(0.0, v_ms + headwind_ms)
314         theta = ca.atan(self._scaled_slope_pct(slope_pct) / 100.0)
315         F_aero = 0.5 * rho * self.p.CdA * v_rel ** 2
316         N = self.p.m * self.p.g * ca.cos(theta)
317         Crr_eff = self.p.Crr
318         if self.p.Crr_per_wheel and self.p.wheel_count:
319             Crr_eff = self.p.Crr_per_wheel * float(self.p.wheel_count)
320         F_roll = Crr_eff * N
321         F_grade = self.p.m * self.p.g * ca.sin(theta)
322         F_total = F_aero + F_roll + F_grade
323         return dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade,
324                    F_total=F_total, theta=theta, v_rel_ms=v_rel,
325                    air_density_kgm3=rho,
326                    normal_force_n=N, Crr_eff=Crr_eff,
327                    slope_scaled_pct=self._scaled_slope_pct(slope_pct))
328     v_rel = max(0.0, float(v_ms) + float(headwind_ms))
329     theta = math.atan(float(self._scaled_slope_pct(slope_pct)) / 100.0)
330     F_aero = 0.5 * float(rho) * self.p.CdA * v_rel ** 2
331     N = self.p.m * self.p.g * math.cos(theta)
332     Crr_eff = self.p.Crr
333     if self.p.Crr_per_wheel and self.p.wheel_count:
334         Crr_eff = self.p.Crr_per_wheel * float(self.p.wheel_count)
335     F_roll = Crr_eff * N
336     F_grade = self.p.m * self.p.g * math.sin(theta)
337     F_total = F_aero + F_roll + F_grade
338     return dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade,
339                F_total=F_total, theta=theta, v_rel_ms=v_rel,
340                air_density_kgm3=float(rho),
341                normal_force_n=N, Crr_eff=Crr_eff,
342                slope_scaled_pct=float(self._scaled_slope_pct(slope_pct)))
343
344 def battery_iv(self, P_pack, z, Tbat_C):
345     OCV = self.ocv_from_soc(z)
346     symbolic = _is_symbolic(P_pack) or _is_symbolic(z) or _is_symbolic(Tbat_C)
347     z_rint = ca.fmin(0.95, ca.fmax(0.1, z)) if symbolic else float(np.clip(z, 0.1, 0.95))
348     Rint = self.R_int(Tbat_C, z_rint)
349     Rline = float(self.p.r_line_ohm)
350     Rpolarization = max(0.0, float(self.p.r_polarization_ohm))
351     # Planner intervals are much longer than the fitted polarization time
352     # constant, so the stateless MPC model uses its steady-state resistance.
353     Rtot = Rint + Rline + Rpolarization
354     a = Rtot
355     b = -OCV
356     c = P_pack
357     if symbolic:
358         disc = ca.fmax(b * b - 4 * a * c, 0.0)
359         I = (OCV - ca.sqrt(disc)) / (2 * Rtot)
360     else:
361         disc = max(float(b * b - 4 * a * c), 0.0)
362         I = (float(OCV) - math.sqrt(disc)) / (2 * float(Rtot))
363     V = OCV - I * Rtot
364     return dict(I=I, V=V, OCV=OCV, Rint=Rint, Rline=Rline,
365                Rpolarization=Rpolarization,
366                Rtotal=Rtot, iv_discriminant=disc)
367
368 def mech_power(
369     self,
370     v_ms,

```

```

371     slope_pct,
372     headwind_ms=0.0,
373     inertial_power_w=0.0,
374     *,
375     ambient_temp_c=None,
376     elevation_m=0.0,
377 ):
378     forces = self.resistive_forces(
379         v_ms,
380         slope_pct,
381         headwind_ms,
382         ambient_temp_c=ambient_temp_c,
383         elevation_m=elevation_m,
384     )
385     # Aerodynamic force depends on air-relative speed, while wheel work is
386     # force times ground speed. This matches the identification model.
387     return forces['F_total'] * v_ms + inertial_power_w
388
389 def auxiliary_power(self, aux_power_w=None):
390     if aux_power_w is not None:
391         return max(0.0, float(aux_power_w))
392     if self.aux_power_override_w is not None:
393         return max(0.0, float(self.aux_power_override_w))
394     return max(0.0, float(self.p.P_aux))
395
396 def scheduled_auxiliary_power(self, *, is_driving: bool, irradiance_wm2: float) -> float:
397     if is_driving:
398         return self.auxiliary_power()
399     if float(irradiance_wm2) <= max(0.0, float(self.p.aux_night_ghi_threshold_wm2)):
400         return max(0.0, float(self.p.P_aux_night))
401     if self.aux_power_override_w is not None:
402         return self.auxiliary_power()
403     return max(0.0, float(self.p.P_aux_stopped))
404
405 def torque_from_mech(self, P_mech, v_ms, wheel_radius=None, ratio=None):
406     if wheel_radius is None:
407         wheel_radius = self.p.wheel_radius
408     if ratio is None:
409         ratio = self.p.gear_ratio
410     eps=1e-3
411     omega_w = v_ms/wheel_radius
412     T_w = P_mech/(omega_w+eps)
413     T_m = T_w/ratio
414     return T_m, omega_w*ratio
415
416 def electrical_balance(self, v_ms, slope_pct, z, Tbat_C, G_poa, Tcell_C,
417     headwind_ms=0.0, aux_power_w=None, inertial_power_w=0.0,
418     ambient_temp_c=None, elevation_m=0.0):
419     pv = self.pv_balance(G_poa, Tcell_C)
420     P_pv = pv['P_pv']
421     road_forces = self.resistive_forces(
422         v_ms,
423         slope_pct,
424         headwind_ms,
425         ambient_temp_c=ambient_temp_c,
426         elevation_m=elevation_m,
427     )
428     P_road_load = road_forces['F_total'] * v_ms
429     P_mech_wheel = P_road_load + inertial_power_w
430     symbolic = (
431         _is_symbolic(P_mech_wheel)
432         or _is_symbolic(z)
433         or _is_symbolic(Tbat_C)
434         or _is_symbolic(inertial_power_w)
435     )
436     if symbolic:
437         P_mech_pos = ca.fmax(P_mech_wheel, 0.0)
438         P_mech_neg = ca.fmax(-P_mech_wheel, 0.0)
439     else:
440         P_mech_pos = max(float(P_mech_wheel), 0.0)
441         P_mech_neg = max(-float(P_mech_wheel), 0.0)
442     Tm_drv, omega_m = self.torque_from_mech(P_mech_pos, v_ms)
443     eff_drv = self.eff_drive(v_ms, Tm_drv)
444     P_dc_to_drv = P_mech_pos/(eff_drv*self.p.gear_eta*self.p.inverter_eta)
445     Tm_reg, _ = self.torque_from_mech(P_mech_neg, v_ms)
446     eff_reg = self.eff_regen(v_ms, Tm_reg)
447     regen_utilization = float(np.clip(self.p.regen_utilization, 0.0, 1.0))
448     P_reg_to_dc = regen_utilization*eff_reg*self.p.gear_eta*self.p.inverter_eta*P_mech_neg
449     P_aux = self.auxiliary_power(aux_power_w)
450     P_pack = P_dc_to_drv - P_reg_to_dc + P_aux - P_pv
451     iv = self.battery_iv(P_pack, z, Tbat_C)
452     OCV = iv['OCV']; Rint = iv['Rint']; Rline = iv['Rline']; Rpolarization = iv['Rpolarization']; Rtot = iv['Rtotal']
453     disc = iv['iv_discriminant']; I = iv['I']; V = iv['V']
454     losses_line = I*I*Rline
455     losses_rint = I*I*Rint
456     losses_polarization = I*I*Rpolarization
457     losses_int = losses_rint + losses_polarization
458     eta_charge = self.charge_efficiency(P_pack)
459     return dict(P_pv=P_pv, P_pv_raw=pv['P_pv_raw'],
460         P_pv_unlimited=pv['P_pv_unlimited'],
461         P_pv_limit_loss=pv['P_pv_limit_loss'],
462         eta_panel=pv['eta_panel'], eta_mppt=pv['eta_mppt'],
463         P_mech=P_mech_wheel, P_mech_wheel=P_mech_wheel,
464         P_road_load=P_road_load, P_inertia=inertial_power_w,
465         P_aux=P_aux,
466         P_pack=P_pack, I=I, V=V,
467         losses_line=losses_line, losses_int=losses_int,
468         losses_rint=losses_rint,
469         losses_polarization=losses_polarization,
470         OCV=OCV, Rint=Rint, Rline=Rline,
471         Rpolarization=Rpolarization, Rtotal=Rtot,
472         iv_discriminant=disc, eta_charge=eta_charge,
473         P_dc_to_drv=P_dc_to_drv, P_reg_to_dc=P_reg_to_dc,

```

```

474         eff_drv=eff_drv, eff_reg=eff_reg,
475         torque_drive_nm=Tm_drv, torque_regen_nm=Tm_reg,
476         omega_motor_radps=omega_m,
477         omega_wheel_radps=omega_m/max(float(self.p.gear_ratio), 1.0e-9))

```

A.10 mpc_solarcar/utils_maps.py

```

1 import numpy as np
2 import pandas as pd
3 def bilinear_interp(xg, yg, Z, x, y):
4     xg = np.asarray(xg); yg=np.asarray(yg); Z=np.asarray(Z)
5     x = np.clip(x, xg[0], xg[-1]); y=np.clip(y, yg[0], yg[-1])
6     i = np.searchsorted(xg, x)-1; i=np.clip(i,0,len(xg)-2)
7     j = np.searchsorted(yg, y)-1; j=np.clip(j,0,len(yg)-2)
8     x0,x1=xg[i],xg[i+1]; y0,y1=yg[j],yg[j+1]
9     Z00=Z[i,j]; Z10=Z[i+1,j]; Z01=Z[i,j+1]; Z11=Z[i+1,j+1]
10    wx=0 if x1==x0 else (x-x0)/(x1-x0)
11    wy=0 if y1==y0 else (y-y0)/(y1-y0)
12    return (1-wx)*(1-wy)*Z00 + wx*(1-wy)*Z10 + (1-wx)*wy*Z01 + wx*wy*Z11
13 def read_eff_map(path):
14     df = pd.read_csv(path, index_col=0)
15     return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
16 def read_Rint_map(path):
17     df = pd.read_csv(path, index_col=0)
18     return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
19
20 def read_map(path):
21     df = pd.read_csv(path, index_col=0)
22     return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
23
24 def read_ld_map(path):
25     df = pd.read_csv(path)
26     if df.shape[1] >= 2:
27         x = df.iloc[:, 0].values.astype(float)
28         y = df.iloc[:, 1].values.astype(float)
29         return x, y
30     df = pd.read_csv(path, index_col=0)
31     x = df.index.values.astype(float)
32     y = df.iloc[:, 0].values.astype(float)
33     return x, y

```

A.11 mpc_solarcar/route_utils.py

```

1 import numpy as np
2 import pandas as pd
3 import math
4
5
6 def _interp_field(d, y, s_km, default=0.0):
7     if len(d) < 2:
8         return float(default)
9     s = np.clip(s_km, d[0], d[-1])
10    i = np.searchsorted(d, s) - 1
11    i = np.clip(i, 0, len(d) - 2)
12    t = 0.0 if d[i + 1] == d[i] else (s - d[i]) / (d[i + 1] - d[i])
13    return float((1 - t) * y[i] + t * y[i + 1])
14
15
16 def interpolate_route(route_df, s_km):
17     d = route_df['dist_km'].values
18     lat = route_df['lat'].values
19     lon = route_df['lon'].values
20     latp = _interp_field(d, lat, s_km, default=float(lat[0]) if len(lat) else 0.0)
21     lonp = _interp_field(d, lon, s_km, default=float(lon[0]) if len(lon) else 0.0)
22     return latp, lonp
23
24
25 def interpolate_route_with_alt(route_df, s_km):
26     lat, lon = interpolate_route(route_df, s_km)
27     alt = None
28     for col in ('alt_m', 'altitude_m', 'elev_m'):
29         if col in route_df.columns:
30             d = route_df['dist_km'].values
31             alt = _interp_field(d, route_df[col].values, s_km, default=float(route_df[col].values[0]))
32             break
33     return lat, lon, alt
34
35
36 def interpolate_profile(route_df, s_km, field: str, default: float = 0.0) -> float:
37     if field not in route_df.columns:
38         return float(default)
39     d = route_df['dist_km'].values
40     return _interp_field(d, route_df[field].values, s_km, default=default)
41
42
43 def average_profile(route_df, s0_km, s1_km, field: str, default: float = 0.0) -> float:
44     """Return the distance average of a piecewise-linear profile field.
45
46     A coarse prediction segment must preserve the integrated road load rather
47     than sample a potentially noisy 100 m grade at one endpoint. Values beyond
48     the table are held at the nearest endpoint, matching ``interpolate_profile``.
49     """

```

```

50 start = float(s0_km)
51 end = float(s1_km)
52 if not np.isfinite(start) or not np.isfinite(end):
53     return float(default)
54 if end < start:
55     start, end = end, start
56 return float(
57     average_profile_segments(
58         route_df,
59         np.asarray([start], dtype=float),
60         np.asarray([end - start], dtype=float),
61         field,
62         default,
63     )[0]
64 )
65
66
67 def average_profile_segments(route_df, starts_km, lengths_km, field: str, default: float = 0.0):
68     """Vector wrapper for prediction meshes with nonuniform segment lengths."""
69     starts = np.asarray(starts_km, dtype=float)
70     lengths = np.asarray(lengths_km, dtype=float)
71     if starts.shape != lengths.shape:
72         raise ValueError('starts_km and lengths_km must have identical shapes')
73     if field not in route_df.columns:
74         return np.full(starts.shape, float(default), dtype=float)
75
76     distance = pd.to_numeric(route_df['dist_km'], errors='coerce').to_numpy(dtype=float)
77     values = pd.to_numeric(route_df[field], errors='coerce').to_numpy(dtype=float)
78     finite = np.isfinite(distance) & np.isfinite(values)
79     if int(finite.sum()) < 2:
80         return np.full(starts.shape, float(default), dtype=float)
81     order = np.argsort(distance[finite], kind='stable')
82     distance = distance[finite][order]
83     values = values[finite][order]
84     distance, unique_index = np.unique(distance, return_index=True)
85     values = values[unique_index]
86     if len(distance) < 2:
87         return np.full(starts.shape, float(default), dtype=float)
88
89     dx = np.diff(distance)
90     segment_slope = np.diff(values) / dx
91     cumulative = np.concatenate(
92         ([0.0], np.cumsum(0.5 * (values[:-1] + values[1:]) * dx))
93     )
94
95     def antiderivative(query):
96         query = np.asarray(query, dtype=float)
97         result = np.empty_like(query)
98         below = query <= distance[0]
99         above = query >= distance[-1]
100         inside = ~(below | above)
101         result[below] = (query[below] - distance[0]) * values[0]
102         result[above] = cumulative[-1] + (query[above] - distance[-1]) * values[-1]
103         if np.any(inside):
104             index = np.searchsorted(distance, query[inside], side='right') - 1
105             index = np.clip(index, 0, len(distance) - 2)
106             local_dx = query[inside] - distance[index]
107             result[inside] = (
108                 cumulative[index]
109                 + values[index] * local_dx
110                 + 0.5 * segment_slope[index] * np.square(local_dx)
111             )
112         return result
113
114     flat_starts = starts.ravel()
115     flat_lengths = np.maximum(0.0, lengths.ravel())
116     flat_ends = flat_starts + flat_lengths
117     integral = antiderivative(flat_ends) - antiderivative(flat_starts)
118     result = np.empty_like(flat_starts)
119     positive = flat_lengths > 1.0e-12
120     result[positive] = integral[positive] / flat_lengths[positive]
121     result[~positive] = np.interp(
122         flat_starts[~positive], distance, values, left=values[0], right=values[-1]
123     )
124     return result.reshape(starts.shape)
125
126
127 def interpolate_route_heading(route_df, s_km, span_km: float = 0.5) -> float:
128     if len(route_df) < 2:
129         return 0.0
130     s0 = float(s_km) - float(span_km)
131     s1 = float(s_km) + float(span_km)
132     lat0, lon0 = interpolate_route(route_df, s0)
133     lat1, lon1 = interpolate_route(route_df, s1)
134     lat0r = math.radians(float(lat0))
135     lat1r = math.radians(float(lat1))
136     dlon = math.radians(float(lon1) - float(lon0))
137     y = math.sin(dlon) * math.cos(lat1r)
138     x = math.cos(lat0r) * math.sin(lat1r) - math.sin(lat0r) * math.cos(lat1r) * math.cos(dlon)
139     return float((math.degrees(math.atan2(y, x)) + 360.0) % 360.0)

```

A.12 mpc_solarcar/schedule_utils.py

```

1 import os
2 from dataclasses import dataclass
3 from datetime import datetime, time as dttime, timedelta, timezone
4 from typing import List, Optional, Tuple
5

```

```

6 import yaml
7
8 try:
9     from zoneinfo import ZoneInfo
10 except Exception: # pragma: no cover - fallback for older Python
11     ZoneInfo = None
12
13
14 def _parse_utc(ts: str) -> Optional[datetime]:
15     try:
16         if ts.endswith('Z'):
17             ts = ts[:-1] + '+00:00'
18         dt = datetime.fromisoformat(ts)
19         if dt.tzinfo is None:
20             dt = dt.replace(tzinfo=datetime.timezone.utc)
21         return dt.astimezone(datetime.timezone.utc)
22     except Exception:
23         return None
24
25
26 def _parse_time_hhmm(s: str) -> Optional[datetime]:
27     try:
28         parts = s.strip().split(':')
29         if len(parts) < 2:
30             return None
31         return datetime(hour=int(parts[0]), minute=int(parts[1]))
32     except Exception:
33         return None
34
35
36 @dataclass
37 class DriveWindow:
38     start_utc: datetime
39     end_utc: datetime
40     v_min_kmh: float
41     v_max_kmh: float
42
43     def contains(self, t_utc: datetime) -> bool:
44         return self.start_utc <= t_utc < self.end_utc
45
46
47 @dataclass
48 class DailyWindow:
49     start_local: datetime
50     end_local: datetime
51     tz: str
52     days: Optional[List[int]]
53     v_min_kmh: float
54     v_max_kmh: float
55
56     def contains(self, t_utc: datetime) -> bool:
57         if ZoneInfo is None:
58             return False
59         try:
60             tzinfo = ZoneInfo(self.tz)
61         except Exception:
62             return False
63         local_dt = t_utc.astimezone(tzinfo)
64         if self.days is not None and local_dt.weekday() not in self.days:
65             return False
66         start = self.start_local
67         end = self.end_local
68         now_t = local_dt.time()
69         if start <= end:
70             return start <= now_t < end
71         # wraps midnight
72         return now_t >= start or now_t < end
73
74
75 class DriveSchedule:
76     def __init__(self, windows: List[DriveWindow], daily: List[DailyWindow], deny_by_default: bool):
77         self.windows = windows
78         self.daily = daily
79         self.deny_by_default = deny_by_default
80
81     @classmethod
82     def from_yaml(cls, path: str) -> Optional['DriveSchedule']:
83         if not path or not os.path.exists(path):
84             return None
85         try:
86             with open(path, 'r', encoding='utf-8') as f:
87                 cfg = yaml.safe_load(f) or {}
88         except Exception:
89             return None
90         deny_by_default = bool(cfg.get('deny_by_default', False))
91         windows = []
92         for w in cfg.get('drive_windows', []) or []:
93             start = _parse_utc(str(w.get('start_utc', '')))
94             end = _parse_utc(str(w.get('end_utc', '')))
95             if start is None or end is None:
96                 continue
97             vmin = float(w.get('v_min_kmh', 0.0))
98             vmax = float(w.get('v_max_kmh', 130.0))
99             windows.append(DriveWindow(start, end, vmin, vmax))
100         daily = []
101         for w in cfg.get('daily_windows', []) or []:
102             start = _parse_time_hhmm(str(w.get('start_local', '')))
103             end = _parse_time_hhmm(str(w.get('end_local', '')))
104             tz = str(w.get('tz', 'UTC'))
105             if start is None or end is None:
106                 continue
107             days = w.get('days', None)
108             if days is not None:

```

```

109         try:
110             days = [int(d) for d in days]
111         except Exception:
112             days = None
113         vmin = float(w.get('v_min_kmh', 0.0))
114         vmax = float(w.get('v_max_kmh', 130.0))
115         daily.append(DailyWindow(start, end, tz, days, vmin, vmax))
116     return cls(windows, daily, deny_by_default)
117
118 def speed_limits(self, t_utc: datetime) -> Optional[Tuple[float, float]]:
119     limits = []
120     for w in self.windows:
121         if w.contains(t_utc):
122             limits.append((w.v_min_kmh, w.v_max_kmh))
123     for w in self.daily:
124         if w.contains(t_utc):
125             limits.append((w.v_min_kmh, w.v_max_kmh))
126     if limits:
127         vmin = max(l[0] for l in limits)
128         vmax = min(l[1] for l in limits)
129         return vmin, vmax
130     if self.deny_by_default:
131         return 0.0, 0.0
132     return None
133
134 def is_drive_time(self, t_utc: datetime) -> bool:
135     limits = self.speed_limits(t_utc)
136     if limits is None:
137         return not self.deny_by_default
138     return limits[1] > 0.0
139
140 def next_drive_start(self, t_utc: datetime) -> datetime:
141     if self.is_drive_time(t_utc):
142         return t_utc
143     candidates = []
144     for w in self.windows:
145         if t_utc < w.start_utc:
146             candidates.append(w.start_utc)
147     for w in self.daily:
148         if ZoneInfo is None:
149             continue
150         try:
151             tzinfo = ZoneInfo(w.tz)
152         except Exception:
153             continue
154         local_dt = t_utc.astimezone(tzinfo)
155         if w.days is not None and local_dt.weekday() not in w.days:
156             # move to next allowed weekday
157             days_ahead = 1
158             while w.days is not None and ((local_dt + timedelta(days=days_ahead)).weekday() not in w.days) and days_ahead < 8:
159                 days_ahead += 1
160             start_date = (local_dt + timedelta(days=days_ahead)).date()
161             start_local = datetime.combine(start_date, w.start_local, tzinfo)
162             candidates.append(start_local.astimezone(timezone.utc))
163             continue
164         now_t = local_dt.time()
165         if w.start_local <= w.end_local:
166             if now_t < w.start_local:
167                 start_local = datetime.combine(local_dt.date(), w.start_local, tzinfo)
168             else:
169                 start_local = datetime.combine(local_dt.date() + timedelta(days=1), w.start_local, tzinfo)
170         else:
171             # wraps midnight
172             if now_t < w.end_local:
173                 start_local = datetime.combine(local_dt.date(), w.start_local, tzinfo)
174             elif now_t < w.start_local:
175                 start_local = datetime.combine(local_dt.date(), w.start_local, tzinfo)
176             else:
177                 start_local = datetime.combine(local_dt.date() + timedelta(days=1), w.start_local, tzinfo)
178             candidates.append(start_local.astimezone(timezone.utc))
179     if candidates:
180         return min(candidates)
181     return t_utc
182
183 def current_drive_window(self, t_utc: datetime):
184     """Return (start_utc, end_utc) if t_utc is inside a drive window, else None."""
185     for w in self.windows:
186         if w.contains(t_utc):
187             return w.start_utc, w.end_utc
188     for w in self.daily:
189         if ZoneInfo is None:
190             continue
191         try:
192             tzinfo = ZoneInfo(w.tz)
193         except Exception:
194             continue
195         local_dt = t_utc.astimezone(tzinfo)
196         if w.days is not None and local_dt.weekday() not in w.days:
197             continue
198         start = w.start_local
199         end = w.end_local
200         now_t = local_dt.time()
201         if start <= end:
202             if not (start <= now_t < end):
203                 continue
204             start_local = datetime.combine(local_dt.date(), start, tzinfo)
205             end_local = datetime.combine(local_dt.date(), end, tzinfo)
206         else:
207             # wraps midnight
208             if not (now_t >= start or now_t < end):
209                 continue
210             if now_t >= start:
211                 start_local = datetime.combine(local_dt.date(), start, tzinfo)

```

```

212         end_local = datetime.combine(local_dt.date() + timedelta(days=1), end, tzinfo)
213     else:
214         start_local = datetime.combine(local_dt.date() - timedelta(days=1), start, tzinfo)
215         end_local = datetime.combine(local_dt.date(), end, tzinfo)
216     return start_local.astimezone(timezone.utc), end_local.astimezone(timezone.utc)
217     return None

```

A.13 mpc_solarcar/forecast_grid.py

```

1  from __future__ import annotations
2
3  from datetime import datetime
4
5  import numpy as np
6  import pandas as pd
7
8
9  FORECAST_GRID_COLUMNS = (
10     "GHI",
11     "DNI",
12     "DHI",
13     "POA_drive",
14     "POA_stop_ideal",
15     "Tamb_C",
16     "Tcell_C",
17     "Tcell_drive_C",
18     "Tcell_stop_ideal_C",
19     "headwind_ms",
20 )
21
22
23 def timestamp_ns(value) -> int:
24     ts = pd.Timestamp(value)
25     if ts.tzinfo is None:
26         ts = ts.tz_localize("UTC")
27     else:
28         ts = ts.tz_convert("UTC")
29     return int(ts.value)
30
31
32 def forecast_distance_column(df: pd.DataFrame) -> str:
33     if not isinstance(df, pd.DataFrame):
34         return ""
35     if "s_km" in df.columns:
36         return "s_km"
37     if "route_progress_km" in df.columns:
38         return "route_progress_km"
39     return ""
40
41
42 def build_forecast_grid_payload(df: pd.DataFrame) -> dict | None:
43     """Build matrices used for bilinear interpolation over UTC time and route distance."""
44     dist_col = forecast_distance_column(df)
45     if not dist_col or "time" not in df.columns:
46         return None
47     work = df.copy()
48     work[dist_col] = pd.to_numeric(work[dist_col], errors="coerce")
49     work = work.dropna(subset=["time", dist_col]).sort_values(["time", dist_col])
50     if work.empty:
51         return None
52     time_index = pd.Index(work["time"].drop_duplicates().sort_values())
53     s_grid = np.array(sorted(work[dist_col].dropna().unique()), dtype=float)
54     if len(time_index) < 2 or len(s_grid) < 2:
55         return None
56     if len(work) <= max(len(time_index), len(s_grid)):
57         return None
58     work = work.drop_duplicates(subset=["time", dist_col], keep="last")
59     matrices = {}
60     for col in FORECAST_GRID_COLUMNS:
61         if col not in work.columns:
62             continue
63         pivot = (
64             work.pivot(index="time", columns=dist_col, values=col)
65             .reindex(index=time_index, columns=s_grid)
66             .apply(pd.to_numeric, errors="coerce")
67             .interpolate(axis=0, limit_direction="both")
68             .interpolate(axis=1, limit_direction="both")
69             .ffill()
70             .bfill()
71             .ffill(axis=1)
72             .bfill(axis=1)
73         )
74         matrices[col] = pivot.to_numpy(dtype=float)
75     if not matrices:
76         return None
77     return {
78         "dist_col": dist_col,
79         "time_ns": np.array([timestamp_ns(value) for value in time_index], dtype=np.int64),
80         "s_grid": s_grid,
81         "matrices": matrices,
82     }
83
84
85 def interp_forecast_grid(
86     payload: dict | None,
87     col: str,
88     t_utc: datetime,
89     s_km: float | None,

```

```

90     default: float,
91 ) -> float:
92     """Bilinearly interpolate one weather field, clamping only outside grid coverage."""
93     if not payload:
94         return float(default)
95     matrix = payload.get("matrices", {}).get(col)
96     tg = payload.get("time_ns")
97     sg = payload.get("s_grid")
98     if matrix is None or tg is None or sg is None or len(tg) == 0 or len(sg) == 0:
99         return float(default)
100     t_ns = int(np.clip(timestamp_ns(t_utc), int(tg[0]), int(tg[-1])))
101     s_val = float(s_km if s_km is not None else sg[0])
102     s_val = float(np.clip(s_val, float(sg[0]), float(sg[-1])))
103
104     i_hi = int(np.searchsorted(tg, t_ns, side="left"))
105     if i_hi <= 0:
106         i0 = i1 = 0
107         wt = 0.0
108     elif i_hi >= len(tg):
109         i0 = i1 = len(tg) - 1
110         wt = 0.0
111     else:
112         i0 = i_hi - 1
113         i1 = i_hi
114         wt = float((t_ns - int(tg[i0])) / max(int(tg[i1]) - int(tg[i0]), 1))
115
116     j_hi = int(np.searchsorted(sg, s_val, side="left"))
117     if j_hi <= 0:
118         j0 = j1 = 0
119         ws = 0.0
120     elif j_hi >= len(sg):
121         j0 = j1 = len(sg) - 1
122         ws = 0.0
123     else:
124         j0 = j_hi - 1
125         j1 = j_hi
126         ws = float((s_val - float(sg[j0])) / max(float(sg[j1]) - float(sg[j0]), 1.0e-9))
127
128     v0 = (1.0 - ws) * float(matrix[i0, j0]) + ws * float(matrix[i0, j1])
129     v1 = (1.0 - ws) * float(matrix[i1, j0]) + ws * float(matrix[i1, j1])
130     return float((1.0 - wt) * v0 + wt * v1)

```

A.14 mpc_solarcar/upper_policy.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import numpy as np
6 import pandas as pd
7
8
9 def load_upper_policy_csv(path: str | Path) -> pd.DataFrame:
10     """Load a distance-indexed upper speed policy with strict schema checks."""
11     policy_path = Path(path)
12     frame = pd.read_csv(policy_path)
13     required = {"s_km", "v_kmh"}
14     if not required.issubset(frame.columns):
15         raise ValueError(f"upper policy must contain {sorted(required)}: {policy_path}")
16     out = frame.loc[:, ["s_km", "v_kmh"]].copy()
17     out["s_km"] = pd.to_numeric(out["s_km"], errors="coerce")
18     out["v_kmh"] = pd.to_numeric(out["v_kmh"], errors="coerce")
19     out = (
20         out.replace([np.inf, -np.inf], np.nan)
21         .dropna()
22         .sort_values("s_km")
23         .drop_duplicates("s_km", keep="last")
24         .reset_index(drop=True)
25     )
26     if len(out) < 2:
27         raise ValueError(f"upper policy needs at least two finite distance points: {policy_path}")
28     if float(out["s_km"].iloc[-1]) <= float(out["s_km"].iloc[0]):
29         raise ValueError(f"upper policy distance must increase: {policy_path}")
30     return out
31
32
33 def interpolate_upper_policy(
34     frame: pd.DataFrame,
35     control_s_km,
36     *,
37     minimum_speed_kmh: float,
38     maximum_speed_kmh: float,
39 ) -> np.ndarray:
40     """Interpolate a learned full-course policy onto the current MPC control mesh."""
41     control_s = np.asarray(control_s_km, dtype=float)
42     if control_s.ndim != 1 or len(control_s) == 0 or not np.all(np.isfinite(control_s)):
43         raise ValueError("control_s_km must be a non-empty finite one-dimensional array")
44     source_s = pd.to_numeric(frame["s_km"], errors="coerce").to_numpy(dtype=float)
45     source_v = pd.to_numeric(frame["v_kmh"], errors="coerce").to_numpy(dtype=float)
46     if len(source_s) < 2 or not np.all(np.isfinite(source_s)) or not np.all(np.isfinite(source_v)):
47         raise ValueError("upper policy contains non-finite or insufficient points")
48     interpolated = np.interp(control_s, source_s, source_v)
49     return np.clip(interpolated, float(minimum_speed_kmh), float(maximum_speed_kmh))

```

A.15 mpc_solarcar/upper_cost.py

```

1 from __future__ import annotations
2
3 from dataclasses import asdict, dataclass
4 from typing import Any, Dict, Optional
5
6
7 @dataclass
8 class UpperCostConfig:
9     objective_mode: str = "weighted"
10     w_wait: float = 1.0
11     w_travel_time: float = 1.0
12     w_terminal_soc_min: float = 30.0
13     w_day_end_soc_min: float = 1.0e5
14     w_soc_day_max: float = 1.0e4
15     w_soc_day_track: float = 0.0
16     w_speed_smooth: float = 30.0
17     w_dv_limit: float = 2.0
18     w_speed_limit: float = 50.0
19     w_drive_window: float = 1.0e5
20     w_current_sq: float = 0.01
21     w_pack_energy: float = 0.0
22     w_joule_loss: float = 0.0
23     w_aero_energy: float = 0.0
24     w_mech_energy: float = 0.0
25     w_speed_quartic: float = 0.0
26     w_solar_headroom: float = 0.0
27     w_progress_lag: float = 0.0
28     w_progress_terminal_lag: float = 0.0
29     w_kinetic_pos: float = 0.0
30     w_pack_power_slew: float = 0.0
31     w_temp: float = 5.0
32     w_soc_terminal: float = 0.0
33     w_soc_floor_barrier: float = 0.0
34     w_uncertainty_reserve: float = 0.0
35     speed_quartic_scale_kmh: float = 80.0
36     progress_lag_deadband_km: float = 0.0
37     soc_solar_headroom_max: float = 0.92
38     solar_headroom_power_scale_w: float = 1000.0
39     soc_floor_barrier_eps: float = 0.01
40     reserve_soc_per_hour: float = 0.0
41     reserve_soc_max_extra: float = 0.0
42     constraint_penalty: float = 1.0e4
43
44     def to_dict(self) -> Dict[str, Any]:
45         return asdict(self)
46
47
48 def _cfg_value(cfg: Optional[dict], key: str, default):
49     if isinstance(cfg, dict) and key in cfg:
50         return cfg[key]
51     return default
52
53
54 def load_upper_cost_config(
55     mpc_cfg: Optional[dict],
56     *,
57     legacy: Optional[dict] = None,
58     default_drive_window: float = 1.0e5,
59 ) -> UpperCostConfig:
60     legacy = legacy or {}
61     nested = {}
62     if isinstance(mpc_cfg, dict):
63         nested = mpc_cfg.get("upper_cost", {}) or {}
64
65     def pick(name: str, default):
66         if name in nested:
67             return nested[name]
68         if name in legacy:
69             return legacy[name]
70         return default
71
72     return UpperCostConfig(
73         objective_mode=str(pick("objective_mode", "weighted")).strip().lower(),
74         w_wait=float(pick("w_wait", 1.0)),
75         w_travel_time=float(pick("w_travel_time", 1.0)),
76         w_terminal_soc_min=float(pick("w_terminal_soc_min", 30.0)),
77         w_day_end_soc_min=float(pick("w_day_end_soc_min", default_drive_window)),
78         w_soc_day_max=float(pick("w_soc_day_max", 1.0e4)),
79         w_soc_day_track=float(pick("w_soc_day_track", 0.0)),
80         w_speed_smooth=float(pick("w_speed_smooth", _cfg_value(legacy, "w_dv", 30.0))),
81         w_dv_limit=float(pick("w_dv_limit", 2.0)),
82         w_speed_limit=float(pick("w_speed_limit", 50.0)),
83         w_drive_window=float(pick("w_drive_window", default_drive_window)),
84         w_current_sq=float(pick("w_current_sq", _cfg_value(legacy, "w_current", 0.01))),
85         w_pack_energy=float(pick("w_pack_energy", 0.0)),
86         w_joule_loss=float(pick("w_joule_loss", 0.0)),
87         w_aero_energy=float(pick("w_aero_energy", 0.0)),
88         w_mech_energy=float(pick("w_mech_energy", 0.0)),
89         w_speed_quartic=float(pick("w_speed_quartic", 0.0)),
90         w_solar_headroom=float(pick("w_solar_headroom", 0.0)),
91         w_progress_lag=float(pick("w_progress_lag", 0.0)),
92         w_progress_terminal_lag=float(pick("w_progress_terminal_lag", 0.0)),
93         w_kinetic_pos=float(pick("w_kinetic_pos", 0.0)),
94         w_pack_power_slew=float(pick("w_pack_power_slew", 0.0)),
95         w_temp=float(pick("w_temp", _cfg_value(legacy, "w_T", 5.0))),
96         w_soc_terminal=float(pick("w_soc_terminal", 0.0)),
97         w_soc_floor_barrier=float(pick("w_soc_floor_barrier", 0.0)),
98         w_uncertainty_reserve=float(pick("w_uncertainty_reserve", 0.0)),
99         speed_quartic_scale_kmh=float(pick("speed_quartic_scale_kmh", 80.0)),
100         progress_lag_deadband_km=float(pick("progress_lag_deadband_km", 0.0)),

```

```

101     soc_solar_headroom_max=float(pick("soc_solar_headroom_max", 0.92)),
102     solar_headroom_power_scale_w=float(pick("solar_headroom_power_scale_w", 1000.0)),
103     soc_floor_barrier_eps=float(pick("soc_floor_barrier_eps", 0.01)),
104     reserve_soc_per_hour=float(pick("reserve_soc_per_hour", 0.0)),
105     reserve_soc_max_extra=float(pick("reserve_soc_max_extra", 0.0)),
106     constraint_penalty=float(pick("constraint_penalty", 1.0e4)),
107 )
108
109
110 def quad_penalty(x: float, cap: float = 1.0e3) -> float:
111     if x <= 0.0:
112         return 0.0
113     if x > cap:
114         x = cap
115     return x * x
116
117
118 def upper_stage_cost(
119     cfg: UpperCostConfig,
120     *,
121     dt_wait: float,
122     dt_travel: float,
123     v_kmh: float,
124     v_prev_kmh: float,
125     vmax_local_kmh: float,
126     drive_limits: Optional[tuple],
127     dv_limit_kmhps: float,
128     I_a: float,
129     V_v: float,
130     P_pv_w: float,
131     P_pack_w: float,
132     P_pack_prev_w: Optional[float],
133     P_mech_wheel_w: float,
134     losses_int_w: float,
135     losses_line_w: float,
136     F_aero_n: float,
137     kinetic_step_wh: float,
138     z_next: float,
139     Tb_next_c: float,
140     term_soc_min: float,
141     soc_min: float,
142     soc_max: float,
143     temp_min_c: float,
144     temp_max_c: float,
145     day_end_soc_min: Optional[float],
146     day_end_crossing: bool,
147     soc_day_end_max: float,
148     soc_day_track_target: Optional[float],
149     soc_day_track_tol: float,
150     I_max: float,
151     I_chg_min: float,
152     V_min: float,
153     V_max: float,
154     time_ahead_h: float,
155     progress_lag_km: float = 0.0,
156 ) -> float:
157     J = 0.0
158     if dt_wait > 0.0:
159         J += cfg.w_wait * dt_wait
160     J += cfg.w_travel_time * dt_travel
161
162     if cfg.objective_mode in {"fastest", "fastest_feasible", "minimum_time"}:
163         c = cfg.constraint_penalty
164         dv = (v_kmh - v_prev_kmh) / max(dt_travel, 1.0e-3)
165         if dv_limit_kmhps > 0.0:
166             J += c * quad_penalty(abs(dv) - dv_limit_kmhps)
167         if drive_limits is not None:
168             vmin_kmh, vmax_kmh = drive_limits
169             J += c * quad_penalty(vmin_kmh - v_kmh)
170             J += c * quad_penalty(v_kmh - vmax_kmh)
171             J += c * quad_penalty(v_kmh - vmax_local_kmh)
172         if day_end_crossing and day_end_soc_min is not None:
173             J += c * quad_penalty(day_end_soc_min - z_next)
174             if soc_day_end_max > 0.0:
175                 J += c * quad_penalty(z_next - soc_day_end_max)
176             J += c * quad_penalty(max(term_soc_min, soc_min) - z_next)
177             J += c * quad_penalty(z_next - soc_max)
178             J += c * quad_penalty(I_a - I_max)
179             J += c * quad_penalty(I_chg_min - I_a)
180             J += c * quad_penalty(V_min - V_v)
181             J += c * quad_penalty(V_v - V_max)
182             J += c * quad_penalty(Tb_next_c - temp_max_c)
183             J += c * quad_penalty(temp_min_c - Tb_next_c)
184             return J
185
186     J += cfg.w_terminal_soc_min * quad_penalty(term_soc_min - z_next)
187
188     if soc_day_track_target is not None:
189         J += cfg.w_soc_day_track * quad_penalty(z_next - (soc_day_track_target + soc_day_track_tol))
190         J += cfg.w_soc_day_track * quad_penalty((soc_day_track_target - soc_day_track_tol) - z_next)
191
192     dv = (v_kmh - v_prev_kmh) / max(dt_travel, 1.0e-3)
193     J += cfg.w_speed_smooth * (v_kmh - v_prev_kmh) ** 2
194     if dv_limit_kmhps > 0.0:
195         J += cfg.w_dv_limit * quad_penalty(abs(dv) - dv_limit_kmhps)
196
197     if drive_limits is not None:
198         vmin_kmh, vmax_kmh = drive_limits
199         J += cfg.w_drive_window * quad_penalty(vmin_kmh - v_kmh)
200         J += cfg.w_drive_window * quad_penalty(v_kmh - vmax_kmh)
201
202     J += cfg.w_speed_limit * quad_penalty(v_kmh - vmax_local_kmh)
203     J += cfg.w_current_sq * (I_a ** 2) * dt_travel

```

```

204
205 e_pack_wh = max(0.0, P_pack_w) * dt_travel / 3600.0
206 e_loss_wh = max(0.0, losses_int_w + losses_line_w) * dt_travel / 3600.0
207 e_aero_wh = max(0.0, F_aero_n) * (v_kmh / 3.6) * dt_travel / 3600.0
208 e_mech_wh = max(0.0, P_mech_wheel_w) * dt_travel / 3600.0
209
210 J += cfg.w_pack_energy * e_pack_wh
211 J += cfg.w_joule_loss * e_loss_wh
212 J += cfg.w_aero_energy * e_aero_wh
213 J += cfg.w_mech_energy * e_mech_wh
214 J += cfg.w_kinetic_pos * max(0.0, kinetic_step_wh)
215
216 if cfg.w_speed_quartic > 0.0:
217     speed_scale = max(1.0, cfg.speed_quartic_scale_kmh)
218     J += cfg.w_speed_quartic * ((max(0.0, v_kmh) / speed_scale) ** 4) * dt_travel
219
220 if cfg.w_pack_power_slew > 0.0 and P_pack_prev_w is not None:
221     d_pack_kw = (float(P_pack_w) - float(P_pack_prev_w)) / 1000.0
222     J += cfg.w_pack_power_slew * (d_pack_kw ** 2) * (dt_travel / 3600.0)
223
224 if cfg.w_progress_lag > 0.0:
225     lag_err = max(0.0, float(progress_lag_km) - float(cfg.progress_lag_deadband_km))
226     J += cfg.w_progress_lag * quad_penalty(lag_err)
227
228 if cfg.w_solar_headroom > 0.0:
229     solar_scale = max(1.0, cfg.solar_headroom_power_scale_w)
230     pv_kw = max(0.0, P_pv_w) / solar_scale
231     if pv_kw > 0.0:
232         J += cfg.w_solar_headroom * pv_kw * quad_penalty(z_next - cfg.soc_solar_headroom_max) * (dt_travel / 3600.0)
233
234 if cfg.w_soc_floor_barrier > 0.0:
235     soc_gap = max(float(z_next) - float(soc_min), float(cfg.soc_floor_barrier_eps))
236     J += cfg.w_soc_floor_barrier / soc_gap
237
238 if cfg.w_uncertainty_reserve > 0.0 and cfg.reserve_soc_per_hour > 0.0:
239     reserve_extra = min(
240         max(0.0, float(cfg.reserve_soc_max_extra)),
241         max(0.0, float(time_ahead_h)) * max(0.0, float(cfg.reserve_soc_per_hour)),
242     )
243     reserve_soc = min(float(soc_max), float(soc_min) + reserve_extra)
244     J += cfg.w_uncertainty_reserve * quad_penalty(reserve_soc - z_next)
245
246 if day_end_crossing and day_end_soc_min is not None:
247     J += cfg.w_day_end_soc_min * quad_penalty(day_end_soc_min - z_next)
248     if soc_day_end_max > 0.0:
249         J += cfg.w_soc_day_max * quad_penalty(z_next - soc_day_end_max)
250
251 c = cfg.constraint_penalty
252 J += c * quad_penalty(I_a - I_max)
253 J += c * quad_penalty(I_chg_min - I_a)
254 J += c * quad_penalty(V_min - V_v)
255 J += c * quad_penalty(V_v - V_max)
256 J += cfg.w_temp * quad_penalty(Tb_next_c - temp_max_c)
257 J += cfg.w_temp * quad_penalty(temp_min_c - Tb_next_c)
258 J += c * quad_penalty(soc_min - z_next)
259 J += c * quad_penalty(z_next - soc_max)
260 return J
261
262
263 def upper_terminal_cost(
264     cfg: UpperCostConfig,
265     *,
266     z_terminal: float,
267     term_soc_min: float,
268     soc_finish_target: float,
269     soc_finish_tol: float = 0.0,
270     progress_terminal_lag_km: float = 0.0,
271 ) -> float:
272     J = cfg.constraint_penalty * quad_penalty(term_soc_min - z_terminal)
273     if soc_finish_target > 0.0:
274         J += cfg.w_soc_terminal * quad_penalty(
275             z_terminal - (soc_finish_target + max(0.0, soc_finish_tol))
276         )
277     if cfg.w_progress_terminal_lag > 0.0:
278         lag_err = max(0.0, float(progress_terminal_lag_km) - float(cfg.progress_lag_deadband_km))
279         J += cfg.w_progress_terminal_lag * quad_penalty(lag_err)
280     return J
281
282
283 def active_upper_cost_terms(cfg: UpperCostConfig, threshold: float = 1.0e-6) -> Dict[str, float]:
284     out = {}
285     for key, value in cfg.to_dict().items():
286         if key.startswith("w_") and abs(float(value)) > threshold:
287             out[key] = float(value)
288     return out

```

A.16 mpc_solarcar/upper_horizon.py

```

1 from __future__ import annotations
2
3 import math
4 from dataclasses import dataclass
5 from typing import Iterable, List
6
7 import numpy as np
8
9
10 MIN_DISTANCE_STEP_KM = 1.0e-6

```

```

11
12
13 @dataclass
14 class UpperDistanceHorizon:
15     ds_seq_km: np.ndarray
16     seg_s_km: np.ndarray
17     ctrl_s_km: np.ndarray
18
19 @property
20 def total_km(self) -> float:
21     return float(np.sum(self.ds_seq_km)) if len(self.ds_seq_km) else 0.0
22
23
24 def _weighted_extra(count: int, growth: float) -> np.ndarray:
25     if count <= 1:
26         return np.ones(max(1, count), dtype=float)
27     growth = max(1.0, float(growth))
28     if abs(growth - 1.0) <= 1.0e-9:
29         return np.ones(count, dtype=float)
30     return np.power(growth, np.arange(count, dtype=float))
31
32
33 def _adaptive_ds_sequence(
34     target_km: float,
35     *,
36     max_steps: int,
37     min_ds_km: float,
38     max_ds_km: float,
39     growth: float,
40 ) -> np.ndarray:
41     target_km = max(MIN_DISTANCE_STEP_KM, float(target_km))
42     min_ds_km = max(MIN_DISTANCE_STEP_KM, float(min_ds_km))
43     if max_steps <= 1:
44         return np.array([target_km], dtype=float)
45     if target_km <= min_ds_km:
46         return np.array([target_km], dtype=float)
47
48     step_count = int(min(max_steps, max(1, math.ceil(target_km / min_ds_km))))
49     base = np.full(step_count, min_ds_km, dtype=float)
50     base_sum = float(base.sum())
51     if target_km <= base_sum + 1.0e-9:
52         return np.full(step_count, target_km / step_count, dtype=float)
53
54     weights = _weighted_extra(step_count, growth)
55     weights = weights / max(float(weights.sum()), 1.0e-9)
56     ds = base.copy()
57     extra_remaining = float(target_km - base_sum)
58     cap_value = max(float(max_ds_km), float(min_ds_km))
59     caps = np.maximum(0.0, cap_value - ds)
60     active = caps > 1.0e-9
61     while extra_remaining > 1.0e-9 and np.any(active):
62         active_idx = np.flatnonzero(active)
63         alloc_weights = weights[active_idx]
64         alloc_weights = alloc_weights / max(float(alloc_weights.sum()), 1.0e-9)
65         proposal = extra_remaining * alloc_weights
66         used = 0.0
67         for j, idx in enumerate(active_idx):
68             take = min(float(proposal[j]), float(caps[idx]))
69             ds[idx] += take
70             caps[idx] -= take
71             used += take
72         extra_remaining -= used
73         active = caps > 1.0e-9
74         if used <= 1.0e-9:
75             break
76
77     if extra_remaining > 1.0e-9:
78         ds[-1] += extra_remaining
79     return ds
80
81
82 def build_upper_distance_horizon(
83     *,
84     mode: str,
85     s0_km: float,
86     race_km: float,
87     ds_km: float,
88     horizon_km: float,
89     max_steps: int,
90     ctrl_km: float,
91     adaptive_min_ds_km: float,
92     adaptive_max_ds_km: float,
93     adaptive_growth: float,
94 ) -> UpperDistanceHorizon:
95     ds_km = max(MIN_DISTANCE_STEP_KM, float(ds_km))
96     horizon_km = max(ds_km, float(horizon_km))
97     remaining_km = max(0.0, float(race_km) - float(s0_km))
98     max_steps = max(1, int(max_steps))
99     mode = str(mode or "fixed").strip().lower()
100
101     if mode in {"adaptive_full_race", "remaining_race", "full_race"}:
102         target_km = max(ds_km, remaining_km if remaining_km > 0.0 else horizon_km)
103         ds_seq = _adaptive_ds_sequence(
104             target_km,
105             max_steps=max_steps,
106             min_ds_km=max(ds_km, float(adaptive_min_ds_km)),
107             max_ds_km=max(float(adaptive_max_ds_km), max(ds_km, float(adaptive_min_ds_km))),
108             growth=float(adaptive_growth),
109         )
110     else:
111         target_km = max(ds_km, min(horizon_km, remaining_km if remaining_km > 0.0 else horizon_km))
112         step_count = int(max(1, math.ceil(target_km / ds_km)))
113         step_count = min(step_count, max_steps)

```

```

114     ds_seq = np.full(step_count, ds_km, dtype=float)
115     covered_before_last = float(ds_km * max(0, step_count - 1))
116     ds_seq[-1] = max(MIN_DISTANCE_STEP_KM, target_km - covered_before_last)
117
118     seg_s = np.concatenate([[0.0], np.cumsum(ds_seq[:-1], dtype=float)])
119     total_km = float(np.sum(ds_seq))
120     control_end_km = float(seg_s[-1]) if len(seg_s) else 0.0
121     ctrl_step_km = float(ctrl_km) if ctrl_km and ctrl_km > 0.0 else float(ds_seq[0])
122     ctrl_step_km = max(
123         MIN_DISTANCE_STEP_KM,
124         min(ctrl_step_km, max(control_end_km, MIN_DISTANCE_STEP_KM)),
125     )
126     ctrl_s = np.arange(0.0, control_end_km + 1.0e-9, ctrl_step_km, dtype=float)
127     if len(ctrl_s) == 0:
128         ctrl_s = np.array([0.0], dtype=float)
129     if len(ctrl_s) > len(ds_seq):
130         ctrl_s = np.array(seg_s, dtype=float)
131     if ctrl_s[-1] < control_end_km - 1.0e-9:
132         ctrl_s = np.append(ctrl_s, control_end_km)
133     return UpperDistanceHorizon(ds_seq_km=ds_seq, seg_s_km=seg_s, ctrl_s_km=ctrl_s)
134
135
136 def plan_segment_index(plan_segments: Iterable[dict], s_km: float) -> int:
137     segments: List[dict] = list(plan_segments)
138     if not segments:
139         return -1
140     s_km = float(s_km)
141     for idx, seg in enumerate(segments):
142         s_end_km = float(seg.get("s_end_km", seg.get("s_start_km", 0.0)))
143         if s_km < s_end_km - 1.0e-9:
144             return idx
145     return len(segments) - 1

```

A.17 mpc_solarcar/upper_solver.py

```

1 from __future__ import annotations
2
3 from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor
4 from itertools import product
5 import multiprocessing as mp
6 import os
7 from typing import Callable, Iterable, List, Sequence, Tuple
8
9 import numpy as np
10 from scipy.optimize import minimize, shgo
11
12 Bounds = Sequence[Tuple[float, float]]
13
14
15 _FORK_COST_FN: Callable[[np.ndarray], float] | None = None
16
17
18 def _fork_finite_cost(vec: np.ndarray) -> float:
19     if _FORK_COST_FN is None:
20         return float("inf")
21     return finite_cost(_FORK_COST_FN, vec)
22
23
24 def clip_to_bounds(vec: np.ndarray, bounds: Bounds) -> np.ndarray:
25     out = np.asarray(vec, dtype=float).copy()
26     for idx, (lo, hi) in enumerate(bounds):
27         out[idx] = float(np.clip(out[idx], lo, hi))
28     return out
29
30
31 def finite_cost(cost_fn: Callable[[np.ndarray], float], vec: np.ndarray) -> float:
32     try:
33         value = float(cost_fn(np.asarray(vec, dtype=float)))
34     except Exception:
35         return float("inf")
36     if not np.isfinite(value):
37         return float("inf")
38     return value
39
40
41 def default_seed_library(x0: np.ndarray, bounds: Bounds) -> List[tuple[str, np.ndarray]]:
42     x0 = np.asarray(x0, dtype=float)
43     lo = np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)
44     hi = np.array([float(hi_i) for _, hi_i in bounds], dtype=float)
45     span = np.maximum(hi - lo, 1.0e-6)
46     seeds: List[tuple[str, np.ndarray]] = [("warm_start", clip_to_bounds(x0, bounds))]
47     for frac in (0.25, 0.40, 0.55, 0.70, 0.85):
48         const = lo + frac * span
49         seeds.append(("const_{frac:.2f}", clip_to_bounds(const, bounds)))
50         seeds.append(("warm_mix_{frac:.2f}", clip_to_bounds(0.5 * x0 + 0.5 * const, bounds)))
51     seeds.append(("ramp_up", clip_to_bounds(np.linspace(lo[0], hi[0], len(x0)), bounds)))
52     seeds.append(("ramp_down", clip_to_bounds(np.linspace(hi[0], lo[0], len(x0)), bounds)))
53     return seeds
54
55
56 def hybrid_bounded_minimize(
57     cost_fn: Callable[[np.ndarray], float],
58     x0: np.ndarray,
59     bounds: Bounds,
60     *,
61     maxiter: int,
62     structured_seeds: Iterable[tuple[str, np.ndarray]] | None = None,

```

```

64  cem_enabled: bool = True,
65  cem_mode: str = "auto",
66  cem_generations: int = 4,
67  cem_population: int = 16,
68  cem_elite: int = 4,
69  local_refine_topk: int = 4,
70  seed_library_mode: str = "full",
71  rng_seed: int = 0,
72  shgo_samples: int = 256,
73  shgo_iters: int = 2,
74  cert_grid_levels: int = 0,
75  cert_grid_values: Sequence[float] | None = None,
76  cert_max_evaluations: int = 250_000,
77  cert_workers: int = 1,
78  progress_callback: Callable[[dict], None] | None = None,
79  cert_progress_interval: int = 25,
80 ) -> tuple[np.ndarray, dict]:
81     x0 = np.asarray(x0, dtype=float)
82     bounds = list(bounds)
83     lo = np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)
84     hi = np.array([float(hi_i) for _, hi_i in bounds], dtype=float)
85     span = np.maximum(hi - lo, 1.0e-6)
86     rng = np.random.default_rng(int(rng_seed))
87
88     evaluated: list[dict] = []
89     seen: set[tuple[float, ...]] = set()
90
91     def add_candidate(label: str, vec: np.ndarray) -> None:
92         clipped = clip_to_bounds(vec, bounds)
93         key = tuple(np.round(clipped, 6))
94         if key in seen:
95             return
96         seen.add(key)
97         value = finite_cost(cost_fn, clipped)
98         evaluated.append(
99             {
100                 "label": str(label),
101                 "x": clipped,
102                 "fun": value,
103                 "source": "seed",
104             }
105         )
106
107     base_seeds = list(structured_seeds or [])
108     seed_library_mode_norm = str(seed_library_mode or "full").strip().lower()
109     if seed_library_mode_norm not in {"full", "realtime", "minimal"}:
110         seed_library_mode_norm = "full"
111     default_seeds = default_seed_library(x0, bounds)
112     if seed_library_mode_norm == "minimal":
113         # Production ROS2 cycles already receive the learned/previous policy as
114         # x0 and a physics-based balance seed from the caller. Evaluating the
115         # full generic library here only repeats expensive full-race rollouts.
116         default_seeds = default_seeds[:1]
117     elif seed_library_mode_norm == "realtime":
118         # Keep one neutral cruise alternative on the very first live cycle.
119         # Subsequent cycles use the previous MPC solution as warm_start, while
120         # the caller always adds its weather-dependent balance seed.
121         nominal = lo + 0.625 * span
122         default_seeds = [default_seeds[0], ("nominal_cruise", clip_to_bounds(nominal, bounds))]
123     for label, vec in default_seeds:
124         add_candidate(label, vec)
125     for label, vec in base_seeds:
126         add_candidate(label, vec)
127
128     finite_pool = [row for row in evaluated if np.isfinite(row["fun"])]
129     if not finite_pool:
130         return clip_to_bounds(x0, bounds), {
131             "success": False,
132             "fun": float("inf"),
133             "label": "fallback_warm_start",
134             "nit": 0,
135             "method": "fallback",
136             "candidates_evaluated": int(len(evaluated)),
137             "cem_used": False,
138         }
139
140     finite_pool.sort(key=lambda item: float(item["fun"]))
141     best = dict(finite_pool[0])
142
143     def local_refine_rows(pool: Sequence[dict]) -> list[dict]:
144         refined: list[dict] = []
145         if maxiter <= 0:
146             return refined
147         for row in pool:
148             res = minimize(cost_fn, row["x"], method="L-BFGS-B", bounds=bounds, options=dict(maxiter=int(maxiter)))
149             candidate_x = row["x"]
150             if hasattr(res, "x") and res.x is not None and len(res.x) == len(x0) and np.all(np.isfinite(res.x)):
151                 candidate_x = clip_to_bounds(np.asarray(res.x, dtype=float), bounds)
152             candidate_fun = finite_cost(cost_fn, candidate_x)
153             refined.append(
154                 {
155                     "label": f"local_refine:{row['label']}",
156                     "x": candidate_x,
157                     "fun": candidate_fun,
158                     "source": "local_refine",
159                     "nit": int(getattr(res, "nit", 0) or 0),
160                     "success": bool(getattr(res, "success", False)),
161                     "message": str(getattr(res, "message", "")),
162                 }
163             )
164         refined.sort(key=lambda item: float(item["fun"]))
165         return refined
166

```

```

167 refine_pool = finite_pool[: max(1, int(local_refine_topk))]
168 refined_rows = local_refine_rows(refine_pool)
169 for row in refined_rows:
170     evaluated.append(row)
171 if refined_rows and float(refined_rows[0]["fun"]) < float(best["fun"]):
172     best = dict(refined_rows[0])
173
174 cem_mode_norm = str(cem_mode or "auto").strip().lower()
175 if cem_mode_norm not in {"always", "auto", "never", "shgo", "certify"}:
176     cem_mode_norm = "auto"
177
178 consensus_spread = float("nan")
179 cem_used = False
180 shgo_used = False
181 shgo_nfev = 0
182 shgo_local_minima = 0
183 discrete_global_proof = False
184 discrete_grid_candidates = 0
185 discrete_grid_nonfinite = 0
186 declared_grid_values = np.asarray(list(cert_grid_values or []), dtype=float)
187 declared_grid_values = declared_grid_values[np.isfinite(declared_grid_values)]
188 if declared_grid_values.size:
189     grid_values = np.unique(np.clip(declared_grid_values, lo[0], hi[0]))
190     discrete_grid_levels = int(len(grid_values))
191 else:
192     grid_values = np.asarray([], dtype=float)
193     discrete_grid_levels = max(0, int(cert_grid_levels))
194 deterministic_seed_candidates = int(len(evaluated))
195 deterministic_seed_nonfinite = int(
196     sum(not np.isfinite(float(row.get("fun", float("inf")))) for row in evaluated)
197 )
198 discrete_grid_best_fun = float("inf")
199 discrete_grid_best_x: list[float] = []
200
201 def emit_progress(payload: dict) -> None:
202     if progress_callback is None:
203         return
204     try:
205         progress_callback(dict(payload))
206     except Exception:
207         # Progress telemetry must never change the optimizer result.
208         return
209
210 if cem_mode_norm == "certify" and discrete_grid_levels >= 2 and len(x0) > 0:
211     requested = int(discrete_grid_levels * len(x0))
212     if requested <= max(1, int(cert_max_evaluations)):
213         emit_progress({"stage": "finite_grid", "completed": 0, "total": requested})
214         axes = [
215             grid_values.copy()
216             if grid_values.size
217             else np.linspace(lo_i, hi_i, discrete_grid_levels)
218             for lo_i, hi_i in bounds
219         ]
220         grid_best = None
221         interval = max(1, int(cert_progress_interval))
222         workers = max(1, int(cert_workers))
223         executor_kind = "serial"
224
225 def record_grid_candidate(idx: int, candidate_x: np.ndarray, candidate_fun: float) -> None:
226     nonlocal discrete_grid_candidates, discrete_grid_nonfinite, grid_best
227     discrete_grid_candidates += 1
228     if not np.isfinite(candidate_fun):
229         discrete_grid_nonfinite += 1
230     if grid_best is None or candidate_fun < float(grid_best["fun"]):
231         grid_best = {
232             "label": f"certified_grid_{idx:08d}",
233             "x": candidate_x,
234             "fun": candidate_fun,
235             "source": "certified_grid",
236             "success": np.isfinite(candidate_fun),
237             "message": "exhaustive finite-grid optimum",
238         }
239     completed = idx + 1
240     if completed == requested or completed % interval == 0:
241         emit_progress(
242             {
243                 "stage": "finite_grid",
244                 "completed": completed,
245                 "total": requested,
246                 "best_fun": float(grid_best["fun"]) if grid_best is not None else float("inf"),
247                 "best_x": (
248                     np.asarray(grid_best["x"], dtype=float).tolist()
249                     if grid_best is not None
250                     else []
251                 ),
252                 "nonfinite": int(discrete_grid_nonfinite),
253                 "workers": workers,
254                 "executor": executor_kind,
255             }
256         )
257
258 grid_candidates = [
259     (idx, np.asarray(values, dtype=float))
260     for idx, values in enumerate(product(*axes))
261 ]
262 if workers == 1:
263     for idx, candidate_x in grid_candidates:
264         record_grid_candidate(idx, candidate_x, finite_cost(cost_fn, candidate_x))
265 else:
266     # executor.map preserves input order, so tie-breaking and the certificate
267     # remain deterministic while independent policy evaluations run in parallel.
268     # POSIX fork inherits a nested simulator cost closure without pickling it;
269     # Windows falls back to threads because spawn cannot serialize that closure.

```

```

270     if os.name == "posix":
271         global _FORK_COST_FN
272         _FORK_COST_FN = cost_fn
273         executor_context = ProcessPoolExecutor(
274             max_workers=workers,
275             mp_context=mp.get_context("fork"),
276         )
277         evaluator = _fork_finite_cost
278         executor_kind = "fork_process"
279     else:
280         executor_context = ThreadPoolExecutor(
281             max_workers=workers,
282             thread_name_prefix="upper-cert",
283         )
284         evaluator = lambda item: finite_cost(cost_fn, item[1])
285         executor_kind = "thread"
286     with executor_context as executor:
287         for (idx, candidate_x), candidate_fun in zip(
288             grid_candidates,
289             executor.map(
290                 evaluator,
291                 (
292                     candidate_x if executor_kind == "fork_process" else (item, candidate_x)
293                     for item, candidate_x in grid_candidates
294                 ),
295                 chunksize=1,
296             ),
297         ):
298             record_grid_candidate(idx, candidate_x, candidate_fun)
299     if grid_best is not None and np.isfinite(float(grid_best["fun"])):
300         discrete_grid_best_fun = float(grid_best["fun"])
301         discrete_grid_best_x = np.asarray(grid_best["x"], dtype=float).tolist()
302         evaluated.append(grid_best)
303         discrete_global_proof = discrete_grid_nonfinite == 0
304         if float(grid_best["fun"]) < float(best["fun"]):
305             best = dict(grid_best)
306
307 if cem_mode_norm in {"shgo", "certify"} and len(x0) > 0 and int(shgo_samples) > 0:
308     shgo_used = True
309     emit_progress({"stage": "shgo", "completed": 0, "total": int(shgo_samples)})
310     try:
311         shgo_result = shgo(
312             cost_fn,
313             bounds,
314             n=max(2 ** len(x0) + 1, int(shgo_samples)),
315             iters=max(1, int(shgo_iters)),
316             minimizer_kwargs={
317                 "method": "L-BFGS-B",
318                 "bounds": bounds,
319                 "options": {"maxiter": max(1, int(maxiter))},
320             },
321             sampling_method="simplicial",
322         )
323     shgo_nfev = int(getattr(shgo_result, "nfev", 0) or 0)
324     x1 = getattr(shgo_result, "x1", None)
325     shgo_local_minima = int(len(x1)) if x1 is not None else 0
326     if getattr(shgo_result, "x", None) is not None:
327         shgo_x = clip_to_bounds(np.asarray(shgo_result.x, dtype=float), bounds)
328         shgo_fun = finite_cost(cost_fn, shgo_x)
329         shgo_row = {
330             "label": "shgo_simplicial",
331             "x": shgo_x,
332             "fun": shgo_fun,
333             "source": "shgo",
334             "nit": int(getattr(shgo_result, "nit", 0) or 0),
335             "success": bool(getattr(shgo_result, "success", False)),
336             "message": str(getattr(shgo_result, "message", "")),
337         }
338         evaluated.append(shgo_row)
339         if np.isfinite(shgo_fun) and shgo_fun < float(best["fun"]):
340             best = dict(shgo_row)
341     emit_progress(
342         {
343             "stage": "shgo",
344             "completed": int(shgo_nfev),
345             "total": int(shgo_samples),
346             "best_fun": float(best["fun"]),
347         }
348     )
349 except Exception as exc:
350     evaluated.append(
351         {
352             "label": "shgo_failed",
353             "x": np.asarray(best["x"], dtype=float),
354             "fun": float(best["fun"]),
355             "source": "shgo",
356             "success": False,
357             "message": str(exc),
358         }
359     )
360     emit_progress({"stage": "shgo_failed", "message": str(exc)})
361
362 global_pool = [row for row in evaluated if np.isfinite(float(row.get("fun", float("inf"))))]
363 global_pool.sort(key=lambda item: float(item["fun"]))
364 refined_rows = local_refine_rows(global_pool[: max(1, int(local_refine_topk))])
365 evaluated.extend(refined_rows)
366 if refined_rows and float(refined_rows[0]["fun"]) < float(best["fun"]):
367     best = dict(refined_rows[0])
368
369 def should_run_cem_from_seed_consensus(rows: Sequence[dict]) -> tuple[bool, float]:
370     usable = [row for row in rows if np.isfinite(float(row.get("fun", float("inf"))))]
371     if len(usable) < 2:
372         return False, 0.0

```

```

373     top = usable[: min(3, len(usable))]
374     mat = np.vstack([np.asarray(row["x"], dtype=float) for row in top])
375     spread = float(np.max(np.std(mat, axis=0) / np.maximum(span, 1.0e-6)))
376     best_fun = float(top[0]["fun"])
377     next_fun = float(top[1]["fun"])
378     gap = abs(next_fun - best_fun) / max(1.0, abs(best_fun))
379     run = bool((spread > 0.08) or (gap > 0.10) or (not bool(top[0].get("success", True))))
380     return run, spread
381
382 use_cem = bool(cem_enabled and len(x0) > 0 and cem_mode_norm not in {"never", "shgo", "certify"})
383 if use_cem and cem_mode_norm == "auto":
384     use_cem, consensus_spread = should_run_cem_from_seed_consensus(refined_rows or refine_pool)
385
386 if use_cem:
387     cem_used = True
388     finite_now = [row for row in evaluated if np.isfinite(row["fun"])]
389     mean = np.asarray((best["x"] if np.isfinite(float(best["fun"])) else (finite_now[0]["x"] if finite_now else x0)), dtype=float).copy()
390     sigma = np.maximum(0.12 * span, 0.5)
391     elite_count = max(1, min(int(cem_elite), int(cem_population)))
392     for gen in range(max(0, int(cem_generations))):
393         pop: list[dict] = []
394         pop.append({"label": f"cem_mean_g{gen:02d}", "x": clip_to_bounds(mean, bounds)})
395         for idx in range(max(0, int(cem_population) - 1)):
396             sample = rng.normal(mean, sigma)
397             pop.append({"label": f"cem_g{gen:02d}_p{idx:02d}", "x": clip_to_bounds(sample, bounds)})
398         for row in pop:
399             row["fun"] = finite_cost(cost_fn, row["x"])
400             row["source"] = "cem"
401             evaluated.append(row)
402         finite_pop = [row for row in pop if np.isfinite(row["fun"])]
403         if not finite_pop:
404             continue
405         finite_pop.sort(key=lambda item: float(item["fun"]))
406         elite = finite_pop[:elite_count]
407         elite_mat = np.vstack([row["x"] for row in elite])
408         mean = np.mean(elite_mat, axis=0)
409         sigma = np.maximum(np.std(elite_mat, axis=0), 0.05 * span)
410
411     finite_pool = [row for row in evaluated if np.isfinite(row["fun"])]
412     finite_pool.sort(key=lambda item: float(item["fun"]))
413     best = dict(finite_pool[0])
414     refine_pool = finite_pool[: max(1, int(local_refine_topk))]
415     refined_rows = local_refine_rows(refine_pool)
416     for row in refined_rows:
417         evaluated.append(row)
418     if refined_rows and float(refined_rows[0]["fun"]) < float(best["fun"]):
419         best = dict(refined_rows[0])
420
421 selected_no_worse_than_grid = bool(
422     discrete_global_proof
423     and np.isfinite(discrete_grid_best_fun)
424     and float(best["fun"]) <= discrete_grid_best_fun + 1.0e-9 * max(1.0, abs(discrete_grid_best_fun))
425 )
426 finite_library_global_proof = bool(
427     discrete_global_proof
428     and deterministic_seed_nonfinite == 0
429     and int(maxiter) <= 0
430     and not cem_used
431     and not shgo_used
432     and selected_no_worse_than_grid
433 )
434 if finite_library_global_proof:
435     certificate_scope = "exact optimum over deterministic seeds plus exhaustive Cartesian speed grid"
436 elif selected_no_worse_than_grid:
437     certificate_scope = "selected candidate is no worse than the exhaustive finite-grid minimum; no continuous proof"
438 elif discrete_global_proof:
439     certificate_scope = "exhaustive finite-grid minimum evaluated; selected candidate has no dominance certificate"
440 else:
441     certificate_scope = "no exhaustive finite-grid certificate"
442
443 return np.asarray(best["x"], dtype=float), {
444     "success": bool(best.get("success", True)),
445     "fun": float(best["fun"]),
446     "label": str(best.get("label", "best")),
447     "nit": int(best.get("nit", 0) or 0),
448     "method": str(best.get("source", "seed")),
449     "message": str(best.get("message", "")),
450     "candidates_evaluated": int(len(evaluated) + discrete_grid_candidates + shgo_nfev),
451     "cem_mode": cem_mode_norm,
452     "cem_used": cem_used,
453     "seed_consensus_spread": consensus_spread,
454     "shgo_used": shgo_used,
455     "shgo_nfev": shgo_nfev,
456     "shgo_local_minima": shgo_local_minima,
457     "discrete_global_proof": discrete_global_proof,
458     "discrete_grid_levels": discrete_grid_levels,
459     "discrete_grid_values": grid_values.tolist() if grid_values.size else [],
460     "discrete_grid_candidates": discrete_grid_candidates,
461     "discrete_grid_nonfinite": discrete_grid_nonfinite,
462     "discrete_grid_best_fun": discrete_grid_best_fun,
463     "discrete_grid_best_x": discrete_grid_best_x,
464     "deterministic_seed_candidates": deterministic_seed_candidates,
465     "deterministic_seed_nonfinite": deterministic_seed_nonfinite,
466     "finite_library_candidates": int(deterministic_seed_candidates + discrete_grid_candidates),
467     "seed_library_mode": seed_library_mode_norm,
468     "selected_x": np.asarray(best["x"], dtype=float).tolist(),
469     "selected_no_worse_than_grid": selected_no_worse_than_grid,
470     "finite_library_global_proof": finite_library_global_proof,
471     "continuous_global_proof": False,
472     "certificate_scope": certificate_scope,
473 }

```

A.18 mpc_solarcar/estimator.py

```

1 import math
2 from collections import deque
3 from dataclasses import dataclass
4 from typing import Optional, Tuple
5
6 import numpy as np
7 from scipy.optimize import minimize
8
9
10 @dataclass
11 class MheInput:
12     v_ms: float
13     slope_pct: float
14     G_poa: float
15     Tcell_C: float
16     Tamb_C: float
17     headwind_ms: float
18     dt: float
19     inertial_power_w: float = 0.0
20     elevation_m: float = 0.0
21
22
23 @dataclass
24 class MheMeas:
25     soc: Optional[float] = None
26     Tb: Optional[float] = None
27     I: Optional[float] = None
28     V: Optional[float] = None
29
30
31 class BatteryMHE:
32     def __init__(
33         self,
34         model,
35         horizon_steps: int = 12,
36         w_soc: float = 50.0,
37         w_tb: float = 5.0,
38         w_i: float = 1.0,
39         w_v: float = 1.0,
40         w_prior: float = 5.0,
41         soc_bounds: Tuple[float, float] = (0.05, 0.98),
42         tb_bounds: Tuple[float, float] = (-10.0, 65.0),
43     ):
44         self.model = model
45         self.samples = deque(maxlen=max(2, int(horizon_steps)))
46         self.w_soc = float(w_soc)
47         self.w_tb = float(w_tb)
48         self.w_i = float(w_i)
49         self.w_v = float(w_v)
50         self.w_prior = float(w_prior)
51         self.soc_bounds = soc_bounds
52         self.tb_bounds = tb_bounds
53
54     def push(self, u: MheInput, y: MheMeas):
55         self.samples.append((u, y))
56
57     def _simulate(self, z0: float, Tb0: float):
58         z = float(z0)
59         Tb = float(Tb0)
60         outputs = []
61         for (u, _) in self.samples:
62             out = self.model.electrical_balance(
63                 u.v_ms,
64                 u.slope_pct,
65                 z,
66                 Tb,
67                 u.G_poa,
68                 u.Tcell_C,
69                 headwind_ms=u.headwind_ms,
70                 inertial_power_w=u.inertial_power_w,
71                 ambient_temp_c=u.Tamb_C,
72                 elevation_m=u.elevation_m,
73             )
74             I = float(out['I'])
75             V = float(out['V'])
76             P_pack = float(out['P_pack'])
77             loss_int = float(out['losses_int'])
78             dt = float(u.dt)
79             z_next = self.model.soc_step(z, P_pack, dt)
80             Tb_next = Tb + (dt / 1800.0) * (u.Tamb_C - Tb) + (loss_int * dt) / 50000.0
81             outputs.append((z_next, Tb_next, I, V))
82             z, Tb = z_next, Tb_next
83         return outputs, z, Tb
84
85     def estimate(self, z_init: float, Tb_init: float) -> Tuple[float, float]:
86         if len(self.samples) < 2:
87             return z_init, Tb_init
88
89         def cost(x):
90             z0, Tb0 = float(x[0]), float(x[1])
91             J = self.w_prior * ((z0 - z_init) ** 2 + (Tb0 - Tb_init) ** 2)
92             outputs, _, _ = self._simulate(z0, Tb0)
93             for (_, meas), (z_pred, Tb_pred, I_pred, V_pred) in zip(self.samples, outputs):
94                 if meas.soc is not None and math.isfinite(meas.soc):
95                     J += self.w_soc * (z_pred - meas.soc) ** 2
96                 if meas.Tb is not None and math.isfinite(meas.Tb):
97                     J += self.w_tb * (Tb_pred - meas.Tb) ** 2
98                 if meas.I is not None and math.isfinite(meas.I):
99                     J += self.w_i * (I_pred - meas.I) ** 2
100                 if meas.V is not None and math.isfinite(meas.V):

```

```

101         J += self.w_v * (V_pred - meas.V) ** 2
102     return J
103
104     x0 = np.array([float(z_init), float(Tb_init)], dtype=float)
105     bounds = [self.soc_bounds, self.tb_bounds]
106     res = minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=80))
107     if not res.success:
108         return z_init, Tb_init
109     z0, Tb0 = float(res.x[0]), float(res.x[1])
110     _, zN, TbN = self._simulate(z0, Tb0)
111     return float(zN), float(TbN)

```

A.19 mpc_solarcar/signal_utils.py

```

1 from __future__ import annotations
2
3 import math
4 import time
5
6
7 def finite_float(value, default: float = math.nan) -> float:
8     try:
9         out = float(value)
10    except Exception:
11        return float(default)
12    return out if math.isfinite(out) else float(default)
13
14
15 def fresh_enough(last_time_sec: float | None, now_sec: float | None, timeout_sec: float) -> bool:
16     if last_time_sec is None or now_sec is None:
17         return False
18     if timeout_sec <= 0.0:
19         return True
20     return (float(now_sec) - float(last_time_sec)) <= float(timeout_sec)
21
22
23 def slew_limit(
24     previous: float,
25     target: float,
26     dt_sec: float,
27     rise_rate: float | None = None,
28     fall_rate: float | None = None,
29 ) -> float:
30     prev = finite_float(previous, 0.0)
31     tgt = finite_float(target, prev)
32     dt = max(0.0, finite_float(dt_sec, 0.0))
33     delta = tgt - prev
34     if delta >= 0.0 and rise_rate is not None and rise_rate > 0.0:
35         delta = min(delta, float(rise_rate) * dt)
36     if delta < 0.0 and fall_rate is not None and fall_rate > 0.0:
37         delta = max(delta, -float(fall_rate) * dt)
38     return prev + delta
39
40
41 class RobustScalarFilter:
42     def __init__(
43         self,
44         *,
45         tau_sec: float = 0.5,
46         initial_value: float = math.nan,
47         min_value: float | None = None,
48         max_value: float | None = None,
49         rise_rate: float | None = None,
50         fall_rate: float | None = None,
51     ) -> None:
52         self.tau_sec = max(0.0, finite_float(tau_sec, 0.5))
53         self.value = finite_float(initial_value, math.nan)
54         self.min_value = min_value
55         self.max_value = max_value
56         self.rise_rate = rise_rate
57         self.fall_rate = fall_rate
58         self.last_time = None
59
60     def reset(self, value: float, *, now: float | None = None) -> float:
61         self.value = finite_float(value, self.value)
62         self.last_time = time.monotonic() if now is None else float(now)
63         return self.value
64
65     def update(self, value: float, *, now: float | None = None) -> float:
66         raw = finite_float(value, math.nan)
67         timestamp = time.monotonic() if now is None else float(now)
68         if not math.isfinite(raw):
69             self.last_time = timestamp
70             return self.value
71         if not math.isfinite(self.value):
72             self.value = raw
73             self.last_time = timestamp
74             return self._clamp(self.value)
75         dt = 0.0 if self.last_time is None else max(0.0, timestamp - self.last_time)
76         if self.tau_sec > 0.0 and dt > 0.0:
77             alpha = 1.0 - math.exp(-dt / self.tau_sec)
78             candidate = self.value + alpha * (raw - self.value)
79         else:
80             candidate = raw
81         candidate = slew_limit(self.value, candidate, dt, self.rise_rate, self.fall_rate)
82         self.value = self._clamp(candidate)
83         self.last_time = timestamp
84         return self.value

```

```

85
86 def _clamp(self, value: float) -> float:
87     out = finite_float(value, self.value)
88     if self.min_value is not None:
89         out = max(float(self.min_value), out)
90     if self.max_value is not None:
91         out = min(float(self.max_value), out)
92     return out
93
94
95 class SmoothRateLimiter:
96     def __init__(
97         self,
98         *,
99         min_value: float | None = None,
100         max_value: float | None = None,
101         tau_sec: float = 0.0,
102         rise_rate: float | None = None,
103         fall_rate: float | None = None,
104         deadband: float = 0.0,
105         quantize_step: float = 0.0,
106         initial_value: float = 0.0,
107     ) -> None:
108         self.min_value = min_value
109         self.max_value = max_value
110         self.tau_sec = max(0.0, finite_float(tau_sec, 0.0))
111         self.rise_rate = rise_rate
112         self.fall_rate = fall_rate
113         self.deadband = max(0.0, finite_float(deadband, 0.0))
114         self.quantize_step = max(0.0, finite_float(quantize_step, 0.0))
115         self.value = self._clamp(finite_float(initial_value, 0.0))
116         self.last_time = None
117
118     def reset(self, value: float, *, now: float | None = None) -> float:
119         self.value = self._clamp(finite_float(value, self.value))
120         self.last_time = time.monotonic() if now is None else float(now)
121         return self.value
122
123     def update(self, target: float, *, now: float | None = None) -> float:
124         timestamp = time.monotonic() if now is None else float(now)
125         raw_target = self._clamp(finite_float(target, self.value))
126         if self.last_time is None:
127             self.last_time = timestamp
128             self.value = raw_target
129             return self._postprocess(self.value, reference=self.value)
130         dt = max(0.0, timestamp - self.last_time)
131         if self.tau_sec > 0.0 and dt > 0.0:
132             alpha = 1.0 - math.exp(-dt / self.tau_sec)
133             shaped = self.value + alpha * (raw_target - self.value)
134         else:
135             shaped = raw_target
136         limited = slew_limit(self.value, shaped, dt, self.rise_rate, self.fall_rate)
137         self.value = self._postprocess(self._clamp(limited), reference=self.value)
138         self.last_time = timestamp
139         return self.value
140
141     def _clamp(self, value: float) -> float:
142         out = finite_float(value, 0.0)
143         if self.min_value is not None:
144             out = max(float(self.min_value), out)
145         if self.max_value is not None:
146             out = min(float(self.max_value), out)
147         return out
148
149     def _postprocess(self, value: float, *, reference: float) -> float:
150         out = float(value)
151         if abs(out - reference) < self.deadband:
152             out = float(reference)
153         if self.quantize_step > 0.0:
154             out = round(out / self.quantize_step) * self.quantize_step
155         return self._clamp(out)

```

A.20 mpc_solarcar/solar_preflight_logic.py

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5
6 @dataclass(frozen=True)
7 class FreshnessResult:
8     state: str
9     health: float
10    diagnostic: str
11
12
13 @dataclass(frozen=True)
14 class CommandGateResult:
15     allowed: bool
16     reason: str
17
18
19 def evaluate_freshness(
20     *,
21     elapsed_sec: float,
22     ages_sec: dict[str, float | None],
23     required: tuple[str, ...],
24     timeout_sec: float,

```

```

25     startup_grace_sec: float,
26 ) -> FreshnessResult:
27     if elapsed_sec < startup_grace_sec:
28         return FreshnessResult("STARTING", 0.25, "waiting for required solar telemetry")
29
30     missing = [name for name in required if ages_sec.get(name) is None]
31     stale = [
32         name
33         for name in required
34         if ages_sec.get(name) is not None and float(ages_sec[name]) > timeout_sec
35     ]
36     if missing:
37         return FreshnessResult("DEGRADED", 0.2, "missing: " + ", ".join(missing))
38     if stale:
39         return FreshnessResult("DEGRADED", 0.4, "stale: " + ", ".join(stale))
40     return FreshnessResult("RUNNING", 1.0, "solar telemetry and planner inputs are fresh")
41
42
43 def evaluate_command_gate(
44     *,
45     elapsed_sec: float,
46     speed_input_age_sec: float | None,
47     system_state: str,
48     system_state_age_sec: float | None,
49     startup_hold_sec: float,
50     input_timeout_sec: float,
51     system_state_timeout_sec: float,
52     require_system_running: bool,
53 ) -> CommandGateResult:
54     if elapsed_sec < max(0.0, startup_hold_sec):
55         return CommandGateResult(False, "startup_hold")
56     if speed_input_age_sec is None:
57         return CommandGateResult(False, "missing_speed_command")
58     if speed_input_age_sec > max(0.0, input_timeout_sec):
59         return CommandGateResult(False, "stale_speed_command")
60     if not require_system_running:
61         return CommandGateResult(True, "ok_without_system_gate")
62     if system_state_age_sec is None:
63         return CommandGateResult(False, "missing_system_state")
64     if system_state_age_sec > max(0.0, system_state_timeout_sec):
65         return CommandGateResult(False, "stale_system_state")
66     if str(system_state).strip().upper() != "RUNNING":
67         return CommandGateResult(False, "system_not_running")
68     return CommandGateResult(True, "ok")

```

A.21 mpc_solarcar/solar_preflight_node.py

```

1 from __future__ import annotations
2
3 import time
4
5 import rclpy
6 from rclpy.node import Node
7 from std_msgs.msg import Float32, Float32MultiArray, String
8
9 from .solar_preflight_logic import evaluate_freshness
10
11
12 class SolarPreflightNode(Node):
13     """Preflight monitor based only on solar-car telemetry freshness."""
14
15     def __init__(self):
16         super().__init__("solar_preflight_node")
17         self.declare_parameter("startup_grace_sec", 10.0)
18         self.declare_parameter("measurement_timeout_sec", 3.0)
19         self.declare_parameter("require_speed", True)
20         self.declare_parameter("require_distance", True)
21         self.declare_parameter("require_battery", True)
22         self.declare_parameter("require_planner", True)
23
24         self.start_mono = time.monotonic()
25         self.last_seen: dict[str, float] = {}
26
27         self.pub_state = self.create_publisher(String, "/system/state", 10)
28         self.pub_health = self.create_publisher(Float32, "/system/health", 10)
29         self.pub_diag = self.create_publisher(String, "/system/diag", 10)
30
31         self.create_subscription(Float32, "/vehicle/speed_kmh", self._seen("speed"), 10)
32         self.create_subscription(Float32, "/vehicle/s_kmh", self._seen("distance"), 10)
33         self.create_subscription(Float32, "/vehicle/batt_soc", self._seen("batt_soc"), 10)
34         self.create_subscription(Float32, "/vehicle/batt_voltage_v", self._seen("batt_voltage"), 10)
35         self.create_subscription(Float32, "/vehicle/batt_current_a", self._seen("batt_current"), 10)
36         self.create_subscription(Float32MultiArray, "/planner/status", self._seen("planner"), 10)
37
38         self.timer = self.create_timer(1.0, self._step)
39         self.get_logger().info("SolarPreflightNode started.")
40
41     def _seen(self, key: str):
42         def callback(_msg):
43             self.last_seen[key] = time.monotonic()
44
45         return callback
46
47     def _required(self) -> tuple[str, ...]:
48         required: list[str] = []
49         if bool(self.get_parameter("require_speed").value):
50             required.append("speed")
51         if bool(self.get_parameter("require_distance").value):

```

```

52         required.append("distance")
53     if bool(self.get_parameter("require_battery").value):
54         required.extend(("batt_soc", "batt_voltage", "batt_current"))
55     if bool(self.get_parameter("require_planner").value):
56         required.append("planner")
57     return tuple(required)
58
59     def _step(self):
60         now = time.monotonic()
61         required = self._required()
62         ages = {
63             key: (now - self.last_seen[key]) if key in self.last_seen else None
64             for key in required
65         }
66         result = evaluate_freshness(
67             elapsed_sec=now - self.start_mono,
68             ages_sec=ages,
69             required=required,
70             timeout_sec=float(self.get_parameter("measurement_timeout_sec").value),
71             startup_grace_sec=float(self.get_parameter("startup_grace_sec").value),
72         )
73         self.pub_state.publish(String(data=result.state))
74         self.pub_health.publish(Float32(data=float(result.health)))
75         self.pub_diag.publish(String(data=result.diagnostic))
76
77
78     def main():
79         rclpy.init()
80         node = SolarPreflightNode()
81         try:
82             rclpy.spin(node)
83         finally:
84             node.destroy_node()
85             rclpy.shutdown()
86
87
88 if __name__ == "__main__":
89     main()

```

A.22 mpc_solarcar/distance_node.py

```

1 import math
2
3 import rclpy
4 from rclpy.node import Node
5 from std_msgs.msg import Float32
6 from std_srvs.srv import Trigger
7
8
9 class DistanceNode(Node):
10     def __init__(self):
11         super().__init__('distance_node')
12         self.declare_parameter('max_dt_sec', 2.5)
13         self.declare_parameter('publish_rate_hz', 1.0)
14
15         self.max_dt_sec = float(self.get_parameter('max_dt_sec').value)
16         publish_rate_hz = float(self.get_parameter('publish_rate_hz').value)
17
18         self.s_kmh = 0.0
19         self.last_time = None
20         self.last_speed = math.nan
21
22         self.sub_speed = self.create_subscription(Float32, '/vehicle/speed_kmh', self.on_speed, 10)
23         self.pub_s = self.create_publisher(Float32, '/vehicle/s_kmh', 10)
24         self.srv_reset = self.create_service(Trigger, '/vehicle/reset_odometry', self.on_reset)
25
26         self.timer = self.create_timer(1.0 / publish_rate_hz, self._publish)
27         self.get_logger().info('DistanceNode started.')
28
29     def _on_speed(self, msg: Float32):
30         v_kmh = float(msg.data)
31         now_sec = self.get_clock().now().nanoseconds / 1e9
32         if self.last_time is not None and math.isfinite(self.last_speed) and math.isfinite(v_kmh):
33             dt = now_sec - self.last_time
34             if 0.0 < dt <= self.max_dt_sec:
35                 v_avg = 0.5 * (self.last_speed + v_kmh)
36                 self.s_kmh += v_avg * (dt / 3600.0)
37         self.last_time = now_sec
38         self.last_speed = v_kmh
39
40     def _on_reset(self, request, response):
41         self.s_kmh = 0.0
42         self.last_time = None
43         self.last_speed = math.nan
44         response.success = True
45         response.message = 'odometry reset'
46         return response
47
48     def _publish(self):
49         msg = Float32()
50         msg.data = float(self.s_kmh)
51         self.pub_s.publish(msg)
52
53
54 def main():
55     rclpy.init()
56     node = DistanceNode()
57     rclpy.spin(node)

```

```

58 node.destroy_node()
59 rclpy.shutdown()

```

A.23 mpc_solarcar/grade_node.py

```

1 import math
2 import time
3
4 import rclpy
5 from rclpy.node import Node
6 from std_msgs.msg import Float32
7 from sensor_msgs.msg import NavSatFix
8
9
10 class GradeNode(Node):
11     def __init__(self):
12         super().__init__('grade_node')
13         self.declare_parameter('min_speed_kmh', 5.0)
14         self.declare_parameter('altitude_alpha', 0.2)
15         self.declare_parameter('min_delta_s_kmh', 0.01)
16         self.declare_parameter('gps_topic', '/sim/gps')
17         self.declare_parameter('altitude_topic', '/vehicle/altitude_m')
18
19         self.min_speed_kmh = float(self.get_parameter('min_speed_kmh').value)
20         self.alpha = float(self.get_parameter('altitude_alpha').value)
21         self.min_delta_s_kmh = float(self.get_parameter('min_delta_s_kmh').value)
22         self.gps_topic = self.get_parameter('gps_topic').value
23         self.altitude_topic = self.get_parameter('altitude_topic').value
24
25         self.altitude_m = math.nan
26         self.altitude_ema = math.nan
27         self.s_kmh = math.nan
28         self.speed_kmh = math.nan
29         self.last_s_kmh = None
30         self.last_alt_ema = None
31         self.grade = math.nan
32         self.last_update = time.monotonic()
33
34         self.create_subscription(Float32, '/vehicle/s_kmh', self._on_s_kmh, 10)
35         self.create_subscription(Float32, '/vehicle/speed_kmh', self._on_speed, 10)
36         self.create_subscription(Float32, self.altitude_topic, self._on_altitude, 10)
37         self.create_subscription(NavSatFix, self.gps_topic, self._on_gps, 10)
38
39         self.pub_grade = self.create_publisher(Float32, '/vehicle/grade', 10)
40         self.timer = self.create_timer(1.0, self._step)
41         self.get_logger().info('GradeNode started.')
42
43     def _on_s_kmh(self, msg: Float32):
44         self.s_kmh = float(msg.data)
45
46     def _on_speed(self, msg: Float32):
47         self.speed_kmh = float(msg.data)
48
49     def _on_altitude(self, msg: Float32):
50         self._update_altitude(float(msg.data))
51
52     def _on_gps(self, msg: NavSatFix):
53         if math.isfinite(msg.altitude):
54             self._update_altitude(float(msg.altitude))
55
56     def _update_altitude(self, altitude_m: float):
57         self.altitude_m = altitude_m
58         if not math.isfinite(self.altitude_ema):
59             self.altitude_ema = altitude_m
60         else:
61             self.altitude_ema = self.alpha * altitude_m + (1.0 - self.alpha) * self.altitude_ema
62
63     def _step(self):
64         if not math.isfinite(self.s_kmh) or not math.isfinite(self.altitude_ema):
65             self.pub_grade.publish(Float32(data=math.nan))
66             return
67         if not math.isfinite(self.speed_kmh) or self.speed_kmh < self.min_speed_kmh:
68             self.pub_grade.publish(Float32(data=self.grade if math.isfinite(self.grade) else math.nan))
69             return
70         if self.last_s_kmh is None or self.last_alt_ema is None:
71             self.last_s_kmh = self.s_kmh
72             self.last_alt_ema = self.altitude_ema
73             self.pub_grade.publish(Float32(data=math.nan))
74             return
75
76         delta_s_kmh = self.s_kmh - self.last_s_kmh
77         if delta_s_kmh < self.min_delta_s_kmh:
78             self.pub_grade.publish(Float32(data=self.grade if math.isfinite(self.grade) else math.nan))
79             return
80
81         delta_alt = self.altitude_ema - self.last_alt_ema
82         grade = delta_alt / (delta_s_kmh * 1000.0)
83         self.grade = float(grade)
84         self.last_s_kmh = self.s_kmh
85         self.last_alt_ema = self.altitude_ema
86         self.pub_grade.publish(Float32(data=self.grade))
87
88
89 def main():
90     rclpy.init()
91     node = GradeNode()
92     rclpy.spin(node)
93     node.destroy_node()

```

```
94 rclpy.shutdown()
```

A.24 mpc_solarcar/weather_utils.py

```
1 from __future__ import annotations
2
3 import json
4 import urllib.parse
5 import urllib.request
6 from pathlib import Path
7
8 import numpy as np
9 import pandas as pd
10
11
12 HOURLY_VARS = [
13     "temperature_2m",
14     "shortwave_radiation_instant",
15     "direct_normal_irradiance_instant",
16     "diffuse_radiation_instant",
17     "direct_radiation_instant",
18     "wind_speed_10m",
19     "wind_direction_10m",
20 ]
21 FORECAST_COLUMNS = [
22     "time",
23     "GHI",
24     "DNI",
25     "DHI",
26     "BHI",
27     "POA_drive",
28     "POA_stop_ideal",
29     "Tamb_C",
30     "Tcell_C",
31     "Tcell_drive_C",
32     "Tcell_stop_ideal_C",
33     "headwind_ms",
34     "wind_speed_ms",
35     "wind_dir_deg",
36     "radiation_temporal_semantics",
37     "weather_source",
38 ]
39
40
41 def _fetch_json(url: str) -> dict:
42     req = urllib.request.Request(url, headers={"User-Agent": "solar-ws-weather-utils/1.0"})
43     with urllib.request.urlopen(req, timeout=60) as resp:
44         return json.load(resp)
45
46
47 def ideal_tracking_poa_wm2(ghi, dni, dhi, *, ground_albedo: float = 0.20):
48     ghi_values = np.clip(np.asarray(ghi, dtype=float), 0.0, None)
49     dni_values = np.clip(np.asarray(dni, dtype=float), 0.0, None)
50     dhi_values = np.clip(np.asarray(dhi, dtype=float), 0.0, None)
51     cos_zenith = np.divide(
52         np.clip(ghi_values - dhi_values, 0.0, None),
53         dni_values,
54         out=np.zeros_like(ghi_values),
55         where=dni_values > 1.0,
56     )
57     cos_zenith = np.clip(cos_zenith, 0.0, 1.0)
58     sky = dhi_values * (1.0 + cos_zenith) * 0.5
59     ground = float(ground_albedo) * ghi_values * (1.0 - cos_zenith) * 0.5
60     return np.where(
61         (ghi_values <= 1.0) & (dni_values <= 1.0),
62         0.0,
63         np.maximum(dni_values + sky + ground, 0.0),
64     )
65
66
67 def fetch_openmeteo_forecast(
68     latitude: float,
69     longitude: float,
70     *,
71     timezone_name: str = "UTC",
72     forecast_days: int = 3,
73     step_minutes: int = 10,
74     tcell_gain: float = 0.03,
75 ) -> pd.DataFrame:
76     params = urllib.parse.urlencode(
77         {
78             "latitude": float(latitude),
79             "longitude": float(longitude),
80             "forecast_days": int(max(1, forecast_days)),
81             "hourly": ", ".join(HOURLY_VARS),
82             "timezone": str(timezone_name or "UTC"),
83             "wind_speed_unit": "ms",
84         }
85     )
86     payload = _fetch_json(f"https://api.open-meteo.com/v1/forecast?{params}")
87     hourly = payload.get("hourly", {}) or {}
88     df = pd.DataFrame(hourly)
89     if df.empty or "time" not in df.columns:
90         return pd.DataFrame(columns=FORECAST_COLUMNS)
91     time_local = pd.to_datetime(df["time"], format="mixed", errors="coerce")
92     work = pd.DataFrame(
93         {
94             "time_local": time_local,
```

```

95         "Tamb_C": pd.to_numeric(df.get("temperature_2m"), errors="coerce"),
96         "GHI": pd.to_numeric(df.get("shortwave_radiation_instant"), errors="coerce"),
97         "DNI": pd.to_numeric(df.get("direct_normal_irradiance_instant"), errors="coerce"),
98         "DHI": pd.to_numeric(df.get("diffuse_radiation_instant"), errors="coerce"),
99         "BHI": pd.to_numeric(df.get("direct_radiation_instant"), errors="coerce"),
100         "wind_speed_ms": pd.to_numeric(df.get("wind_speed_10m"), errors="coerce"),
101         "wind_dir_deg": pd.to_numeric(df.get("wind_direction_10m"), errors="coerce"),
102     }
103     ).dropna(subset=["time_local"])
104     if work.empty:
105         return pd.DataFrame(columns=FORECAST_COLUMNS)
106     work = work.set_index("time_local").sort_index()
107     freq = f"{max(1, int(step_minutes))}min"
108     work = work.resample(freq).interpolate(method="time").ffill().bfill()
109     work["POA_drive"] = work["GHI"].fillna(0.0)
110     work["POA_stop_ideal"] = ideal_tracking_poa_wm2(work["GHI"], work["DNI"], work["DHI"])
111     work["Tcell_drive_C"] = work["Tamb_C"] + float(tcell_gain) * work["POA_drive"]
112     work["Tcell_stop_ideal_C"] = work["Tamb_C"] + float(tcell_gain) * work["POA_stop_ideal"]
113     work["Tcell_C"] = work["Tcell_drive_C"]
114     work["headwind_ms"] = 0.0
115     work["radiation_temporal_semantics"] = "instant_at_timestamp"
116     work["weather_source"] = "openmeteo_forecast_instant"
117     work = work.reset_index()
118     work["time"] = work["time_local"].dt.strftime("%Y-%m-%d %H:%M:%S")
119     return work[FORECAST_COLUMNS]
120
121
122 def write_forecast_csv(df: pd.DataFrame, out_csv: str | Path) -> None:
123     out_path = Path(out_csv)
124     out_path.parent.mkdir(parents=True, exist_ok=True)
125     work = df.copy()
126     if "time" in work.columns:
127         work = work.sort_values("time")
128     work.to_csv(out_path, index=False)

```

A.25 mpc_solarcar/weather_fetch_node.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import rclpy
6 from rclpy.node import Node
7
8 from sensor_msgs.msg import NavSatFix
9 from std_msgs.msg import String
10
11 from .solar_profile import get_path, get_section, load_profile
12 from .weather_utils import fetch_openmeteo_forecast, write_forecast_csv
13
14
15 class WeatherFetchNode(Node):
16     def __init__(self) -> None:
17         super().__init__("weather_fetch_node")
18         self.declare_parameter("profile_yaml", "")
19         self.declare_parameter("output_csv", "")
20         self.declare_parameter("raw_forecast_csv", "")
21         self.declare_parameter("gps_topic", "/chase/gps")
22         self.declare_parameter("fetch_period_sec", 3600.0)
23         self.declare_parameter("forecast_days", 3)
24         self.declare_parameter("step_minutes", 10)
25         self.declare_parameter("timezone_name", "Australia/Darwin")
26         self.declare_parameter("fallback_latitude", -12.4634)
27         self.declare_parameter("fallback_longitude", 130.8456)
28         self.declare_parameter("tcell_gain", 0.03)
29
30     profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
31     if profile_yaml:
32         profile_path, cfg = load_profile(profile_yaml)
33         runtime_cfg = get_section(cfg, "runtime")
34         live_cfg = get_section(cfg, "live")
35         weather_cfg = get_section(live_cfg, "weather")
36         if not self.get_parameter("output_csv").value:
37             self.set_parameters(
38                 [
39                     rclpy.parameter.Parameter("output_csv", value=get_path(cfg, profile_path, "forecast_csv")),
40                     rclpy.parameter.Parameter(
41                         "raw_forecast_csv",
42                         value=str(weather_cfg.get("raw_forecast_csv", "") or ""),
43                     ),
44                     rclpy.parameter.Parameter(
45                         "gps_topic",
46                         value=str(weather_cfg.get("gps_topic", "/chase/gps")),
47                     ),
48                     rclpy.parameter.Parameter(
49                         "fetch_period_sec",
50                         value=float(weather_cfg.get("fetch_period_sec", 3600.0)),
51                     ),
52                     rclpy.parameter.Parameter(
53                         "forecast_days",
54                         value=int(weather_cfg.get("forecast_days", 3)),
55                     ),
56                     rclpy.parameter.Parameter(
57                         "step_minutes",
58                         value=int(weather_cfg.get("step_minutes", 10)),
59                     ),
60                     rclpy.parameter.Parameter(
61                         "timezone_name",

```

```

62         value=str(
63             weather_cfg.get(
64                 "timezone_name",
65                 live_cfg.get("forecast_time_tz", runtime_cfg.get("forecast_time_tz", "Australia/Darwin")),
66             )
67         ),
68     ),
69     rclpy.parameter.Parameter(
70         "fallback_latitude",
71         value=float(weather_cfg.get("fallback_latitude", -12.4634)),
72     ),
73     rclpy.parameter.Parameter(
74         "fallback_longitude",
75         value=float(weather_cfg.get("fallback_longitude", 130.8456)),
76     ),
77     rclpy.parameter.Parameter(
78         "tcell_gain",
79         value=float(weather_cfg.get("tcell_gain", 0.03)),
80     ),
81 ]
82 )
83
84 raw_out = str(self.get_parameter("raw_forecast_csv").value or "").strip()
85 self.output_csv = Path(str(self.get_parameter("output_csv").value or "")).expanduser()
86 self.raw_forecast_csv = Path(raw_out).expanduser() if raw_out else None
87 self.gps_topic = str(self.get_parameter("gps_topic").value)
88 self.fetch_period_sec = max(60.0, float(self.get_parameter("fetch_period_sec").value))
89 self.forecast_days = max(1, int(self.get_parameter("forecast_days").value))
90 self.step_minutes = max(1, int(self.get_parameter("step_minutes").value))
91 self.timezone_name = str(self.get_parameter("timezone_name").value or "Australia/Darwin")
92 self.fallback_lat = float(self.get_parameter("fallback_latitude").value)
93 self.fallback_lon = float(self.get_parameter("fallback_longitude").value)
94 self.tcell_gain = float(self.get_parameter("tcell_gain").value)
95
96 self.latest_lat = None
97 self.latest_lon = None
98 self.pub_status = self.create_publisher(String, "/weather/fetch_status", 10)
99 self.create_subscription(NavSatFix, self.gps_topic, self._on_gps, 10)
100 self.timer = self.create_timer(self.fetch_period_sec, self._fetch_once)
101 self.get_logger().info(f"WeatherFetchNode started: gps_topic={self.gps_topic} -> {self.output_csv}")
102 self._fetch_once()
103
104 def _on_gps(self, msg: NavSatFix) -> None:
105     try:
106         self.latest_lat = float(msg.latitude)
107         self.latest_lon = float(msg.longitude)
108     except Exception:
109         return
110
111 def _fetch_once(self) -> None:
112     lat = self.latest_lat if self.latest_lat is not None else self.fallback_lat
113     lon = self.latest_lon if self.latest_lon is not None else self.fallback_lon
114     try:
115         df = fetch_openmeteo_forecast(
116             lat,
117             lon,
118             timezone_name=self.timezone_name,
119             forecast_days=self.forecast_days,
120             step_minutes=self.step_minutes,
121             tcell_gain=self.tcell_gain,
122         )
123         write_forecast_csv(df, self.output_csv)
124         if self.raw_forecast_csv:
125             raw_path = self.raw_forecast_csv
126             if not raw_path.is_absolute():
127                 raw_path = (self.output_csv.parent / raw_path).resolve()
128             write_forecast_csv(df, raw_path)
129             status = f"ok lat={lat:.5f} lon={lon:.5f} rows={len(df)} out={self.output_csv}"
130             self.pub_status.publish(String(data=status))
131             self.get_logger().info(status)
132     except Exception as exc:
133         status = f"error weather fetch failed: {exc}"
134         self.pub_status.publish(String(data=status))
135         self.get_logger().error(status)
136
137
138 def main() -> None:
139     rclpy.init()
140     node = WeatherFetchNode()
141     rclpy.spin(node)
142     node.destroy_node()
143     rclpy.shutdown()

```

A.26 mpc_solarcar/solar_autocal_logic.py

```

1 """Pure helpers for bounded online solar-car calibration."""
2
3 from __future__ import annotations
4
5 import math
6
7
8 def daytime_stationary_aux_estimate(
9     *,
10     ghi_wm2: float,
11     day_ghi_threshold_wm2: float,
12     speed_kmh: float,
13     stationary_speed_kmh: float,

```

```

14     pack_power_w: float,
15     solar_power_w: float,
16 ) -> float | None:
17     values = (ghi_wm2, speed_kmh, pack_power_w, solar_power_w)
18     if not all(math.isfinite(float(value)) for value in values):
19         return None
20     if float(ghi_wm2) < float(day_ghi_threshold_wm2):
21         return None
22     if abs(float(speed_kmh)) > float(stationary_speed_kmh):
23         return None
24     return max(0.0, float(pack_power_w) + float(solar_power_w))

```

A.27 mpc_solarcar/solar_autocal_node.py

```

1  from __future__ import annotations
2
3  import math
4
5  import rclpy
6  from rclpy.node import Node
7
8  from std_msgs.msg import Float32, Float32MultiArray, String
9
10 from .solar_autocal_logic import daytime_stationary_aux_estimate
11
12
13 def _clamp(value: float, lo: float, hi: float) -> float:
14     return max(float(lo), min(float(hi), float(value)))
15
16
17 class SolarAutocalNode(Node):
18     def __init__(self) -> None:
19         super().__init__("solar_autocal_node")
20         self.declare_parameter("publish_period_sec", 30.0)
21         self.declare_parameter("stationary_speed_kmh", 2.0)
22         self.declare_parameter("drive_speed_kmh", 25.0)
23         self.declare_parameter("day_ghi_threshold", 150.0)
24         self.declare_parameter("alpha", 0.2)
25         self.declare_parameter("solar_gain_init", 1.0)
26         self.declare_parameter("drive_power_gain_init", 1.0)
27         self.declare_parameter("aux_power_w_init", 8.0)
28         self.declare_parameter("solar_gain_min", 0.5)
29         self.declare_parameter("solar_gain_max", 1.5)
30         self.declare_parameter("drive_power_gain_min", 0.7)
31         self.declare_parameter("drive_power_gain_max", 1.4)
32         self.declare_parameter("aux_power_w_min", 0.0)
33         self.declare_parameter("aux_power_w_max", 300.0)
34
35         self.period = max(1.0, float(self.get_parameter("publish_period_sec").value))
36         self.stationary_speed = float(self.get_parameter("stationary_speed_kmh").value)
37         self.drive_speed = float(self.get_parameter("drive_speed_kmh").value)
38         self.day_ghi = float(self.get_parameter("day_ghi_threshold").value)
39         self.alpha = _clamp(float(self.get_parameter("alpha").value), 0.01, 1.0)
40         self.solar_gain = float(self.get_parameter("solar_gain_init").value)
41         self.drive_gain = float(self.get_parameter("drive_power_gain_init").value)
42         self.aux_power = float(self.get_parameter("aux_power_w_init").value)
43         self.solar_gain_min = float(self.get_parameter("solar_gain_min").value)
44         self.solar_gain_max = float(self.get_parameter("solar_gain_max").value)
45         self.drive_gain_min = float(self.get_parameter("drive_power_gain_min").value)
46         self.drive_gain_max = float(self.get_parameter("drive_power_gain_max").value)
47         self.aux_min = float(self.get_parameter("aux_power_w_min").value)
48         self.aux_max = float(self.get_parameter("aux_power_w_max").value)
49
50         self.speed_kmh = math.nan
51         self.pack_power_w = math.nan
52         self.solar_obs_w = math.nan
53         self.ghi = math.nan
54         self.pred_solar_w = math.nan
55         self.pred_pack_w = math.nan
56
57         self.pub_solar = self.create_publisher(Float32, "/calib/solar_gain", 10)
58         self.pub_drive = self.create_publisher(Float32, "/calib/drive_power_gain", 10)
59         self.pub_aux = self.create_publisher(Float32, "/calib/aux_power_w", 10)
60         self.pub_status = self.create_publisher(String, "/calib/status", 10)
61
62         self.create_subscription(Float32, "/vehicle/speed_kmh", self._on_speed, 10)
63         self.create_subscription(Float32, "/vehicle/batt_current_a", self._on_current, 10)
64         self.create_subscription(Float32, "/vehicle/batt_voltage_v", self._on_voltage, 10)
65         self.create_subscription(Float32, "/vehicle/solar_power_w", self._on_solar, 10)
66         self.create_subscription(Float32MultiArray, "/planner/env", self._on_env, 10)
67         self.create_subscription(Float32MultiArray, "/planner/metrics", self._on_metrics, 10)
68
69         self._latest_current_a = math.nan
70         self._latest_voltage_v = math.nan
71         self.timer = self.create_timer(self.period, self._step)
72         self.get_logger().info("SolarAutocalNode started.")
73
74     def _on_speed(self, msg: Float32) -> None:
75         self.speed_kmh = float(msg.data)
76
77     def _on_current(self, msg: Float32) -> None:
78         self._latest_current_a = float(msg.data)
79         if math.isfinite(self._latest_voltage_v):
80             self.pack_power_w = self._latest_current_a * self._latest_voltage_v
81
82     def _on_voltage(self, msg: Float32) -> None:
83         self._latest_voltage_v = float(msg.data)
84         if math.isfinite(self._latest_current_a):

```

```

85         self.pack_power_w = self._latest_current_a * self._latest_voltage_v
86
87     def _on_solar(self, msg: Float32) -> None:
88         self.solar_obs_w = float(msg.data)
89
90     def _on_env(self, msg: Float32MultiArray) -> None:
91         data = list(msg.data)
92         if len(data) >= 1:
93             self.ghi = float(data[0])
94
95     def _on_metrics(self, msg: Float32MultiArray) -> None:
96         data = list(msg.data)
97         if len(data) >= 6:
98             self.pred_solar_w = float(data[5])
99         if len(data) >= 9:
100             self.pred_pack_w = float(data[8])
101
102     def _ema(self, current: float, estimate: float, lo: float, hi: float) -> float:
103         blended = (1.0 - self.alpha) * float(current) + self.alpha * float(estimate)
104         return _clamp(blended, lo, hi)
105
106     def _step(self) -> None:
107         # The team switches auxiliaries off at night, so a night stationary
108         # sample is not an estimate of the daytime auxiliary load. At a
109         # daytime stop P_pack = P_aux - P_pv, hence P_aux = P_pack + P_pv.
110         aux_estimate = daytime_stationary_aux_estimate(
111             ghi_wm2=self.ghi,
112             day_ghi_threshold_wm2=self.day_ghi,
113             speed_kmh=self.speed_kmh,
114             stationary_speed_kmh=self.stationary_speed,
115             pack_power_w=self.pack_power_w,
116             solar_power_w=self.solar_obs_w,
117         )
118         if aux_estimate is not None:
119             self.aux_power = self._ema(self.aux_power, aux_estimate, self.aux_min, self.aux_max)
120
121         if (
122             math.isfinite(self.ghi)
123             and self.ghi >= self.day_ghi
124             and math.isfinite(self.solar_obs_w)
125             and math.isfinite(self.pred_solar_w)
126             and abs(self.pred_solar_w) >= 20.0
127         ):
128             gain_est = self.solar_obs_w / self.pred_solar_w
129             self.solar_gain = self._ema(self.solar_gain, gain_est, self.solar_gain_min, self.solar_gain_max)
130
131         if (
132             math.isfinite(self.speed_kmh)
133             and self.speed_kmh >= self.drive_speed
134             and math.isfinite(self.pack_power_w)
135             and math.isfinite(self.pred_pack_w)
136             and abs(self.pred_pack_w) >= 50.0
137             and self.pack_power_w > 0.0
138         ):
139             gain_est = self.pack_power_w / max(1.0, self.pred_pack_w)
140             self.drive_gain = self._ema(self.drive_gain, gain_est, self.drive_gain_min, self.drive_gain_max)
141
142         self.pub_solar.publish(Float32(data=float(self.solar_gain)))
143         self.pub_drive.publish(Float32(data=float(self.drive_gain)))
144         self.pub_aux.publish(Float32(data=float(self.aux_power)))
145         status = (
146             f"solar_gain={self.solar_gain:.3f} "
147             f"drive_power_gain={self.drive_gain:.3f} "
148             f"aux_power_w={self.aux_power:.1f}"
149         )
150         self.pub_status.publish(String(data=status))
151
152
153     def main() -> None:
154         rclpy.init()
155         node = SolarAutocalNode()
156         rclpy.spin(node)
157         node.destroy_node()
158         rclpy.shutdown()

```

A.28 mpc_solarcar/speed_command_bridge_node.py

```

1 from __future__ import annotations
2
3 import json
4 import socket
5 import time
6
7 import rclpy
8 from rclpy.node import Node
9
10 from std_msgs.msg import Float32, String
11
12 from .signal_utils import SmoothRateLimiter
13 from .solar_preflight_logic import evaluate_command_gate
14 from .telemetry_protocol import utc_iso_now
15
16
17 class SpeedCommandBridgeNode(Node):
18     def __init__(self) -> None:
19         super().__init__("speed_command_bridge_node")
20         self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
21         self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")

```

```

22     self.declare_parameter("udp_enabled", False)
23     self.declare_parameter("udp_host", "127.0.0.1")
24     self.declare_parameter("udp_port", 50050)
25     self.declare_parameter("publish_rate_hz", 5.0)
26     self.declare_parameter("input_timeout_sec", 3.0)
27     self.declare_parameter("safe_speed_kmh", 0.0)
28     self.declare_parameter("startup_hold_sec", 2.0)
29     self.declare_parameter("filter_tau_sec", 1.0)
30     self.declare_parameter("accel_limit_kmhps", 1.2)
31     self.declare_parameter("decel_limit_kmhps", 3.5)
32     self.declare_parameter("speed_deadband_kmh", 0.1)
33     self.declare_parameter("speed_quantize_step_kmh", 0.1)
34     self.declare_parameter("max_output_speed_kmh", 120.0)
35     self.declare_parameter("drive_mode_min_hold_sec", 5.0)
36     self.declare_parameter("require_system_running", True)
37     self.declare_parameter("system_state_timeout_sec", 2.5)
38
39     self.output_speed_topic = str(self.get_parameter("output_speed_topic").value)
40     self.output_drive_mode_topic = str(self.get_parameter("output_drive_mode_topic").value)
41     self.udp_enabled = bool(self.get_parameter("udp_enabled").value)
42     self.udp_host = str(self.get_parameter("udp_host").value)
43     self.udp_port = int(self.get_parameter("udp_port").value)
44     self.publish_rate_hz = max(1.0, float(self.get_parameter("publish_rate_hz").value))
45     self.input_timeout_sec = max(0.1, float(self.get_parameter("input_timeout_sec").value))
46     self.safe_speed_kmh = float(self.get_parameter("safe_speed_kmh").value)
47     self.startup_hold_sec = max(0.0, float(self.get_parameter("startup_hold_sec").value))
48     self.drive_mode_min_hold_sec = max(0.0, float(self.get_parameter("drive_mode_min_hold_sec").value))
49     self.require_system_running = bool(self.get_parameter("require_system_running").value)
50     self.system_state_timeout_sec = max(
51         0.1, float(self.get_parameter("system_state_timeout_sec").value)
52     )
53
54     self.limiter = SmoothRateLimiter(
55         min_value=0.0,
56         max_value=float(self.get_parameter("max_output_speed_kmh").value),
57         tau_sec=float(self.get_parameter("filter_tau_sec").value),
58         rise_rate=float(self.get_parameter("accel_limit_kmhps").value),
59         fall_rate=float(self.get_parameter("decel_limit_kmhps").value),
60         deadband=float(self.get_parameter("speed_deadband_kmh").value),
61         quantize_step=float(self.get_parameter("speed_quantize_step_kmh").value),
62         initial_value=self.safe_speed_kmh,
63     )
64     self.limiter.reset(self.safe_speed_kmh)
65
66     self.speed_pub = self.create_publisher(Float32, self.output_speed_topic, 10)
67     self.mode_pub = self.create_publisher(String, self.output_drive_mode_topic, 10)
68     self.create_subscription(Float32, "/planner/speed_cmd", self._on_speed_cmd, 10)
69     self.create_subscription(Float32, "/planner/upper_speed_cmd", self._on_upper_speed_cmd, 10)
70     self.create_subscription(String, "/planner/drive_mode", self._on_drive_mode, 10)
71     self.create_subscription(String, "/system/state", self._on_system_state, 10)
72
73     self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) if self.udp_enabled else None
74     self.start_time = time.monotonic()
75     self.last_speed_input_time = None
76     self.last_mode_input_time = None
77     self.last_system_state_time = None
78     self.system_state = "STARTING"
79     self.last_speed_cmd = self.safe_speed_kmh
80     self.last_upper_speed_cmd = self.safe_speed_kmh
81     self.last_drive_mode = "stop"
82     self.applied_drive_mode = "stop"
83     self.last_drive_mode_change = self.start_time
84     self.last_udp_error_log_time = float("-inf")
85     self.last_gate_reason = "startup_hold"
86     self.timer = self.create_timer(1.0 / self.publish_rate_hz, self._step)
87     self.get_logger().info("SpeedCommandBridgeNode started.")
88
89     def _on_speed_cmd(self, msg: Float32) -> None:
90         self.last_speed_cmd = float(msg.data)
91         self.last_speed_input_time = time.monotonic()
92
93     def _on_upper_speed_cmd(self, msg: Float32) -> None:
94         self.last_upper_speed_cmd = float(msg.data)
95
96     def _on_drive_mode(self, msg: String) -> None:
97         self.last_drive_mode = str(msg.data or "stop")
98         self.last_mode_input_time = time.monotonic()
99
100     def _on_system_state(self, msg: String) -> None:
101         self.system_state = str(msg.data or "").strip().upper()
102         self.last_system_state_time = time.monotonic()
103
104     def _step(self) -> None:
105         now = time.monotonic()
106         speed_target = self.safe_speed_kmh
107         gate = evaluate_command_gate(
108             elapsed_sec=now - self.start_time,
109             speed_input_age_sec=(
110                 None if self.last_speed_input_time is None else now - self.last_speed_input_time
111             ),
112             system_state=self.system_state,
113             system_state_age_sec=(
114                 None if self.last_system_state_time is None else now - self.last_system_state_time
115             ),
116             startup_hold_sec=self.startup_hold_sec,
117             input_timeout_sec=self.input_timeout_sec,
118             system_state_timeout_sec=self.system_state_timeout_sec,
119             require_system_running=self.require_system_running,
120         )
121         self.last_gate_reason = gate.reason
122         if gate.allowed:
123             speed_target = self.last_speed_cmd
124         speed_out = float(self.limiter.update(speed_target, now=now))

```

```

125
126     mode_is_fresh = (
127         self.last_mode_input_time is not None
128         and (now - self.last_mode_input_time) <= self.input_timeout_sec
129     )
130     requested_mode = self.last_drive_mode if gate.allowed and mode_is_fresh else "stop"
131     if requested_mode == "stop":
132         self.applied_drive_mode = "stop"
133         self.last_drive_mode_change = now
134     elif requested_mode != self.applied_drive_mode and (now - self.last_drive_mode_change) >= self.drive_mode_min_hold_sec:
135         self.applied_drive_mode = requested_mode
136         self.last_drive_mode_change = now
137
138     self.speed_pub.publish(Float32(data=speed_out))
139     self.mode_pub.publish(String(data=self.applied_drive_mode))
140     if self.sock is not None:
141         payload = {
142             "type": "planner_command",
143             "ts_unix": time.time(),
144             "timestamp_utc": utc_iso_now(),
145             "planner": {
146                 "speed_cmd_kmh": speed_out,
147                 "upper_speed_cmd_kmh": self.last_upper_speed_cmd,
148                 "drive_mode": self.applied_drive_mode,
149                 "command_gate": self.last_gate_reason,
150             },
151         }
152     try:
153         self.sock.sendto(json.dumps(payload, ensure_ascii=False).encode("utf-8"), (self.udp_host, self.udp_port))
154     except OSError as exc:
155         if (now - self.last_udp_error_log_time) >= 5.0:
156             self.get_logger().error(
157                 f"Planner command UDP send failed to {self.udp_host}:{self.udp_port}: {exc}"
158             )
159             self.last_udp_error_log_time = now
160
161
162 def main() -> None:
163     rclpy.init()
164     node = SpeedCommandBridgeNode()
165     rclpy.spin(node)
166     node.destroy_node()
167     rclpy.shutdown()

```

A.29 mpc_solarcar/telemetry_protocol.py

```

1  """Timestamp validation shared by the WiFi telemetry receiver and tests."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from datetime import datetime, timezone
7  import math
8
9
10 @dataclass(frozen=True)
11 class TimestampValidation:
12     accepted: bool
13     source_unix: float | None
14     age_sec: float | None
15     reason: str
16
17
18 def parse_source_timestamp(payload: dict) -> float | None:
19     """Return a UTC Unix timestamp from the supported wire-format fields."""
20     for key in ("ts_unix", "timestamp_unix", "time_unix"):
21         if key not in payload:
22             continue
23         try:
24             value = float(payload[key])
25         except (TypeError, ValueError):
26             continue
27         if math.isfinite(value):
28             return value
29
30     for key in ("timestamp_utc", "ts_utc", "time_utc"):
31         raw = str(payload.get(key, "")).strip()
32         if not raw:
33             continue
34         try:
35             parsed = datetime.fromisoformat(raw[:-1] + "+00:00" if raw.endswith("Z") else raw)
36         except ValueError:
37             continue
38         if parsed.tzinfo is None:
39             continue
40         return parsed.astimezone(timezone.utc).timestamp()
41     return None
42
43
44 def validate_source_timestamp(
45     payload: dict,
46     *,
47     now_unix: float,
48     last_source_unix: float | None,
49     required: bool,
50     max_age_sec: float,
51     max_future_skew_sec: float,
52     max_out_of_order_sec: float,

```

```

53 ) -> TimestampValidation:
54     """Reject stale, future, duplicate, or excessively reordered UDP packets."""
55     source_unix = parse_source_timestamp(payload)
56     if source_unix is None:
57         if required:
58             return TimestampValidation(False, None, None, "missing_or_invalid_timestamp")
59         return TimestampValidation(True, None, None, "timestamp_not_required")
60
61     age_sec = float(now_unix) - source_unix
62     if age_sec > max(0.0, float(max_age_sec)):
63         return TimestampValidation(False, source_unix, age_sec, "stale_packet")
64     if age_sec < -max(0.0, float(max_future_skew_sec)):
65         return TimestampValidation(False, source_unix, age_sec, "future_packet")
66     if last_source_unix is not None:
67         tolerance = max(0.0, float(max_out_of_order_sec))
68         if source_unix == last_source_unix:
69             return TimestampValidation(False, source_unix, age_sec, "duplicate_packet")
70         if source_unix < last_source_unix - tolerance:
71             return TimestampValidation(False, source_unix, age_sec, "out_of_order_packet")
72     return TimestampValidation(True, source_unix, age_sec, "ok")
73
74
75 def utc_iso_now() -> str:
76     return datetime.now(timezone.utc).isoformat().replace("+00:00", "Z")

```

A.30 mpc_solarcar/telemetry_text_bridge_node.py

```

1 from __future__ import annotations
2
3 import json
4 import math
5 import socket
6 import time
7 from statistics import NormalDist
8
9 import rclpy
10 from rclpy.node import Node
11
12 from sensor_msgs.msg import NavSatFix
13 from std_msgs.msg import Float32, Float64, String
14
15 from .signal_utils import RobustScalarFilter
16 from .telemetry_protocol import utc_iso_now, validate_source_timestamp
17
18
19 def _safe_float(payload: dict, *keys: str) -> float | None:
20     for key in keys:
21         if key not in payload:
22             continue
23         try:
24             value = float(payload[key])
25         except Exception:
26             continue
27         if math.isfinite(value):
28             return value
29     return None
30
31
32 def _parse_key_value_text(text: str) -> dict:
33     payload: dict[str, object] = {}
34     for token in text.replace(";", " ").replace("\n", " ").split(" "):
35         token = token.strip()
36         if not token:
37             continue
38         sep = "=" if "=" in token else (":" if ":" in token else "")
39         if not sep:
40             continue
41         key, value = token.split(sep, 1)
42         key = key.strip()
43         value = value.strip()
44         if not key:
45             continue
46         try:
47             payload[key] = float(value)
48         except Exception:
49             payload[key] = value
50     return payload
51
52
53 def parse_text_payload(text: str) -> dict:
54     raw = str(text or "").strip()
55     if not raw:
56         return {}
57     try:
58         parsed = json.loads(raw)
59         if isinstance(parsed, dict):
60             return parsed
61     except Exception:
62         pass
63     return _parse_key_value_text(raw)
64
65
66 def headwind_component_ms(payload: dict) -> float | None:
67     direct = _safe_float(payload, "headwind_ms")
68     if direct is not None:
69         return direct
70     wind_speed = _safe_float(payload, "wind_speed_ms", "wind_ms")
71     wind_dir = _safe_float(payload, "wind_dir_deg")

```

```

72     course_deg = _safe_float(payload, "course_deg", "heading_deg")
73     if wind_speed is None or wind_dir is None or course_deg is None:
74         return None
75     rel = math.radians((wind_dir - course_deg) % 360.0)
76     return float(wind_speed * math.cos(rel))
77
78
79 class TelemetryTextBridgeNode(Node):
80     def __init__(self) -> None:
81         super().__init__("telemetry_text_bridge_node")
82         self.declare_parameter("enable_inbound", True)
83         self.declare_parameter("enable_outbound", True)
84         self.declare_parameter("bind_host", "0.0.0.0")
85         self.declare_parameter("bind_port", 52001)
86         self.declare_parameter("publish_period_sec", 1.0)
87         self.declare_parameter("solar_remote_host", "192.168.50.21")
88         self.declare_parameter("solar_remote_port", 52002)
89         self.declare_parameter("chase_remote_host", "192.168.50.22")
90         self.declare_parameter("chase_remote_port", 52003)
91         self.declare_parameter("send_to_solar", True)
92         self.declare_parameter("send_to_chase", True)
93         self.declare_parameter("prefer_direct_distance", False)
94         self.declare_parameter("speed_filter_tau_sec", 0.6)
95         self.declare_parameter("speed_max_kmh", 130.0)
96         self.declare_parameter("speed_max_accel_kmhps", 12.0)
97         self.declare_parameter("speed_max_decel_kmhps", 20.0)
98         self.declare_parameter("distance_max_rate_kmps", 0.06)
99         self.declare_parameter("distance_max_backtrack_km", 0.02)
100        self.declare_parameter("battery_filter_tau_sec", 1.0)
101        self.declare_parameter("wind_filter_tau_sec", 1.0)
102        self.declare_parameter("headwind_filter_tau_sec", 0.8)
103        self.declare_parameter("max_abs_headwind_ms", 25.0)
104        self.declare_parameter("timestamp_required", True)
105        self.declare_parameter("max_packet_age_sec", 5.0)
106        self.declare_parameter("max_future_skew_sec", 2.0)
107        self.declare_parameter("max_out_of_order_sec", 0.0)
108        self.declare_parameter("solar_power_gain_to_pack", 1.0)
109
110        self.enable_inbound = bool(self.get_parameter("enable_inbound").value)
111        self.enable_outbound = bool(self.get_parameter("enable_outbound").value)
112        self.publish_period_sec = max(0.1, float(self.get_parameter("publish_period_sec").value))
113        self.prefer_direct_distance = bool(self.get_parameter("prefer_direct_distance").value)
114        self.max_abs_headwind_ms = max(0.0, float(self.get_parameter("max_abs_headwind_ms").value))
115        self.timestamp_required = bool(self.get_parameter("timestamp_required").value)
116        self.max_packet_age_sec = max(0.0, float(self.get_parameter("max_packet_age_sec").value))
117        self.max_future_skew_sec = max(0.0, float(self.get_parameter("max_future_skew_sec").value))
118        self.max_out_of_order_sec = max(0.0, float(self.get_parameter("max_out_of_order_sec").value))
119        self.solar_power_gain_to_pack = max(
120            0.0, float(self.get_parameter("solar_power_gain_to_pack").value)
121        )
122        self.last_source_unix = {"solar": None, "chase": None}
123
124        self.speed_filter = RobustScalarFilter(
125            tau_sec=float(self.get_parameter("speed_filter_tau_sec").value),
126            min_value=0.0,
127            max_value=float(self.get_parameter("speed_max_kmh").value),
128            rise_rate=float(self.get_parameter("speed_max_accel_kmhps").value),
129            fall_rate=float(self.get_parameter("speed_max_decel_kmhps").value),
130            initial_value=0.0,
131        )
132        self.soc_filter = RobustScalarFilter(tau_sec=float(self.get_parameter("battery_filter_tau_sec").value), min_value=0.0, max_value=1.0)
133        self.tb_filter = RobustScalarFilter(tau_sec=float(self.get_parameter("battery_filter_tau_sec").value), min_value=-40.0, max_value=120.0)
134        self.i_filter = RobustScalarFilter(tau_sec=float(self.get_parameter("battery_filter_tau_sec").value))
135        self.v_filter = RobustScalarFilter(tau_sec=float(self.get_parameter("battery_filter_tau_sec").value), min_value=0.0, max_value=1000.0)
136        self.wind_filter = RobustScalarFilter(tau_sec=float(self.get_parameter("wind_filter_tau_sec").value))
137        self.headwind_filter = RobustScalarFilter(
138            tau_sec=float(self.get_parameter("headwind_filter_tau_sec").value),
139            min_value=-self.max_abs_headwind_ms,
140            max_value=self.max_abs_headwind_ms,
141        )
142
143        self.pub_raw_solar = self.create_publisher(String, "/telemetry/raw_solar", 10)
144        self.pub_raw_chase = self.create_publisher(String, "/telemetry/raw_chase", 10)
145        self.pub_vehicle_speed = self.create_publisher(Float32, "/vehicle/speed_kmh", 10)
146        self.pub_vehicle_soc = self.create_publisher(Float32, "/vehicle/batt_soc", 10)
147        self.pub_vehicle_tb = self.create_publisher(Float32, "/vehicle/batt_temp_c", 10)
148        self.pub_vehicle_i = self.create_publisher(Float32, "/vehicle/batt_current_a", 10)
149        self.pub_vehicle_v = self.create_publisher(Float32, "/vehicle/batt_voltage_v", 10)
150        self.pub_vehicle_solar = self.create_publisher(Float32, "/vehicle/solar_power_w", 10)
151        self.pub_vehicle_alt = self.create_publisher(Float32, "/vehicle/altitude_m", 10)
152        self.pub_vehicle_s = self.create_publisher(Float32, "/vehicle/s_kmh", 10)
153        self.pub_chase_gps = self.create_publisher(NavSatFix, "/vehicle/gps", 10)
154        self.pub_chase_gps = self.create_publisher(NavSatFix, "/chase/gps", 10)
155        self.pub_chase_alt = self.create_publisher(Float32, "/chase/altitude_m", 10)
156        self.pub_headwind = self.create_publisher(Float32, "/weather/headwind_meas_ms", 10)
157        self.pub_wind_speed = self.create_publisher(Float32, "/weather/wind_speed_ms", 10)
158        self.pub_wind_dir = self.create_publisher(Float32, "/weather/wind_dir_deg", 10)
159        self.pub_course_dir = self.create_publisher(Float32, "/weather/course_deg", 10)
160        self.pub_status = self.create_publisher(String, "/telemetry/bridge_status", 10)
161        self.pub_solar_source_time = self.create_publisher(Float64, "/telemetry/solar_source_ts_unix", 10)
162        self.pub_chase_source_time = self.create_publisher(Float64, "/telemetry/chase_source_ts_unix", 10)
163
164        self.sock = None
165        if self.enable_inbound:
166            self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
167            self.sock.bind((str(self.get_parameter("bind_host").value), int(self.get_parameter("bind_port").value)))
168            self.sock.setblocking(False)
169        self.tx_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) if self.enable_outbound else None
170
171        self.solar_remote = (str(self.get_parameter("solar_remote_host").value), int(self.get_parameter("solar_remote_port").value))
172        self.chase_remote = (str(self.get_parameter("chase_remote_host").value), int(self.get_parameter("chase_remote_port").value))
173        self.send_to_solar = bool(self.get_parameter("send_to_solar").value)
174        self.send_to_chase = bool(self.get_parameter("send_to_chase").value)

```

```

175
176     self.outbound_state = {
177         "speed_cmd_kmh": 0.0,
178         "upper_speed_cmd_kmh": 0.0,
179         "drive_mode": "stop",
180         "speed_meas_kmh": math.nan,
181         "soc": math.nan,
182         "s_kmh": math.nan,
183     }
184     self.create_subscription(Float32, "/planner/speed_cmd", lambda msg: self._set_outbound("speed_cmd_kmh", float(msg.data)), 10)
185     self.create_subscription(Float32, "/planner/upper_speed_cmd", lambda msg: self._set_outbound("upper_speed_cmd_kmh", float(msg.data)), 10)
186     self.create_subscription(String, "/planner/drive_mode", lambda msg: self._set_outbound("drive_mode", str(msg.data)), 10)
187     self.create_subscription(Float32, "/vehicle/speed_kmh", lambda msg: self._set_outbound("speed_meas_kmh", float(msg.data)), 10)
188     self.create_subscription(Float32, "/vehicle/batt_soc", lambda msg: self._set_outbound("soc", float(msg.data)), 10)
189     self.create_subscription(Float32, "/vehicle/s_kmh", lambda msg: self._set_outbound("s_kmh", float(msg.data)), 10)
190
191     self.rx_timer = self.create_timer(0.05, self._poll_socket)
192     self.tx_timer = self.create_timer(self.publish_period_sec, self._send_outbound)
193     self.get_logger().info("TelemetryTextBridgeNode started.")
194
195 def _set_outbound(self, key: str, value) -> None:
196     self.outbound_state[key] = value
197
198 def _poll_socket(self) -> None:
199     if self.sock is None:
200         return
201     while True:
202         try:
203             data, _addr = self.sock.recvfrom(4096)
204         except BlockingIOError:
205             break
206         except Exception:
207             break
208         text = data.decode("utf-8", errors="replace")
209         payload = parse_text_payload(text)
210         if not payload:
211             continue
212         payload_type = str(payload.get("type", "")).lower()
213         source = "chase" if payload_type == "chase_state" else "solar"
214         validation = validate_source_timestamp(
215             payload,
216             now_unix=time.time(),
217             last_source_unix=self.last_source_unix[source],
218             required=self.timestamp_required,
219             max_age_sec=self.max_packet_age_sec,
220             max_future_skew_sec=self.max_future_skew_sec,
221             max_out_of_order_sec=self.max_out_of_order_sec,
222         )
223         if not validation.accepted:
224             self.pub_status.publish(String(data=f"rejected source={source} reason={validation.reason}"))
225             continue
226         if validation.source_unix is not None:
227             self.last_source_unix[source] = validation.source_unix
228             pub = self.pub_chase_source_time if source == "chase" else self.pub_solar_source_time
229             pub.publish(Float64(data=validation.source_unix))
230
231         if source == "chase":
232             self.pub_raw_chase.publish(String(data=text))
233             self._handle_chase_payload(payload, validation.source_unix)
234         else:
235             self.pub_raw_solar.publish(String(data=text))
236             self._handle_vehicle_payload(payload, validation.source_unix)
237
238 def _publish_gps(self, pub, lat: float | None, lon: float | None, alt: float | None, source_unix: float | None) -> None:
239     if lat is None or lon is None:
240         return
241     msg = NavSatFix()
242     if source_unix is None:
243         msg.header.stamp = self.get_clock().now().to_msg()
244     else:
245         nanoseconds = max(0, int(source_unix * 1.0e9))
246         msg.header.stamp.sec = nanoseconds // 1_000_000_000
247         msg.header.stamp.nanosec = nanoseconds % 1_000_000_000
248     msg.latitude = float(lat)
249     msg.longitude = float(lon)
250     msg.altitude = float(alt) if alt is not None else 0.0
251     pub.publish(msg)
252
253 def _handle_vehicle_payload(self, payload: dict, source_unix: float | None) -> None:
254     now = time.monotonic()
255     speed = _safe_float(payload, "speed_kmh", "vehicle_speed_kmh")
256     soc = _safe_float(payload, "soc", "batt_soc")
257     tb = _safe_float(payload, "batt_temp_c", "tb_c")
258     current = _safe_float(payload, "batt_current_a", "current_a")
259     voltage = _safe_float(payload, "batt_voltage_v", "voltage_v")
260     solar = _safe_float(payload, "solar_power_w", "pv_w")
261     s_kmh = _safe_float(payload, "s_kmh", "distance_kmh")
262     lat = _safe_float(payload, "lat", "latitude")
263     lon = _safe_float(payload, "lon", "longitude")
264     alt = _safe_float(payload, "alt_m", "altitude_m")
265     wind_speed = _safe_float(payload, "wind_speed_ms", "wind_ms")
266     wind_dir = _safe_float(payload, "wind_dir_deg")
267     course_deg = _safe_float(payload, "course_deg", "heading_deg")
268     headwind = headwind_component_ms(payload)
269
270     if speed is not None:
271         self.pub_vehicle_speed.publish(Float32(data=float(self.speed_filter.update(speed, now=now))))
272     if soc is not None:
273         self.pub_vehicle_soc.publish(Float32(data=float(self.soc_filter.update(soc, now=now))))
274     if tb is not None:
275         self.pub_vehicle_tb.publish(Float32(data=float(self.tb_filter.update(tb, now=now))))
276     if current is not None:
277         self.pub_vehicle_i.publish(Float32(data=float(self.i_filter.update(current, now=now))))

```

```

278     if voltage is not None:
279         self.pub_vehicle_v.publish(Float32(data=float(self.v_filter.update(voltage, now=now))))
280     if solar is not None:
281         self.pub_vehicle_solar.publish(
282             Float32(data=float(solar) * self.solar_power_gain_to_pack)
283         )
284     if alt is not None:
285         self.pub_vehicle_alt.publish(Float32(data=float(alt)))
286     if self.prefer_direct_distance and s_km is not None:
287         self.pub_vehicle_s.publish(Float32(data=float(s_km)))
288     self._publish_gps(self.pub_vehicle_gps, lat, lon, alt, source_unix)
289     self._publish_weather(payload, wind_speed, wind_dir, course_deg, headwind, now)
290
291 def _handle_chase_payload(self, payload: dict, source_unix: float | None) -> None:
292     now = time.monotonic()
293     lat = _safe_float(payload, "lat", "latitude")
294     lon = _safe_float(payload, "lon", "longitude")
295     alt = _safe_float(payload, "alt_m", "altitude_m")
296     wind_speed = _safe_float(payload, "wind_speed_ms", "wind_ms")
297     wind_dir = _safe_float(payload, "wind_dir_deg")
298     course_deg = _safe_float(payload, "course_deg", "heading_deg")
299     headwind = headwind_component_ms(payload)
300     self._publish_gps(self.pub_chase_gps, lat, lon, alt, source_unix)
301     if alt is not None:
302         self.pub_chase_alt.publish(Float32(data=float(alt)))
303     self._publish_weather(payload, wind_speed, wind_dir, course_deg, headwind, now)
304
305 def _publish_weather(self, payload: dict, wind_speed, wind_dir, course_deg, headwind, now: float) -> None:
306     if wind_speed is not None:
307         self.pub_wind_speed.publish(Float32(data=float(self.wind_filter.update(wind_speed, now=now))))
308     if wind_dir is not None:
309         self.pub_wind_dir.publish(Float32(data=float(wind_dir)))
310     if course_deg is not None:
311         self.pub_course_dir.publish(Float32(data=float(course_deg)))
312     if headwind is not None:
313         filtered = self.headwind_filter.update(headwind, now=now)
314         self.pub_headwind.publish(Float32(data=float(filtered)))
315         self.pub_status.publish(String(data=f"headwind_ms={filtered:.2f} type={payload.get('type', 'vehicle_state')}"))
316
317 def _send_payload(self, addr: tuple[str, int], payload: dict) -> None:
318     if self.tx_sock is None:
319         return
320     try:
321         self.tx_sock.sendto(json.dumps(payload, ensure_ascii=False).encode("utf-8"), addr)
322     except Exception:
323         return
324
325 def _send_outbound(self) -> None:
326     if not self.enable_outbound:
327         return
328     payload = {
329         "type": "planner_command",
330         "ts_unix": time.time(),
331         "timestamp_utc": utc_iso_now(),
332         "planner": {
333             "speed_cmd_kmh": self.outbound_state["speed_cmd_kmh"],
334             "upper_speed_cmd_kmh": self.outbound_state["upper_speed_cmd_kmh"],
335             "drive_mode": self.outbound_state["drive_mode"],
336         },
337         "vehicle": {
338             "speed_kmh": self.outbound_state["speed_meas_kmh"],
339             "soc": self.outbound_state["soc"],
340             "s_km": self.outbound_state["s_km"],
341         },
342     }
343     if self.send_to_solar:
344         self._send_payload(self.solar_remote, payload)
345     if self.send_to_chase:
346         self._send_payload(self.chase_remote, payload)
347
348
349 def main() -> None:
350     rclpy.init()
351     node = TelemetryTextBridgeNode()
352     rclpy.spin(node)
353     node.destroy_node()
354     rclpy.shutdown()

```

A.31 mpc_solarcar/wind_correction_node.py

```

1 from __future__ import annotations
2
3 import math
4 import os
5 from datetime import datetime, timezone
6 from pathlib import Path
7 from statistics import NormalDist
8
9 import pandas as pd
10 import rclpy
11 from rclpy.node import Node
12
13 from std_msgs.msg import Float32, String
14
15
16 def _timestamp_ns(value) -> int:
17     return int(pd.Timestamp(value).value)
18

```

```

19
20 class WindCorrectionNode(Node):
21     def __init__(self) -> None:
22         super().__init__("wind_correction_node")
23         self.declare_parameter("forecast_csv", "")
24         self.declare_parameter("corrected_forecast_csv", "outputs/runtime/live_forecast_corrected.csv")
25         self.declare_parameter("forecast_time_tz", "Australia/Darwin")
26         self.declare_parameter("publish_period_sec", 30.0)
27         self.declare_parameter("measurement_sigma_ms", 1.0)
28         self.declare_parameter("correlation_distance_km", 300.0)
29         self.declare_parameter("fallback_correlation_time_h", 3.0)
30         self.declare_parameter("forecast_sigma0_ms", 1.5)
31         self.declare_parameter("forecast_variance_growth_per_hour", 0.05)
32         self.declare_parameter("planning_quantile", 0.5)
33         self.declare_parameter("confidence_z", 1.96)
34         self.declare_parameter("min_sigma_ms", 0.2)
35         self.declare_parameter("preferred_source", "auto")
36         self.declare_parameter("use_exp_distance_decay", True)
37
38         self.forecast_csv = Path(str(self.get_parameter("forecast_csv").value or "")).expanduser()
39         self.corrected_csv = Path(str(self.get_parameter("corrected_forecast_csv").value or "")).expanduser()
40         self.forecast_time_tz = str(self.get_parameter("forecast_time_tz").value or "Australia/Darwin")
41         self.measurement_sigma = max(0.0, float(self.get_parameter("measurement_sigma_ms").value))
42         self.correlation_distance_km = max(1.0, float(self.get_parameter("correlation_distance_km").value))
43         self.correlation_time_h = max(0.25, float(self.get_parameter("fallback_correlation_time_h").value))
44         self.forecast_sigma0 = max(0.0, float(self.get_parameter("forecast_sigma0_ms").value))
45         self.variance_growth = max(0.0, float(self.get_parameter("forecast_variance_growth_per_hour").value))
46         self.planning_quantile = min(0.999, max(0.001, float(self.get_parameter("planning_quantile").value)))
47         self.confidence_z = max(0.1, float(self.get_parameter("confidence_z").value))
48         self.min_sigma = max(0.0, float(self.get_parameter("min_sigma_ms").value))
49         self.use_exp_distance_decay = bool(self.get_parameter("use_exp_distance_decay").value)
50         self.quantile_z = float(NormalDist().inv_cdf(self.planning_quantile))
51
52         self.latest_headwind = math.nan
53         self.latest_s_km = math.nan
54         self.latest_bias = 0.0
55         self.base_df = None
56         self.base_mtime = None
57         self.pub_headwind = self.create_publisher(Float32, "/weather/headwind_corrected_ms", 10)
58         self.pub_status = self.create_publisher(String, "/weather/wind_correction_status", 10)
59         self.create_subscription(Float32, "/weather/headwind_meas_ms", self._on_headwind, 10)
60         self.create_subscription(Float32, "/vehicle/s_km", self._on_s, 10)
61         self.timer = self.create_timer(max(1.0, float(self.get_parameter("publish_period_sec").value)), self._step)
62         self.get_logger().info(f"WindCorrectionNode started: {self.forecast_csv} -> {self.corrected_csv}")
63
64     def _on_headwind(self, msg: Float32) -> None:
65         self.latest_headwind = float(msg.data)
66
67     def _on_s(self, msg: Float32) -> None:
68         self.latest_s_km = float(msg.data)
69
70     def _load_base(self) -> None:
71         try:
72             mtime = os.path.getmtime(self.forecast_csv)
73         except Exception:
74             return
75         if self.base_mtime is not None and mtime <= self.base_mtime and self.base_df is not None:
76             return
77         df = pd.read_csv(self.forecast_csv)
78         if "time" in df.columns:
79             t = pd.to_datetime(df["time"], format="mixed", errors="coerce")
80             if getattr(t.dt, "tz", None) is None:
81                 try:
82                     t = t.dt.tz_localize(self.forecast_time_tz, ambiguous="NaT", nonexistent="NaT").dt.tz_convert("UTC")
83                 except Exception:
84                     t = t.dt.tz_localize("UTC")
85             else:
86                 t = t.dt.tz_convert("UTC")
87             df["time"] = t
88         self.base_df = df
89         self.base_mtime = mtime
90
91     def _interp_headwind_now(self, now_utc: datetime) -> float:
92         if self.base_df is None or self.base_df.empty:
93             return 0.0
94         work = self.base_df
95         if "time" in work.columns and work["time"].notna().any():
96             idx = int(max(0, min(len(work) - 1, work["time"].searchsorted(pd.Timestamp(now_utc), side="right") - 1)))
97             row = work.iloc[idx]
98         else:
99             row = work.iloc[0]
100         try:
101             return float(row.get("headwind_ms", 0.0))
102         except Exception:
103             return 0.0
104
105     def _step(self) -> None:
106         self._load_base()
107         if self.base_df is None or self.base_df.empty:
108             return
109         now_utc = datetime.now(timezone.utc)
110         base_now = self._interp_headwind_now(now_utc)
111         if math.isfinite(self.latest_headwind):
112             innovation = float(self.latest_headwind - base_now)
113             self.latest_bias = 0.7 * float(self.latest_bias) + 0.3 * innovation
114         corrected_now = base_now + self.latest_bias
115         self.pub_headwind.publish(Float32(data=float(corrected_now)))
116
117         df = self.base_df.copy()
118         if "time" not in df.columns:
119             df["time"] = [now_utc + pd.Timedelta(minutes=10 * i) for i in range(len(df))]
120         if "s_km" not in df.columns:
121             df["s_km"] = float(self.latest_s_km) if math.isfinite(self.latest_s_km) else 0.0

```

```

122     current_s = float(self.latest_s_km) if math.isfinite(self.latest_s_km) else float(df["s_km"].iloc[0])
123
124     corrected_values = []
125     sigma_values = []
126     for row in df.itertuples(index=False):
127         t_utc = getattr(row, "time")
128         dt_h = max(0.0, (pd.Timestamp(t_utc).to_pydatetime().astimezone(timezone.utc) - now_utc).total_seconds() / 3600.0)
129         ds = abs(float(getattr(row, "s_km", current_s)) - current_s)
130         time_decay = math.exp(-dt_h / self.correlation_time_h)
131         dist_decay = math.exp(-ds / self.correlation_distance_km) if self.use_exp_distance_decay else 1.0 / (1.0 + ds / self.
correlation_distance_km)
132         decay = time_decay * dist_decay
133         base = float(getattr(row, "headwind_ms", 0.0))
134         mean = base + self.latest_bias * decay
135         sigma = max(self.min_sigma, math.sqrt(self.forecast_sigma0 ** 2 + self.variance_growth * dt_h) + self.measurement_sigma * (1.0 - decay
))
136         corrected = mean + self.quantile_z * sigma
137         corrected_values.append(corrected)
138         sigma_values.append(sigma)
139
140     df["headwind_base_ms"] = pd.to_numeric(df.get("headwind_ms", 0.0), errors="coerce").fillna(0.0)
141     df["headwind_mean_ms"] = corrected_values
142     df["headwind_sigma_ms"] = sigma_values
143     df["headwind_lower_ms"] = df["headwind_mean_ms"] - self.confidence_z * df["headwind_sigma_ms"]
144     df["headwind_upper_ms"] = df["headwind_mean_ms"] + self.confidence_z * df["headwind_sigma_ms"]
145     df["headwind_ms"] = corrected_values
146     out = df.copy()
147     if pd.api.types.is_datetime64_any_dtype(out["time"]):
148         out["time"] = out["time"].dt.tz_convert(self.forecast_time_tz).dt.strftime("%Y-%m-%d %H:%M:%S")
149     self.corrected_csv.parent.mkdir(parents=True, exist_ok=True)
150     out.to_csv(self.corrected_csv, index=False)
151     self.pub_status.publish(String(data=f"bias={self.latest_bias:.2f} corrected_now={corrected_now:.2f} out={self.corrected_csv}"))
152
153
154 def main() -> None:
155     rclpy.init()
156     node = WindCorrectionNode()
157     rclpy.spin(node)
158     node.destroy_node()
159     rclpy.shutdown()

```

A.32 mpc_solarcar/solar_logger_node.py

```

1 import csv
2 import math
3 import os
4 from datetime import datetime, timezone
5
6 import rclpy
7 from rclpy.node import Node
8 from std_msgs.msg import Float32, Float32MultiArray, Float64, String
9
10
11 FLOAT_TOPICS = {
12     "speed_kmh": "/vehicle/speed_kmh",
13     "s_km": "/vehicle/s_km",
14     "altitude_m": "/vehicle/altitude_m",
15     "grade_pct": "/vehicle/grade",
16     "batt_soc": "/vehicle/batt_soc",
17     "batt_temp_c": "/vehicle/batt_temp_c",
18     "batt_current_a": "/vehicle/batt_current_a",
19     "batt_voltage_v": "/vehicle/batt_voltage_v",
20     "solar_power_w": "/vehicle/solar_power_w",
21     "headwind_meas_ms": "/weather/headwind_meas_ms",
22     "headwind_corrected_ms": "/weather/headwind_corrected_ms",
23     "wind_speed_ms": "/weather/wind_speed_ms",
24     "wind_dir_deg": "/weather/wind_dir_deg",
25     "course_deg": "/weather/course_deg",
26     "speed_cmd_kmh": "/planner/speed_cmd",
27     "upper_speed_cmd_kmh": "/planner/upper_speed_cmd",
28     "throttle_cmd_pct": "/planner/throttle_cmd_pct",
29     "calib_solar_gain": "/calib/solar_gain",
30     "calib_drive_power_gain": "/calib/drive_power_gain",
31     "calib_aux_power_w": "/calib/aux_power_w",
32     "system_health": "/system/health",
33 }
34
35 FLOAT64_TOPICS = {
36     "solar_source_ts_unix": "/telemetry/solar_source_ts_unix",
37     "chase_source_ts_unix": "/telemetry/chase_source_ts_unix",
38 }
39
40 STRING_TOPICS = {
41     "drive_mode": "/planner/drive_mode",
42     "system_state": "/system/state",
43     "system_diag": "/system/diag",
44     "mpc_state": "/system/mpc_state",
45     "telemetry_bridge_status": "/telemetry/bridge_status",
46     "wind_correction_status": "/weather/wind_correction_status",
47     "weather_fetch_status": "/weather/fetch_status",
48     "autocal_status": "/calib/status",
49     "raw_solar": "/telemetry/raw_solar",
50     "raw_chase": "/telemetry/raw_chase",
51 }
52
53 ARRAY_TOPICS = {
54     "planner/status": [
55         "planner_soc",

```

```

56     "planner_temp_c",
57     "planner_s_km",
58     "planner_step",
59     "planner_sec_to_next",
60     "planner_control_stop_hold",
61     "planner_control_stop_remaining_sec",
62     "planner_control_stop_completed_count",
63 ],
64 "/planner/metrics": [
65     "model_pack_voltage_v",
66     "model_pack_current_a",
67     "model_soc",
68     "model_motor_power_w",
69     "model_motor_current_a",
70     "model_pv_power_w",
71     "model_speed_kmh",
72     "model_mech_power_w",
73     "model_pack_power_w",
74 ],
75 "/planner/env": [
76     "env_poa_wm2",
77     "env_cell_temp_c",
78     "env_ambient_temp_c",
79     "env_grade_pct",
80     "env_headwind_ms",
81 ],
82 }
83
84
85 class SolarLoggerNode(Node):
86     def __init__(self):
87         super().__init__("solar_logger_node")
88         self.declare_parameter("log_dir", "outputs/logs")
89         self.declare_parameter("file_prefix", "solar_live")
90         self.declare_parameter("log_rate_hz", 2.0)
91         self.declare_parameter("flush_every_rows", 1)
92         self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
93         self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")
94
95         log_dir = os.fspath(self.get_parameter("log_dir").value)
96         prefix = str(self.get_parameter("file_prefix").value)
97         rate_hz = max(0.1, float(self.get_parameter("log_rate_hz").value))
98         self.flush_every_rows = max(1, int(self.get_parameter("flush_every_rows").value))
99         self.rows_since_flush = 0
100
101         os.makedirs(log_dir, exist_ok=True)
102         stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
103         self.log_path = os.path.join(log_dir, f"{prefix}_{stamp}.csv")
104
105         array_fields = [field for fields in ARRAY_TOPICS.values() for field in fields]
106         self.fields = ["t_ros_sec", "t_wall_utc"] + list(FLOAT_TOPICS) + list(FLOAT64_TOPICS) + [
107             "output_speed_cmd_kmh",
108         ] + list(String_TOPICS) + ["output_drive_mode"] + array_fields
109         self.latest = {field: math.nan for field in self.fields}
110         for field in ["t_wall_utc", *String_TOPICS, "output_drive_mode"]:
111             self.latest[field] = ""
112
113         self._topic_subscriptions = []
114         for field, topic in FLOAT_TOPICS.items():
115             self._topic_subscriptions.append(self.create_subscription(Float32, topic, self._set_float(field), 10))
116         for field, topic in FLOAT64_TOPICS.items():
117             self._topic_subscriptions.append(self.create_subscription(Float64, topic, self._set_float(field), 10))
118         for field, topic in String_TOPICS.items():
119             self._topic_subscriptions.append(self.create_subscription(String, topic, self._set_string(field), 10))
120         for topic, fields in ARRAY_TOPICS.items():
121             self._topic_subscriptions.append(self.create_subscription(Float32MultiArray, topic, self._set_array(fields), 10))
122
123         output_speed_topic = str(self.get_parameter("output_speed_topic").value)
124         output_mode_topic = str(self.get_parameter("output_drive_mode_topic").value)
125         self._topic_subscriptions.append(
126             self.create_subscription(Float32, output_speed_topic, self._set_float("output_speed_cmd_kmh"), 10)
127         )
128         self._topic_subscriptions.append(
129             self.create_subscription(String, output_mode_topic, self._set_string("output_drive_mode"), 10)
130         )
131
132         self.csv_file = open(self.log_path, "w", encoding="utf-8", newline="")
133         self.writer = csv.DictWriter(self.csv_file, fieldnames=self.fields)
134         self.writer.writeheader()
135         self.csv_file.flush()
136         self.timer = self.create_timer(1.0 / rate_hz, self._write_row)
137         self.get_logger().info(f"Solar logger started: {self.log_path}")
138
139     def _set_float(self, field):
140         def handler(msg):
141             try:
142                 self.latest[field] = float(msg.data)
143             except (TypeError, ValueError):
144                 self.latest[field] = math.nan
145
146         return handler
147
148     def _set_string(self, field):
149         def handler(msg):
150             self.latest[field] = str(msg.data).replace("\n", "").replace("\n", "\\n")
151
152         return handler
153
154     def _set_array(self, fields):
155         def handler(msg):
156             values = list(msg.data)
157             for idx, field in enumerate(fields):
158                 self.latest[field] = float(values[idx]) if idx < len(values) else math.nan

```

```

159
160     return handler
161
162     @staticmethod
163     def _clean_float(value):
164         try:
165             value = float(value)
166         except (TypeError, ValueError):
167             return ""
168         return value if math.isfinite(value) else ""
169
170     def _write_row(self):
171         self.latest["t_ros_sec"] = self.get_clock().now().nanoseconds / 1.0e9
172         self.latest["t_wall_utc"] = datetime.now(timezone.utc).isoformat()
173         row = {}
174         string_fields = {"t_wall_utc", *STRING_TOPICS, "output_drive_mode"}
175         for field in self.fields:
176             row[field] = str(self.latest.get(field, "")) if field in string_fields else self._clean_float(self.latest.get(field))
177         self.writer.writerow(row)
178         self.rows_since_flush += 1
179         if self.rows_since_flush >= self.flush_every_rows:
180             self.csv_file.flush()
181             self.rows_since_flush = 0
182
183     def destroy_node(self):
184         try:
185             self.csv_file.flush()
186             self.csv_file.close()
187         except Exception:
188             pass
189         super().destroy_node()
190
191
192 def main():
193     rclpy.init()
194     node = SolarLoggerNode()
195     try:
196         rclpy.spin(node)
197     finally:
198         node.destroy_node()
199         rclpy.shutdown()

```

A.33 mpc_solarcar/dashboard_node.py

```

1 import json
2 import math
3 import os
4 import threading
5 import time
6 from functools import partial
7 from http.server import SimpleHTTPRequestHandler, ThreadingHTTPServer
8
9 import rclpy
10 from rclpy.node import Node
11
12 from std_msgs.msg import Float32, Float32MultiArray, String
13 from sensor_msgs.msg import NavSatFix
14 from nav_msgs.msg import Path
15
16 from ament_index_python.packages import get_package_share_directory
17
18
19 class DashboardHandler(SimpleHTTPRequestHandler):
20     def __init__(self, *args, node=None, directory=None, **kwargs):
21         self._node = node
22         super().__init__(*args, directory=directory, **kwargs)
23
24     def do_GET(self):
25         if self.path.startswith('/api/state'):
26             self._send_json(self._node.get_state())
27             return
28         if self.path.startswith('/metrics'):
29             self._send_metrics(self._node.get_prometheus_metrics())
30             return
31         super().do_GET()
32
33     def log_message(self, format, *args):
34         # quiet
35         return
36
37     def _send_json(self, data):
38         payload = json.dumps(data).encode('utf-8')
39         self.send_response(200)
40         self.send_header('Content-Type', 'application/json')
41         self.send_header('Content-Length', str(len(payload)))
42         self.end_headers()
43         self.wfile.write(payload)
44
45     def _send_metrics(self, payload):
46         body = payload.encode('utf-8')
47         self.send_response(200)
48         self.send_header('Content-Type', 'text/plain; version=0.0.4; charset=utf-8')
49         self.send_header('Cache-Control', 'no-store')
50         self.send_header('Content-Length', str(len(body)))
51         self.end_headers()
52         self.wfile.write(body)
53
54

```

```

55 class DashboardNode(Node):
56     def __init__(self):
57         super().__init__('dashboard_node')
58         self.declare_parameter('host', '0.0.0.0')
59         self.declare_parameter('port', 8080)
60         self.declare_parameter('static_dir', '')
61         self.declare_parameter('dummy_csv', '')
62         self.declare_parameter('dummy_rate_hz', 5.0)
63
64         static_dir = str(self.get_parameter('static_dir').value).strip()
65         if not static_dir:
66             try:
67                 pkg_share = get_package_share_directory('mpc_solarcar')
68                 static_dir = os.path.join(pkg_share, 'dashboard')
69             except Exception:
70                 static_dir = os.path.join(os.path.dirname(__file__), '..', 'dashboard')
71         static_dir = os.path.abspath(static_dir)
72
73         self._lock = threading.Lock()
74         self._state = {
75             'ts': time.time(),
76             'speed_cmd_kmh': None,
77             'upper_speed_cmd_kmh': None,
78             'speed_meas_kmh': None,
79             'model_speed_kmh': None,
80             'throttle_cmd_pct': None,
81             'drive_mode': None,
82             'soc': None,
83             'soc_meas': None,
84             'soc_est': None,
85             'Tb_C': None,
86             's_kmh': None,
87             'batt_current_a': None,
88             'batt_voltage_v': None,
89             'motor_w': None,
90             'motor_a': None,
91             'solar_w': None,
92             'wheel_w': None,
93             'pack_w': None,
94             'G_poa': None,
95             'Tcell_C': None,
96             'Tamb_C': None,
97             'headwind_ms': None,
98             'slope_pct': None,
99             'plan_dt': None,
100             'lower_dt': None,
101             'plan_upper': None,
102             'plan_lower': None,
103             'forecast_k': None,
104             'sec_to_next': None,
105             'control_stop_hold': None,
106             'control_stop_remaining_sec': None,
107             'control_stop_completed_count': None,
108             'system_state': None,
109             'system_diag': None,
110             'mpc_state': None,
111             'system_health': None,
112             'gps_lat': None,
113             'gps_lon': None,
114         }
115         self._soc_meas_monotonic = None
116
117         self.create_subscription(Float32, '/planner/speed_cmd', self._on_speed_cmd, 10)
118         self.create_subscription(Float32, '/planner/upper_speed_cmd', self._on_upper_speed_cmd, 10)
119         self.create_subscription(Float32, '/vehicle/speed_kmh', self._on_speed_meas, 10)
120         self.create_subscription(Float32, '/planner/throttle_cmd_pct', self._on_throttle_cmd, 10)
121         self.create_subscription(String, '/planner/drive_mode', self._on_drive_mode, 10)
122         self.create_subscription(Float32, '/vehicle/batt_soc', self._on_soc, 10)
123         self.create_subscription(Float32, '/vehicle/batt_temp_c', self._on_tb, 10)
124         self.create_subscription(Float32, '/vehicle/batt_current_a', self._on_ibatt, 10)
125         self.create_subscription(Float32, '/vehicle/batt_voltage_v', self._on_vbatt, 10)
126         self.create_subscription(Float32, '/vehicle/s_kmh', self._on_s_kmh, 10)
127         self.create_subscription(Float32MultiArray, '/planner/status', self._on_status, 10)
128         self.create_subscription(Float32MultiArray, '/planner/env', self._on_env, 10)
129         self.create_subscription(Float32MultiArray, '/planner/metrics', self._on_metrics, 10)
130         self.create_subscription(Float32MultiArray, '/planner/upper_plan', self._on_upper_plan, 10)
131         self.create_subscription(Float32MultiArray, '/planner/lower_plan', self._on_lower_plan, 10)
132         self.create_subscription(String, '/system/state', self._on_sys_state, 10)
133         self.create_subscription(String, '/system/diag', self._on_sys_diag, 10)
134         self.create_subscription(String, '/system/mpc_state', self._on_mpc_state, 10)
135         self.create_subscription(Float32, '/system/health', self._on_health, 10)
136         self.create_subscription(NavSatFix, '/sim/gps', self._on_gps, 10)
137         self.create_subscription(Path, '/planner/trajectory', self._on_path, 10)
138
139         dummy_csv = str(self.get_parameter('dummy_csv').value).strip()
140         if dummy_csv:
141             self._init_dummy(dummy_csv, float(self.get_parameter('dummy_rate_hz').value))
142
143         self._server = None
144         self._server_thread = None
145         host = str(self.get_parameter('host').value)
146         port = int(self.get_parameter('port').value)
147         handler = partial(DashboardHandler, node=self, directory=static_dir)
148         try:
149             self._server = ThreadingHTTPServer((host, port), handler)
150             self._server_thread = threading.Thread(target=self._server.serve_forever, daemon=True)
151             self._server_thread.start()
152             self.get_logger().info(f'Dashboard server at http://{host}:{port} (dir={static_dir})')
153         except Exception as exc:
154             self.get_logger().error(f'Failed to start dashboard server: {exc}')
155
156     def destroy_node(self):
157         if self._server is not None:

```

```

158         try:
159             self._server.shutdown()
160             self._server.server_close()
161         except Exception:
162             pass
163         return super().destroy_node()
164
165     def get_state(self):
166         with self._lock:
167             return dict(self._state)
168
169     def get_prometheus_metrics(self):
170         metric_keys = {
171             'speed_cmd_kmh': ('solarcar_speed_command_kmh', 'Lower speed command'),
172             'upper_speed_cmd_kmh': ('solarcar_speed_upper_command_kmh', 'Upper planner speed command'),
173             'speed_meas_kmh': ('solarcar_speed_measured_kmh', 'Measured vehicle speed'),
174             'model_speed_kmh': ('solarcar_speed_model_kmh', 'Speed used for planner model metrics'),
175             'throttle_cmd_pct': ('solarcar_throttle_command_percent', 'Lower controller throttle command'),
176             'soc': ('solarcar_battery_soc_ratio', 'Selected battery state of charge'),
177             'soc_meas': ('solarcar_battery_soc_measured_ratio', 'Measured battery state of charge'),
178             'soc_est': ('solarcar_battery_soc_estimated_ratio', 'Estimated battery state of charge'),
179             'Tb_C': ('solarcar_battery_temperature_celsius', 'Battery temperature'),
180             's_kmh': ('solarcar_distance_kmh', 'Race distance'),
181             'batt_current_a': ('solarcar_battery_current_ampere', 'Battery current'),
182             'batt_voltage_v': ('solarcar_battery_voltage_volt', 'Battery terminal voltage'),
183             'motor_w': ('solarcar_motor_power_watt', 'Motor electrical power'),
184             'motor_a': ('solarcar_motor_current_ampere', 'Motor current'),
185             'solar_w': ('solarcar_pv_power_watt', 'PV power'),
186             'wheel_w': ('solarcar_wheel_power_watt', 'Wheel mechanical power'),
187             'pack_w': ('solarcar_pack_power_watt', 'Battery pack power'),
188             'G_poa': ('solarcar_irradiance_poa_watt_per_m2', 'Plane-of-array irradiance'),
189             'Tcell_C': ('solarcar_cell_temperature_celsius', 'PV cell temperature'),
190             'Tamb_C': ('solarcar_ambient_temperature_celsius', 'Ambient temperature'),
191             'headwind_ms': ('solarcar_headwind_meter_per_second', 'Corrected headwind'),
192             'slope_pct': ('solarcar_route_slope_percent', 'Route slope'),
193             'forecast_k': ('solarcar_forecast_index', 'Forecast row index'),
194             'sec_to_next': ('solarcar_seconds_to_next_update', 'Seconds to next planner update'),
195             'control_stop_hold': ('solarcar_control_stop_hold', 'Control-stop approach or dwell is active'),
196             'control_stop_remaining_sec': ('solarcar_control_stop_remaining_seconds', 'Remaining mandatory control-stop dwell'),
197             'control_stop_completed_count': ('solarcar_control_stop_completed_total', 'Completed mandatory control stops'),
198             'system_health': ('solarcar_system_health_ratio', 'System health ratio'),
199             'gps_lat': ('solarcar_gps_latitude_degree', 'GNSS latitude'),
200             'gps_lon': ('solarcar_gps_longitude_degree', 'GNSS longitude'),
201             'path_points': ('solarcar_path_points', 'Published trajectory point count'),
202         }
203         with self._lock:
204             state = dict(self._state)
205             lines = []
206             for state_key, (metric_name, help_text) in metric_keys.items():
207                 value = state.get(state_key)
208                 try:
209                     number = float(value)
210                 except (TypeError, ValueError):
211                     continue
212                 if not math.isfinite(number):
213                     continue
214                 lines.extend((f'# HELP {metric_name} {help_text}', f'# TYPE {metric_name} gauge', f'{metric_name} {number:.12g}'))
215             age_sec = max(0.0, time.time() - float(state.get('ts', time.time())))
216             lines.extend((
217                 '# HELP solarcar_telemetry_age_seconds Age of the newest dashboard state',
218                 '# TYPE solarcar_telemetry_age_seconds gauge',
219                 f'solarcar_telemetry_age_seconds {age_sec:.6f}',
220                 '# HELP solarcar_telemetry_fresh Whether dashboard telemetry is newer than five seconds',
221                 '# TYPE solarcar_telemetry_fresh gauge',
222                 f'solarcar_telemetry_fresh {1 if age_sec <= 5.0 else 0}',
223             ))
224             for key, metric_name in (
225                 ('system_state', 'solarcar_system_state_info'),
226                 ('mpc_state', 'solarcar_mpc_state_info'),
227                 ('drive_mode', 'solarcar_drive_mode_info'),
228             ):
229                 label = str(state.get(key) or 'unknown').replace('\\', '\\\\').replace("'", '\\\'').replace('\n', ' ')
230                 lines.extend((
231                     f'# TYPE {metric_name} gauge',
232                     f'{metric_name}{{value="{label}"}} 1',
233                 ))
234             return '\n'.join(lines) + '\n'
235
236     def update(self, key, value):
237         with self._lock:
238             self._state[key] = value
239             self._state['ts'] = time.time()
240
241     def _on_speed_cmd(self, msg: Float32):
242         self._update('speed_cmd_kmh', float(msg.data))
243
244     def _on_upper_speed_cmd(self, msg: Float32):
245         self._update('upper_speed_cmd_kmh', float(msg.data))
246
247     def _on_speed_meas(self, msg: Float32):
248         self._update('speed_meas_kmh', float(msg.data))
249
250     def _on_throttle_cmd(self, msg: Float32):
251         self._update('throttle_cmd_pct', float(msg.data))
252
253     def _on_drive_mode(self, msg: String):
254         self._update('drive_mode', str(msg.data))
255
256     def _on_soc(self, msg: Float32):
257         value = float(msg.data)
258         with self._lock:
259             self._state['soc'] = value
260             self._state['soc_meas'] = value

```

```

261         self._state['ts'] = time.time()
262         self._soc_meas_monotonic = time.monotonic()
263
264     def _set_estimated_soc(self, value):
265         self._state['soc_est'] = float(value)
266         measured_is_fresh = (
267             self._soc_meas_monotonic is not None
268             and (time.monotonic() - self._soc_meas_monotonic) <= 5.0
269         )
270         if not measured_is_fresh:
271             self._state['soc'] = float(value)
272
273     def _on_tb(self, msg: Float32):
274         self._update('Tb_C', float(msg.data))
275
276     def _on_ibatt(self, msg: Float32):
277         self._update('batt_current_a', float(msg.data))
278
279     def _on_vbatt(self, msg: Float32):
280         self._update('batt_voltage_v', float(msg.data))
281
282     def _on_s_km(self, msg: Float32):
283         self._update('s_km', float(msg.data))
284
285     def _on_status(self, msg: Float32MultiArray):
286         data = list(msg.data)
287         with self._lock:
288             if len(data) >= 5:
289                 self._set_estimated_soc(data[0])
290                 self._state['Tb_C'] = float(data[1])
291                 self._state['s_km'] = float(data[2])
292                 self._state['forecast_k'] = float(data[3])
293                 self._state['sec_to_next'] = float(data[4])
294             if len(data) >= 8:
295                 self._state['control_stop_hold'] = float(data[5])
296                 self._state['control_stop_remaining_sec'] = float(data[6])
297                 self._state['control_stop_completed_count'] = float(data[7])
298             self._state['status_raw'] = data
299             self._state['ts'] = time.time()
300
301     def _on_env(self, msg: Float32MultiArray):
302         data = list(msg.data)
303         if len(data) < 5:
304             return
305         with self._lock:
306             self._state['G_poa'] = float(data[0])
307             self._state['Tcell_C'] = float(data[1])
308             self._state['Tamb_C'] = float(data[2])
309             self._state['slope_pct'] = float(data[3])
310             self._state['headwind_ms'] = float(data[4])
311             self._state['ts'] = time.time()
312
313     def _on_metrics(self, msg: Float32MultiArray):
314         data = list(msg.data)
315         if len(data) < 8:
316             return
317         with self._lock:
318             self._state['batt_voltage_v'] = float(data[0])
319             self._state['batt_current_a'] = float(data[1])
320             self._set_estimated_soc(data[2])
321             self._state['motor_w'] = float(data[3])
322             self._state['motor_a'] = float(data[4])
323             self._state['solar_w'] = float(data[5])
324             # Metrics carry the speed used for model evaluation, not the lower
325             # controller command. Keep it separate so callback ordering cannot
326             # overwrite /planner/speed_cmd on the dashboard.
327             self._state['model_speed_kmh'] = float(data[6])
328             self._state['wheel_w'] = float(data[7])
329             if len(data) >= 9:
330                 self._state['pack_w'] = float(data[8])
331             self._state['ts'] = time.time()
332
333     def _on_upper_plan(self, msg: Float32MultiArray):
334         data = list(msg.data)
335         if len(data) < 2:
336             return
337         dt = float(data[0])
338         speeds = [float(v) for v in data[1:]]
339         with self._lock:
340             self._state['plan_dt'] = dt
341             self._state['plan_upper'] = speeds
342             self._state['ts'] = time.time()
343
344     def _on_lower_plan(self, msg: Float32MultiArray):
345         data = list(msg.data)
346         if len(data) < 2:
347             return
348         dt = float(data[0])
349         speeds = [float(v) for v in data[1:]]
350         with self._lock:
351             self._state['lower_dt'] = dt
352             self._state['plan_lower'] = speeds
353             self._state['ts'] = time.time()
354
355     def _on_sys_state(self, msg: String):
356         self._update('system_state', str(msg.data))
357
358     def _on_sys_diag(self, msg: String):
359         self._update('system_diag', str(msg.data))
360
361     def _on_mpc_state(self, msg: String):
362         self._update('mpc_state', str(msg.data))
363

```

```

364 def _on_health(self, msg: Float32):
365     self._update('system_health', float(msg.data))
366
367 def _on_gps(self, msg: NavSatFix):
368     with self._lock:
369         self._state['gps_lat'] = float(msg.latitude)
370         self._state['gps_lon'] = float(msg.longitude)
371         self._state['ts'] = time.time()
372
373 def _on_path(self, msg: Path):
374     # Keep last path length as a simple sanity signal
375     with self._lock:
376         self._state['path_points'] = len(msg.poses)
377         self._state['ts'] = time.time()
378
379 def _init_dummy(self, path: str, rate_hz: float):
380     try:
381         import pandas as pd
382         df = pd.read_csv(path)
383     except Exception as exc:
384         self.get_logger().error(f'Failed to load dummy_csv: {exc}')
385         return
386     if df.empty:
387         self.get_logger().error('dummy_csv is empty.')
388         return
389     self._dummy_rows = df.to_dict(orient='records')
390     self._dummy_idx = 0
391     period = 1.0 / max(rate_hz, 0.5)
392     self._dummy_timer = self.create_timer(period, self._tick_dummy)
393     self.get_logger().info(f'Dummy mode enabled: {path}^{len(self._dummy_rows)} rows')
394
395 def _tick_dummy(self):
396     if not hasattr(self, '_dummy_rows') or not self._dummy_rows:
397         return
398     row = self._dummy_rows[self._dummy_idx]
399     self._dummy_idx = (self._dummy_idx + 1) % len(self._dummy_rows)
400     with self._lock:
401         for key, val in row.items():
402             try:
403                 self._state[key] = float(val)
404             except Exception:
405                 self._state[key] = val
406         self._state['ts'] = time.time()
407
408
409 def main():
410     rclpy.init()
411     node = DashboardNode()
412     rclpy.spin(node)
413     node.destroy_node()
414     rclpy.shutdown()
415
416
417 if __name__ == '__main__':
418     main()

```

A.34 dashboard/index.html

```

1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1" />
6   <title>Solar MPC Dashboard</title>
7   <link rel="stylesheet" href="style.css" />
8 </head>
9 <body>
10  <div class="bg">
11    <div class="scan"></div>
12    <div class="glow"></div>
13  </div>
14  <div class="shell">
15    <header class="topbar">
16      <div class="brand">
17        <div class="tag">SOLAR CAR EMS</div>
18        <div class="title">SOLAR MPC COMMAND</div>
19      </div>
20      <div class="center">
21        <div class="pill">UPPER <span id="upper_speed">--</span> km/h</div>
22        <div class="pill">LOWER <span id="speed">--</span> km/h</div>
23        <div class="pill">MEAS <span id="speed_meas">--</span> km/h</div>
24      </div>
25      <div class="right">
26        <div id="clock" class="clock">--:--:--</div>
27        <div class="state" id="sys_state">--</div>
28      </div>
29    </header>
30
31    <main class="grid">
32      <section class="card speed-card">
33        <div class="label">Speed Control</div>
34        <div class="speed-main">
35          <span id="speed_big" class="speed">--</span>
36          <span class="unit">km/h</span>
37        </div>
38        <div class="mode-row">
39          <div id="drive_mode" class="mode">MODE</div>
40          <div class="throttle">THROTTLE <span id="throttle">--</span>%</div>

```

```
41     </div>
42     <canvas id="speedCanvas" class="plot"></canvas>
43 </section>
44
45 <section class="card soc-card">
46   <div class="label">Battery Core</div>
47   <div class="soc-ring" id="socRing">
48     <div class="soc-text">
49       <div class="soc-value"><span id="soc">--</span>%</div>
50       <div class="soc-sub">SoC</div>
51     </div>
52   </div>
53   <div class="metrics">
54     <div>Temp <span id="tb">--</span>°C</div>
55     <div>Current <span id="ibatt">--</span>A</div>
56     <div>Voltage <span id="vbatt">--</span>V</div>
57     <div>Distance <span id="s_km">--</span> km</div>
58   </div>
59 </section>
60
61 <section class="card env-card">
62   <div class="label">Environment</div>
63   <div class="env-grid">
64     <div>GHI <span id="ghi">--</span> W/m²</div>
65     <div>Tcell <span id="tcell">--</span>°C</div>
66     <div>Tamb <span id="tamb">--</span>°C</div>
67     <div>Headwind <span id="wind">--</span> m/s</div>
68     <div>Slope <span id="slope">--</span> %</div>
69   </div>
70   <canvas id="throttleCanvas" class="plot"></canvas>
71 </section>
72
73 <section class="card battery-card">
74   <div class="label">Battery</div>
75   <div class="env-grid">
76     <div>Voltage <span id="batt_v">--</span> V</div>
77     <div>Current <span id="batt_a">--</span> A</div>
78     <div>SoC <span id="soc_small">--</span>%</div>
79     <div>Pack W <span id="pack_w">--</span> W</div>
80   </div>
81   <canvas id="batteryCanvas" class="plot"></canvas>
82 </section>
83
84 <section class="card motor-card">
85   <div class="label">Motor</div>
86   <div class="env-grid">
87     <div>Motor W <span id="motor_w">--</span> W</div>
88     <div>Motor A <span id="motor_a">--</span> A</div>
89   </div>
90   <canvas id="motorCanvas" class="plot"></canvas>
91 </section>
92
93 <section class="card solar-card">
94   <div class="label">Solar</div>
95   <div class="env-grid">
96     <div>Solar W <span id="solar_w">--</span> W</div>
97   </div>
98   <canvas id="solarCanvas" class="plot"></canvas>
99 </section>
100
101 <section class="card speedpower-card">
102   <div class="label">Speed / Wheel Power</div>
103   <div class="env-grid">
104     <div>Speed <span id="speed_kmh">--</span> km/h</div>
105     <div>Wheel W <span id="wheel_w">--</span> W</div>
106   </div>
107   <canvas id="speedPowerCanvas" class="plot"></canvas>
108 </section>
109
110 <section class="card plan-card">
111   <div class="label">Upper Plan</div>
112   <canvas id="planCanvas" class="plot"></canvas>
113   <div class="plan-meta">
114     <span>dt <span id="plan_dt">--</span>s</span>
115     <span>k <span id="forecast_k">--</span></span></span>
116     <span>next <span id="sec_next">--</span>s</span>
117   </div>
118 </section>
119
120 <section class="card lower-card">
121   <div class="label">Lower Plan</div>
122   <canvas id="lowerCanvas" class="plot"></canvas>
123   <div class="plan-meta">
124     <span>dt <span id="lower_dt">--</span>s</span>
125   </div>
126 </section>
127
128 <section class="card sys-card">
129   <div class="label">System Status</div>
130   <div class="sys-grid">
131     <div>Diag <span id="sys_diag">--</span></div>
132     <div>MPC <span id="mpc_state">--</span></div>
133     <div>Health <span id="health">--</span></div>
134   </div>
135   <div class="badges">
136     <span class="badge" id="mpc_badge">MPC</span>
137     <span class="badge" id="drive_badge">DRIVE</span>
138   </div>
139 </section>
140 </main>
141
142 <footer class="footer">
143   <div class="footer-left">SolarCar MPC • Hierarchical</div>
```

```

144     <div class="footer-right">Web Dashboard Node</div>
145   </footer>
146 </div>
147
148 <script src="app.js"></script>
149 </body>
150 </html>

```

A.35 dashboard/app.js

```

1  const els = {
2    speed: document.getElementById('speed'),
3    speedBig: document.getElementById('speed_big'),
4    upper: document.getElementById('upper_speed'),
5    speedMeas: document.getElementById('speed_meas'),
6    throttle: document.getElementById('throttle'),
7    driveMode: document.getElementById('drive_mode'),
8    soc: document.getElementById('soc'),
9    tb: document.getElementById('tb'),
10   ibatt: document.getElementById('ibatt'),
11   vbatt: document.getElementById('vbatt'),
12   battV: document.getElementById('batt_v'),
13   battA: document.getElementById('batt_a'),
14   socSmall: document.getElementById('soc_small'),
15   packW: document.getElementById('pack_w'),
16   motorW: document.getElementById('motor_w'),
17   motorA: document.getElementById('motor_a'),
18   solarW: document.getElementById('solar_w'),
19   speedKmh: document.getElementById('speed_kmh'),
20   wheelW: document.getElementById('wheel_w'),
21   sKmh: document.getElementById('s_kmh'),
22   ghi: document.getElementById('ghi'),
23   tcell: document.getElementById('tcell'),
24   tamb: document.getElementById('tamb'),
25   wind: document.getElementById('wind'),
26   slope: document.getElementById('slope'),
27   planDt: document.getElementById('plan_dt'),
28   lowerDt: document.getElementById('lower_dt'),
29   forecastK: document.getElementById('forecast_k'),
30   secNext: document.getElementById('sec_next'),
31   sysState: document.getElementById('sys_state'),
32   sysDiag: document.getElementById('sys_diag'),
33   mpcState: document.getElementById('mpc_state'),
34   health: document.getElementById('health'),
35   clock: document.getElementById('clock'),
36   socRing: document.getElementById('socRing'),
37   mpcBadge: document.getElementById('mpc_badge'),
38   driveBadge: document.getElementById('drive_badge'),
39 };
40
41  const canvases = {
42    speed: document.getElementById('speedCanvas'),
43    throttle: document.getElementById('throttleCanvas'),
44    plan: document.getElementById('planCanvas'),
45    lower: document.getElementById('lowerCanvas'),
46    battery: document.getElementById('batteryCanvas'),
47    motor: document.getElementById('motorCanvas'),
48    solar: document.getElementById('solarCanvas'),
49    speedPower: document.getElementById('speedPowerCanvas'),
50  };
51
52  const history = {
53    speed: [],
54    upper: [],
55    throttle: [],
56    soc: [],
57    battV: [],
58    battA: [],
59    packW: [],
60    motorW: [],
61    motorA: [],
62    solarW: [],
63    wheelW: [],
64    max: 900,
65  };
66
67  function fmt(val, digits = 1) {
68    if (val === null || val === undefined || !Number.isFinite(val)) return '--';
69    return val.toFixed(digits);
70  }
71
72  function pushHistory(arr, val) {
73    if (!Number.isFinite(val)) return;
74    arr.push(val);
75    if (arr.length > history.max) arr.shift();
76  }
77
78  function drawPlot(canvas, seriesList, colors) {
79    const ctx = canvas.getContext('2d');
80    const w = canvas.clientWidth;
81    const h = canvas.clientHeight;
82    if (w === 0 || h === 0) return;
83    canvas.width = w;
84    canvas.height = h;
85    ctx.clearRect(0, 0, w, h);
86    ctx.fillStyle = 'rgba(10,14,20,0.8)';
87    ctx.fillRect(0, 0, w, h);
88    ctx.strokeStyle = 'rgba(255,255,255,0.08)';

```

```

89   ctx.lineWidth = 1;
90   for (let i = 1; i < 4; i++) {
91     const y = (h / 4) * i;
92     ctx.beginPath();
93     ctx.moveTo(0, y);
94     ctx.lineTo(w, y);
95     ctx.stroke();
96   }
97   let min = Infinity;
98   let max = -Infinity;
99   seriesList.forEach((s) => {
100     s.forEach((v) => {
101       min = Math.min(min, v);
102       max = Math.max(max, v);
103     });
104   });
105   if (!Number.isFinite(min) || !Number.isFinite(max)) return;
106   if (Math.abs(max - min) < 1e-6) {
107     max += 1;
108     min -= 1;
109   }
110   seriesList.forEach((series, idx) => {
111     if (series.length < 2) return;
112     ctx.strokeStyle = colors[idx] || '#fff';
113     ctx.lineWidth = 2;
114     ctx.beginPath();
115     series.forEach((v, i) => {
116       const x = (i / (series.length - 1)) * (w - 6) + 3;
117       const y = h - ((v - min) / (max - min)) * (h - 10) - 5;
118       if (i === 0) ctx.moveTo(x, y);
119       else ctx.lineTo(x, y);
120     });
121     ctx.stroke();
122   });
123 }
124
125 function updateClock() {
126   const now = new Date();
127   const hh = String(now.getHours()).padStart(2, '0');
128   const mm = String(now.getMinutes()).padStart(2, '0');
129   const ss = String(now.getSeconds()).padStart(2, '0');
130   els.clock.textContent = `${hh}:${mm}:${ss}`;
131 }
132
133 function applyState(data) {
134   const vLower = data.speed_cmd_kmh;
135   const vUpper = data.upper_speed_cmd_kmh;
136   const vMeas = data.speed_meas_kmh;
137   const throttle = data.throttle_cmd_pct;
138   let soc = data.soc;
139   if (Number.isFinite(soc) && soc > 1.5) soc = soc / 100.0;
140
141   els.speed.textContent = fmt(vLower, 1);
142   els.speedBig.textContent = fmt(vLower, 0);
143   els.upper.textContent = fmt(vUpper, 1);
144   els.speedMeas.textContent = fmt(vMeas, 1);
145   els.throttle.textContent = fmt(throttle, 1);
146   els.driveMode.textContent = data.drive_mode || '--';
147   els.soc.textContent = fmt(soc * 100, 1);
148   els.tb.textContent = fmt(data.Tb_C, 1);
149   els.ibatt.textContent = fmt(data.batt_current_a, 1);
150   els.vbatt.textContent = fmt(data.batt_voltage_v, 1);
151   els.battV.textContent = fmt(data.batt_voltage_v, 1);
152   els.battA.textContent = fmt(data.batt_current_a, 1);
153   els.socSmall.textContent = fmt(soc * 100, 1);
154   els.packW.textContent = fmt(data.pack_w, 0);
155   els.motorW.textContent = fmt(data.motor_w, 0);
156   els.motorA.textContent = fmt(data.motor_a, 1);
157   els.solarW.textContent = fmt(data.solar_w, 0);
158   els.speedKmh.textContent = fmt(vLower, 1);
159   els.wheelW.textContent = fmt(data.wheel_w, 0);
160   els.sKm.textContent = fmt(data.s_km, 1);
161   els.ghi.textContent = fmt(data.G_poa, 0);
162   els.tcell.textContent = fmt(data.Tcell_C, 1);
163   els.tamb.textContent = fmt(data.Tamb_C, 1);
164   els.wind.textContent = fmt(data.headwind_ms, 1);
165   els.slope.textContent = fmt(data.slope_pct, 2);
166   els.planDt.textContent = fmt(data.plan_dt, 0);
167   els.lowerDt.textContent = fmt(data.lower_dt, 2);
168   els.forecastK.textContent = fmt(data.forecast_k, 0);
169   els.secNext.textContent = fmt(data.sec_to_next, 0);
170   els.sysState.textContent = data.system_state || '--';
171   els.sysDiag.textContent = data.system_diag || '--';
172   els.mpcState.textContent = data.mpc_state || '--';
173   els.health.textContent = fmt(data.system_health, 2);
174
175   els.socRing.style.setProperty('--soc', Math.max(0, Math.min(1, soc || 0)));
176   els.mpcBadge.textContent = data.mpc_state || 'MPC';
177   els.driveBadge.textContent = data.drive_mode || 'DRIVE';
178
179   pushHistory(history.speed, vLower);
180   pushHistory(history.upper, vUpper);
181   pushHistory(history.throttle, throttle);
182   pushHistory(history.soc, (soc || 0) * 100);
183   pushHistory(history.battV, data.batt_voltage_v);
184   pushHistory(history.battA, data.batt_current_a);
185   pushHistory(history.packW, data.pack_w);
186   pushHistory(history.motorW, data.motor_w);
187   pushHistory(history.motorA, data.motor_a);
188   pushHistory(history.solarW, data.solar_w);
189   pushHistory(history.wheelW, data.wheel_w);
190
191   drawPlot(canvases.speed, [history.speed, history.upper], ['#ff2d2d', '#2de2e6']);

```

```

192 drawPlot(canvases.throttle, [history.throttle], ['#f7c14a']);
193 drawPlot(canvases.battery, [history.battV, history.battA, history.soc], ['#2de2e6', '#f7c14a', '#ff2d2d']);
194 drawPlot(canvases.motor, [history.motorW, history.motorA], ['#ff2d2d', '#2de2e6']);
195 drawPlot(canvases.solar, [history.solarW], ['#f7c14a']);
196 drawPlot(canvases.speedPower, [history.speed, history.wheelW], ['#2de2e6', '#ff2d2d']);
197
198 if (Array.isArray(data.plan_upper)) {
199   drawPlot(canvases.plan, [data.plan_upper], ['#2de2e6']);
200 }
201 if (Array.isArray(data.plan_lower)) {
202   drawPlot(canvases.lower, [data.plan_lower], ['#ff2d2d']);
203 }
204 }
205
206 async function fetchState() {
207   try {
208     const res = await fetch('/api/state', { cache: 'no-store' });
209     const data = await res.json();
210     applyState(data);
211   } catch (err) {
212     // ignore fetch errors
213   }
214 }
215
216 setInterval(fetchState, 200);
217 setInterval(updateClock, 1000);
218 updateClock();

```

A.36 dashboard/style.css

```

1 @import url('https://fonts.googleapis.com/css2?family=Oxanium:wght@400;600;700&family=Space+Grotesk:wght@400;600&display=swap');
2
3 :root {
4   --bg: #0b0f14;
5   --bg2: #121826;
6   --panel: #0f1624;
7   --panel-2: #0c111b;
8   --accent: #ff2d2d;
9   --accent-2: #2de2e6;
10  --accent-3: #f7c14a;
11  --text: #e9eef5;
12  --muted: #8b97a8;
13  --grid-gap: 18px;
14 }
15
16 * { box-sizing: border-box; }
17
18 body {
19   margin: 0;
20   font-family: 'Space Grotesk', sans-serif;
21   color: var(--text);
22   background: var(--bg);
23   overflow-x: hidden;
24 }
25
26 .bg {
27   position: fixed;
28   inset: 0;
29   background: radial-gradient(1200px 600px at 80% -10%, rgba(45, 226, 230, 0.15), transparent 60%),
30     radial-gradient(900px 500px at -10% 20%, rgba(247, 193, 74, 0.12), transparent 60%),
31     linear-gradient(180deg, #090c11, #111824 60%, #0a0e14);
32   z-index: -2;
33 }
34
35 .bg .scan {
36   position: absolute;
37   inset: 0;
38   background: repeating-linear-gradient(
39     180deg,
40     rgba(255, 255, 255, 0.03) 0px,
41     rgba(255, 255, 255, 0.03) 1px,
42     transparent 1px,
43     transparent 6px
44   );
45   opacity: 0.3;
46 }
47
48 .bg .glow {
49   position: absolute;
50   width: 60vw;
51   height: 60vw;
52   top: 10%;
53   right: -20%;
54   background: radial-gradient(circle, rgba(255, 45, 45, 0.2), transparent 60%);
55   filter: blur(40px);
56 }
57
58 .shell {
59   min-height: 100vh;
60   display: flex;
61   flex-direction: column;
62   padding: 22px;
63   gap: 16px;
64 }
65
66 .topbar {
67   display: grid;
68   grid-template-columns: 1fr 2fr 1fr;

```

```

69   align-items: center;
70   gap: 16px;
71   padding: 16px 20px;
72   background: linear-gradient(120deg, rgba(255,45,45,0.18), transparent 60%),
73             rgba(15, 22, 36, 0.9);
74   border: 1px solid rgba(255, 255, 255, 0.08);
75   border-radius: 16px;
76   backdrop-filter: blur(6px);
77 }
78
79 .brand .tag {
80   font-family: 'Oxanium', sans-serif;
81   font-weight: 600;
82   letter-spacing: 0.2em;
83   font-size: 11px;
84   color: var(--accent-3);
85   text-transform: uppercase;
86 }
87
88 .brand .title {
89   font-family: 'Oxanium', sans-serif;
90   font-weight: 700;
91   font-size: 20px;
92   margin-top: 6px;
93 }
94
95 .center {
96   display: flex;
97   justify-content: center;
98   gap: 10px;
99   flex-wrap: wrap;
100 }
101
102 .pill {
103   padding: 6px 12px;
104   border-radius: 999px;
105   border: 1px solid rgba(45, 226, 230, 0.4);
106   color: var(--accent-2);
107   font-size: 12px;
108   letter-spacing: 0.08em;
109   text-transform: uppercase;
110 }
111
112 .right {
113   text-align: right;
114 }
115
116 .clock {
117   font-family: 'Oxanium', sans-serif;
118   font-size: 20px;
119   font-weight: 600;
120 }
121
122 .state {
123   color: var(--accent);
124   font-weight: 600;
125   letter-spacing: 0.1em;
126 }
127
128 .grid {
129   display: grid;
130   grid-template-columns: repeat(12, 1fr);
131   grid-auto-rows: minmax(140px, auto);
132   gap: var(--grid-gap);
133 }
134
135 .card {
136   background: linear-gradient(160deg, rgba(19, 27, 43, 0.95), rgba(12, 16, 26, 0.95));
137   border: 1px solid rgba(255, 255, 255, 0.06);
138   border-radius: 18px;
139   padding: 18px;
140   position: relative;
141   overflow: hidden;
142   box-shadow: 0 18px 40px rgba(0, 0, 0, 0.35);
143 }
144
145 .card::after {
146   content: '';
147   position: absolute;
148   top: 0;
149   left: 0;
150   width: 60px;
151   height: 3px;
152   background: linear-gradient(90deg, var(--accent), transparent);
153 }
154
155 .label {
156   font-size: 12px;
157   letter-spacing: 0.2em;
158   text-transform: uppercase;
159   color: var(--muted);
160   margin-bottom: 10px;
161 }
162
163 .speed-card { grid-column: span 5; }
164 .soc-card { grid-column: span 3; }
165 .env-card { grid-column: span 4; }
166 .battery-card { grid-column: span 3; }
167 .motor-card { grid-column: span 3; }
168 .solar-card { grid-column: span 3; }
169 .speedpower-card { grid-column: span 3; }
170 .plan-card { grid-column: span 6; }
171 .lower-card { grid-column: span 6; }

```

```
172 .sys-card { grid-column: span 6; }
173
174 .speed-main {
175   display: flex;
176   align-items: baseline;
177   gap: 12px;
178 }
179
180 .speed {
181   font-family: 'Oxanium', sans-serif;
182   font-size: 72px;
183   font-weight: 700;
184   color: var(--accent);
185   text-shadow: 0 0 18px rgba(255, 45, 45, 0.35);
186 }
187
188 .unit {
189   font-size: 18px;
190   color: var(--muted);
191 }
192
193 .mode-row {
194   display: flex;
195   justify-content: space-between;
196   align-items: center;
197   margin-top: 6px;
198 }
199
200 .mode {
201   padding: 6px 12px;
202   border-radius: 10px;
203   background: rgba(45, 226, 230, 0.15);
204   color: var(--accent-2);
205   font-family: 'Oxanium', sans-serif;
206   letter-spacing: 0.08em;
207 }
208
209 .throttle {
210   color: var(--accent-3);
211   font-weight: 600;
212 }
213
214 .plot {
215   width: 100%;
216   height: 120px;
217   margin-top: 12px;
218   background: rgba(8, 12, 18, 0.7);
219   border-radius: 12px;
220 }
221
222 .soc-ring {
223   width: 160px;
224   height: 160px;
225   margin: 6px auto 10px;
226   border-radius: 50%;
227   background: conic-gradient(var(--accent-2) calc(var(--soc, 0.0) * 360deg), rgba(255,255,255,0.08) 0deg);
228   display: flex;
229   align-items: center;
230   justify-content: center;
231   position: relative;
232 }
233
234 .soc-ring::after {
235   content: '';
236   position: absolute;
237   inset: 14px;
238   background: var(--panel);
239   border-radius: 50%;
240   border: 1px solid rgba(255, 255, 255, 0.08);
241 }
242
243 .soc-text {
244   position: relative;
245   text-align: center;
246   z-index: 1;
247 }
248
249 .soc-value {
250   font-family: 'Oxanium', sans-serif;
251   font-size: 32px;
252   color: var(--accent-2);
253 }
254
255 .soc-sub { color: var(--muted); font-size: 12px; }
256
257 .metrics {
258   display: grid;
259   gap: 6px;
260   font-size: 13px;
261   color: var(--muted);
262 }
263
264 .metrics span { color: var(--text); font-weight: 600; }
265
266 .env-grid, .sys-grid {
267   display: grid;
268   grid-template-columns: repeat(2, 1fr);
269   gap: 6px 12px;
270   font-size: 13px;
271   color: var(--muted);
272 }
273
274 .env-grid span, .sys-grid span {
```

```

275   color: var(--text);
276   font-weight: 600;
277 }
278
279 .plan-meta {
280   display: flex;
281   justify-content: space-between;
282   font-size: 12px;
283   color: var(--muted);
284   margin-top: 8px;
285 }
286
287 .badges {
288   margin-top: 12px;
289   display: flex;
290   gap: 8px;
291 }
292
293 .badge {
294   padding: 6px 10px;
295   border-radius: 999px;
296   border: 1px solid rgba(255, 255, 255, 0.12);
297   font-size: 11px;
298   letter-spacing: 0.1em;
299 }
300
301 .footer {
302   display: flex;
303   justify-content: space-between;
304   color: var(--muted);
305   font-size: 12px;
306   padding: 0 6px;
307 }
308
309 @media (min-width: 1101px) and (max-height: 1100px) {
310   :root { --grid-gap: 10px; }
311   .shell { padding: 12px; gap: 10px; }
312   .topbar { padding: 10px 16px; border-radius: 12px; }
313   .brand .title { font-size: 18px; margin-top: 3px; }
314   .clock { font-size: 17px; }
315   .grid { grid-auto-rows: minmax(96px, auto); }
316   .card { padding: 12px; border-radius: 14px; }
317   .label { margin-bottom: 6px; }
318   .speed { font-size: 52px; }
319   .mode-row { margin-top: 2px; }
320   .plot { height: 72px; margin-top: 6px; }
321   .soc-ring { width: 116px; height: 116px; margin: 0 auto 6px; }
322   .soc-ring::after { inset: 10px; }
323   .soc-value { font-size: 26px; }
324   .metrics, .env-grid, .sys-grid { gap: 3px 10px; font-size: 12px; }
325   .plan-meta { margin-top: 4px; }
326   .badges { margin-top: 6px; }
327   .badge { padding: 4px 8px; }
328 }
329
330 @media (max-width: 1100px) {
331   .topbar {
332     grid-template-columns: 1fr;
333     text-align: center;
334   }
335   .right { text-align: center; }
336   .grid { grid-template-columns: repeat(6, 1fr); }
337   .speed-card, .soc-card, .env-card, .battery-card, .motor-card, .solar-card, .speedpower-card, .plan-card, .lower-card, .sys-card {
338     grid-column: span 6;
339   }
340 }

```

A.37 grafana/docker-compose.yml

```

1 services:
2   prometheus:
3     image: prom/prometheus:v3.5.0
4     container_name: solarcar-prometheus
5     restart: unless-stopped
6     command:
7       - --config.file=/etc/prometheus/prometheus.yml
8       - --storage.tsdb.retention.time=30d
9       - --web.enable-lifecycle
10    ports:
11      - "9090:9090"
12    extra_hosts:
13      - "host.docker.internal:host-gateway"
14    volumes:
15      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
16      - prometheus_data:/prometheus
17
18    grafana:
19      image: grafana/grafana:12.1.0
20      container_name: solarcar-grafana
21      restart: unless-stopped
22      depends_on:
23        - prometheus
24      ports:
25        - "3000:3000"
26      environment:
27        GF_AUTH_ANONYMOUS_ENABLED: "true"
28        GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
29        GF_AUTH_DISABLE_LOGIN_FORM: "true"

```

```

30     GF_USERS_DEFAULT_THEME: dark
31     GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/solarcar-ems.json
32     volumes:
33       - grafana_data:/var/lib/grafana
34       - ./provisioning:/etc/grafana/provisioning:ro
35       - ./dashboards:/var/lib/grafana/dashboards:ro
36
37     volumes:
38       prometheus_data:
39       grafana_data:

```

A.38 grafana/prometheus.yml

```

1 global:
2   scrape_interval: 1s
3   evaluation_interval: 1s
4
5 scrape_configs:
6   - job_name: solarcar_ems
7     metrics_path: /metrics
8     static_configs:
9       - targets:
10         - host.docker.internal:8080
11     labels:
12       system: YATA

```

A.39 grafana/provisioning/dashboards/dashboards.yml

```

1 apiVersion: 1
2
3 providers:
4   - name: SolarCar EMS
5     orgId: 1
6     folder: YATA
7     type: file
8     disableDeletion: true
9     allowUiUpdates: false
10    updateIntervalSeconds: 10
11    options:
12      path: /var/lib/grafana/dashboards

```

A.40 grafana/provisioning/datasources/prometheus.yml

```

1 apiVersion: 1
2
3 datasources:
4   - name: SolarCar Prometheus
5     uid: solar-prometheus
6     type: prometheus
7     access: proxy
8     url: http://prometheus:9090
9     isDefault: true
10    editable: false
11    jsonData:
12      httpMethod: POST
13      timeInterval: 1s

```

A.41 grafana/dashboards/solarcar-ems.json

```

1 {
2   "annotations": {"list": []},
3   "editable": false,
4   "fiscalYearStartMonth": 0,
5   "graphTooltip": 1,
6   "id": null,
7   "links": [],
8   "liveNow": true,
9   "panels": [
10     {
11       "type": "stat", "title": "SPEED / MEASURED", "id": 1,
12       "gridPos": {"h": 4, "w": 4, "x": 0, "y": 0},
13       "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
14       "targets": [{"expr": "solarcar_speed_measured_kmh", "instant": true, "refId": "A"}],
15       "fieldConfig": {"defaults": {"unit": "velocitykmh", "decimals": 1, "color": {"mode": "fixedColor", "fixedColor": "semi-dark-blue"}, "thresholds": {"mode": "absolute", "steps": [{"color": "dark-blue", "value": null}, {"color": "orange", "value": 90}, {"color": "red", "value": 105}]}}, "overrides": []},
16       "options": {"colorMode": "background", "graphMode": "area", "justifyMode": "center", "orientation": "horizontal", "reduceOptions": {"calcs": ["lastNotNull"], "fields": "", "values": false}, "textMode": "value_and_name", "wideLayout": true}
17     },
18     {
19       "type": "stat", "title": "SPEED / COMMAND", "id": 2,

```

```

20   "gridPos": {"h": 4, "w": 4, "x": 4, "y": 0},
21   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
22   "targets": [{"expr": "solarcar_speed_command_kmh", "instant": true, "refId": "A"}],
23   "fieldConfig": {"defaults": {"unit": "velocitykmh", "decimals": 1, "color": {"mode": "fixedColor", "fixedColor": "semi-dark-cyan"}}, "
  overrides": []},
24   "options": {"colorMode": "background", "graphMode": "area", "justifyMode": "center", "orientation": "horizontal", "reduceOptions": {"calcs":
    ["lastNotNull"], "fields": "", "values": false}, "textMode": "value_and_name", "wideLayout": true}
25 },
26 {
27   "type": "stat", "title": "BATTERY SOC", "id": 3,
28   "gridPos": {"h": 4, "w": 4, "x": 8, "y": 0},
29   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
30   "targets": [{"expr": "solarcar_battery_soc_ratio * 100", "instant": true, "refId": "A"}],
31   "fieldConfig": {"defaults": {"unit": "percent", "min": 0, "max": 100, "decimals": 1, "color": {"mode": "thresholds"}, "thresholds": {"mode":
    "absolute", "steps": [{"color": "red", "value": null}, {"color": "orange", "value": 20}, {"color": "green", "value": 40}]}}, "overrides":
    []},
32   "options": {"colorMode": "background", "graphMode": "area", "justifyMode": "center", "orientation": "horizontal", "reduceOptions": {"calcs":
    ["lastNotNull"], "fields": "", "values": false}, "textMode": "value_and_name", "wideLayout": true}
33 },
34 {
35   "type": "stat", "title": "DISTANCE", "id": 4,
36   "gridPos": {"h": 4, "w": 4, "x": 12, "y": 0},
37   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
38   "targets": [{"expr": "solarcar_distance_kmh", "instant": true, "refId": "A"}],
39   "fieldConfig": {"defaults": {"unit": "lengthkm", "decimals": 1, "color": {"mode": "fixedColor", "fixedColor": "dark-green"}}, "overrides":
    []},
40   "options": {"colorMode": "background", "graphMode": "area", "justifyMode": "center", "orientation": "horizontal", "reduceOptions": {"calcs":
    ["lastNotNull"], "fields": "", "values": false}, "textMode": "value_and_name", "wideLayout": true}
41 },
42 {
43   "type": "stat", "title": "SYSTEM HEALTH", "id": 5,
44   "gridPos": {"h": 4, "w": 4, "x": 16, "y": 0},
45   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
46   "targets": [{"expr": "solarcar_system_health_ratio * 100", "instant": true, "refId": "A"}],
47   "fieldConfig": {"defaults": {"unit": "percent", "min": 0, "max": 100, "decimals": 0, "color": {"mode": "thresholds"}, "thresholds": {"mode":
    "absolute", "steps": [{"color": "red", "value": null}, {"color": "orange", "value": 70}, {"color": "green", "value": 90}]}}, "overrides":
    []},
48   "options": {"colorMode": "background", "graphMode": "none", "justifyMode": "center", "orientation": "horizontal", "reduceOptions": {"calcs":
    ["lastNotNull"], "fields": "", "values": false}, "textMode": "value_and_name", "wideLayout": true}
49 },
50 {
51   "type": "stat", "title": "TELEMETRY AGE", "id": 6,
52   "gridPos": {"h": 4, "w": 4, "x": 20, "y": 0},
53   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
54   "targets": [{"expr": "solarcar_telemetry_age_seconds", "instant": true, "refId": "A"}],
55   "fieldConfig": {"defaults": {"unit": "s", "decimals": 1, "color": {"mode": "thresholds"}, "thresholds": {"mode": "absolute", "steps": [{"
    color": "green", "value": null}, {"color": "orange", "value": 2}, {"color": "red", "value": 5}]}}, "overrides": []},
56   "options": {"colorMode": "background", "graphMode": "area", "justifyMode": "center", "orientation": "horizontal", "reduceOptions": {"calcs":
    ["lastNotNull"], "fields": "", "values": false}, "textMode": "value_and_name", "wideLayout": true}
57 },
58 {
59   "type": "timeseries", "title": "Speed tracking", "id": 7,
60   "gridPos": {"h": 6, "w": 8, "x": 0, "y": 4},
61   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
62   "targets": [
63     {"expr": "solarcar_speed_upper_command_kmh", "legendFormat": "upper", "refId": "A"},
64     {"expr": "solarcar_speed_command_kmh", "legendFormat": "lower", "refId": "B"},
65     {"expr": "solarcar_speed_measured_kmh", "legendFormat": "measured", "refId": "C"}
66   ],
67   "fieldConfig": {"defaults": {"unit": "velocitykmh", "custom": {"drawStyle": "line", "lineInterpolation": "smooth", "lineWidth": 2, "
    fillOpacity": 8, "showPoints": "never", "spanNulls": true}}, "overrides": []},
68   "options": {"legend": {"displayMode": "list", "placement": "bottom", "showLegend": true}, "tooltip": {"mode": "multi", "sort": "desc"}}
69 },
70 {
71   "type": "timeseries", "title": "Energy flow", "id": 8,
72   "gridPos": {"h": 6, "w": 8, "x": 8, "y": 4},
73   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
74   "targets": [
75     {"expr": "solarcar_pv_power_watt", "legendFormat": "PV", "refId": "A"},
76     {"expr": "solarcar_motor_power_watt", "legendFormat": "motor", "refId": "B"},
77     {"expr": "solarcar_pack_power_watt", "legendFormat": "pack", "refId": "C"},
78     {"expr": "solarcar_wheel_power_watt", "legendFormat": "wheel", "refId": "D"}
79   ],
80   "fieldConfig": {"defaults": {"unit": "watt", "custom": {"drawStyle": "line", "lineInterpolation": "smooth", "lineWidth": 2, "fillOpacity":
    10, "showPoints": "never", "spanNulls": true}}, "overrides": []},
81   "options": {"legend": {"displayMode": "list", "placement": "bottom", "showLegend": true}, "tooltip": {"mode": "multi", "sort": "desc"}}
82 },
83 {
84   "type": "timeseries", "title": "Battery electrical state", "id": 9,
85   "gridPos": {"h": 6, "w": 8, "x": 16, "y": 4},
86   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
87   "targets": [
88     {"expr": "solarcar_battery_voltage_volt", "legendFormat": "voltage [V]", "refId": "A"},
89     {"expr": "solarcar_battery_current_ampere", "legendFormat": "current [A]", "refId": "B"},
90     {"expr": "solarcar_battery_soc_ratio * 100", "legendFormat": "SoC [%]", "refId": "C"}
91   ],
92   "fieldConfig": {"defaults": {"custom": {"drawStyle": "line", "lineInterpolation": "smooth", "lineWidth": 2, "fillOpacity": 8, "showPoints":
    "never", "spanNulls": true}}, "overrides": [
93     {"matcher": {"id": "byName", "options": "voltage [V]"}, "properties": [{"id": "unit", "value": "volt"}, {"id": "custom.axisPlacement", "
    value": "left"}]},
94     {"matcher": {"id": "byName", "options": "current [A]"}, "properties": [{"id": "unit", "value": "amp"}, {"id": "custom.axisPlacement", "
    value": "right"}]},
95     {"matcher": {"id": "byName", "options": "SoC [%]"}, "properties": [{"id": "unit", "value": "percent"}, {"id": "custom.axisPlacement", "
    value": "right"}]}
96   ],
97   "options": {"legend": {"displayMode": "list", "placement": "bottom", "showLegend": true}, "tooltip": {"mode": "multi", "sort": "desc"}}
98 },
99 {
100   "type": "timeseries", "title": "Solar and route environment", "id": 10,
101   "gridPos": {"h": 6, "w": 12, "x": 0, "y": 10},
102   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
103   "targets": [
104     {"expr": "solarcar_irradiance_poa_watt_per_m2", "legendFormat": "POA [W/m2]", "refId": "A"},

```

```

105     {"expr": "solarcar_headwind_meter_per_second", "legendFormat": "headwind [m/s]", "refId": "B"},
106     {"expr": "solarcar_route_slope_percent", "legendFormat": "slope [%]", "refId": "C"}
107   ],
108   "fieldConfig": {"defaults": {"custom": {"drawStyle": "line", "lineInterpolation": "smooth", "lineWidth": 2, "fillOpacity": 8, "showPoints":
"never", "spanNulls": true}}, "overrides": [
109     {"matcher": {"id": "byName", "options": "POA [W/m2]"}, "properties": [{"id": "unit", "value": "wattm2"}, {"id": "custom.axisPlacement", "
value": "left"}]},
110     {"matcher": {"id": "byName", "options": "headwind [m/s]"}, "properties": [{"id": "unit", "value": "velocityms"}, {"id": "custom.
axisPlacement", "value": "right"}]},
111     {"matcher": {"id": "byName", "options": "slope [%]"}, "properties": [{"id": "unit", "value": "percent"}, {"id": "custom.axisPlacement", "
value": "right"}]}
112   ]},
113   "options": {"legend": {"displayMode": "list", "placement": "bottom", "showLegend": true}, "tooltip": {"mode": "multi", "sort": "desc"}}
114 },
115 {
116   "type": "timeseries", "title": "Thermal state", "id": 11,
117   "gridPos": {"h": 6, "w": 6, "x": 12, "y": 10},
118   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
119   "targets": [
120     {"expr": "solarcar_battery_temperature_celsius", "legendFormat": "battery", "refId": "A"},
121     {"expr": "solarcar_cell_temperature_celsius", "legendFormat": "cell", "refId": "B"},
122     {"expr": "solarcar_ambient_temperature_celsius", "legendFormat": "ambient", "refId": "C"}
123   ],
124   "fieldConfig": {"defaults": {"unit": "celsius", "custom": {"drawStyle": "line", "lineInterpolation": "smooth", "lineWidth": 2, "fillOpacity
": 8, "showPoints": "never", "spanNulls": true}}, "overrides": []},
125   "options": {"legend": {"displayMode": "list", "placement": "bottom", "showLegend": true}, "tooltip": {"mode": "multi", "sort": "desc"}}
126 },
127 {
128   "type": "gauge", "title": "Freshness / health / control-stop state", "id": 12,
129   "gridPos": {"h": 6, "w": 6, "x": 18, "y": 10},
130   "datasource": {"type": "prometheus", "uid": "solar-prometheus"},
131   "targets": [
132     {"expr": "solarcar_telemetry_fresh * 100", "instant": true, "legendFormat": "telemetry fresh", "refId": "A"},
133     {"expr": "solarcar_system_health_ratio * 100", "instant": true, "legendFormat": "system health", "refId": "B"},
134     {"expr": "solarcar_control_stop_hold * 100", "instant": true, "legendFormat": "control stop active", "refId": "C"}
135   ],
136   "fieldConfig": {"defaults": {"unit": "percent", "min": 0, "max": 100, "color": {"mode": "thresholds"}, "thresholds": {"mode": "absolute", "
steps": [{"color": "red", "value": null}, {"color": "orange", "value": 70}, {"color": "green", "value": 95}]}, "overrides": [
137     {"matcher": {"id": "byName", "options": "control stop active"}, "properties": [{"id": "color", "value": {"mode": "fixedColor", "fixedColor
": "dark-blue"}}]}
138   ]},
139   "options": {"minVizHeight": 75, "minVizWidth": 75, "orientation": "auto", "reduceOptions": {"calcs": ["lastNotNull"], "fields": "", "values
": false}, "showThresholdLabels": false, "showThresholdMarkers": true, "sizing": "auto"}
140 }
141 ],
142 "refresh": "1s",
143 "schemaVersion": 41,
144 "tags": ["YATA", "BWSC", "MPC", "solarcar"],
145 "templating": {"list": []},
146 "time": {"from": "now-30m", "to": "now"},
147 "timepicker": {"refresh_intervals": ["1s", "2s", "5s", "10s", "30s", "1m"]},
148 "timezone": "browser",
149 "title": "YATA SolarCar EMS",
150 "uid": "solarcar-ems",
151 "version": 1,
152 "weekStart": "monday"
153 }

```

A.42 config/solar/bwsc_2027_demo.yaml

```

1 meta:
2   name: bwsc_2027_demo
3   purpose: Default wrapper profile that points to the blank BWSC 2027 template package.
4   notes:
5     - This profile exists so SolarSim.ps1 has a valid default profile path in the source tree.
6     - The referenced package is intentionally blank. Replace route, weather, race, map, model,
7       and network values before operation.
8     - Simulation and live modes are expected to fail until the required CSV and YAML
9       files are filled.
10  paths:
11    route_waypoints_csv: ../../project_packages/bwsc2027_template/data/route/route_waypoints.csv
12    route_profile_csv: ../../project_packages/bwsc2027_template/data/route/route_profile.csv
13    speed_profile_csv: ../../project_packages/bwsc2027_template/data/route/speed_profile.csv
14    forecast_csv: ../../project_packages/bwsc2027_template/data/weather/forecast_10min.csv
15    stop_yaml: ../../project_packages/bwsc2027_template/data/race/control_stops.yaml
16    drive_schedule_yaml: ../../project_packages/bwsc2027_template/data/race/drive_schedule.yaml
17    drive_eff_map: ../../project_packages/bwsc2027_template/maps/drive_eff_map.csv
18    regen_eff_map: ../../project_packages/bwsc2027_template/maps/regen_eff_map.csv
19    rint_map: ../../project_packages/bwsc2027_template/maps/rint_map.csv
20    panel_eff_map: ../../project_packages/bwsc2027_template/maps/panel_eff_map.csv
21    mppt_eff_map: ../../project_packages/bwsc2027_template/maps/mppt_eff_map.csv
22    drive_map_eco: ../../project_packages/bwsc2027_template/maps/drive_eff_map_eco.csv
23    drive_map_power: ../../project_packages/bwsc2027_template/maps/drive_eff_map_power.csv
24    regen_map_eco: ../../project_packages/bwsc2027_template/maps/regen_eff_map_eco.csv
25    regen_map_power: ../../project_packages/bwsc2027_template/maps/regen_eff_map_power.csv
26    ocv_soc_map: ../../project_packages/bwsc2027_template/maps/ocv_soc_curve.csv
27  runtime:
28    dashboard_host: 0.0.0.0
29    dashboard_port: 8080
30    forecast_time_mode: relative
31    forecast_time_tz: Australia/Darwin
32  logging:
33    log_dir: ../../project_packages/bwsc2027_template/outputs/logs
34    log_rate_hz: 2.0
35  simulation:
36    gps_rate_hz: 1.0
37    gps_init_speed_kmh: 45.0
38    output_dir: ../../project_packages/bwsc2027_template/outputs/prerace

```

```

39  output_prefix: bwsc2027_template
40  auto_version_outputs: true
41  latest_manifest_json: ../../project_packages/bwsc2027_template/outputs/prerace/latest_simulation_run.json
42  start_utc: '2027-08-22T22:30:00Z'
43  start_s_km: 0.0
44  soc0: 0.95
45  Tb0: 30.0
46  v0_kmh: 45.0
47  energy_budget: false
48  logging:
49    file_prefix: solar_prerace
50  measurement:
51    use_distance_node: true
52    distance:
53      publish_rate_hz: 2.0
54      max_dt_sec: 2.5
55    use_grade_node: true
56    grade:
57      gps_topic: /vehicle/gps
58      altitude_topic: /vehicle/altitude_m
59      min_speed_kmh: 5.0
60      altitude_alpha: 0.2
61      min_delta_s_km: 0.01
62    preflight:
63      startup_grace_sec: 10.0
64      measurement_timeout_sec: 3.0
65      require_speed: true
66      require_distance: true
67      require_battery: true
68      require_planner: true
69    logging:
70      file_prefix: solar_measurement
71  identification:
72    input_dir: ../../project_packages/bwsc2027_template/data/identification/raw
73    output_dir: ../../project_packages/bwsc2027_template/outputs/identification
74  live:
75    use_distance_node: true
76    distance:
77      publish_rate_hz: 2.0
78      max_dt_sec: 2.5
79    use_grade_node: true
80    forecast_time_mode: absolute
81    forecast_time_tz: Australia/Darwin
82    grade:
83      gps_topic: /vehicle/gps
84      altitude_topic: /vehicle/altitude_m
85      min_speed_kmh: 5.0
86      altitude_alpha: 0.2
87      min_delta_s_km: 0.01
88    weather:
89      enabled: true
90      provider: openmeteo
91      gps_topic: /chase/gps
92      fetch_period_sec: 3600.0
93      forecast_days: 3
94      step_minutes: 10
95      timezone_name: Australia/Darwin
96      fallback_latitude: -12.4634
97      fallback_longitude: 130.8456
98      tccl_gain: 0.03
99      raw_forecast_csv: ../../project_packages/bwsc2027_template/outputs/runtime/live_forecast_raw.csv
100  autocal:
101    enabled: true
102    publish_period_sec: 30.0
103    stationary_speed_kmh: 2.0
104    drive_speed_kmh: 25.0
105    day_ghi_threshold: 150.0
106    alpha: 0.2
107    solar_gain_init: 1.0
108    drive_power_gain_init: 1.0
109    aux_power_w_init: 0.0
110    solar_gain_min: 0.5
111    solar_gain_max: 1.5
112    drive_power_gain_min: 0.7
113    drive_power_gain_max: 1.4
114    aux_power_w_min: 0.0
115    aux_power_w_max: 300.0
116  command_bridge:
117    enabled: true
118    output_speed_topic: /vehicle/speed_cmd_kmh
119    output_drive_mode_topic: /vehicle/drive_mode_cmd
120    udp_enabled: false
121    udp_host: 127.0.0.1
122    udp_port: 50050
123    publish_rate_hz: 5.0
124    input_timeout_sec: 3.0
125    safe_speed_kmh: 0.0
126    startup_hold_sec: 2.0
127    filter_tau_sec: 1.0
128    accel_limit_kmhps: 1.2
129    decel_limit_kmhps: 3.5
130    speed_deadband_kmh: 0.1
131    speed_quantize_step_kmh: 0.1
132    max_output_speed_kmh: 120.0
133    drive_mode_min_hold_sec: 5.0
134    require_system_running: true
135    system_state_timeout_sec: 2.5
136  wifi_bridge:
137    enabled: true
138    enable_inbound: true
139    enable_outbound: true
140    bind_host: 0.0.0.0
141    bind_port: 52001

```

```

142     publish_period_sec: 1.0
143     solar_remote_host: 192.168.50.21
144     solar_remote_port: 52002
145     chase_remote_host: 192.168.50.22
146     chase_remote_port: 52003
147     send_to_solar: true
148     send_to_chase: true
149     speed_filter_tau_sec: 0.6
150     speed_max_kmh: 130.0
151     speed_max_accel_kmhps: 12.0
152     speed_max_decel_kmhps: 20.0
153     distance_max_rate_kmps: 0.06
154     distance_max_backtrack_km: 0.02
155     battery_filter_tau_sec: 1.0
156     wind_filter_tau_sec: 1.0
157     headwind_filter_tau_sec: 0.8
158     max_abs_headwind_ms: 25.0
159     timestamp_required: true
160     max_packet_age_sec: 5.0
161     max_future_skew_sec: 2.0
162     max_out_of_order_sec: 0.0
163     wind_model:
164         enabled: true
165         corrected_forecast_csv: ../../project_packages/bwsc2027_template/outputs/runtime/live_forecast_corrected.csv
166         publish_period_sec: 30.0
167         measurement_sigma_ms: 1.0
168         correlation_distance_km: 300.0
169         fallback_correlation_time_h: 3.0
170         forecast_sigma0_ms: 1.5
171         forecast_variance_growth_per_hour: 0.05
172         planning_quantile: 0.5
173         confidence_z: 1.96
174         min_sigma_ms: 0.2
175         preferred_source: auto
176         use_exp_distance_decay: true
177     logging:
178         file_prefix: solar_live
179     model:
180         m: 0.0
181         CdA: 0.0
182         Crr: 0.0
183         Crr_per_wheel: 0.0
184         wheel_count: 3
185         motor_count: 2
186         motor_type: inwheel
187         gear_eta: 1.0
188         gear_ratio: 1.0
189         wheel_radius: 0.0
190         inverter_eta: 1.0
191         ocv_soc_map: ''
192         P_aux: 0.0
193         P_aux_stopped: 0.0
194         P_aux_night: 0.0
195         aux_night_ghi_threshold_wm2: 20.0
196         pv_area: 0.0
197         pv_eta_ref: 0.0
198         pv_mu_p: -0.0045
199         mppt_eta: 0.95
200         E_nom_Wh: 0.0
201         Q_nom_Ah: 0.0
202         r_polarization_ohm: 0.0
203         polarization_tau_sec: 60.0
204         V_min: 0.0
205         V_max: 0.0
206         I_max: 0.0
207         I_chg_min: 0.0
208         T_min: -20.0
209         T_max: 60.0
210     mpc:
211         v_max_kmh: 110.0
212         terminal_soc_min: 0.2
213         soc_finish_target: 0.2
214         soc_finish_tol: 0.005
215         soc_min: 0.2
216         soc_max: 0.98
217         w_current: 0.02
218         w_dv: 0.05
219         w_dv_limit: 2.0
220         dv_max_kmhps: 5.0
221         upper_mode: distance
222         upper_ds_km: 10.0
223         upper_horizon_km: 3000.0
224         upper_max_steps: 400
225         upper_max_iter: 40
226         upper_global_search_enabled: true
227         upper_global_search_mode: auto
228         upper_cem_generations: 4
229         upper_cem_population: 16
230         upper_cem_elite: 4
231         upper_local_refine_topk: 4
232         upper_shgo_samples: 256
233         upper_shgo_iters: 2
234         upper_cert_grid_levels: 0
235         upper_cert_max_evaluations: 250000
236         upper_day_end_soc_min: 0.2
237         soc_guard_margin: 0.01
238         soc_guard_mode: stop
239         hierarchical: true
240         lower_dt: 1.0
241         lower_horizon_steps: 30
242         lower_rate_hz: 5.0
243         w_track: 6.0
244         w_throttle: 0.2

```

```

245 throttle_rate_limit: 30.0
246 dt: 600.0
247 horizon_steps: 6
248 race_km: 3035.5
249 upper_cost:
250     objective_mode: fastest_feasible
251     w_wait: 1.0
252     w_travel_time: 1.0
253     w_soc_terminal: 1.0e10
254     constraint_penalty: 1.0e10

```

A.43 package.xml

```

1 <?xml version="1.0"?>
2 <package format="3">
3   <name>mpc_solarcar</name>
4   <version>0.1.0</version>
5   <description>ROS2 nodes for solar car MPC with simulated GPS and a simple control panel.</description>
6   <maintainer email="you@example.com">you</maintainer>
7   <license>MIT</license>
8
9   <buildtool_depend>ament_python</buildtool_depend>
10
11   <exec_depend>rcclpy</exec_depend>
12   <exec_depend>std_msgs</exec_depend>
13   <exec_depend>sensor_msgs</exec_depend>
14   <exec_depend>geometry_msgs</exec_depend>
15   <exec_depend>nav_msgs</exec_depend>
16   <exec_depend>std_srvs</exec_depend>
17   <exec_depend>rccl_interfaces</exec_depend>
18   <exec_depend>ament_index_python</exec_depend>
19   <exec_depend>launch</exec_depend>
20   <exec_depend>launch_ros</exec_depend>
21   <exec_depend>python3-numpy</exec_depend>
22   <exec_depend>python3-scipy</exec_depend>
23   <exec_depend>python3-pandas</exec_depend>
24   <exec_depend>python3-yaml</exec_depend>
25   <exec_depend>python3-matplotlib</exec_depend>
26   <exec_depend>python3-casadi</exec_depend>
27
28   <export>
29     <build_type>ament_python</build_type>
30   </export>
31 </package>

```

A.44 setup.py

```

1 import os
2
3 from setuptools import setup
4
5 package_name = "mpc_solarcar"
6
7
8 def collect_data_files(src_dir, *, excluded_dirs=()):
9     collected = []
10    if not os.path.isdir(src_dir):
11        return collected
12    excluded = {"__pycache__", *excluded_dirs}
13    for root, dirs, files in os.walk(src_dir):
14        dirs[:] = [d for d in dirs if d not in excluded]
15        if not files:
16            continue
17        collected.append((
18            os.path.join("share", package_name, root),
19            [os.path.join(root, f) for f in files],
20        ))
21    return collected
22
23
24 data_files = [
25     ("share/ament_index/resource_index/packages", ["resource/" + package_name]),
26     ("share/" + package_name, ["package.xml"]),
27 ]
28
29 for directory in ("launch", "maps", "templates", "dashboard", "config", "data"):
30     data_files.extend(collect_data_files(directory))
31 # Source evidence is retained under inputs/external_docs, but ROS nodes never
32 # read it at runtime. Excluding it avoids copying large ZIP/PPTX/PDF evidence on
33 # every colcon build, especially across the WSL/OneDrive filesystem boundary.
34 data_files.extend(collect_data_files("inputs", excluded_dirs=("external_docs",)))
35 # Offline scripts and manuals are intentionally consumed from the GitHub/ZIP
36 # source tree. Installing them into share/ duplicates hundreds of generated
37 # files but provides no ROS runtime capability.
38
39 setup(
40     name=package_name,
41     version="0.1.0",
42     packages=[package_name],
43     data_files=data_files,
44     install_requires=[
45         "setuptools",

```

```
46     "numpy",
47     "scipy",
48     "pandas",
49     "pyyaml",
50     "matplotlib",
51     "casadi",
52 ],
53 zip_safe=True,
54 maintainer="you",
55 maintainer_email="you@example.com",
56 description="ROS2 nodes for solar car MPC with simulated GPS and a simple control panel.",
57 license="MIT",
58 tests_require=["pytest"],
59 entry_points={
60     "console_scripts": [
61         "gps_sim_node = mpc_solarcar.gps_sim_node:main",
62         "mpc_node = mpc_solarcar.mpc_node:main",
63         "dashboard_node = mpc_solarcar.dashboard_node:main",
64         "distance_node = mpc_solarcar.distance_node:main",
65         "solar_logger_node = mpc_solarcar.solar_logger_node:main",
66         "solar_preflight_node = mpc_solarcar.solar_preflight_node:main",
67         "grade_node = mpc_solarcar.grade_node:main",
68         "weather_fetch_node = mpc_solarcar.weather_fetch_node:main",
69         "solar_autocal_node = mpc_solarcar.solar_autocal_node:main",
70         "speed_command_bridge_node = mpc_solarcar.speed_command_bridge_node:main",
71         "telemetry_text_bridge_node = mpc_solarcar.telemetry_text_bridge_node:main",
72         "wind_correction_node = mpc_solarcar.wind_correction_node:main",
73         "solar_state_node = mpc_solarcar.solar_state_node:main",
74     ],
75 },
76 )
```